

Efficient Extraction from Effectful E-Graphs



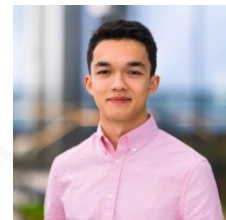
WPLSE



WPLSE



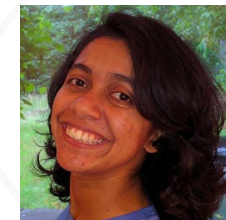
WPLSE



WPLSE



WPLSE



WPLSE
certora



Google



WPLSE



WPLSE

Oliver Flatt, Anjali Pal, Yihong Zhang, Ryan Tjoa, Kirsten Graham, Alex Fischman,
Chandrakana Nandi, Eli Rosenthal, Zachary Tatlock, Haobin Ni

Efficient Extraction from Effectful E-Graphs

side-effects (e.g. mutating memory, I/O, etc.)



WPLSE



WPLSE



WPLSE



WPLSE



WPLSE



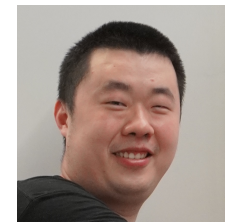
WPLSE
certora



Google

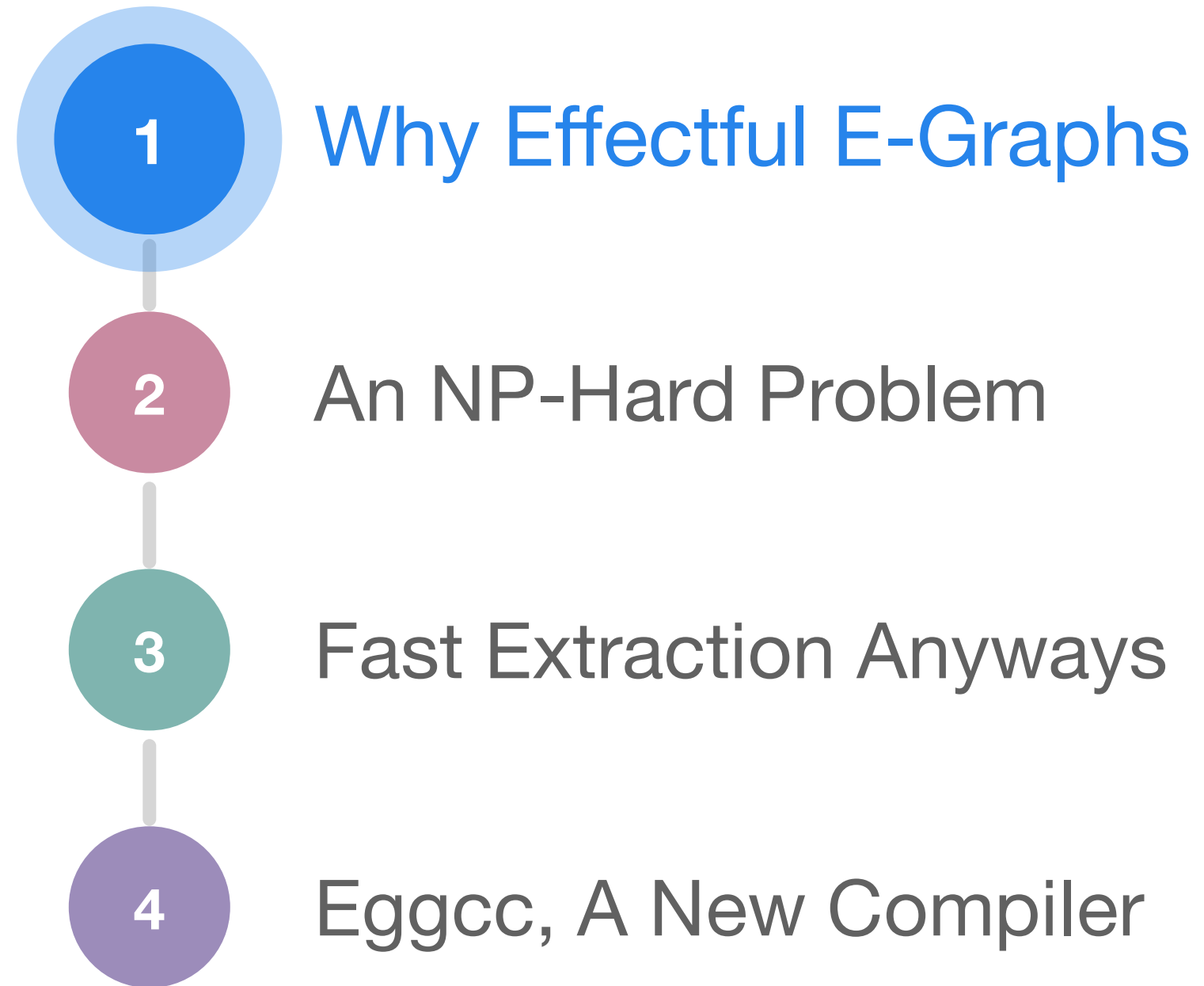


WPLSE



WPLSE

Oliver Flatt, Anjali Pal, Yihong Zhang, Ryan Tjoa, Kirsten Graham, Alex Fischman,
Chandrakana Nandi, Eli Rosenthal, Zachary Tatlock, Haobin Ni

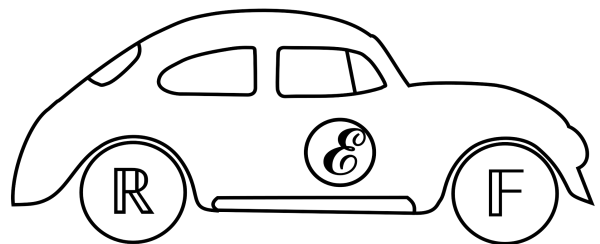




E-Graphs are Super Useful!

E-Graphs are Super Useful!

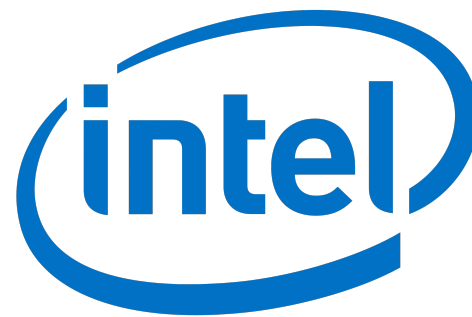
Floating point



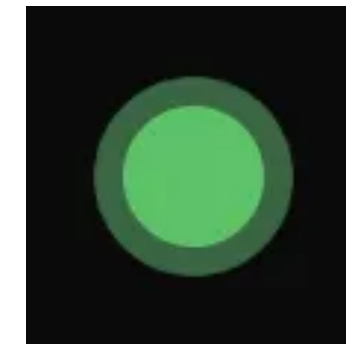
Python Simulations



RTL Optimization



ML Kernels



Luminal

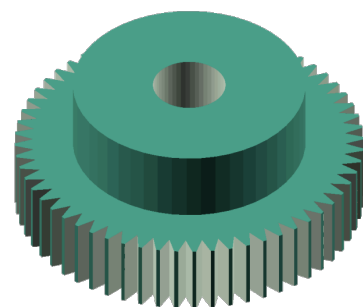
Verification



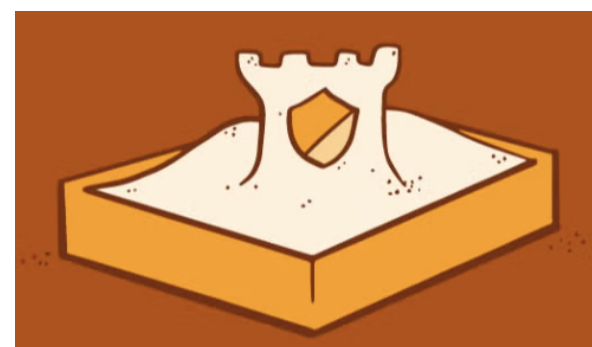
SQL



CAD



WASM

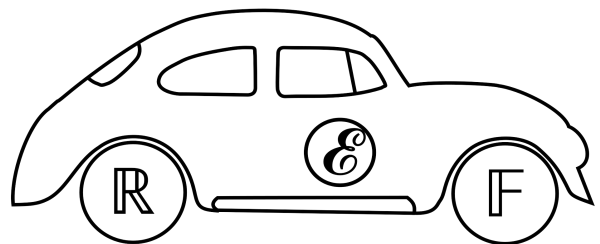


User DSL
Optimizations



E-Graphs are Super Useful!

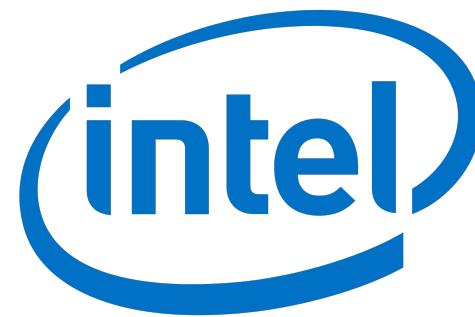
Floating point



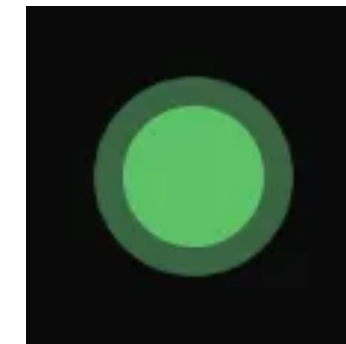
Python Simulations



RTL Optimization



ML Kernels



Luminal

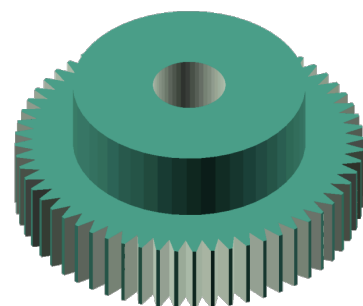
Verification



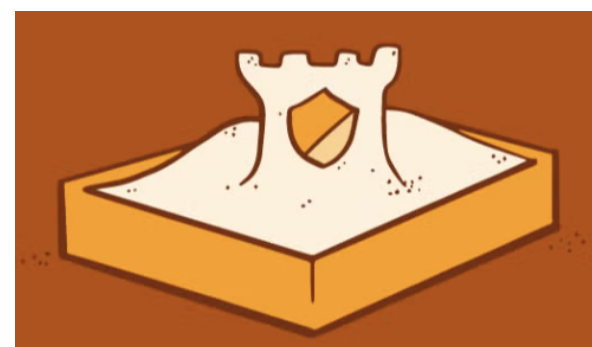
SQL



CAD



WASM



User DSL
Optimizations



...and more!

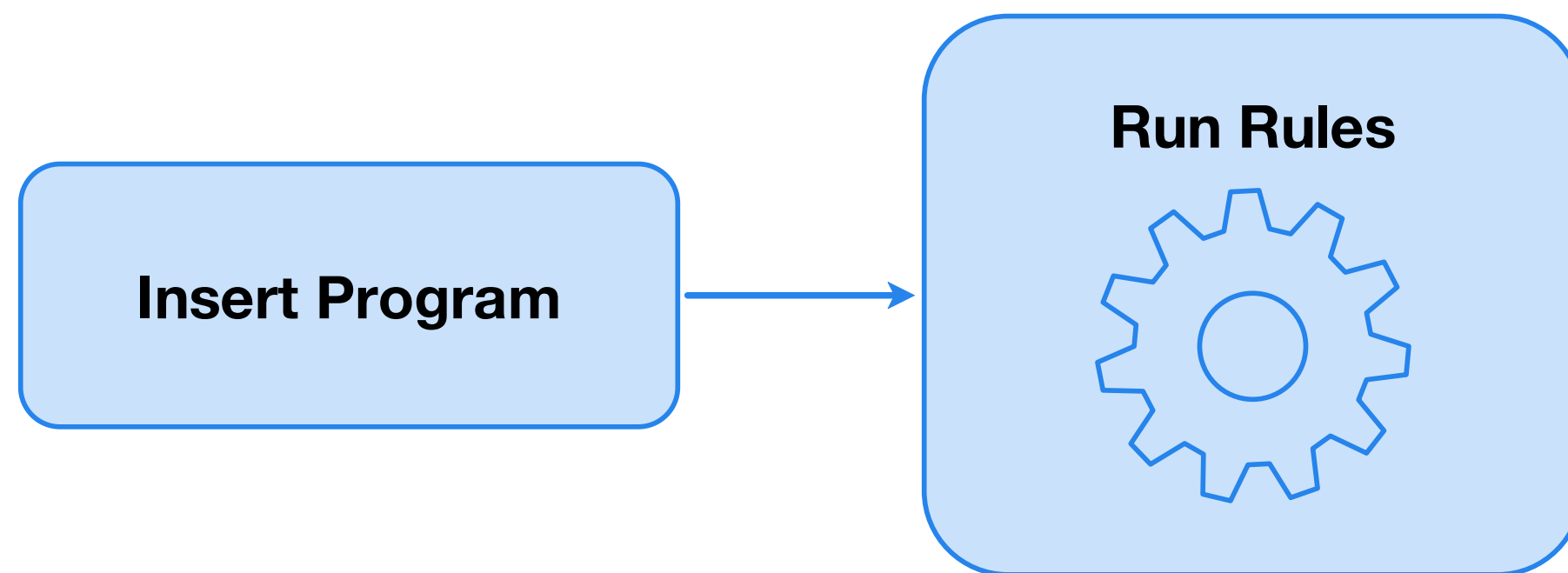


E-Graphs Explore a Big Space

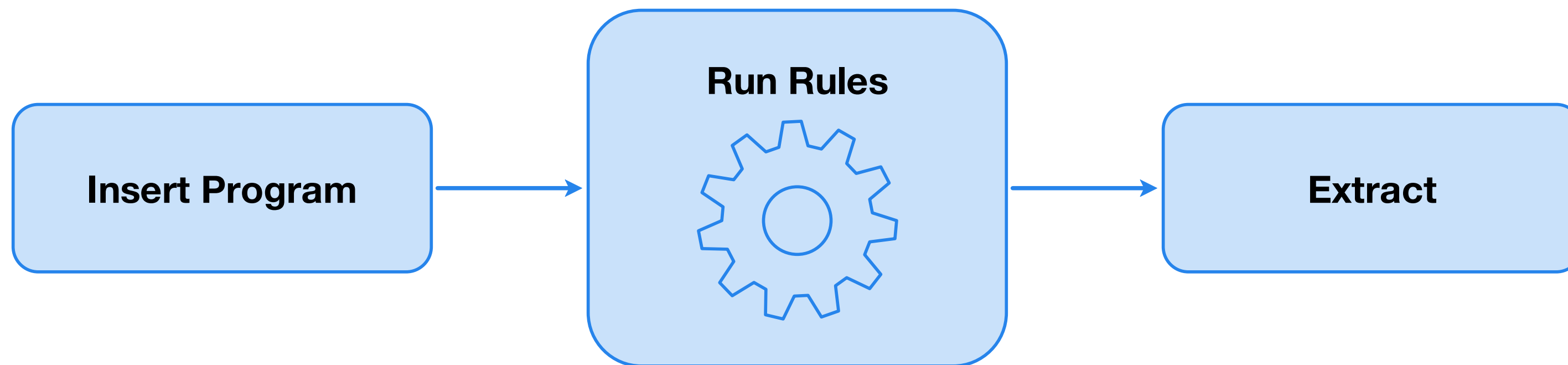
E-Graphs Explore a Big Space

Insert Program

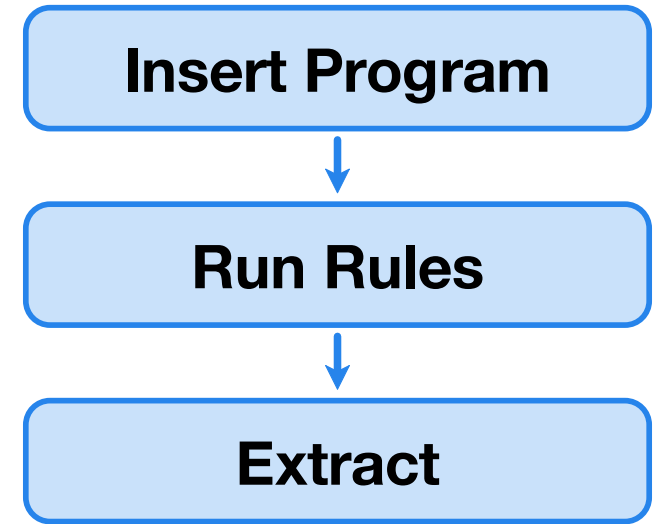
E-Graphs Explore a Big Space



E-Graphs Explore a Big Space



E-Graphs Explore a Big Space



E-Graphs Explore a Big Space

Insert Program



Run Rules



Extract

E-Graphs Explore a Big Space

$(a + 1) - 1$

Insert Program



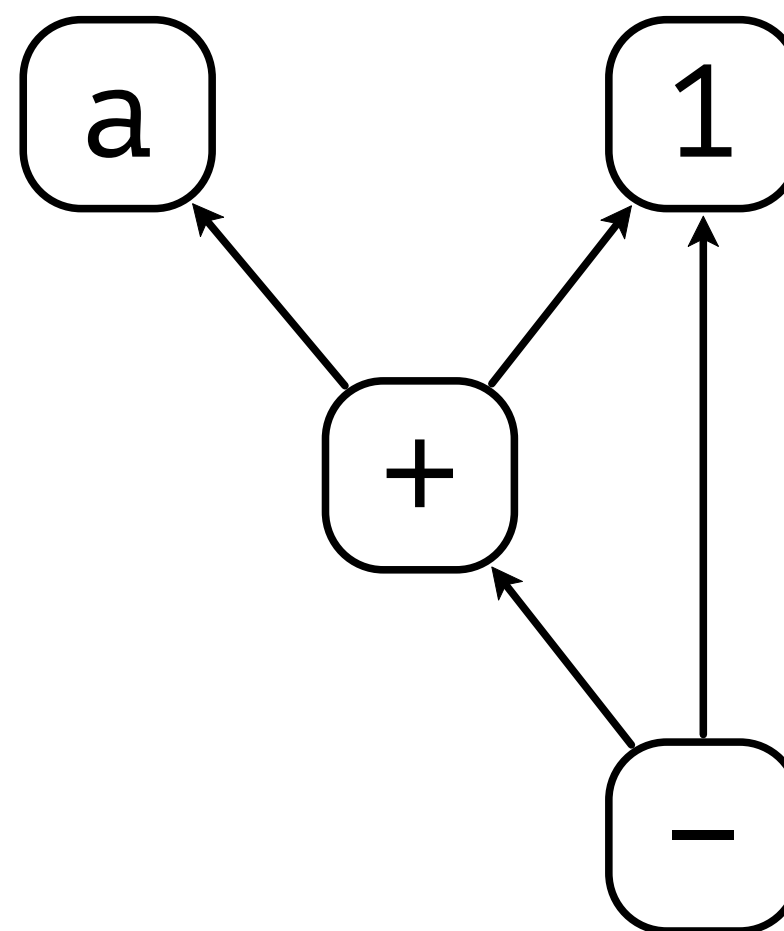
Run Rules



Extract

E-Graphs Explore a Big Space

$$(a + 1) - 1$$



Insert Program



Run Rules



Extract

E-Graphs Explore a Big Space

$$(a + 1) - 1$$

Insert Program

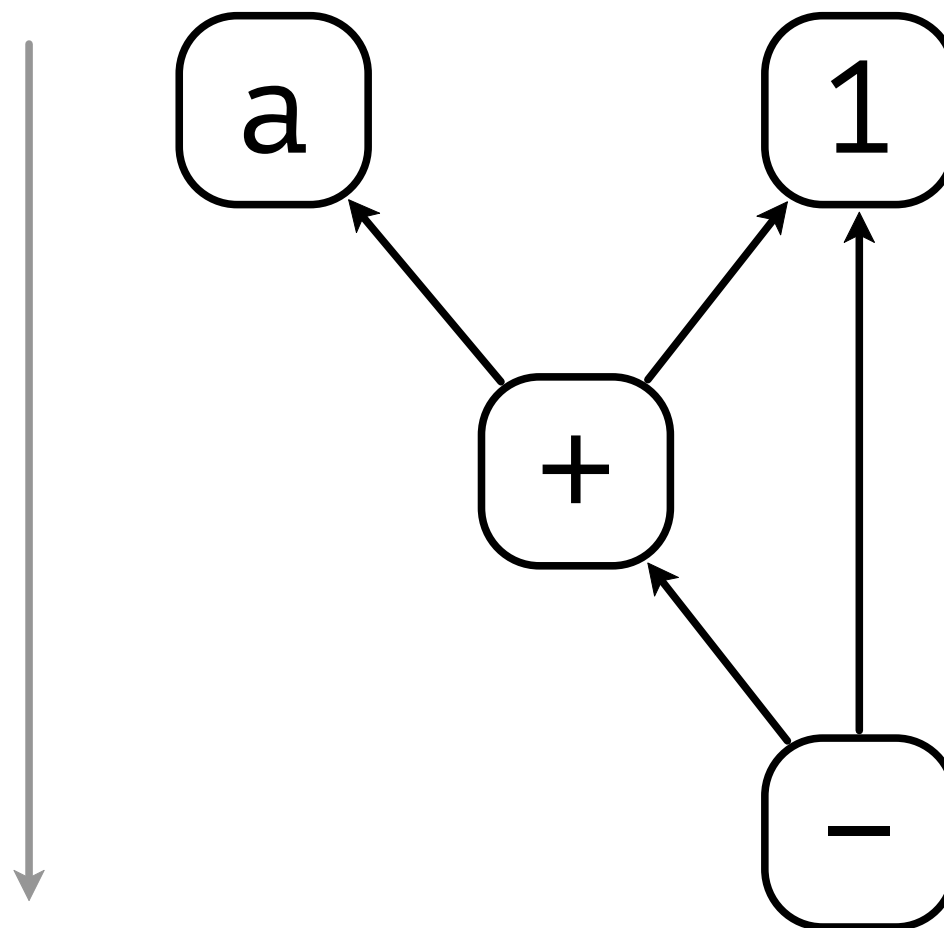


Run Rules



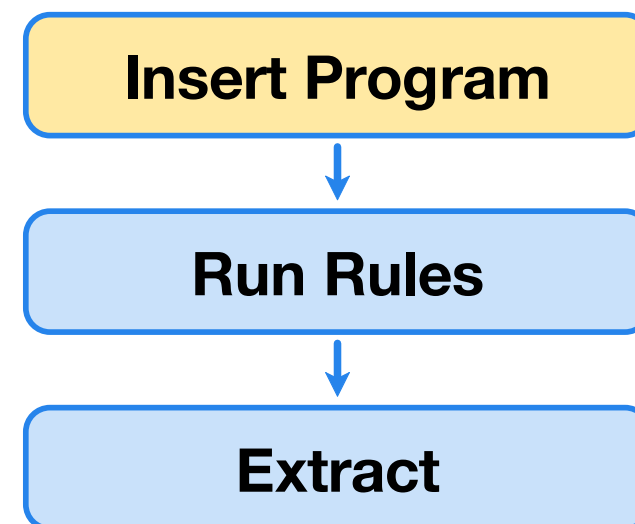
Extract

data flow

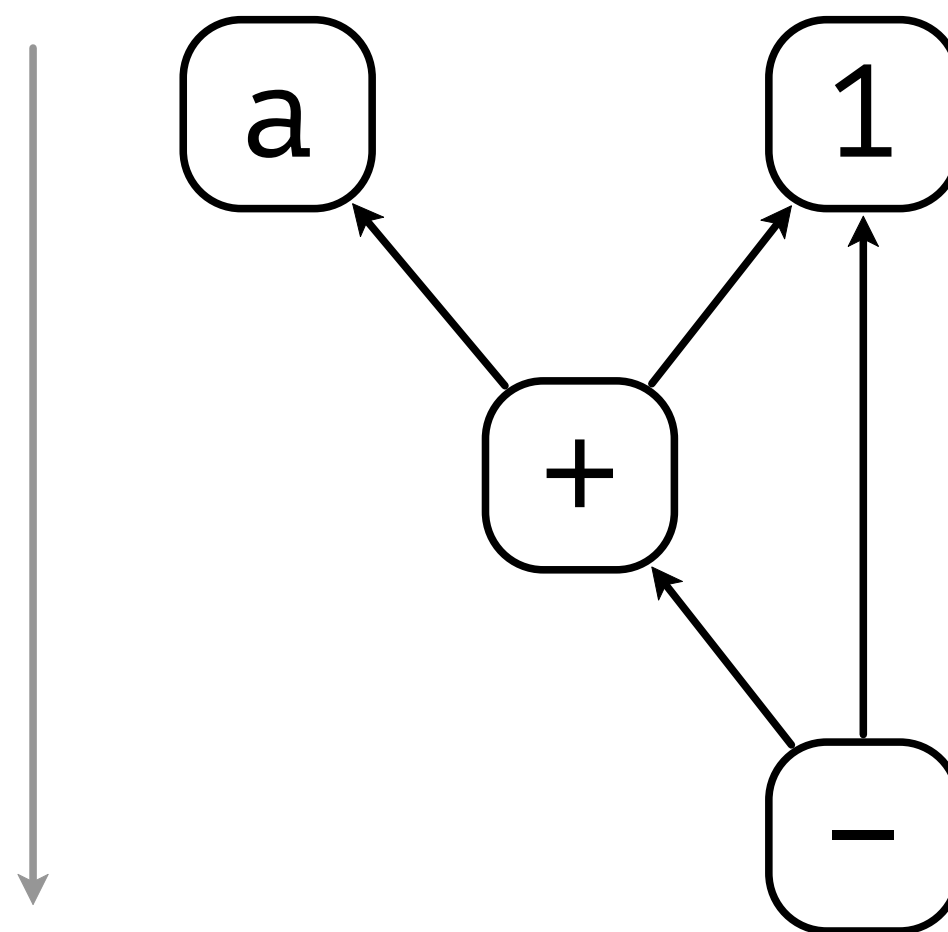


E-Graphs Explore a Big Space

$(a + 1) - 1$



data flow



E-Graphs Explore a Big Space

$(a + 1) - 1$

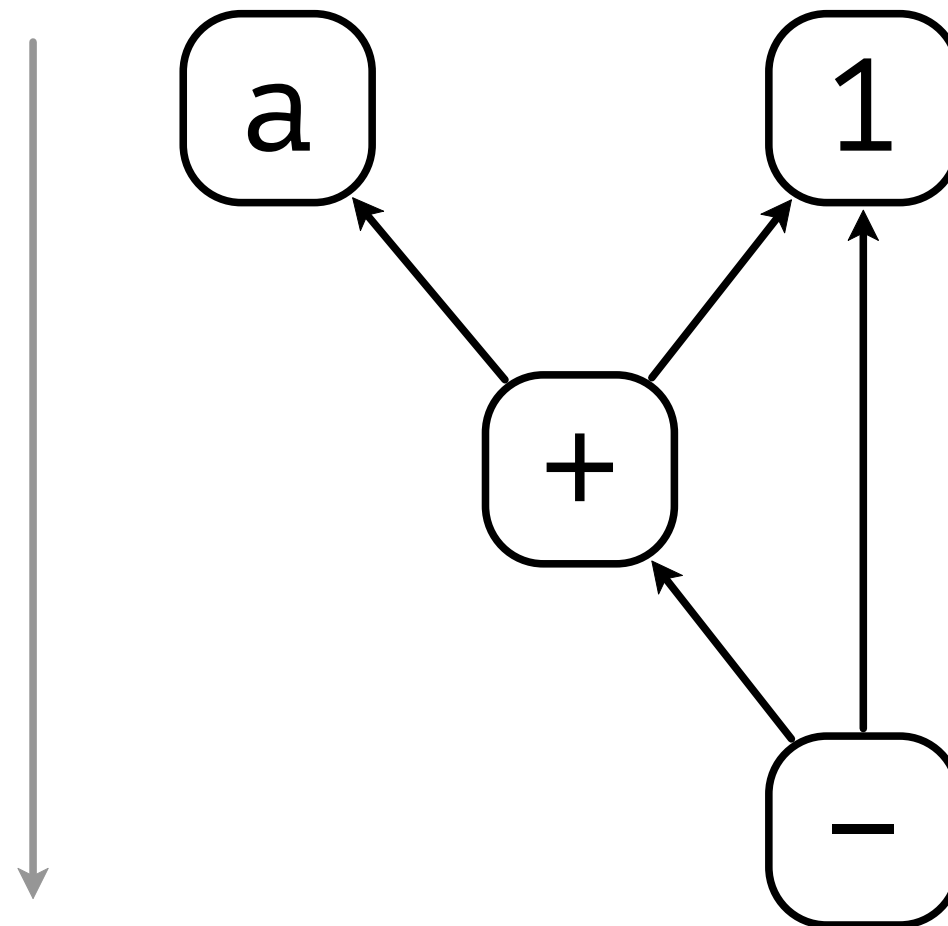
$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

Insert Program

Run Rules

Extract

data flow

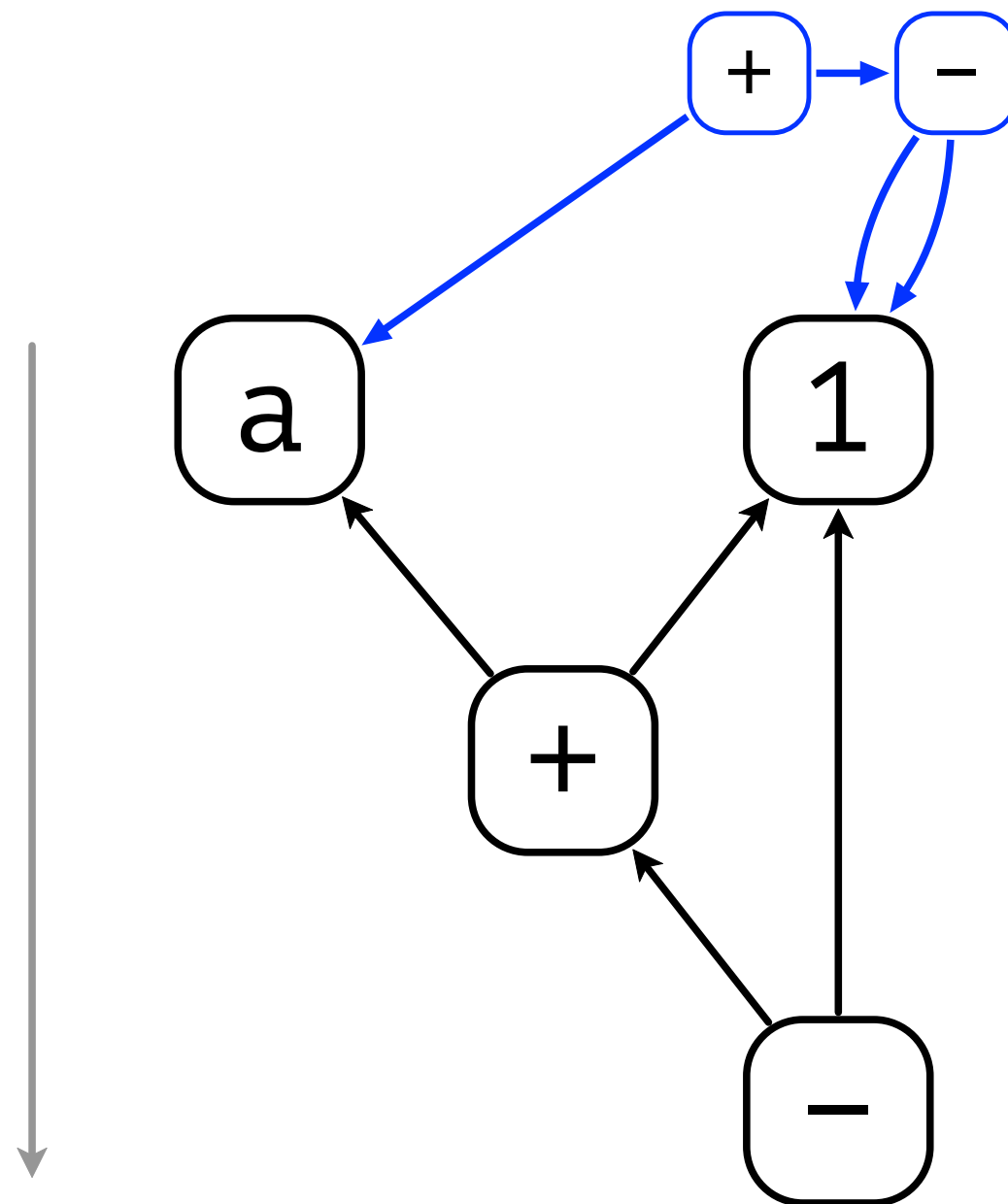


E-Graphs Explore a Big Space

$(a + 1) - 1$

$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

data flow



Insert Program

Run Rules

Extract

E-Graphs Explore a Big Space

$$(a + 1) - 1$$

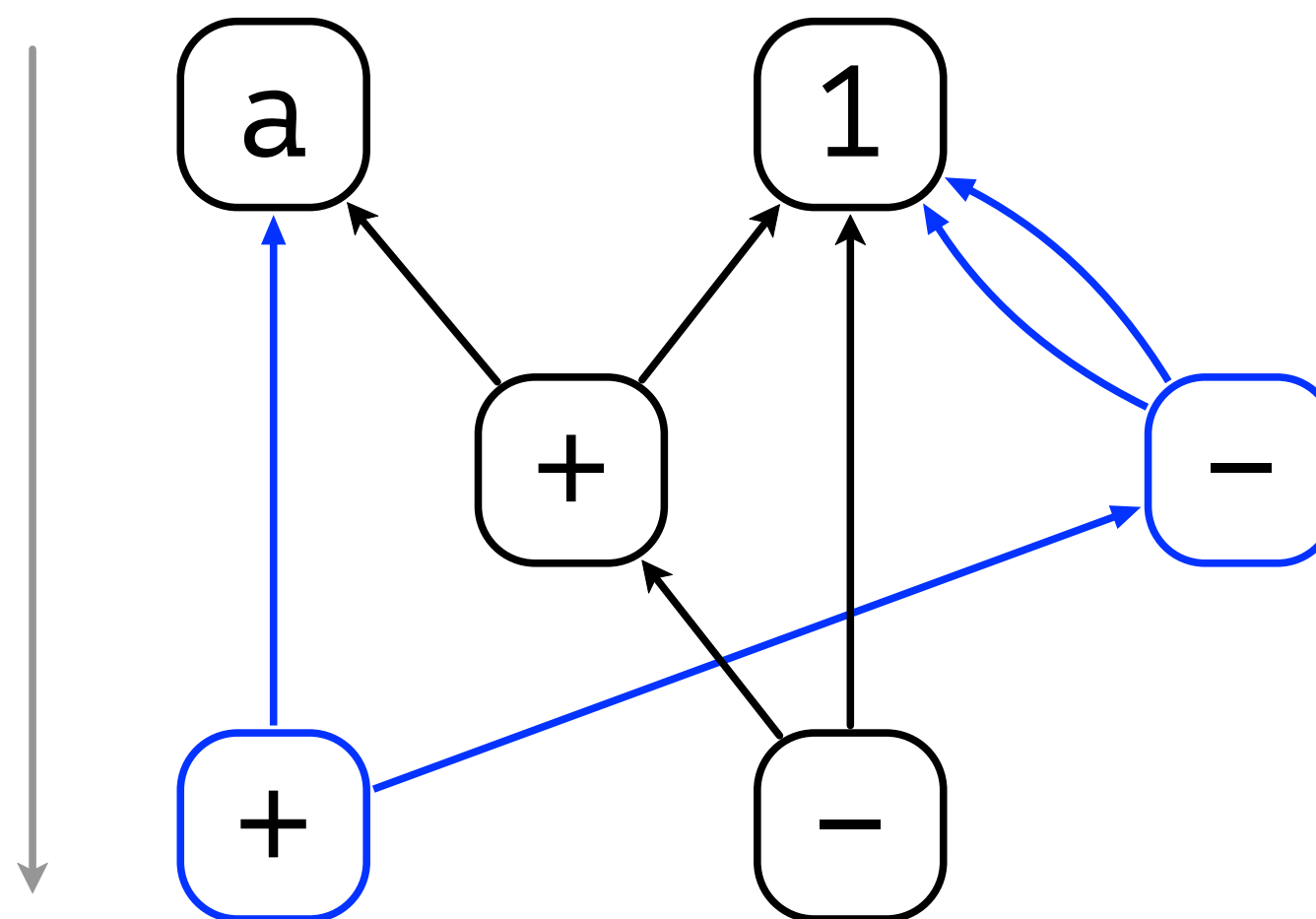
$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

Insert Program

Run Rules

Extract

data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

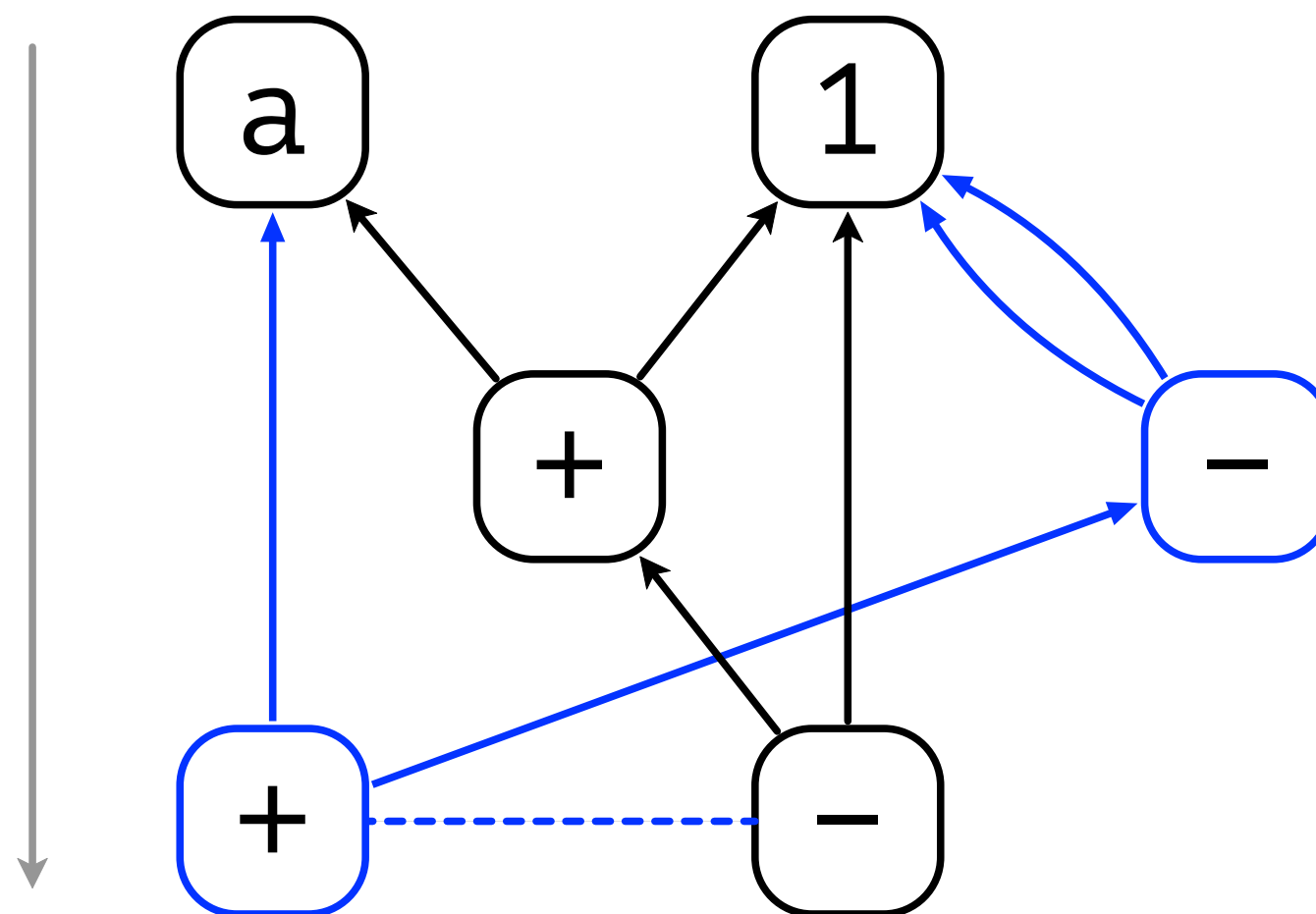
$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

Insert Program

Run Rules

Extract

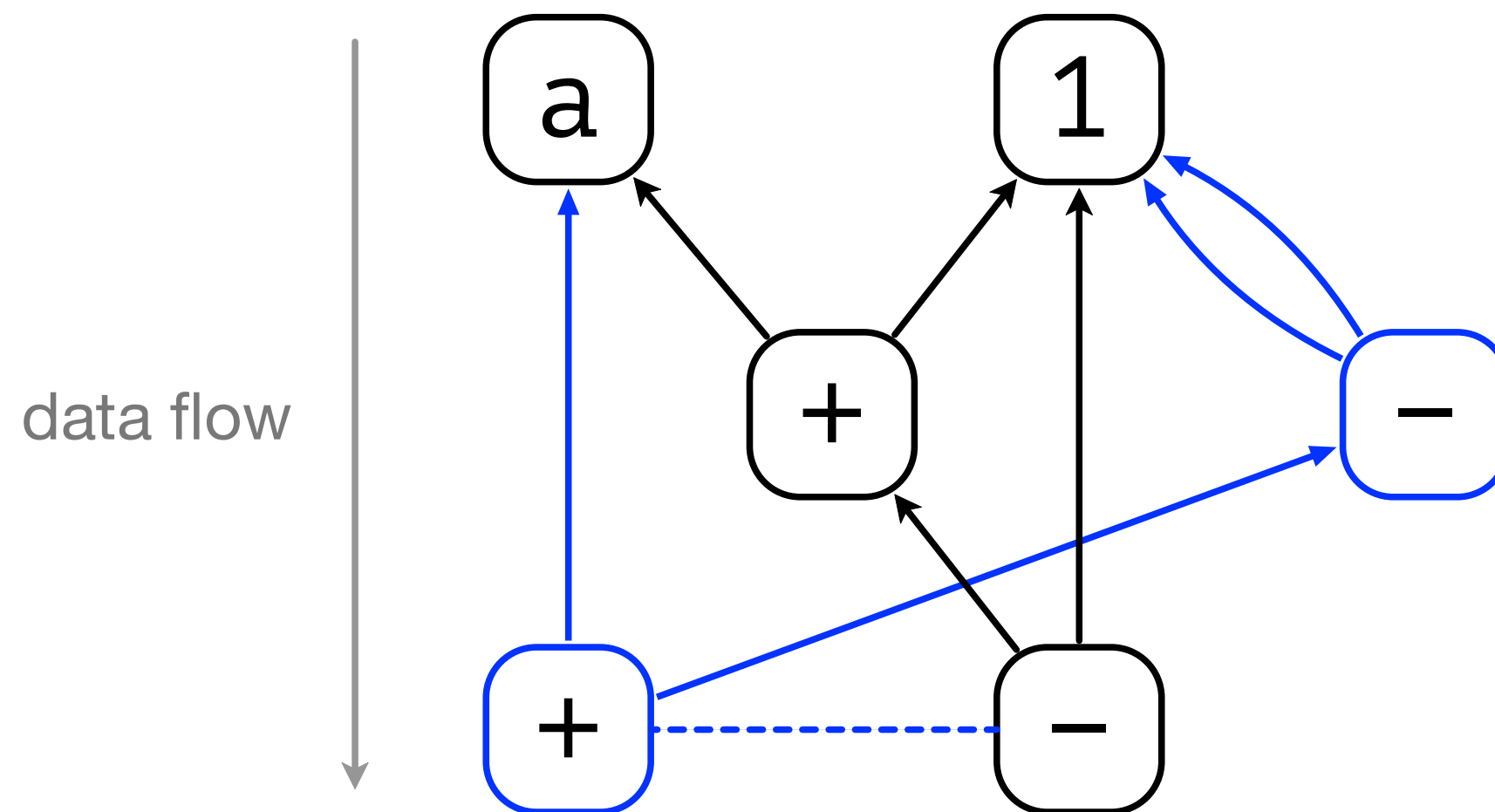
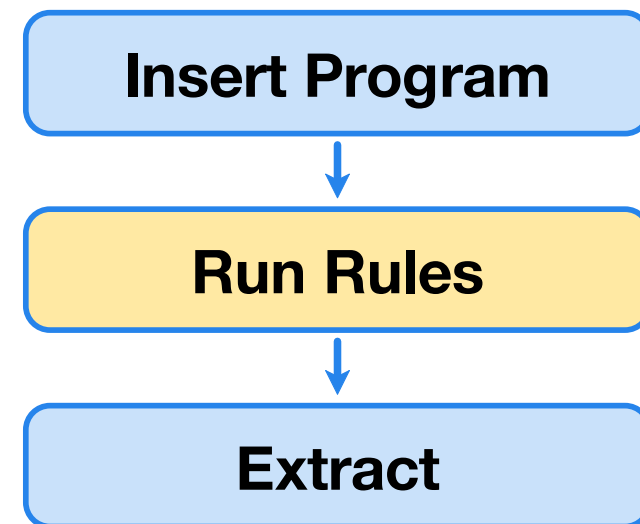
data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha - \alpha \rightarrow 0$$

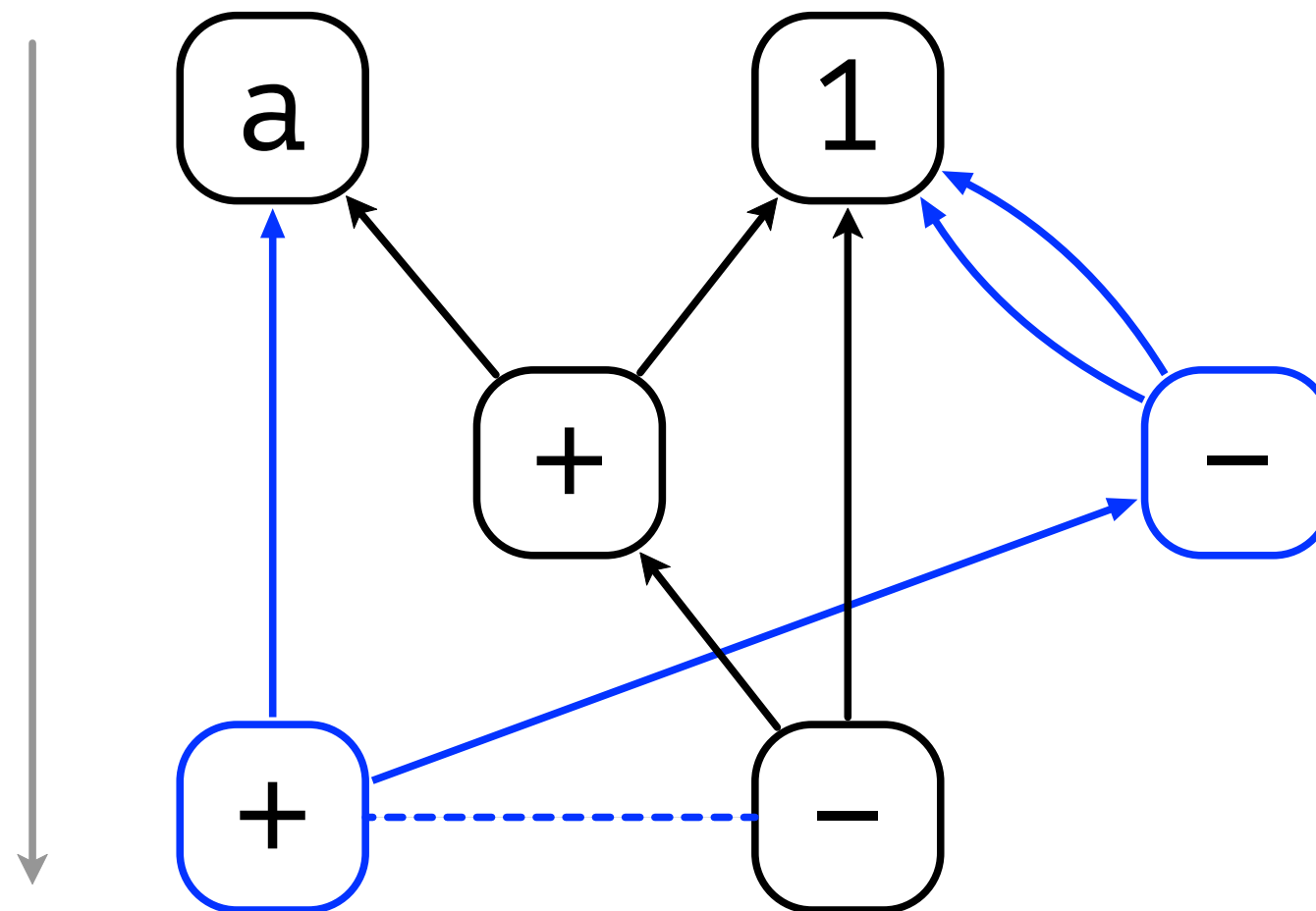
$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

Insert Program

Run Rules

Extract

data flow



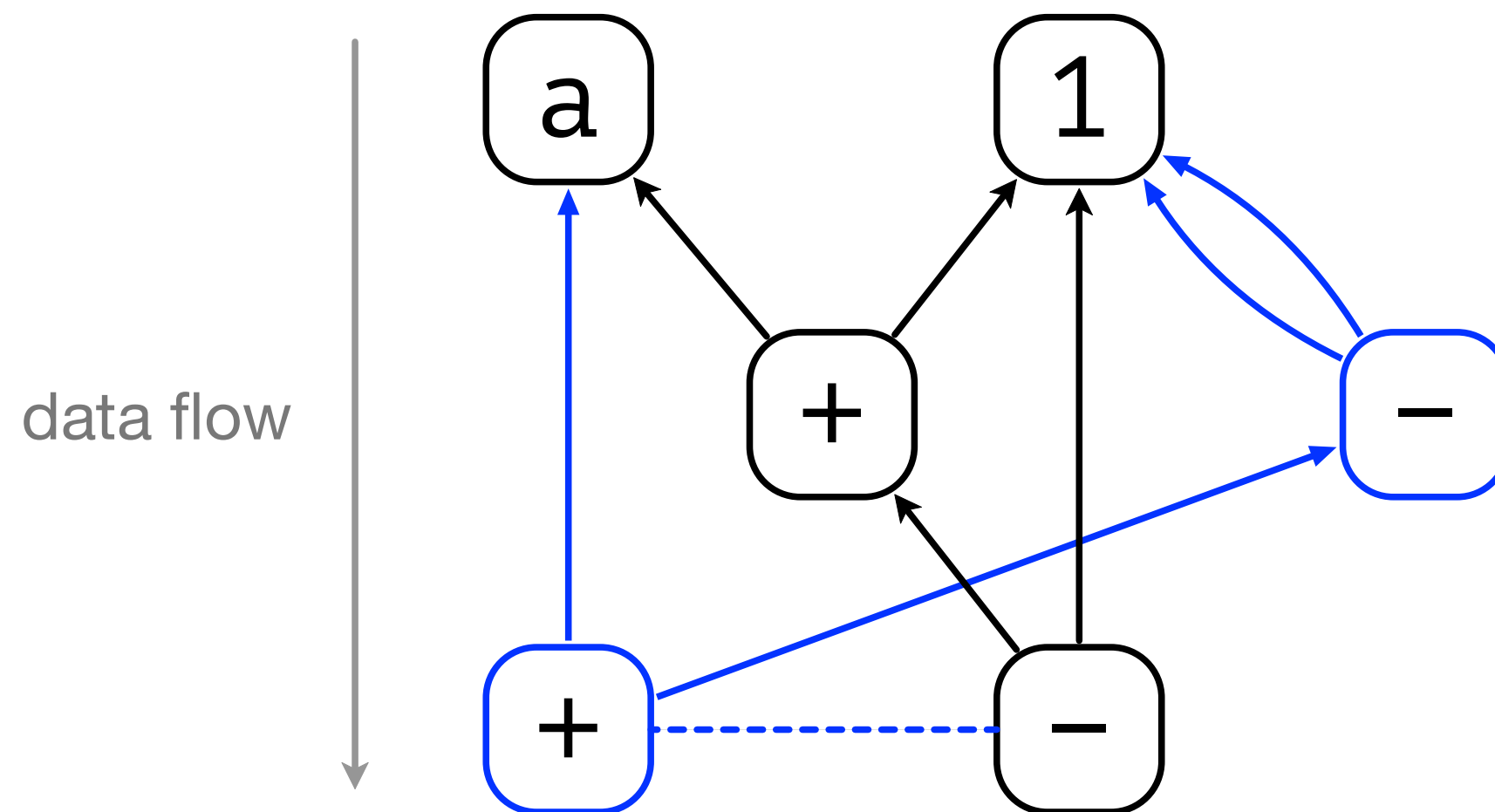
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$0$$



Insert Program

Run Rules

Extract

E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha - \alpha \rightarrow 0$$

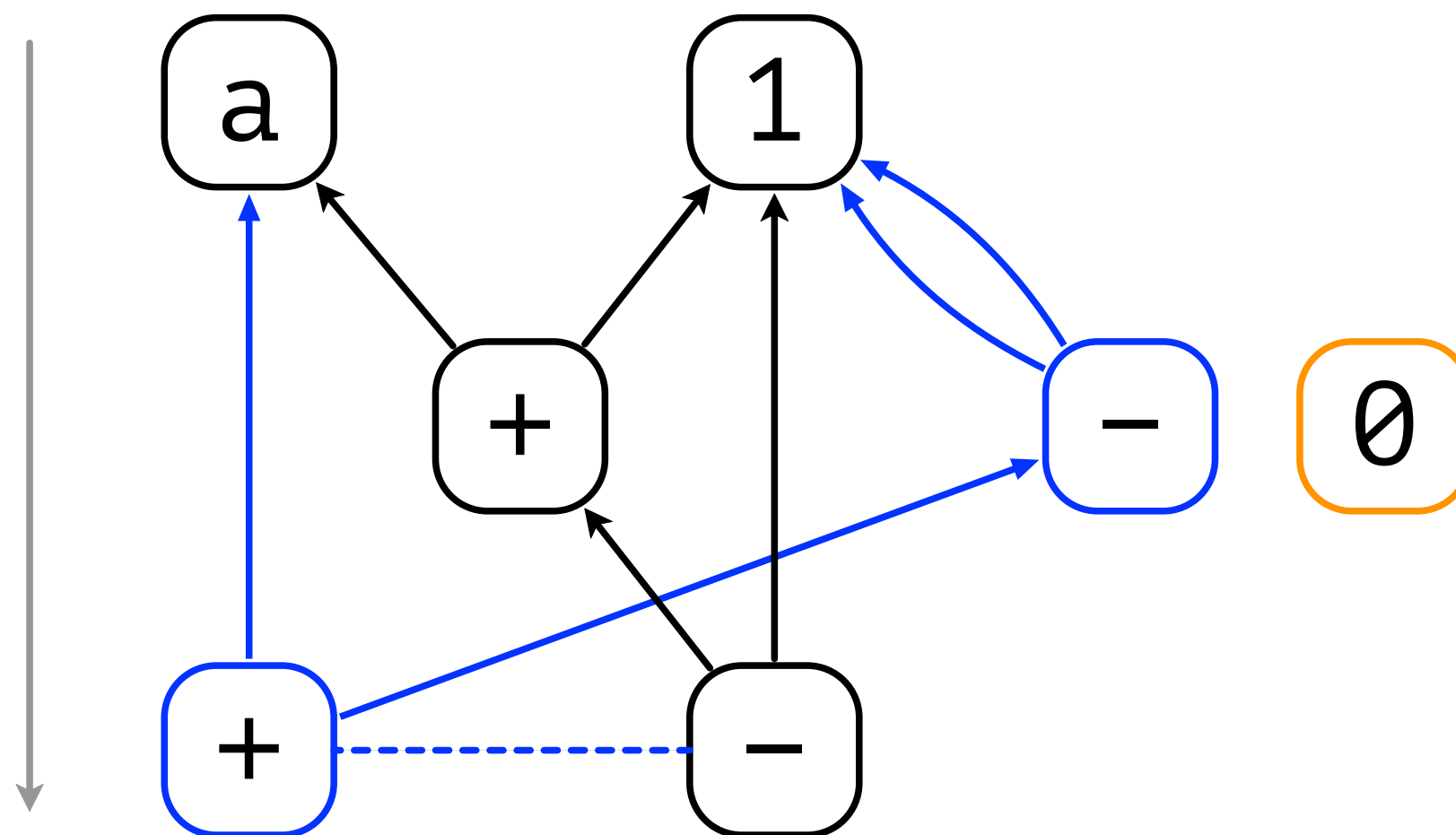
$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

Insert Program

Run Rules

Extract

data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha - \alpha \rightarrow 0$$

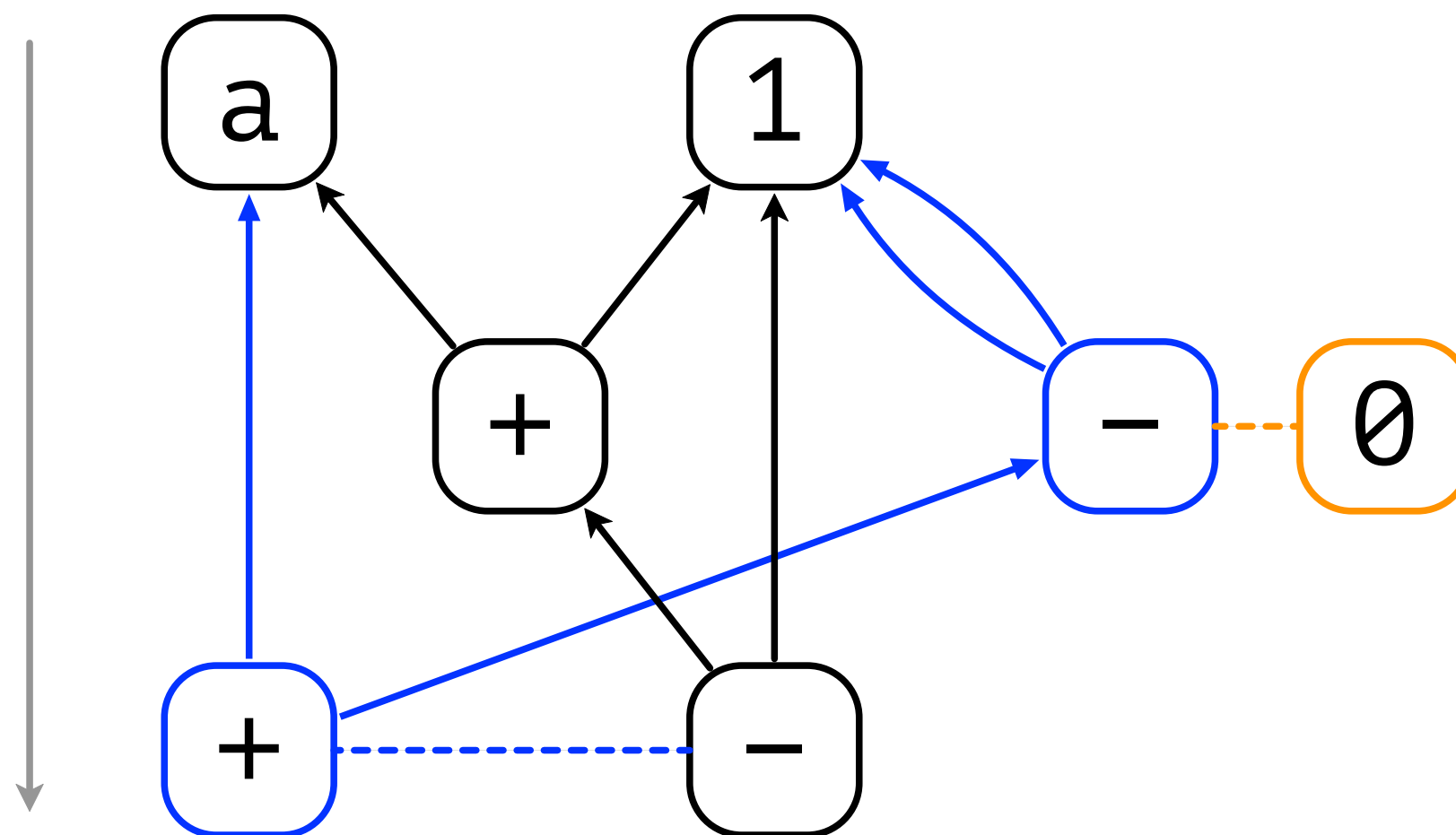
$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

Insert Program

Run Rules

Extract

data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

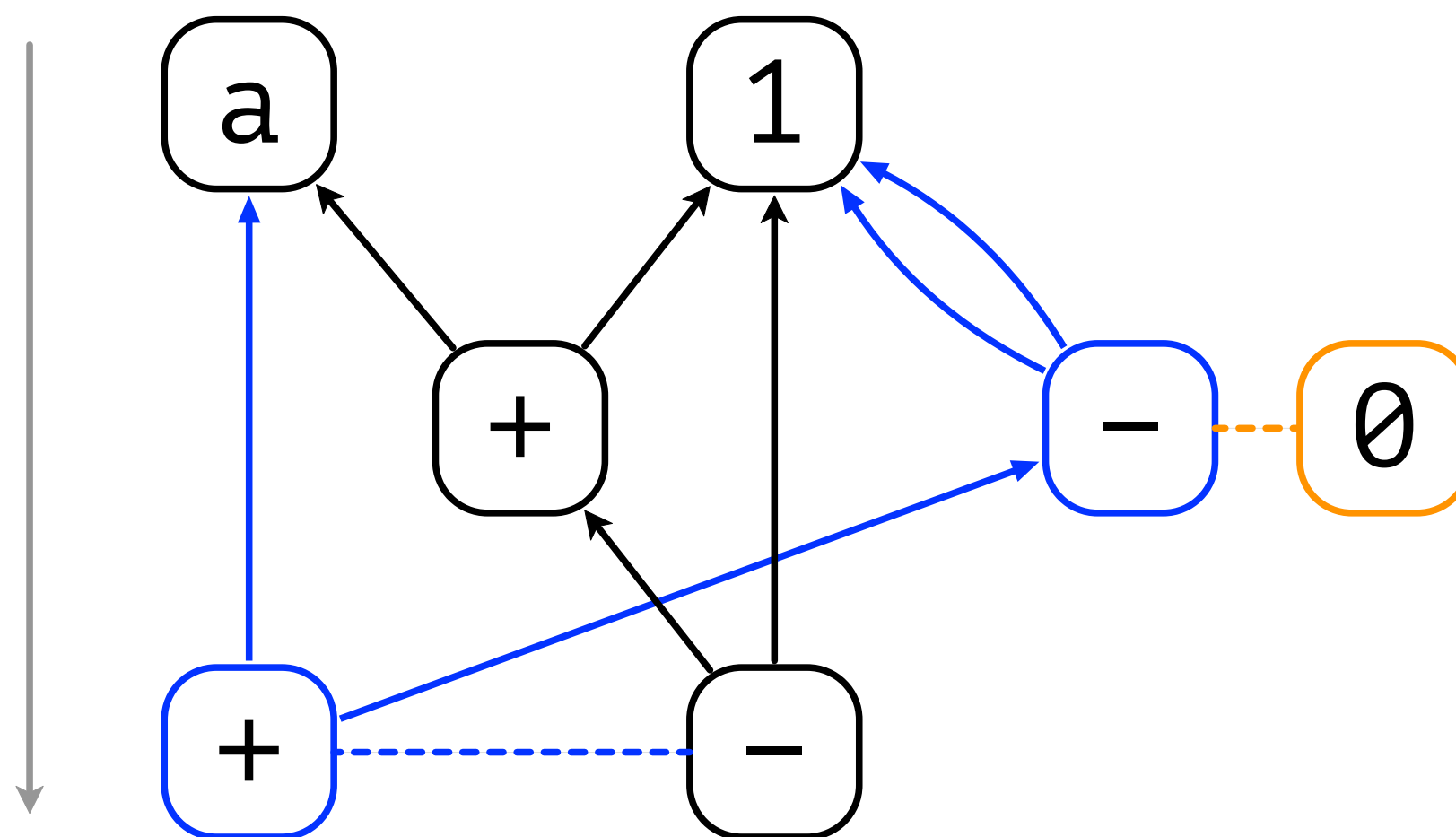
$$\alpha - \alpha \rightarrow 0$$

Insert Program

Run Rules

Extract

data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha + 0 \rightarrow \alpha$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

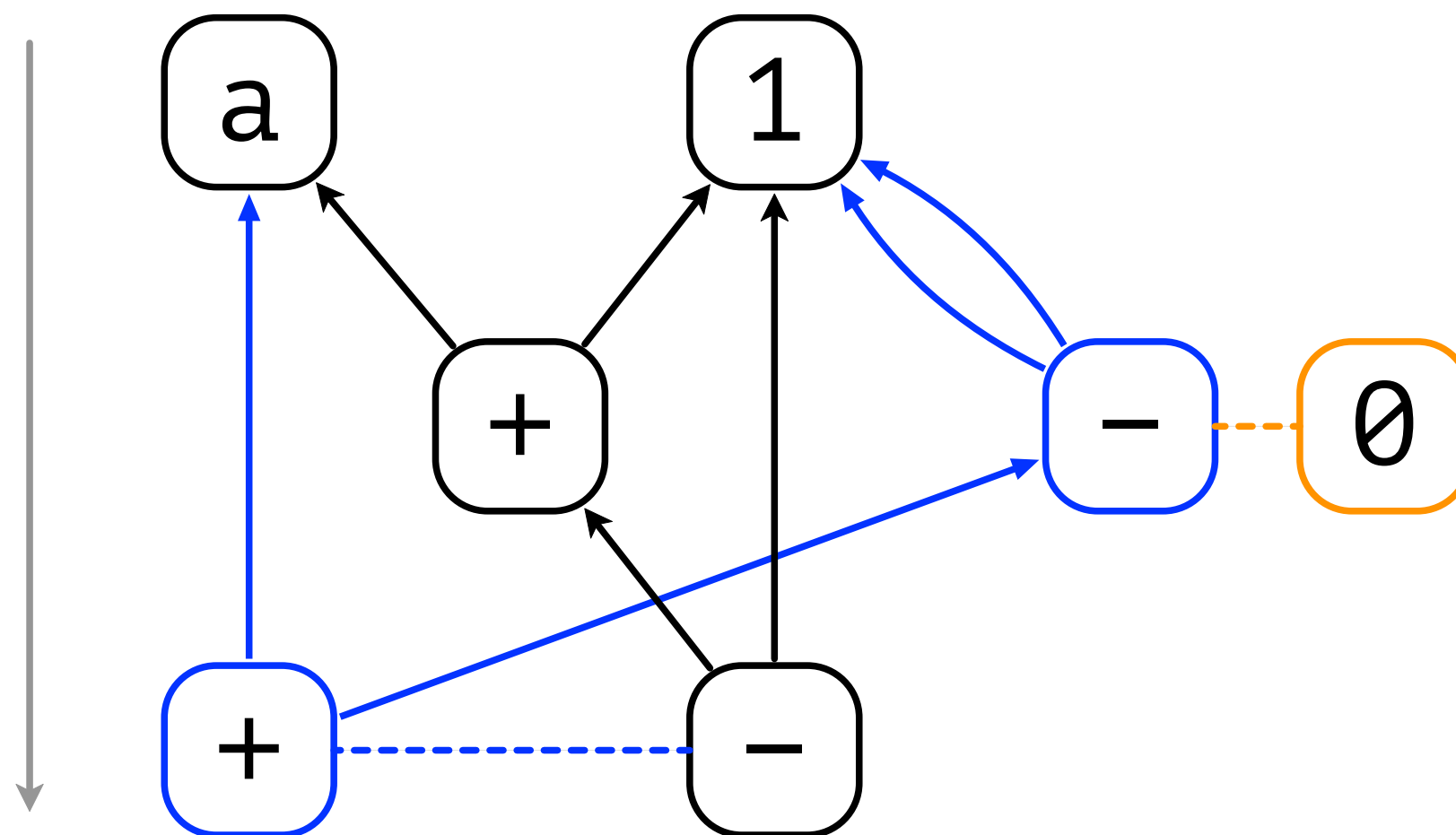
$$\alpha - \alpha \rightarrow 0$$

Insert Program

Run Rules

Extract

data flow



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$\alpha + 0 \rightarrow \alpha$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

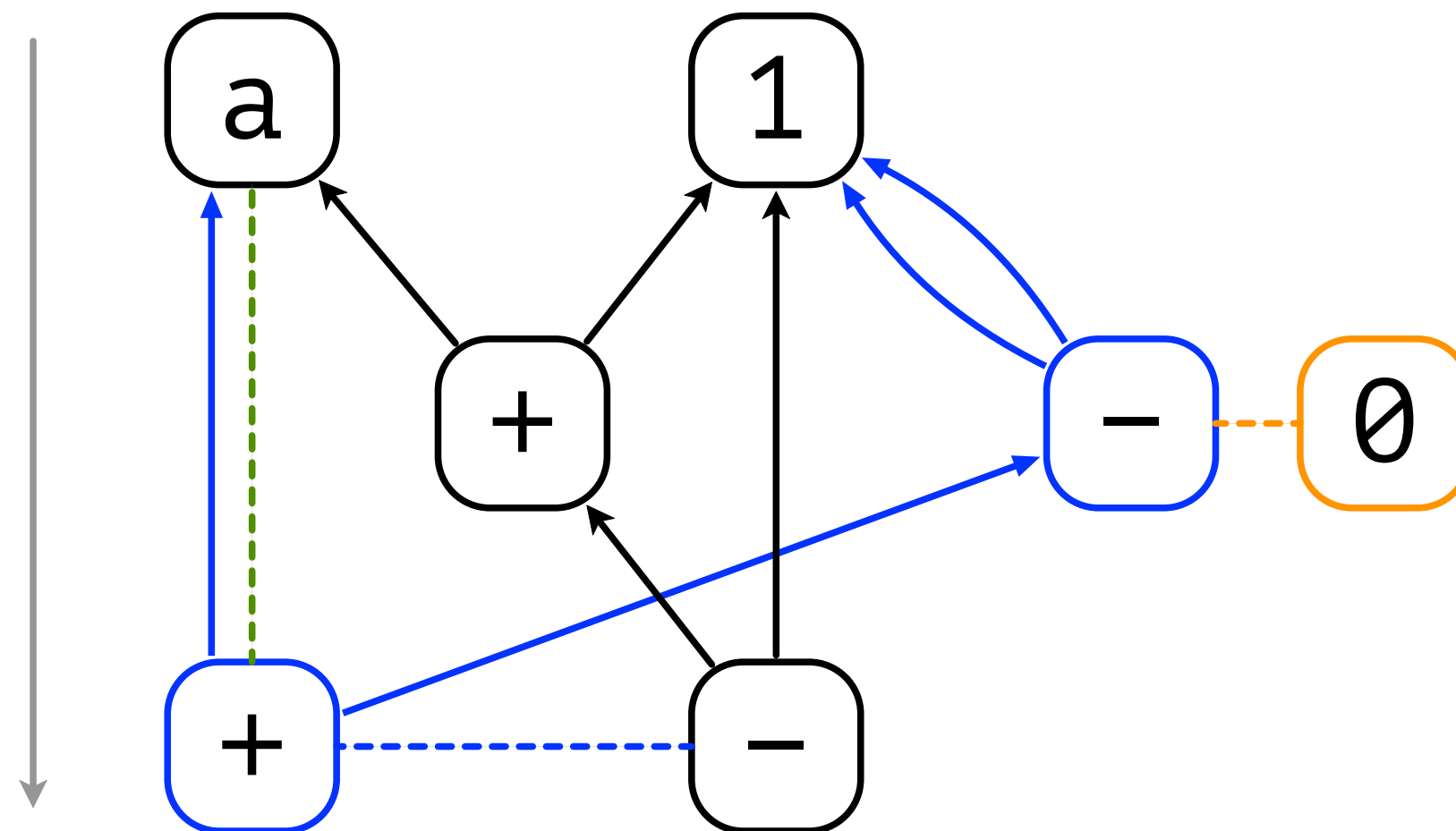
$$\alpha - \alpha \rightarrow 0$$

Insert Program

Run Rules

Extract

data flow



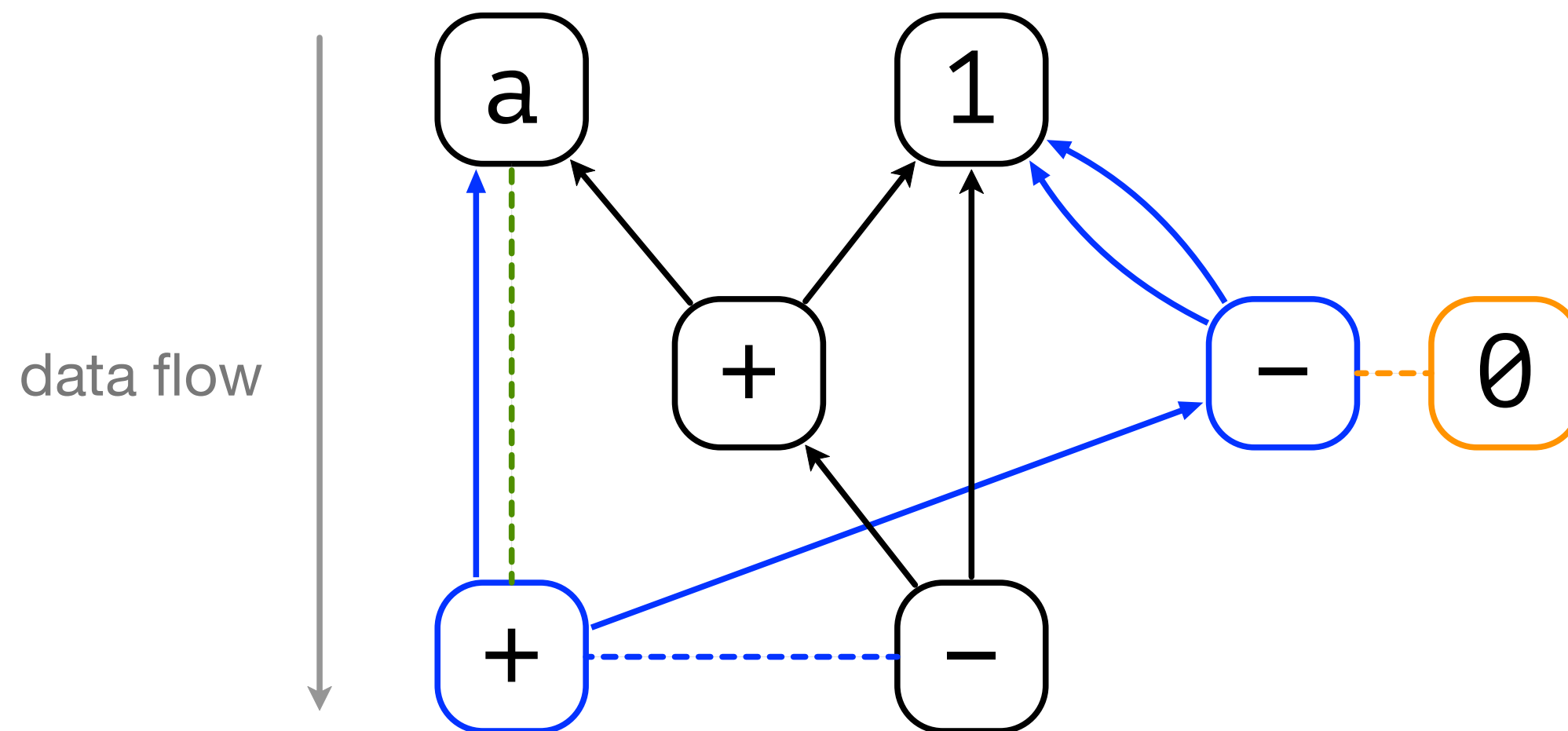
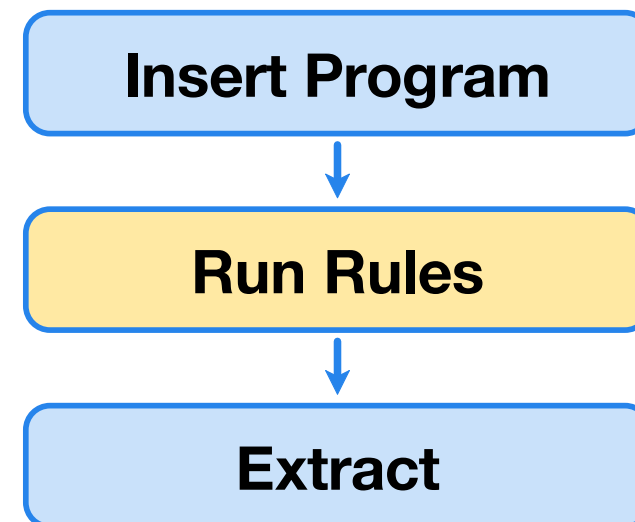
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$



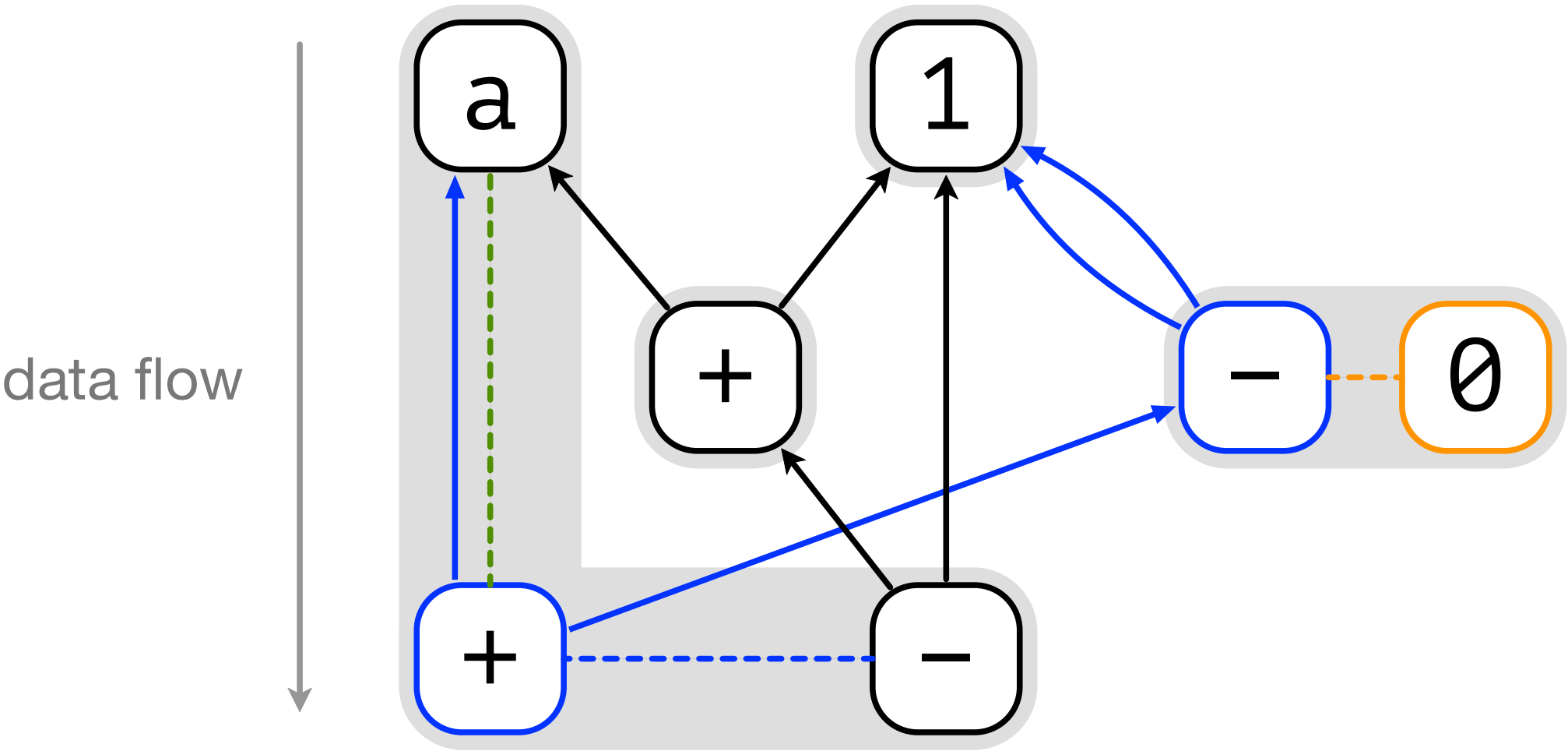
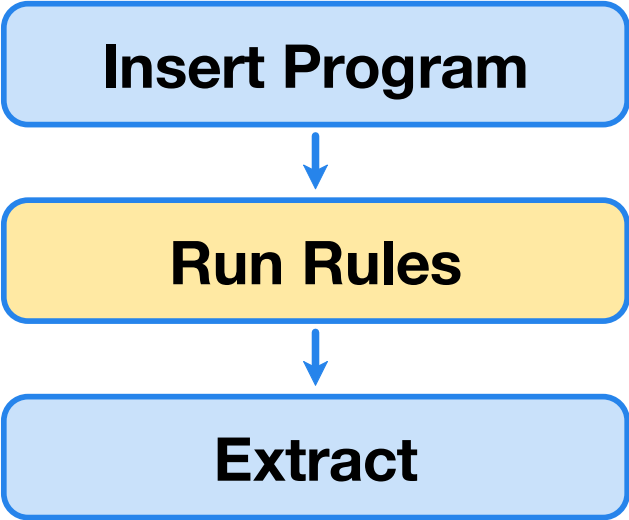
E-Graphs Explore a Big Space

$(a + 1) - 1$

$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$

$\alpha - \alpha \rightarrow 0$

$\alpha + 0 \rightarrow \alpha$



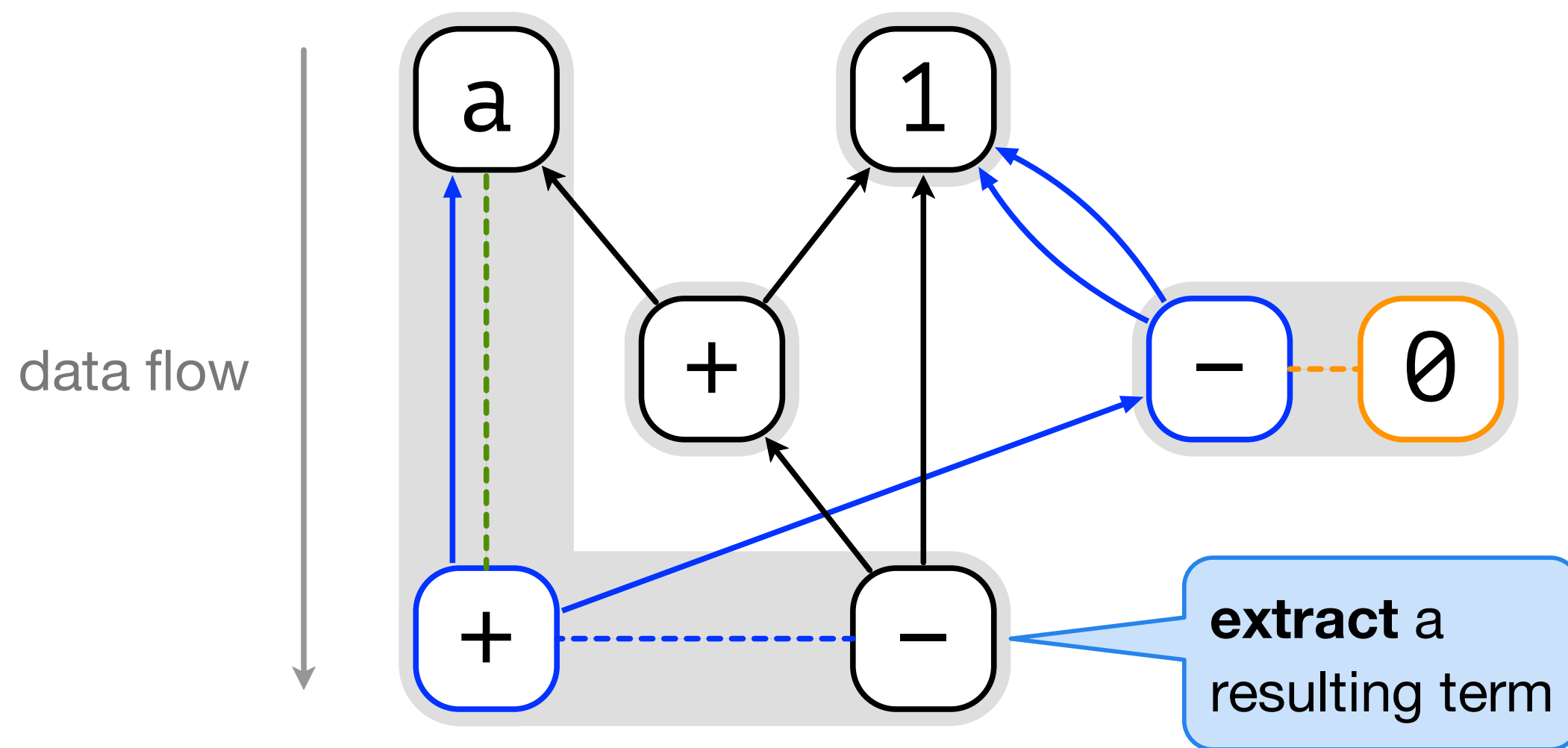
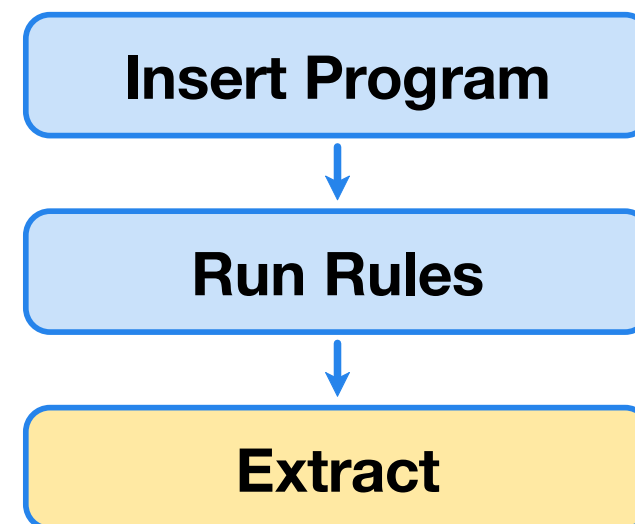
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$



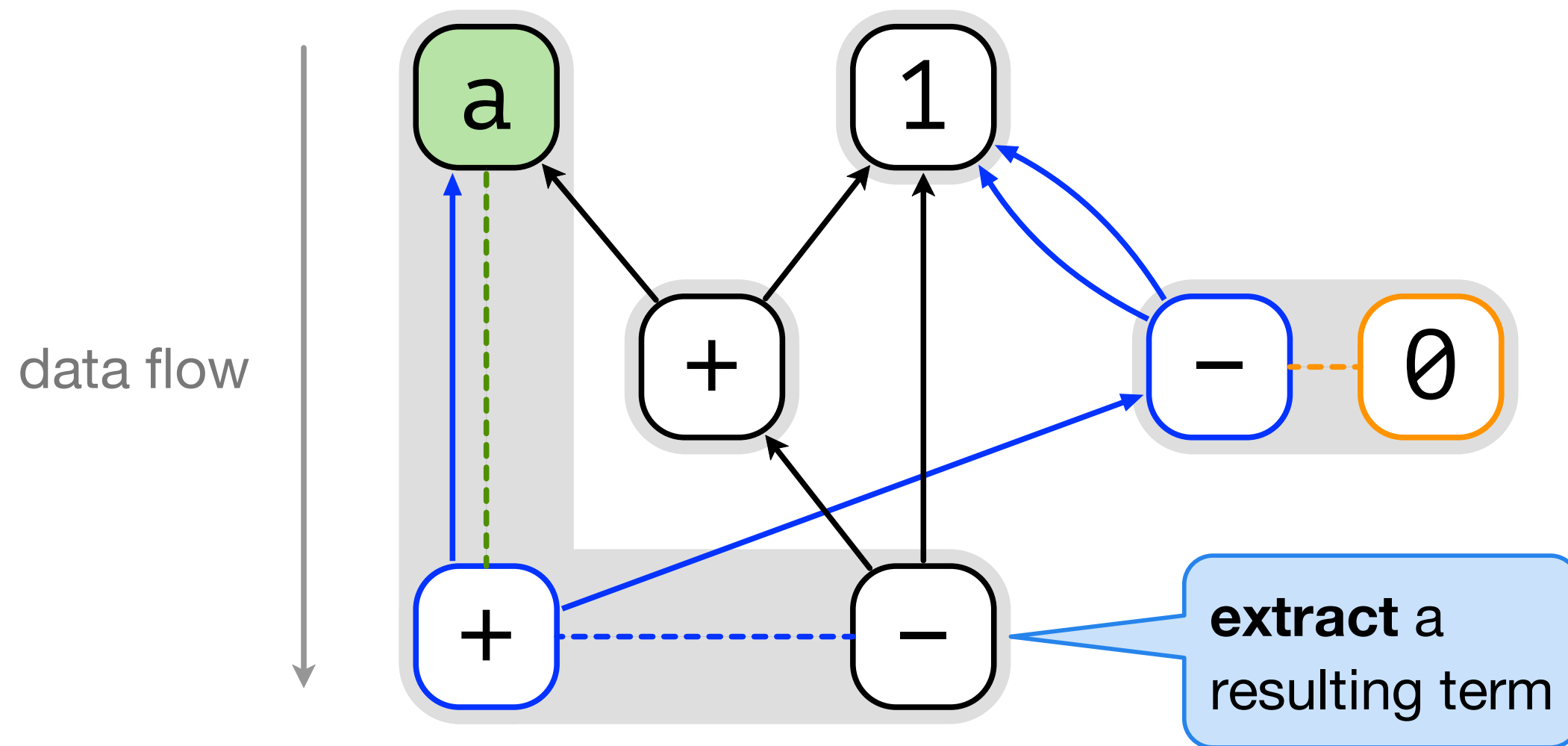
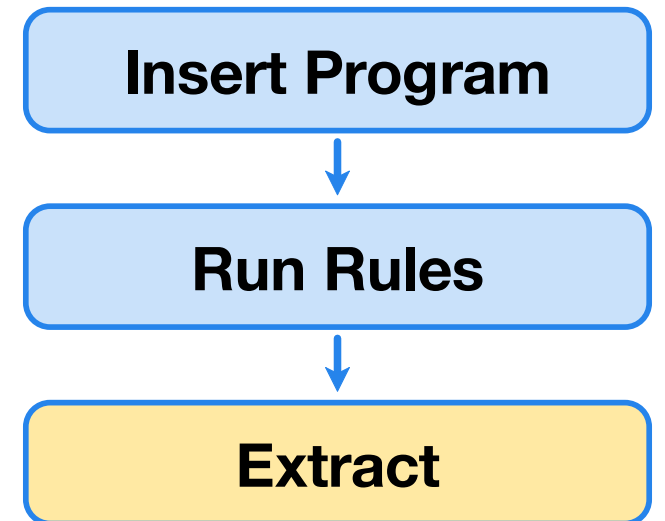
E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$



E-Graphs Explore a Big Space

$$(a + 1) - 1$$

$$(\alpha + \beta) - \gamma \rightarrow \alpha + (\beta - \gamma)$$

$$\alpha - \alpha \rightarrow 0$$

$$\alpha + 0 \rightarrow \alpha$$

a

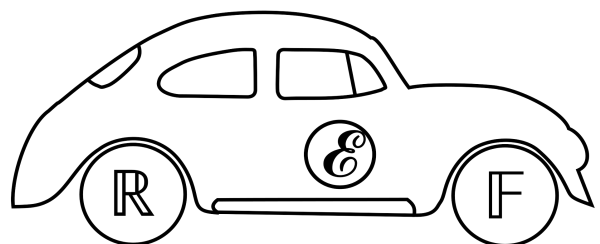
Insert Program

Run Rules

Extract

E-Graphs are Super Useful!

Floating point

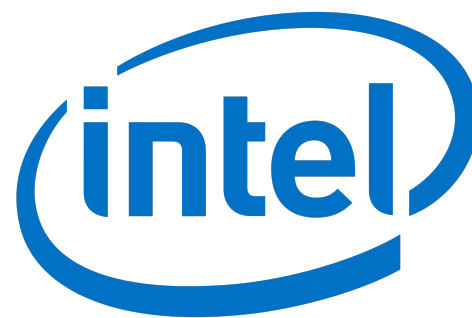


(Herbie)

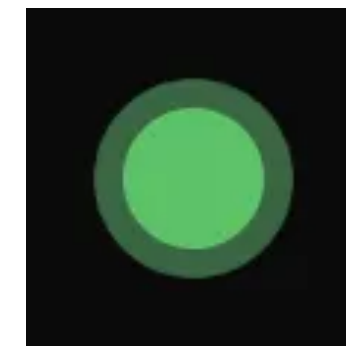
Python Simulations



RTL Optimization



ML Kernels



Luminal

Verification

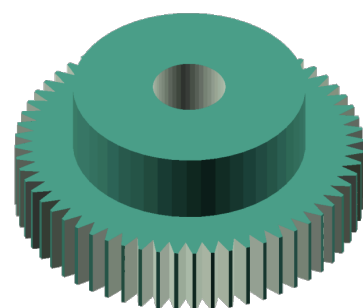


SQL



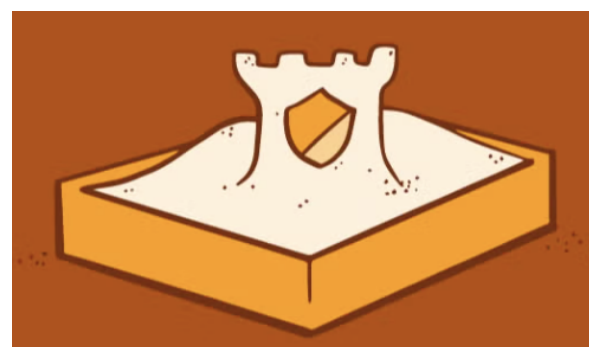
RisingWave

CAD



(Szalinski)

WASM



(Cranelift)

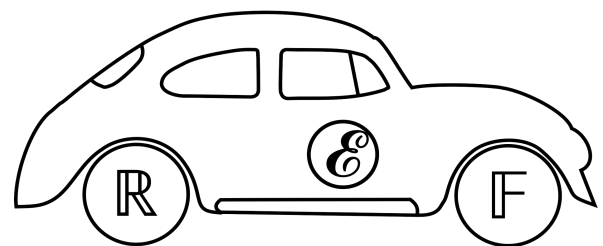
User DSL
Optimizations



...and more!

E-Graphs are Super Useful!

Floating point

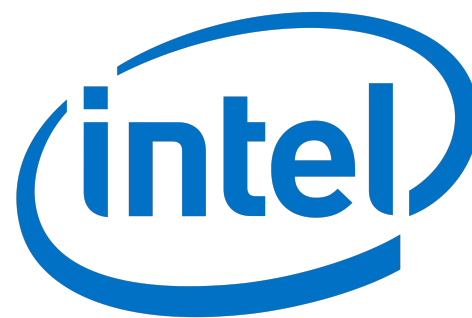


(Herbie)

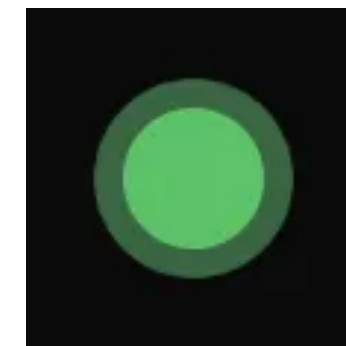
Python Simulations



RTL Optimization



ML Kernels



Luminal

Verification

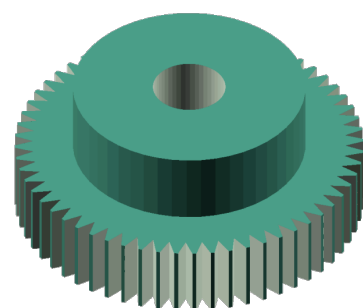


SQL



RisingWave

CAD



(Szalinski)

WASM

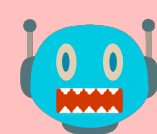


(Cranelift)

User DSL
Optimizations



...and more!

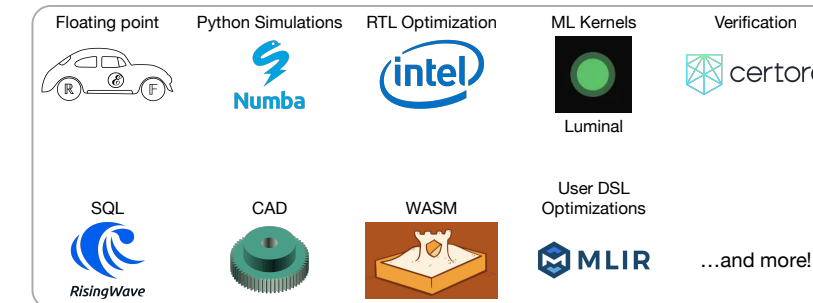


limited to pure
optimizations!



Why Effectful E-Graphs

- 1 Why Effectful E-Graphs
- 2 An NP-Hard Problem
- 3 Fast Extraction Anyways
- 4 Eggcc, A New Compiler



Update

Update

```
fun foo(a, x):  
    v1 = update(a, x)  
    v2 = update(a, x+1)  
    return v1
```

Update

```
fun foo(a, x):  
    v1 = update(a, x)  
    v2 = update(a, x+1)  
    return v1
```

Write **x** to address **a**,
return the previous value at **a**

Update

```
fun foo(a, x, ω):  
    (v1, ω1) = update(a, x, ω)  
    (v2, ω2) = update(a, x+1, ω1)  
    return (v1, ω2)
```

Write **x** to address **a**,
return the previous value at **a**

Update

```
fun foo(a, x,  $\omega$ ):  
    ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
    ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
    return ( $v_1$ ,  $\omega_2$ )
```

Write **x** to address **a**,
return the previous value at **a**

ω_1 is the state of memory
after the update

Update

```
fun foo(a, x,  $\omega$ ):  
    ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
    ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
    return ( $v_1$ ,  $\omega_2$ )
```

Write **x** to address **a**,
return the previous value at **a**

ω_1 is the state of memory
after the update

Update

```
fun foo(a, x,  $\omega$ ):  
    ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
    ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
    return ( $v_1$ ,  $\omega_2$ )
```

Write x to address a ,
return the previous value at a

ω_1 is the state of memory
after the update

uses memory **linearly**

Update

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

Write **x** to address **a**,
return the previous value at **a**

ω_1 is the state of memory
after the update

uses memory **linearly**

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

Update

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

Write **x** to address **a**,
return the previous value at **a**

ω_1 is the state of memory
after the update

uses memory **linearly**

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

Update

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

Write x to address a ,
return the previous value at a

ω_1 is the state of memory
after the update

uses memory **linearly**

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

equivalent to original
under dataflow semantics

Update

```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```

Write **x** to address **a**,
return the previous value at **a**

ω₁ is the state of memory
after the update

uses memory **linearly**

```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω)
  return (v1, ω2)
  
```

equivalent to original
under dataflow semantics

Update

```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```

Write **x** to address **a**,
return the previous value at **a**

ω₁ is the state of memory
after the update

uses memory **linearly**

```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω)
  return (v1, ω2)
  
```

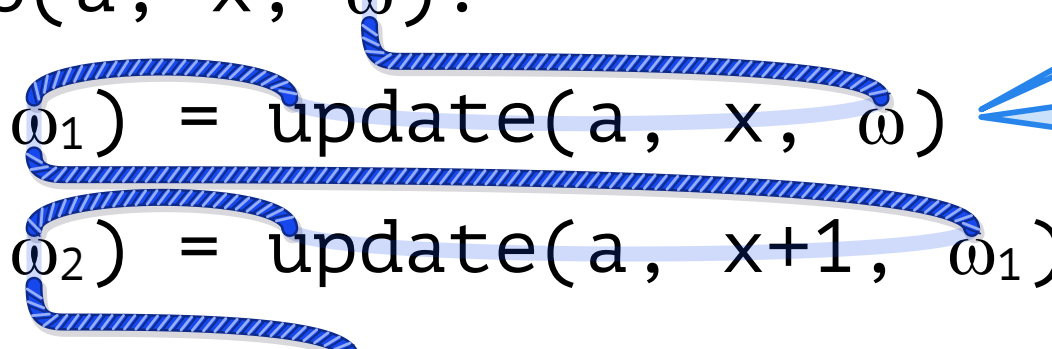
equivalent to original
under dataflow semantics

uses memory **non-linearly**

Update

```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```



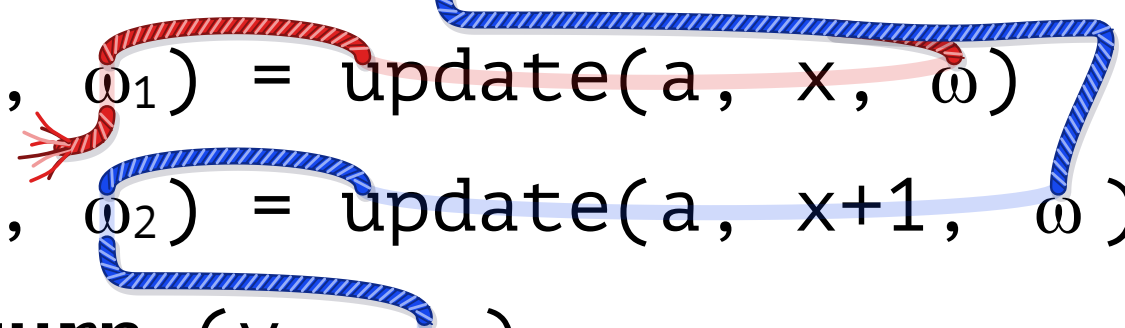
Write **x** to address **a**,
return the previous value at **a**

ω₁ is the state of memory
after the update

uses memory **linearly**

```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω)
  return (v1, ω2)
  
```



equivalent to original
under dataflow semantics

uses memory **non-linearly**

central challenge:
non-linear programs unavoidable in the e-graph!

Update

```
fun foo(a, x,  $\omega$ ):  
    ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
    ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
    return ( $v_1$ ,  $\omega_2$ )
```

The Effectful E-Graph

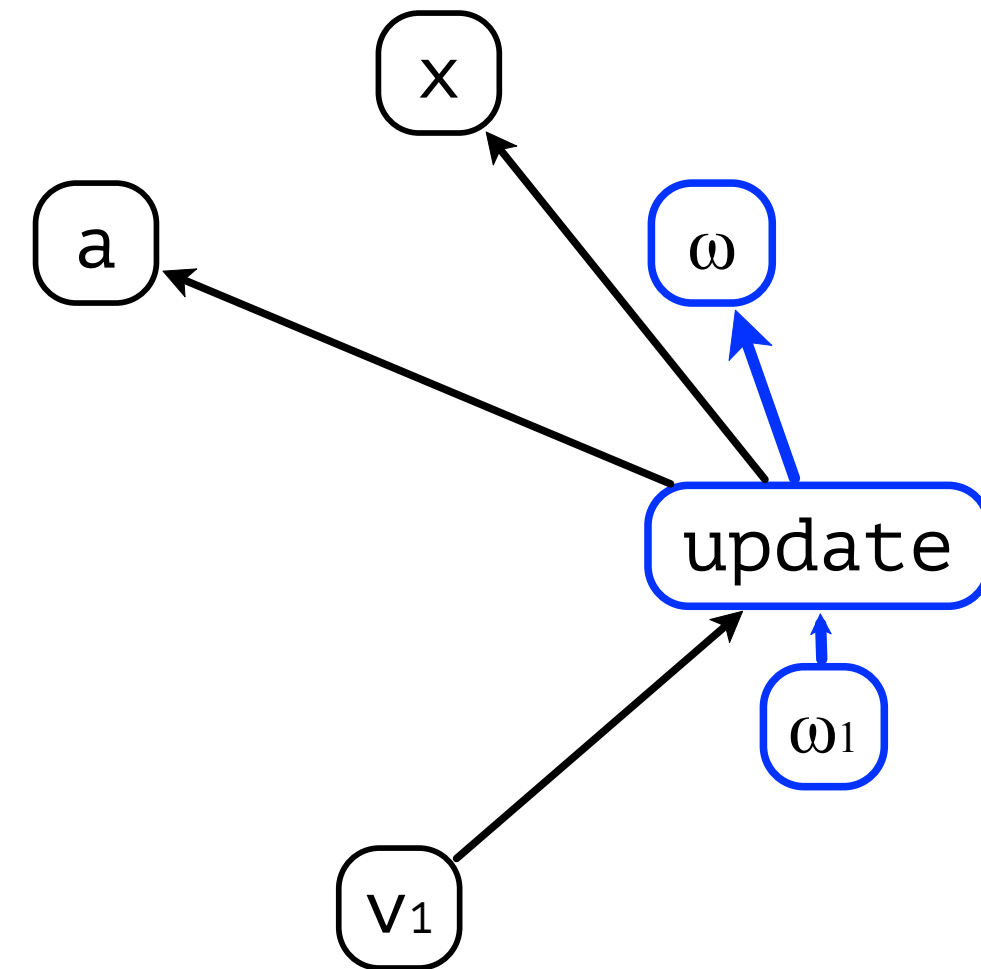
```
fun foo(a, x,  $\omega$ ):  
    ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
    ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
    return ( $v_1$ ,  $\omega_2$ )
```

The Effectful E-Graph

```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```

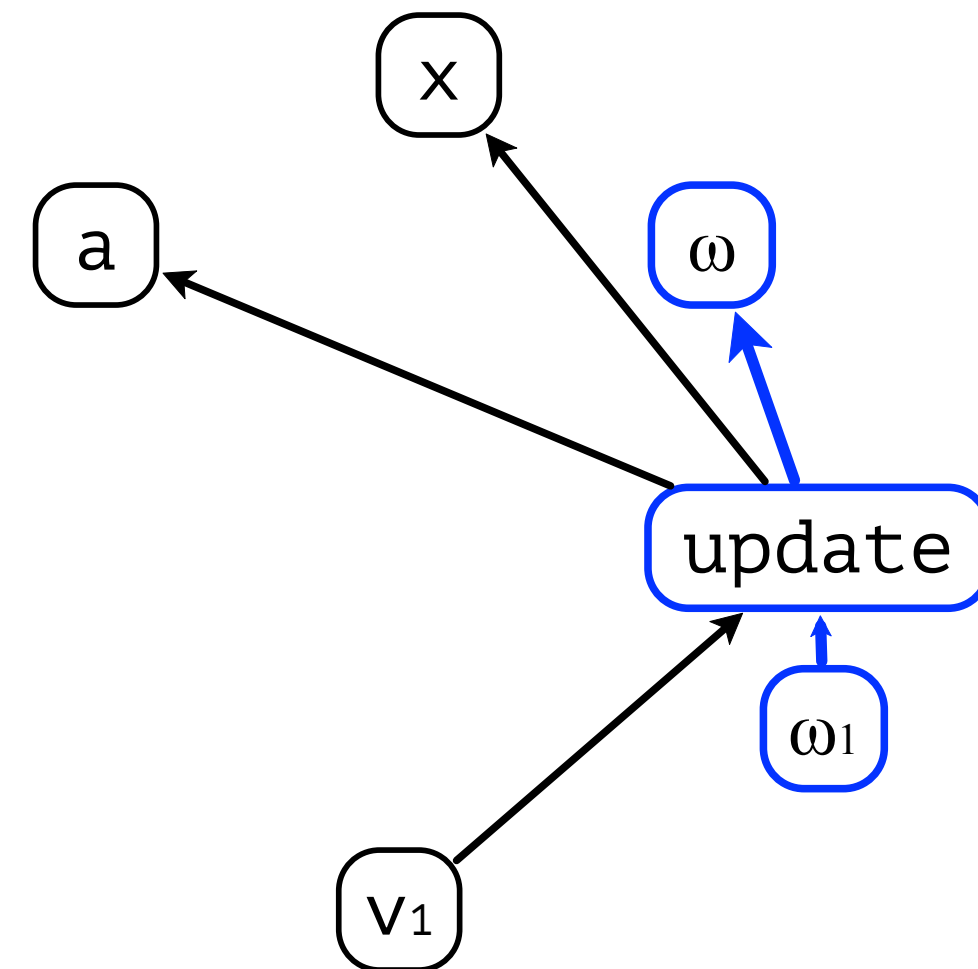
The Effectful E-Graph

```
fun foo(a, x, ω):  
  (v1, ω1) = update(a, x, ω)  
  (v2, ω2) = update(a, x+1, ω1)  
  return (v1, ω2)
```



The Effectful E-Graph

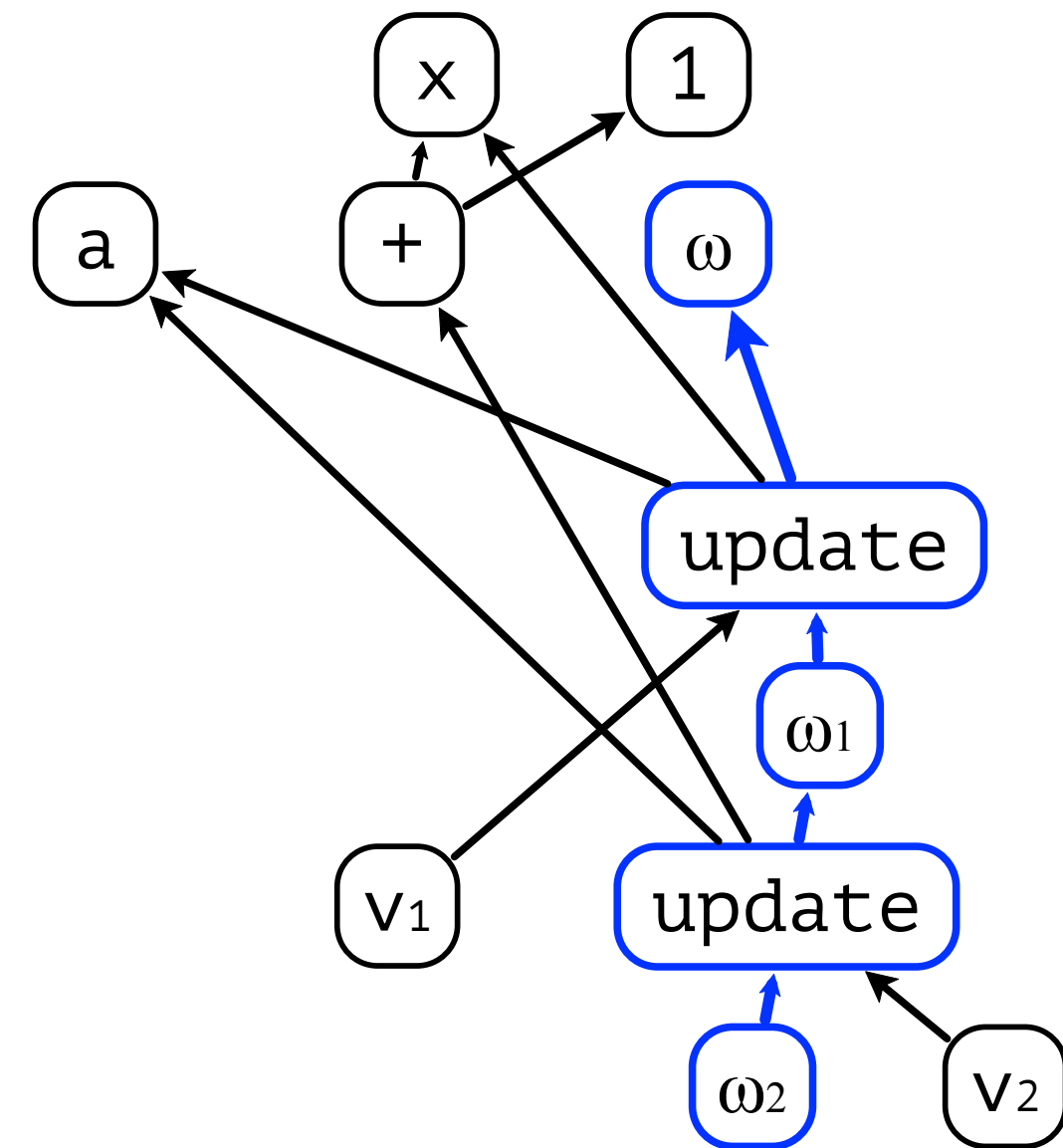
```
fun foo(a, x,  $\omega$ ):  
  ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
  ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
  return ( $v_1$ ,  $\omega_2$ )
```



The Effectful E-Graph

```

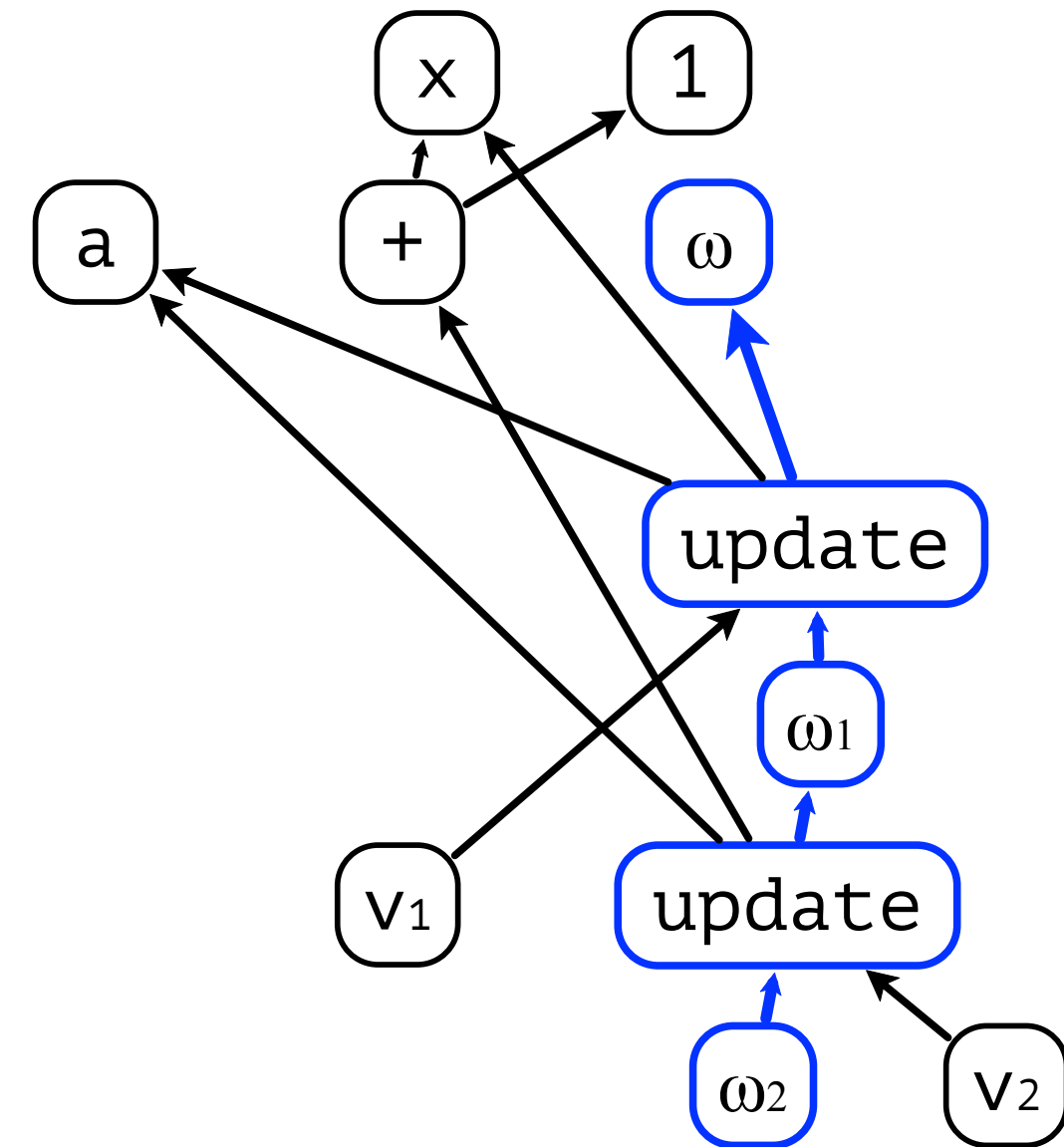
fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```



The Effectful E-Graph

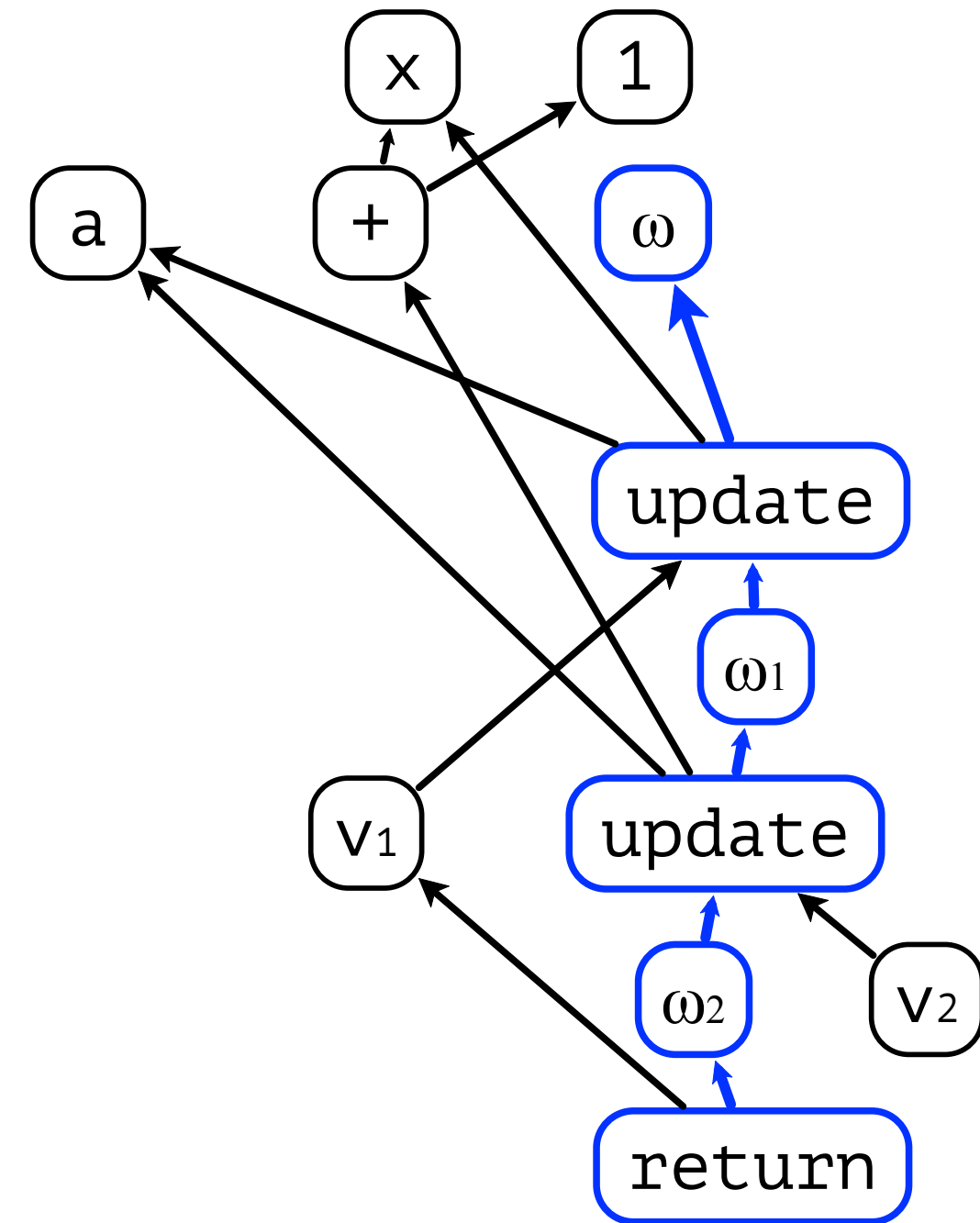
```

fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```



The Effectful E-Graph

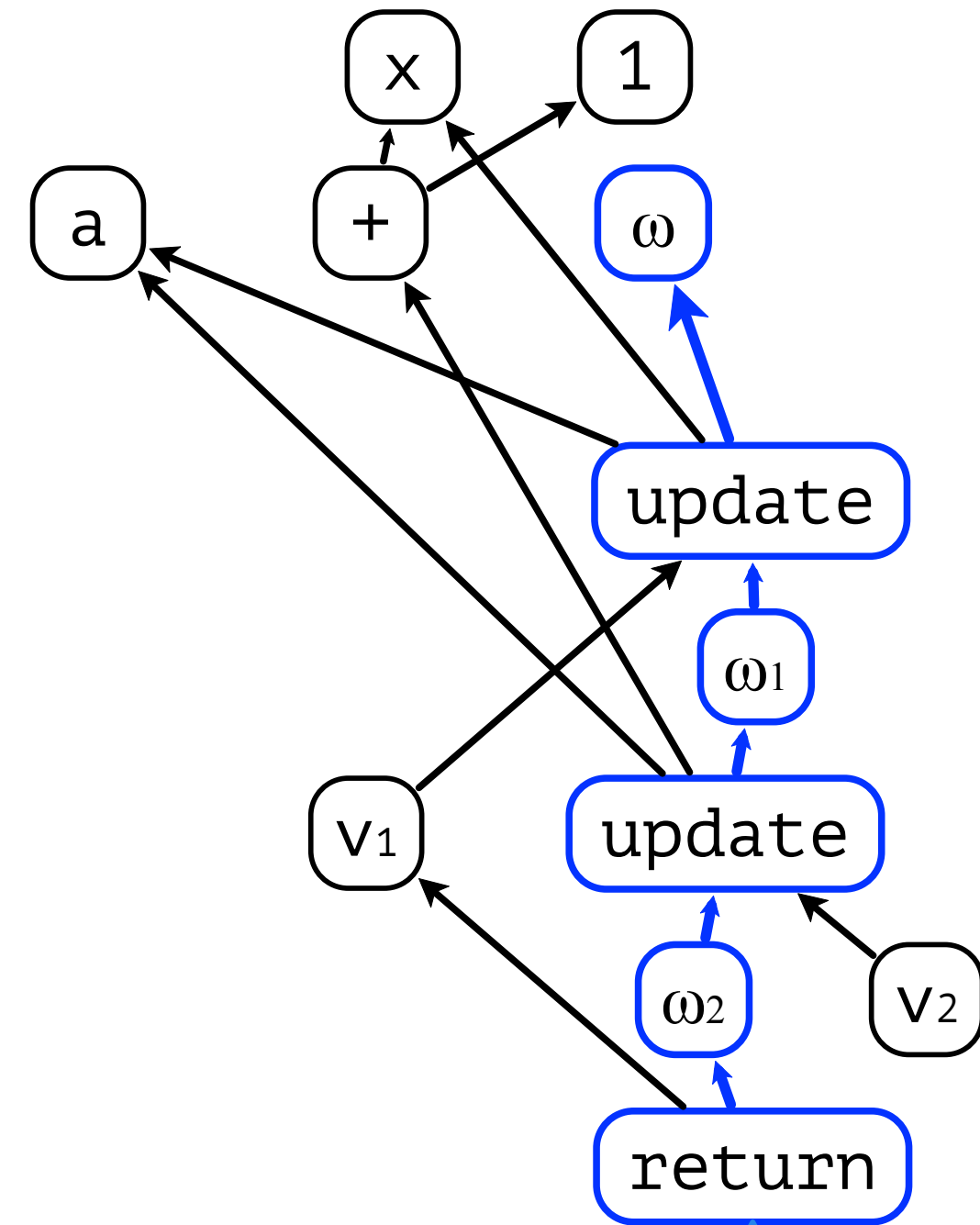
```
fun foo(a, x,  $\omega$ ):  
    ( $v_1$ ,  $\omega_1$ ) = update(a, x,  $\omega$ )  
    ( $v_2$ ,  $\omega_2$ ) = update(a, x+1,  $\omega_1$ )  
    return ( $v_1$ ,  $\omega_2$ )
```



The Effectful E-Graph

```

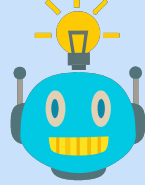
fun foo(a, x, ω):
  (v1, ω1) = update(a, x, ω)
  (v2, ω2) = update(a, x+1, ω1)
  return (v1, ω2)
  
```

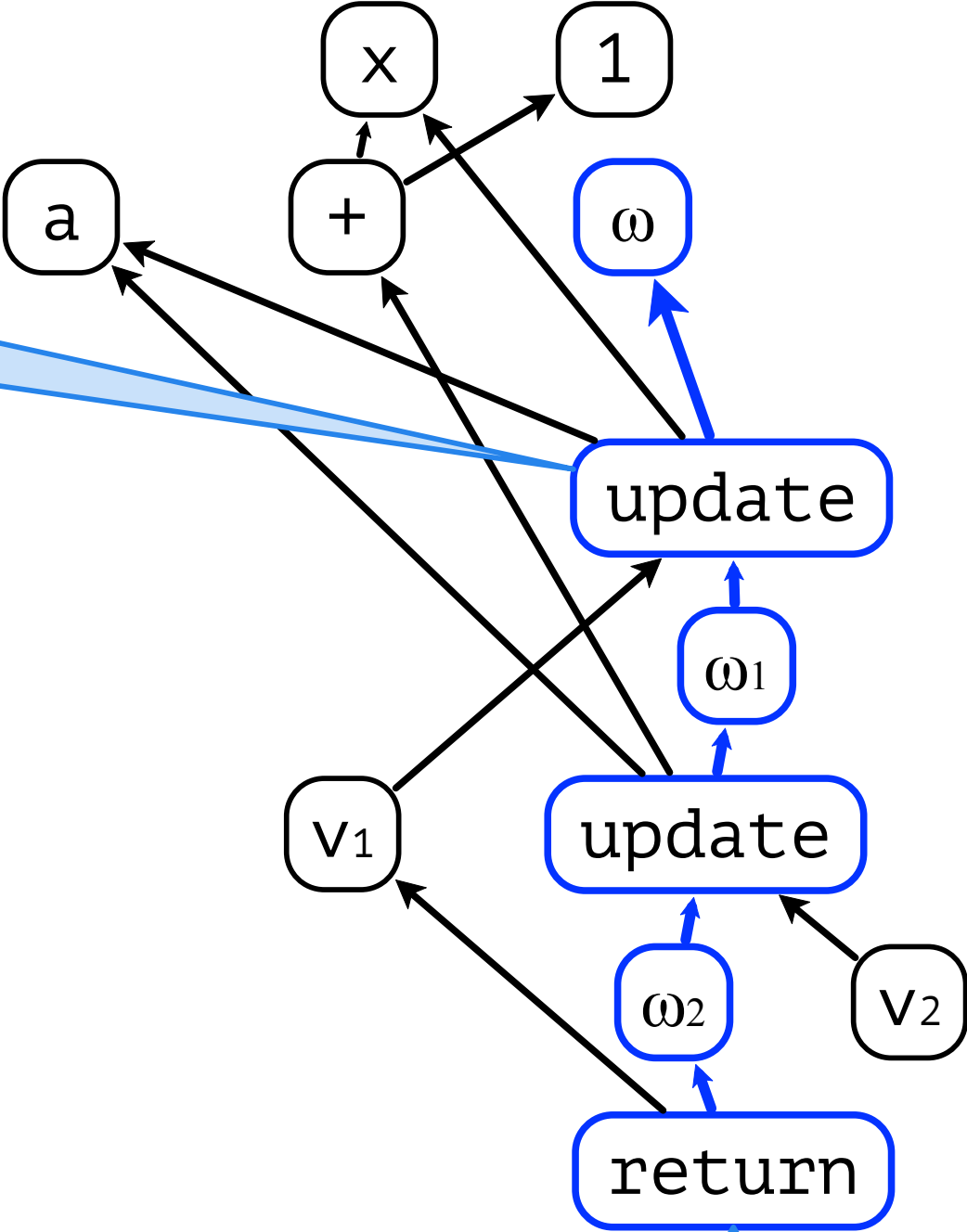


state walk uses memory linearly

The Effective E-Graph

```
fun foo(a, x, ω):  
  (v1, ω1) = update(a, x, ω)  
  (v2, ω2) = update(a, x+1, ω1)  
  return (v1, ω2)
```


**Redundant
write**



state walk uses memory linearly

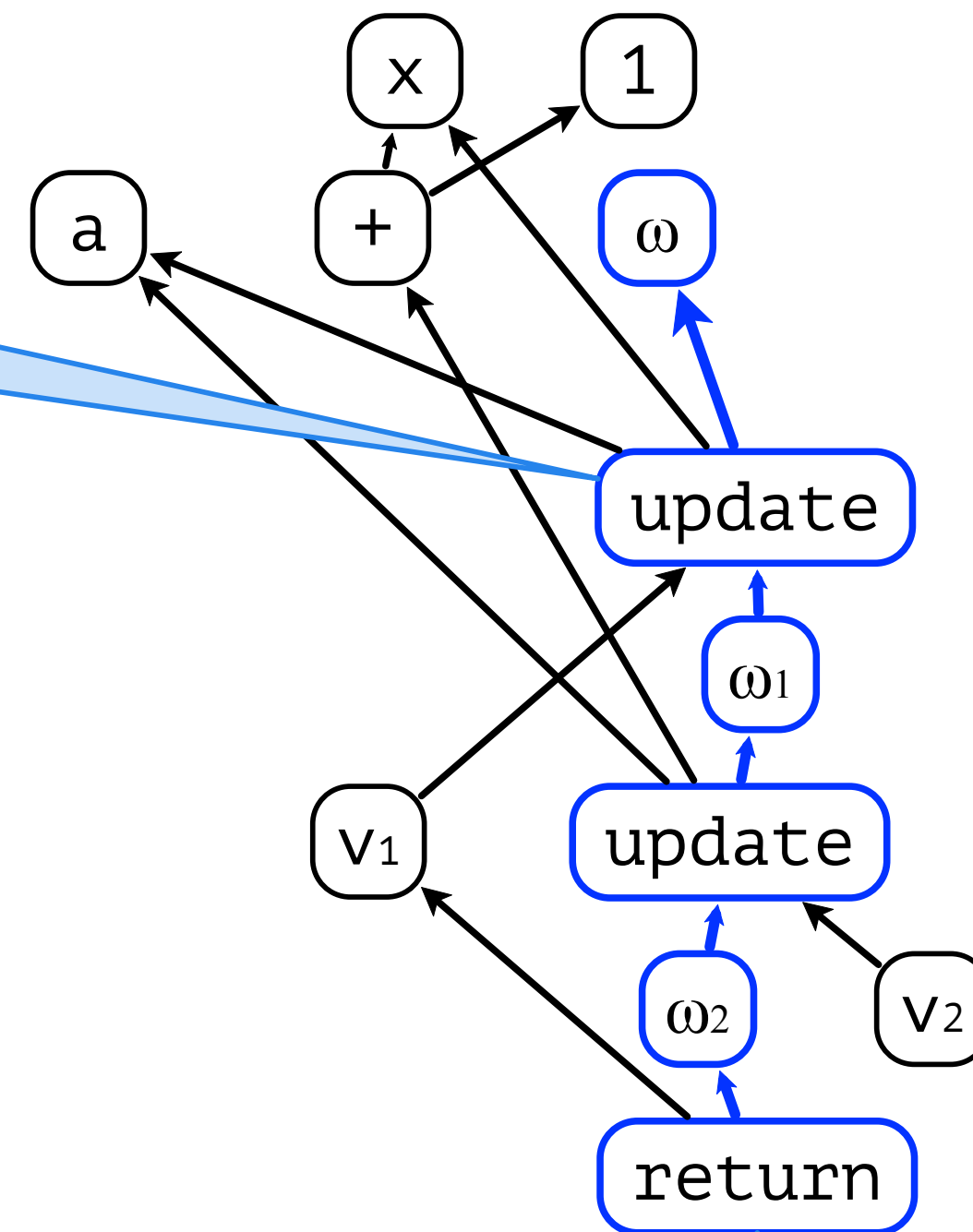
The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```



state walk uses memory linearly

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

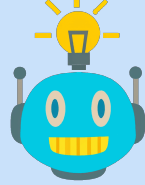
```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$

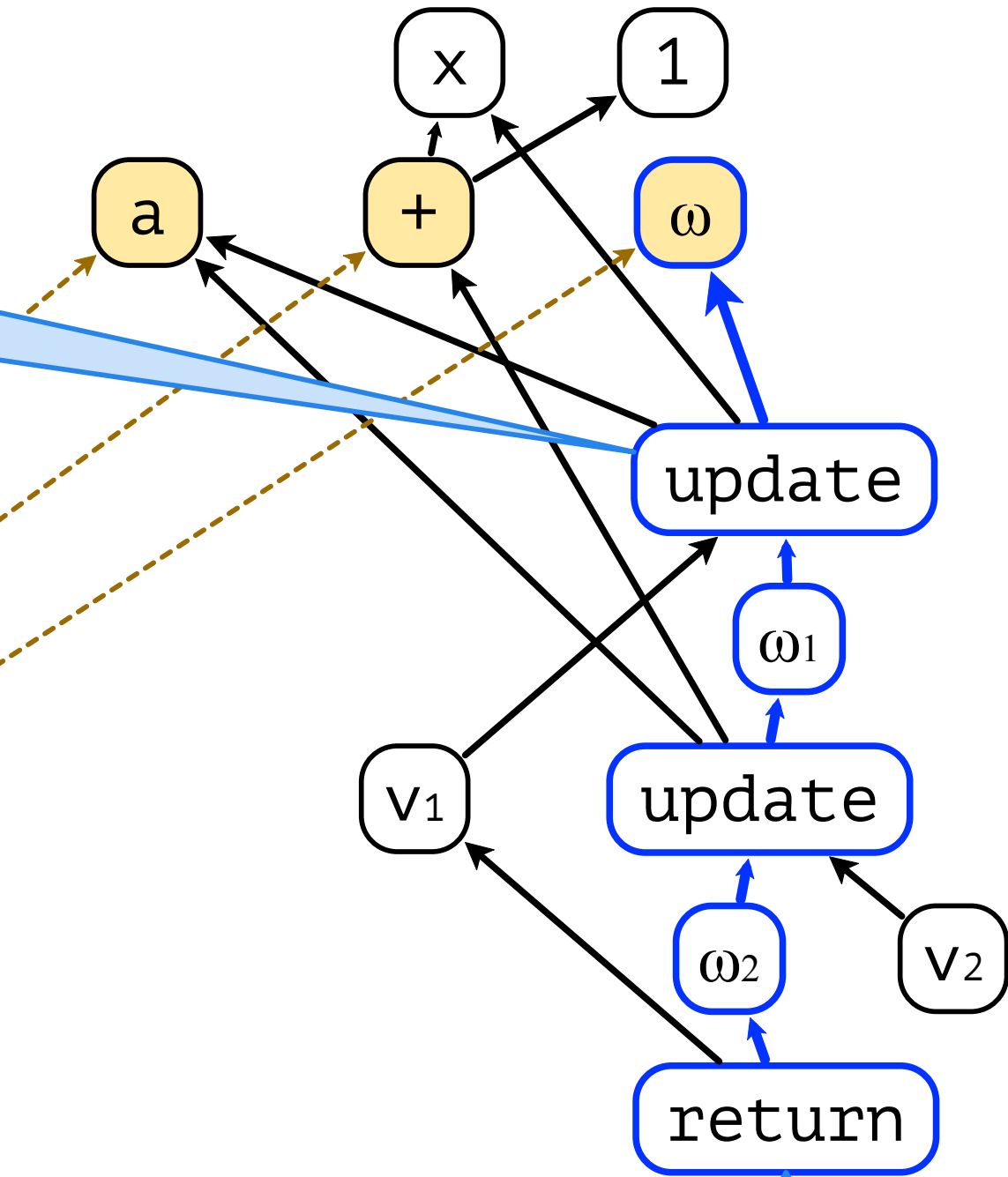
The Effectful E-Graph

```
fun foo(a, x, ω):  
  (v1, ω1) = update(a, x, ω)  
  (v2, ω2) = update(a, x+1, ω1)  
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)  
(v2, ω2) = update(a, y, ω1)  
-----  
(v3, ω3) = update(a, y, ω)  
v1 ≡ v3      ω2 ≡ ω3
```

 **Redundant write**

state walk uses memory linearly



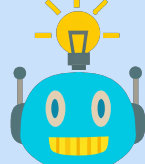
The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

 **Redundant write**

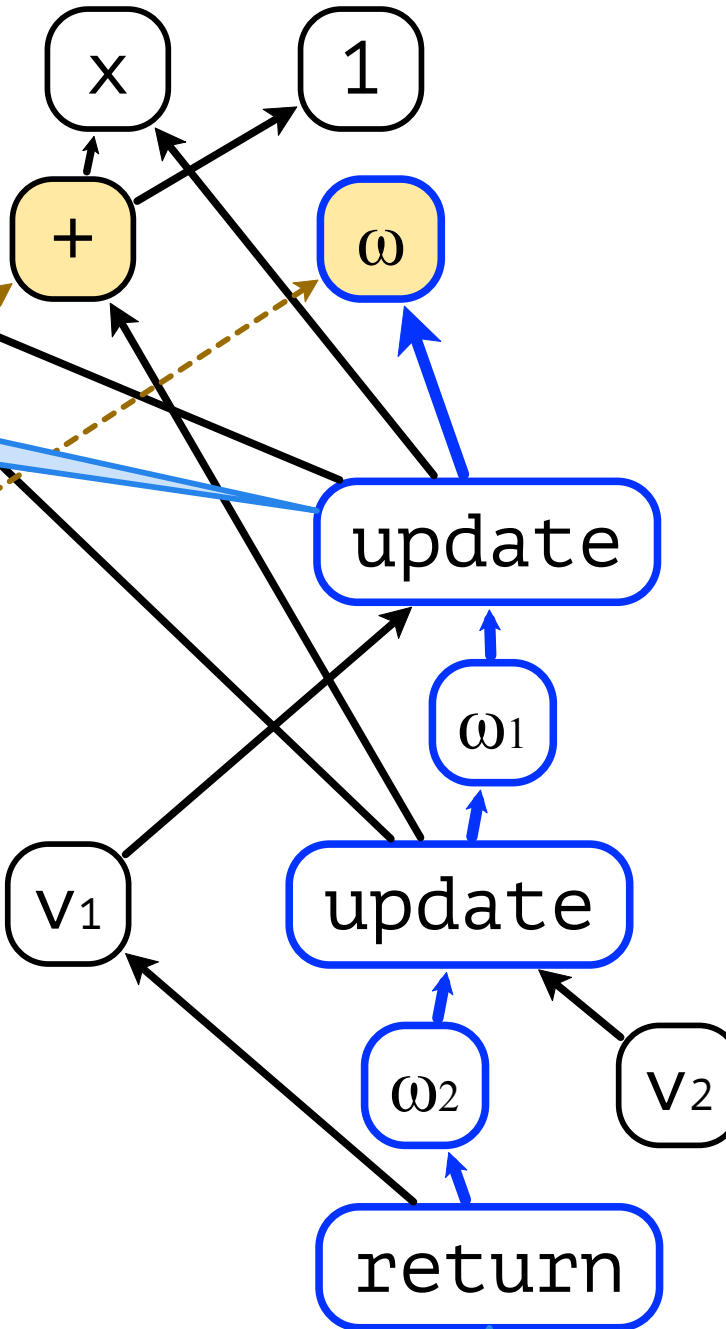
```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$

$\omega_2 \equiv \omega_3$



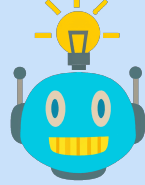
state walk uses memory linearly

The Effectful E-Graph

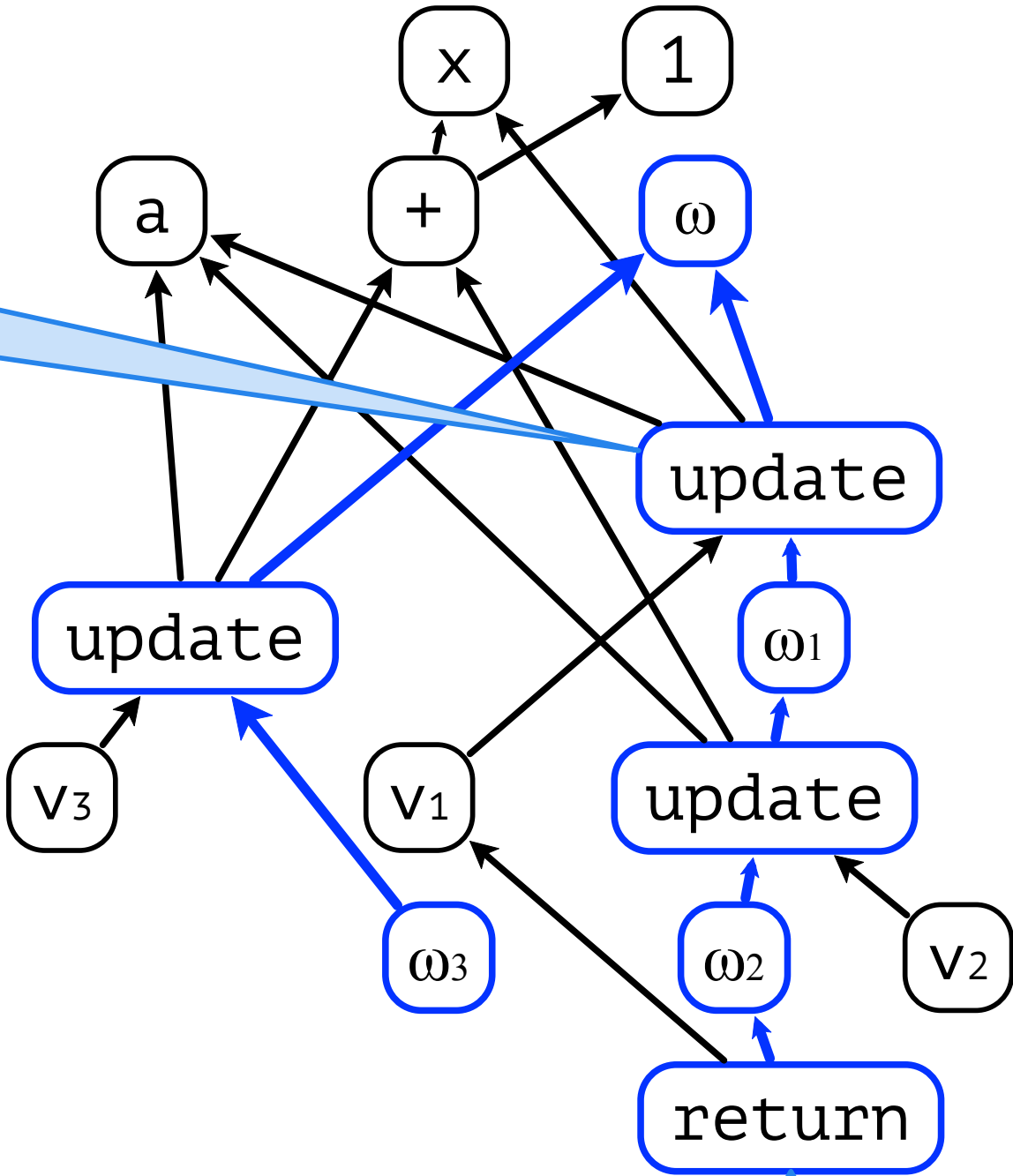
```
fun foo(a, x, ω):  
  (v1, ω1) = update(a, x, ω)  
  (v2, ω2) = update(a, x+1, ω1)  
  return (v1, ω2)
```

$(v_1, \omega_1) = \text{update}(a, x, \omega)$
 $(v_2, \omega_2) = \text{update}(a, y, \omega_1)$

$(v_3, \omega_3) = \text{update}(a, y, \omega)$
 $v_1 \equiv v_3 \quad \omega_2 \equiv \omega_3$

 **Redundant write**

state walk uses memory linearly



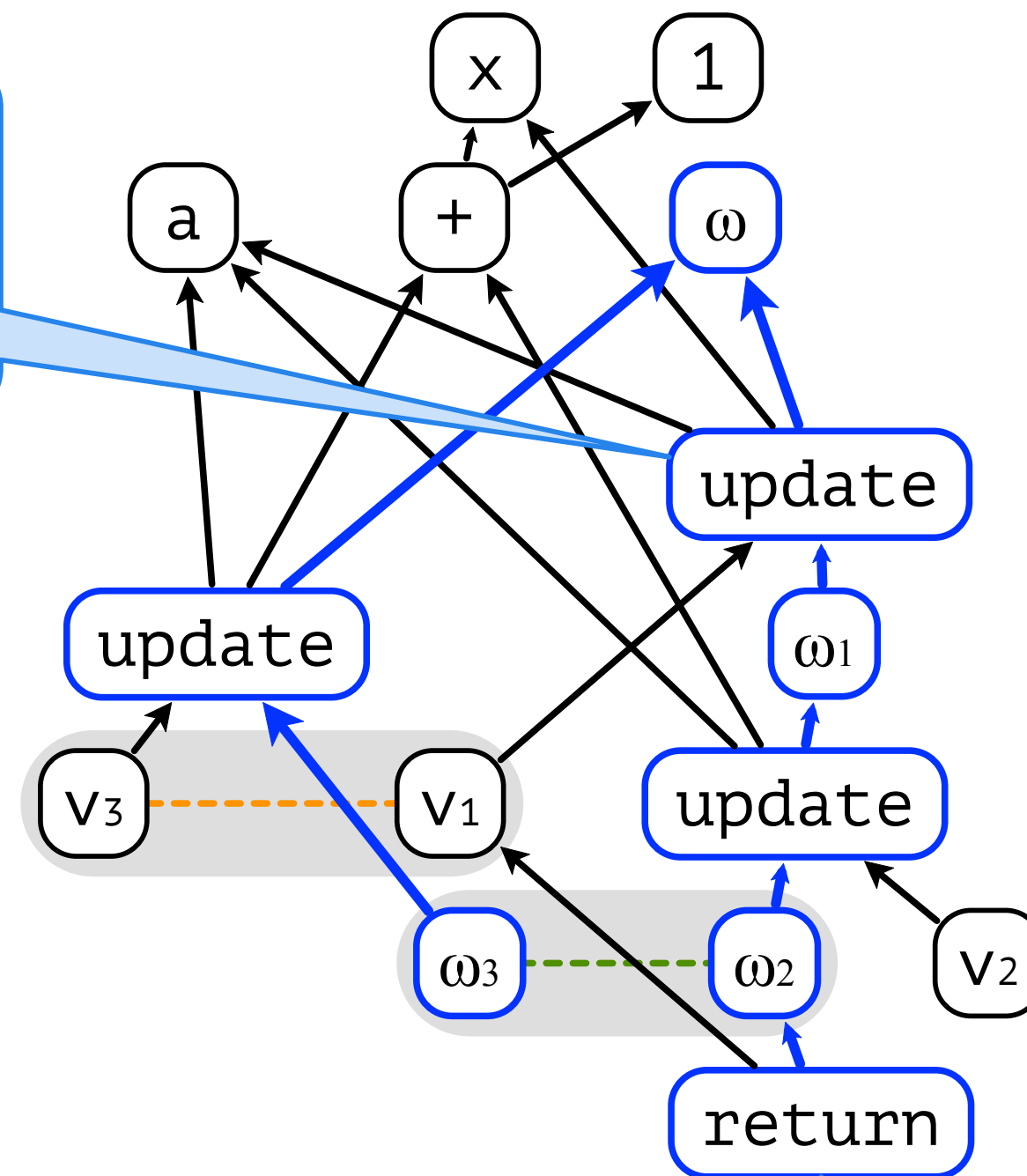
The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```



state walk uses memory linearly

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

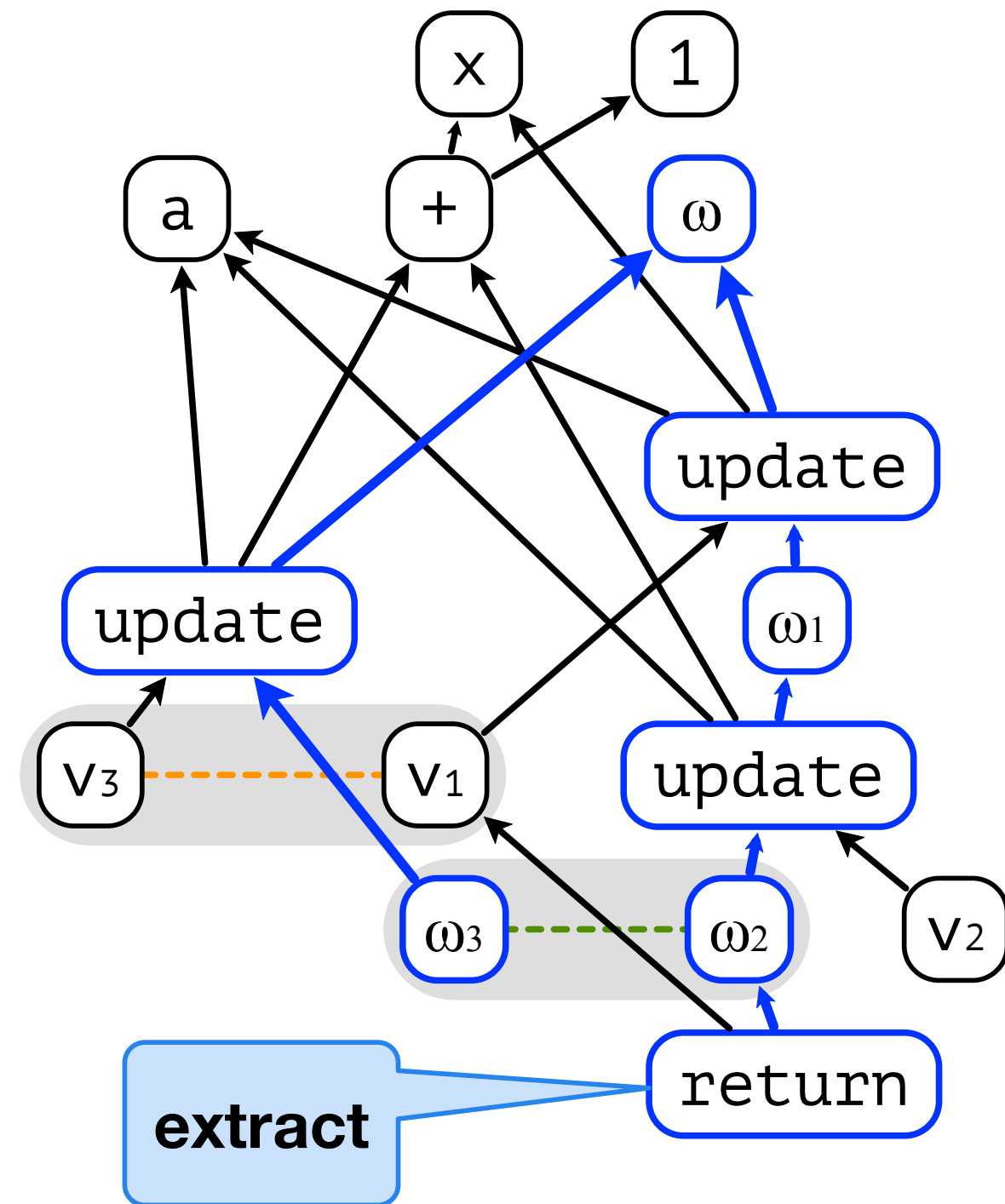
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

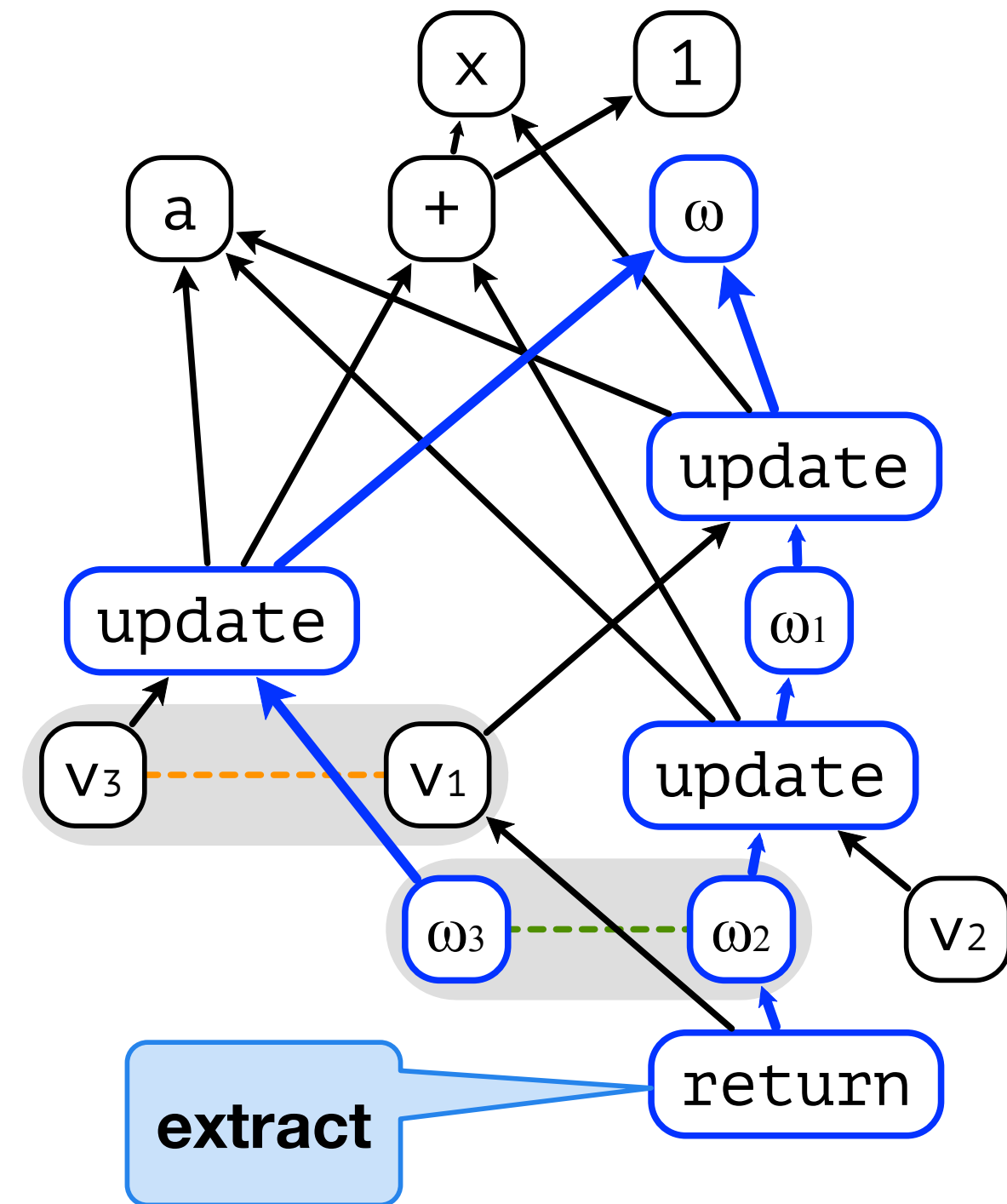
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$

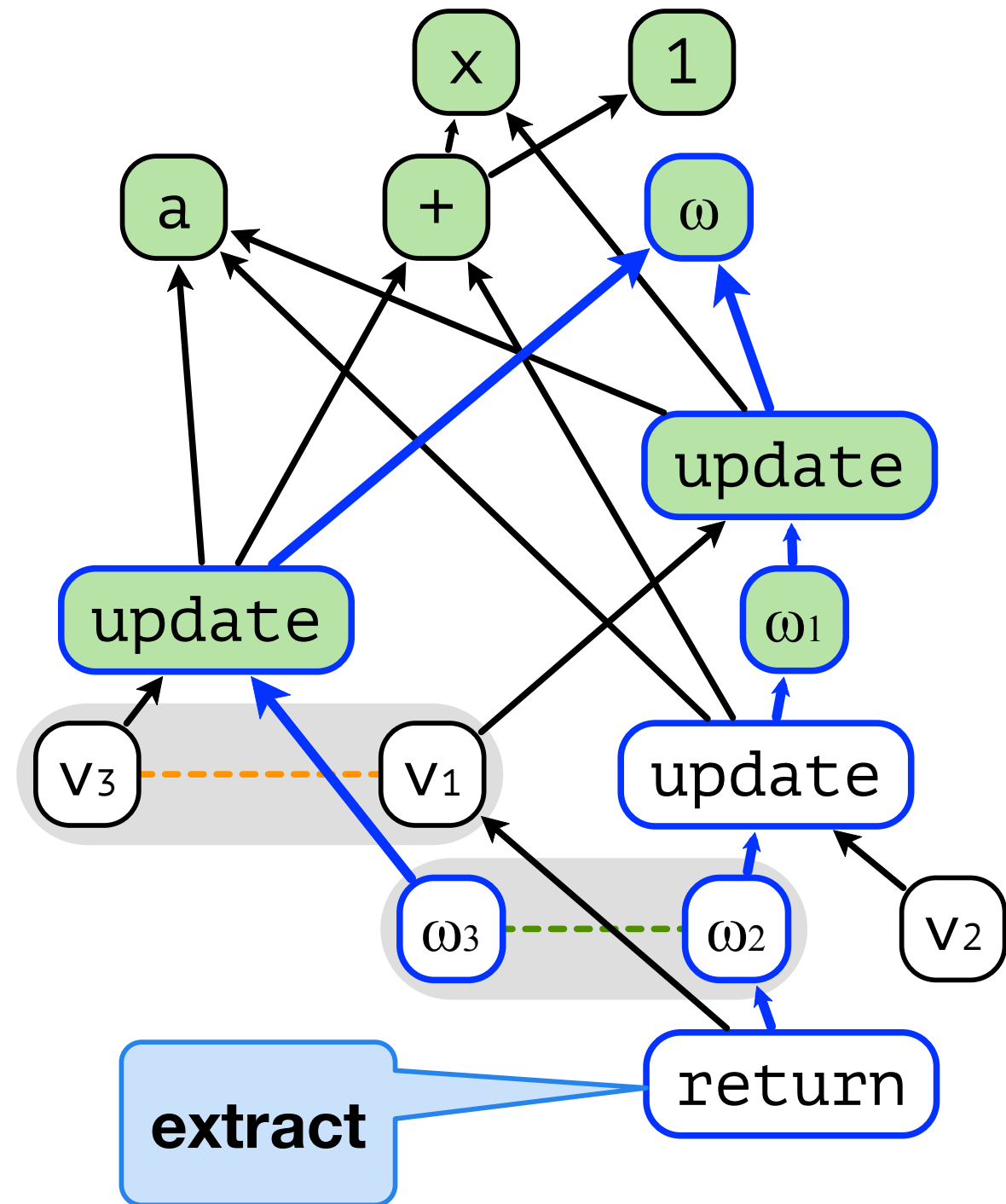


The Effectful E-Graph

```
fun foo(a, x, ω):
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$
$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

```
return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$
$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$
$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$
$$V_1 \equiv V_3 \qquad \omega_2 \equiv \omega_3$$


The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

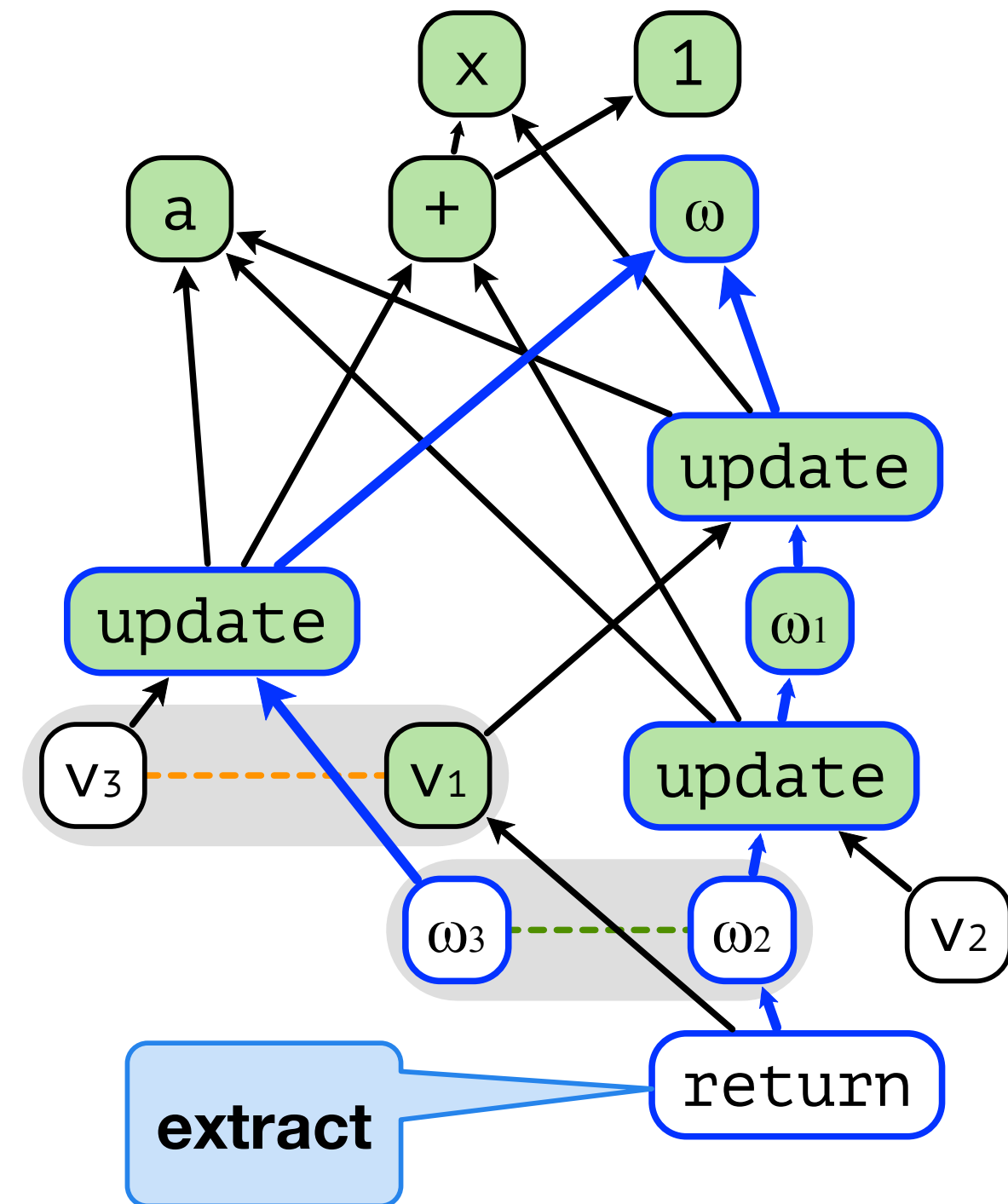
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

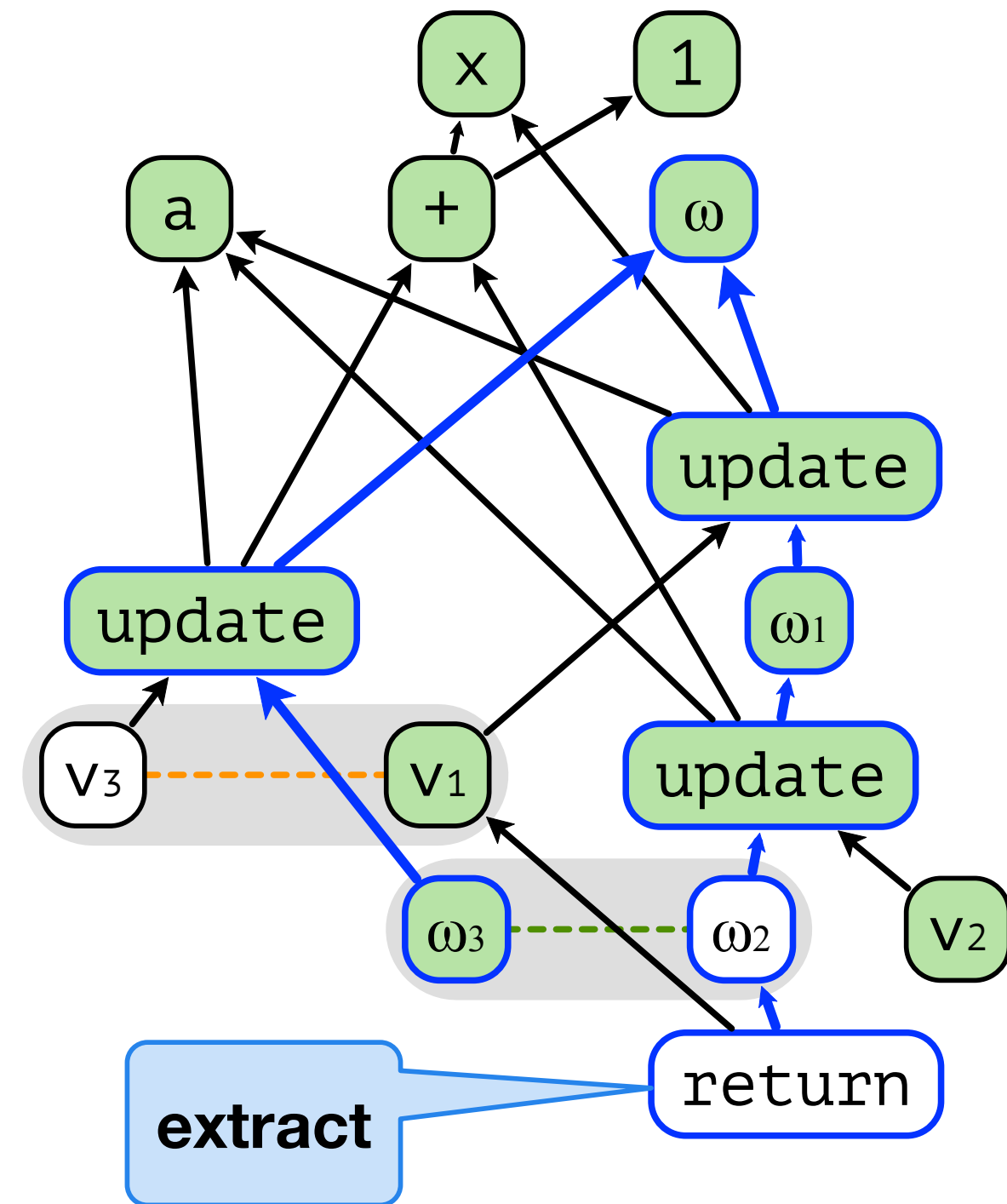
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

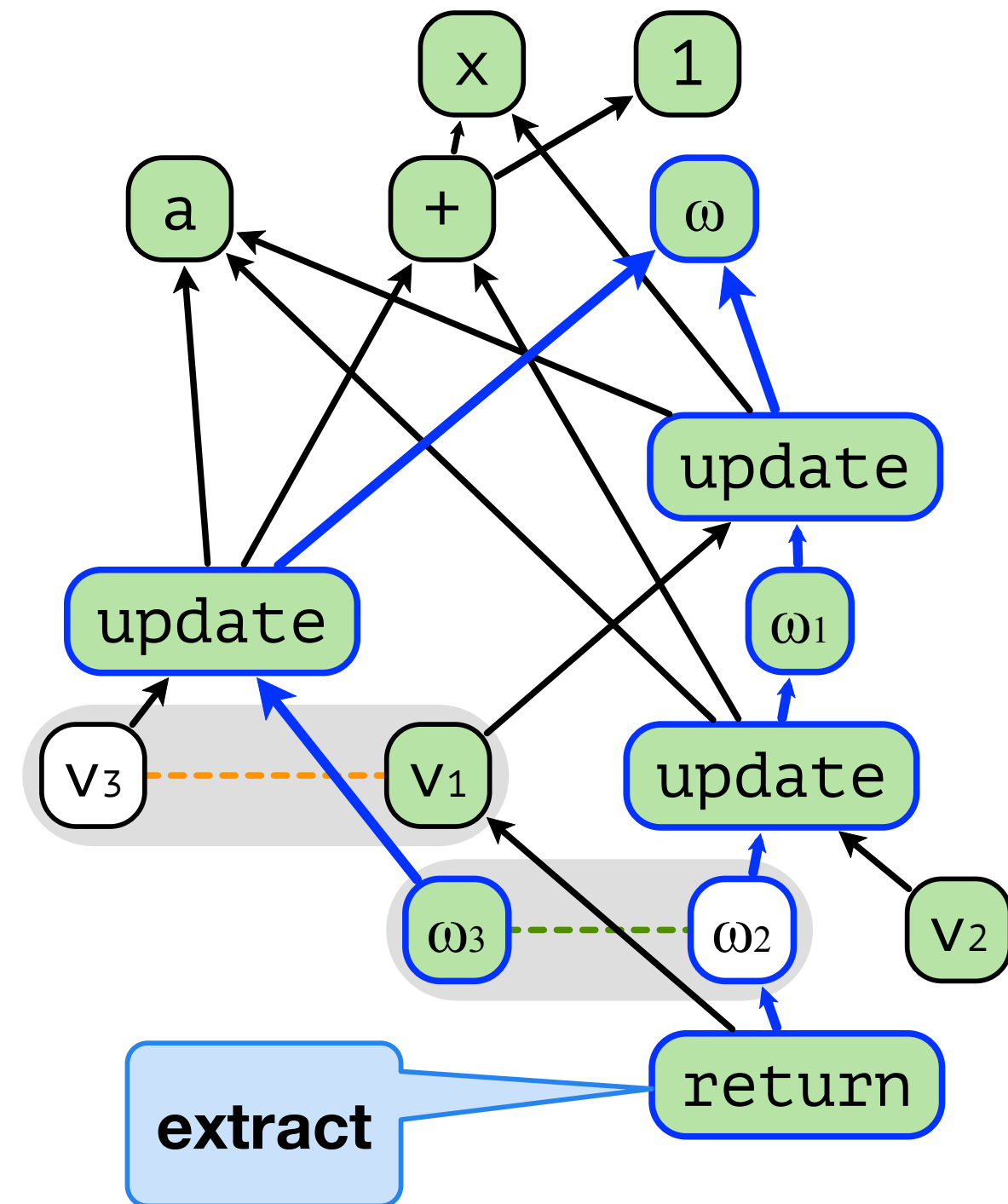
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

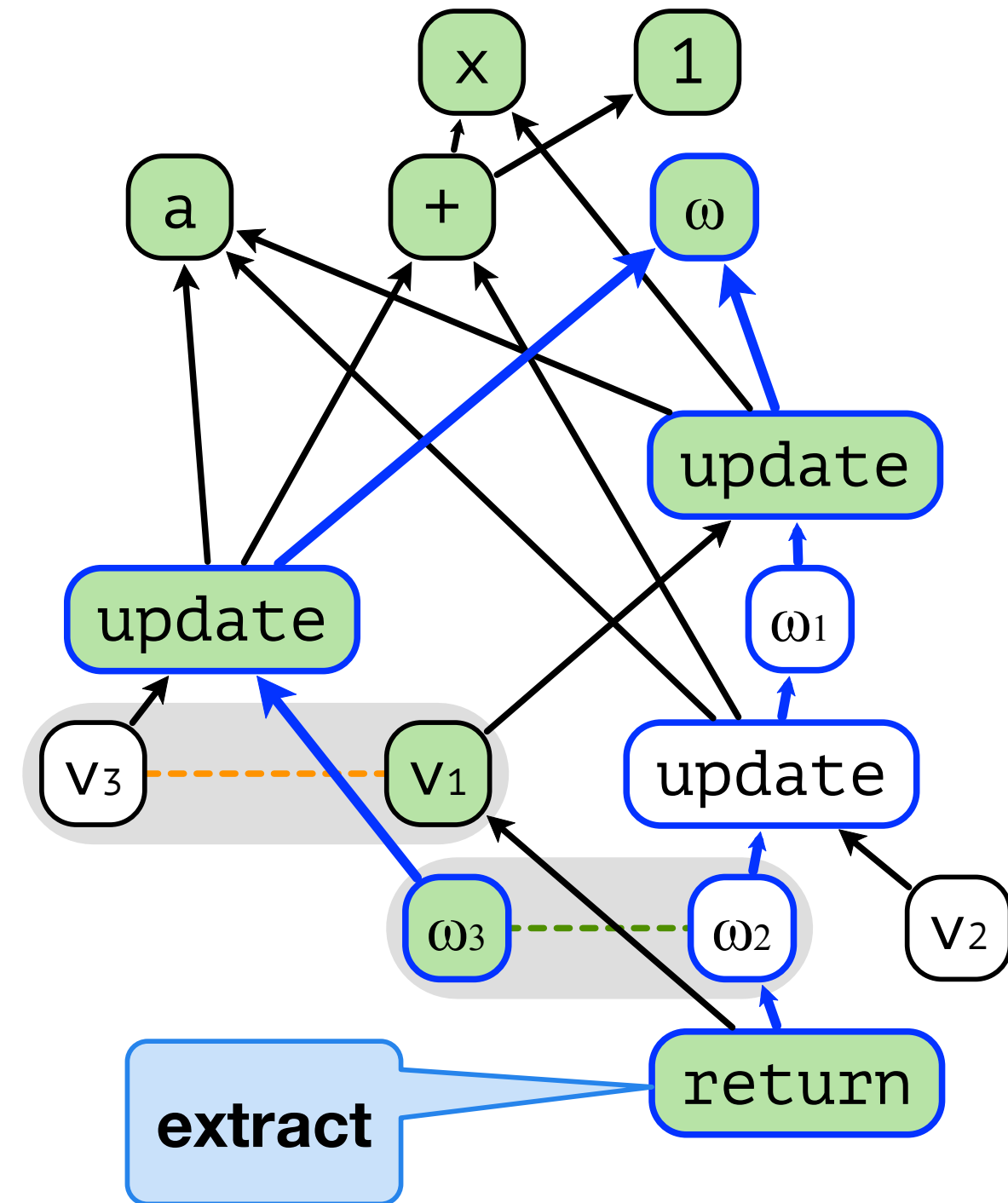
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

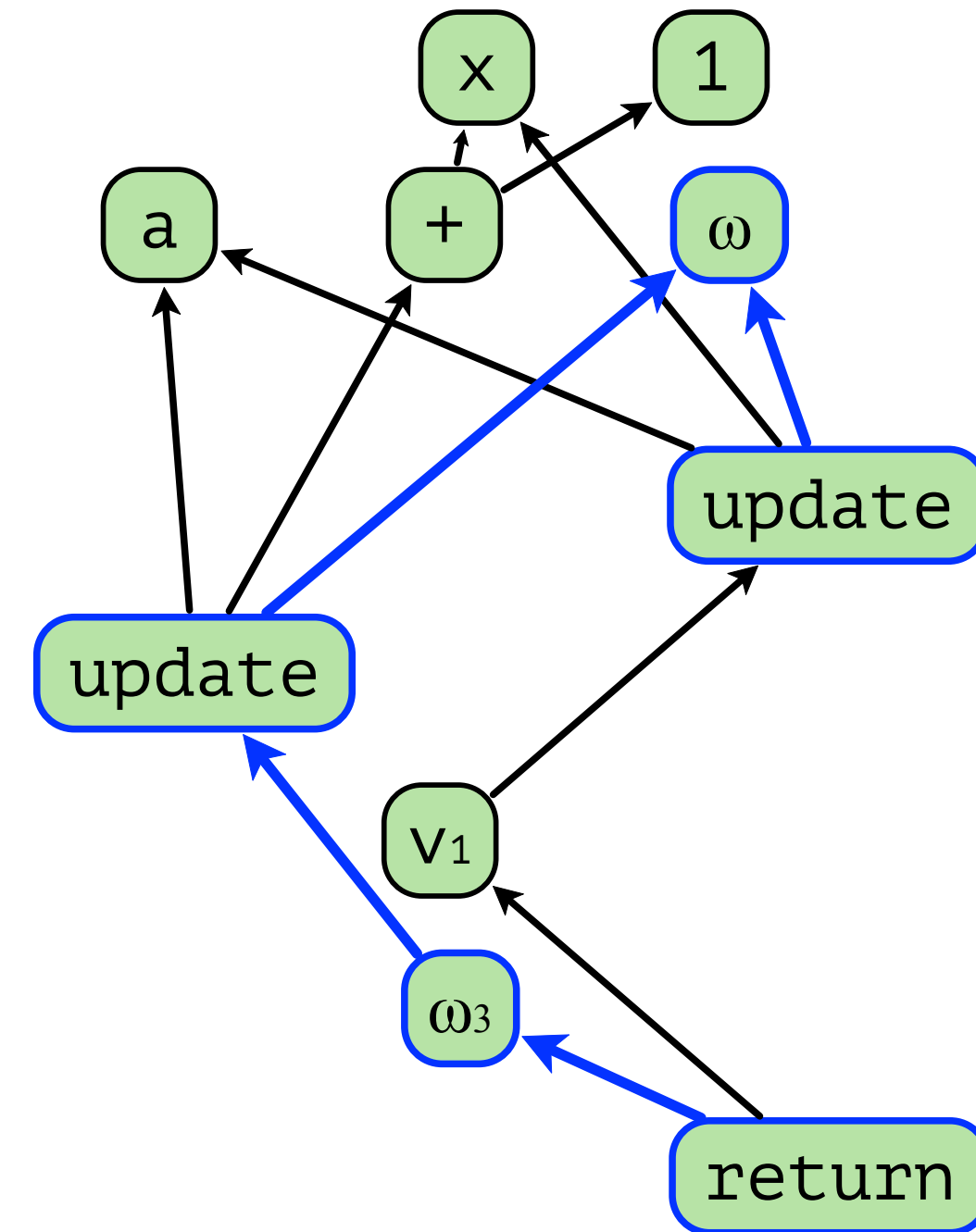
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$$v_1 \equiv v_3 \quad \omega_2 \equiv \omega_3$$


The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

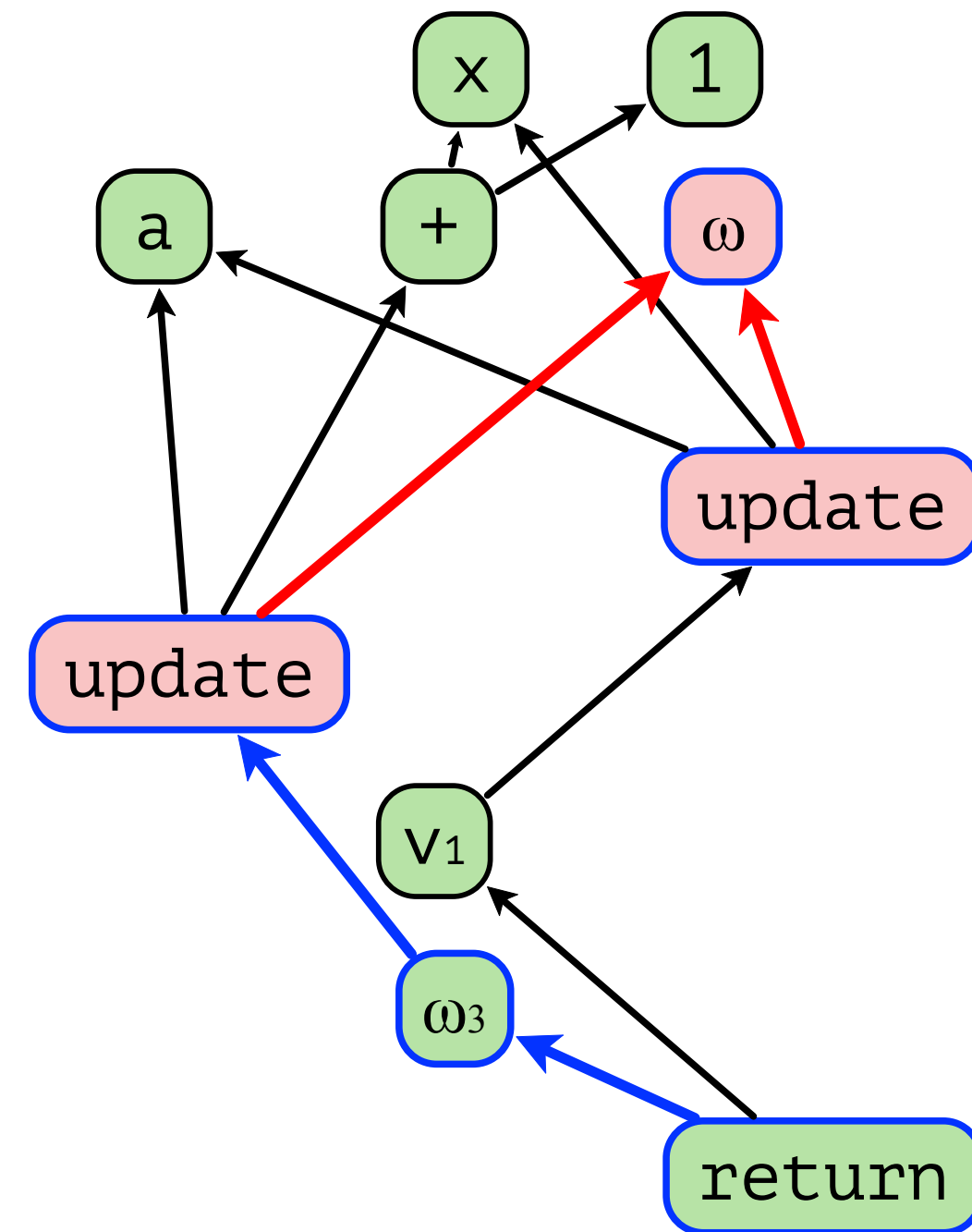
```
  (v2, ω2) = update(a, x+1, ω1)
```

```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$$v_1 \equiv v_3 \quad \omega_2 \equiv \omega_3$$


The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

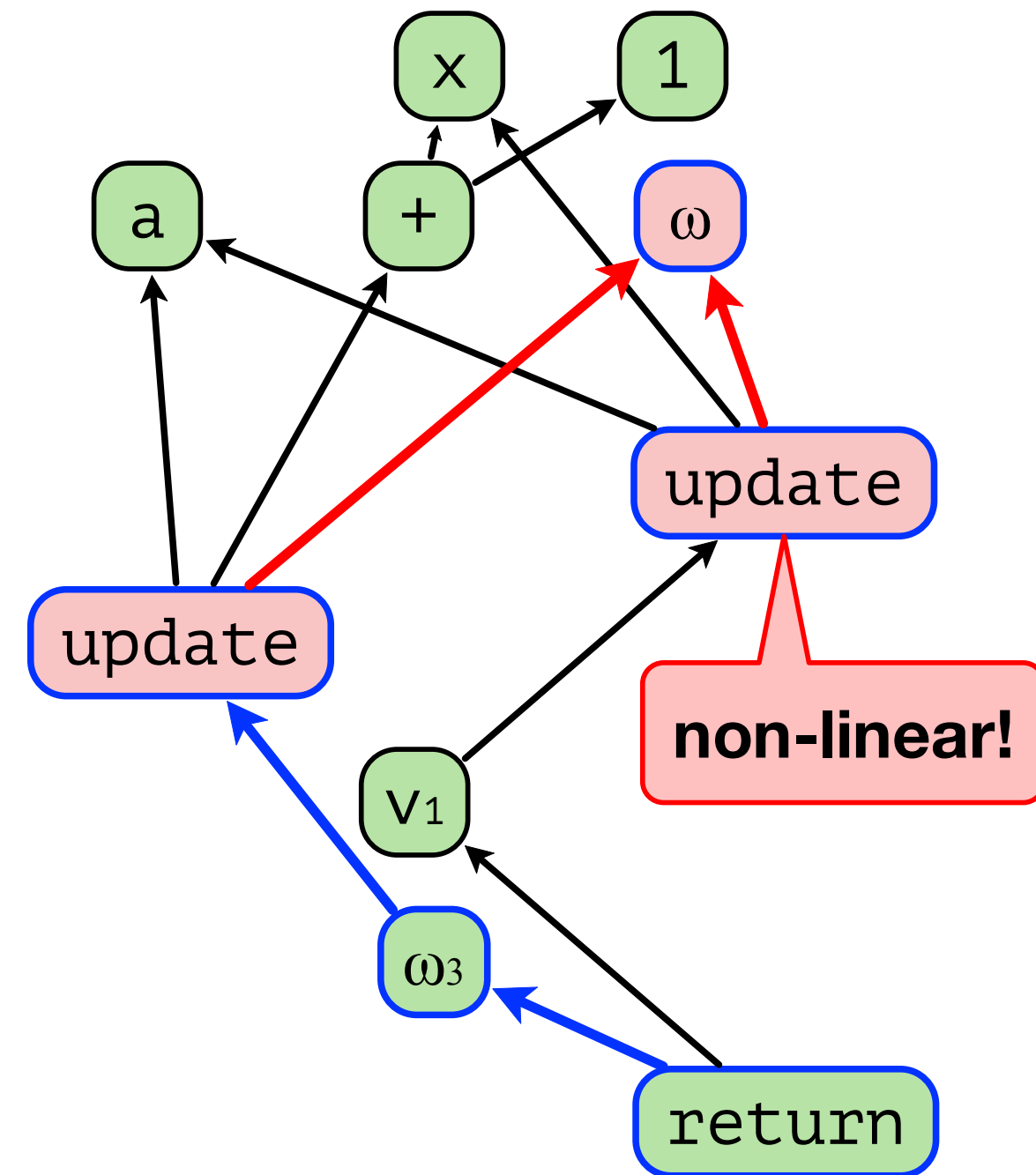
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

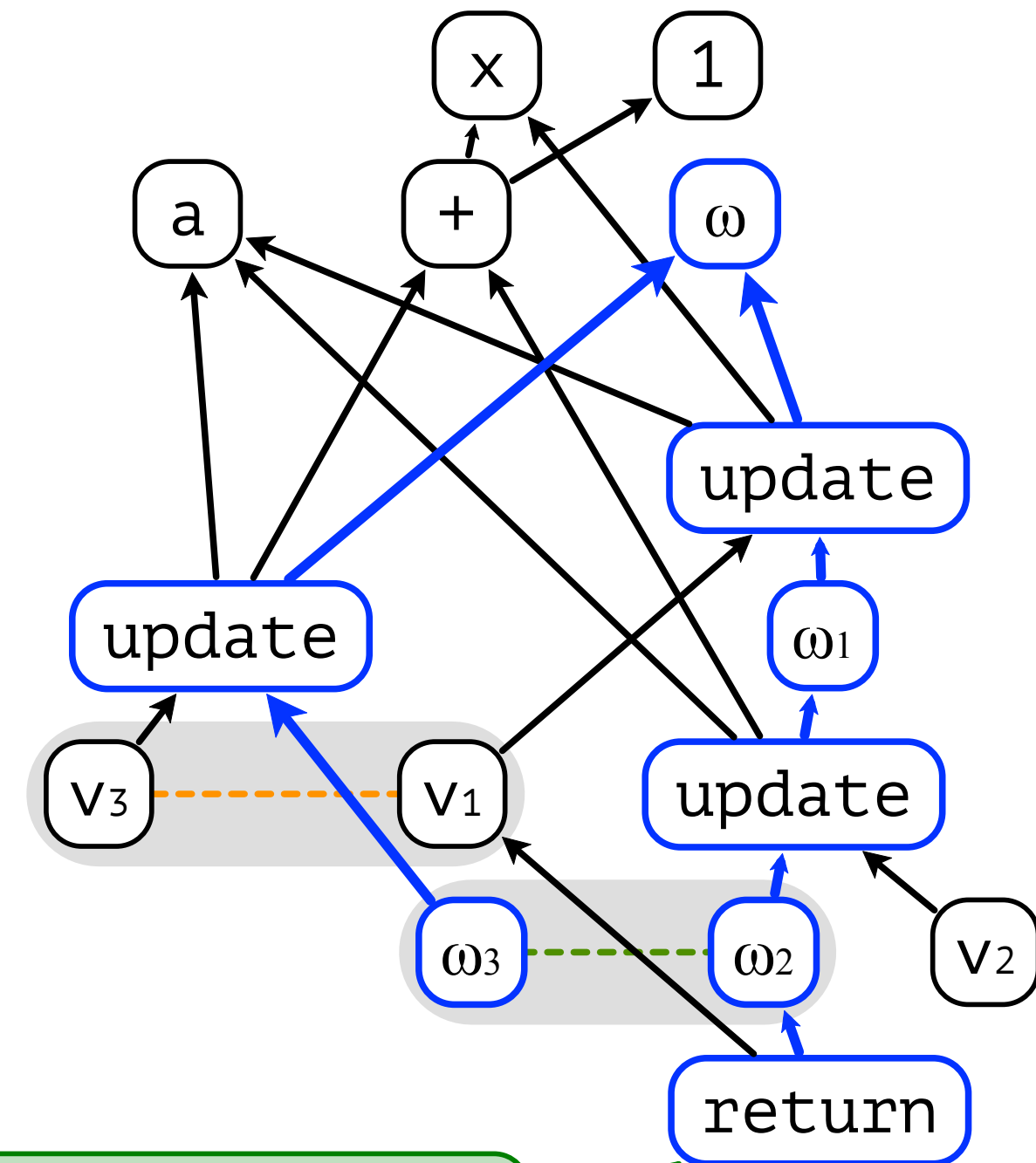
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

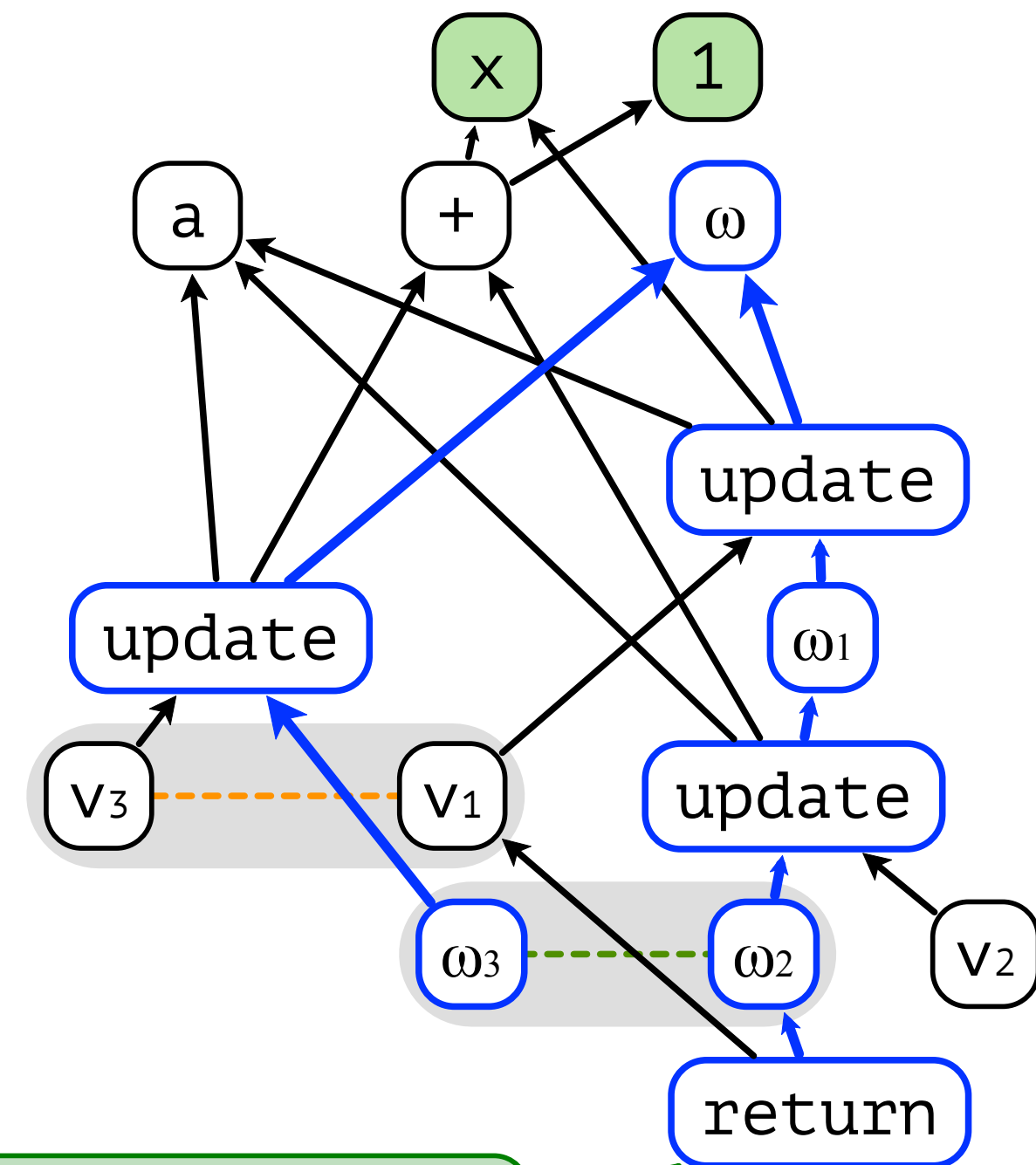
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

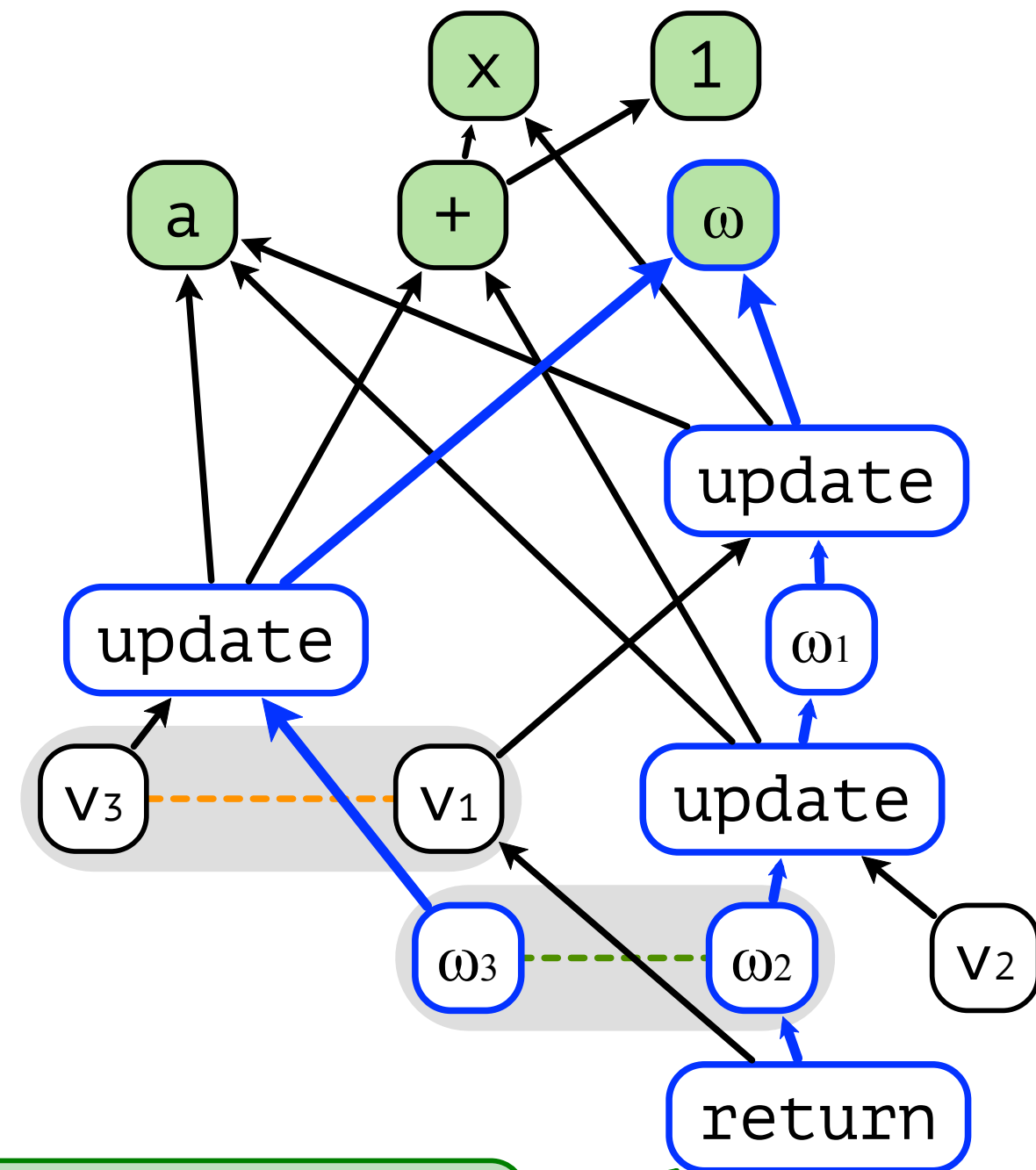
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

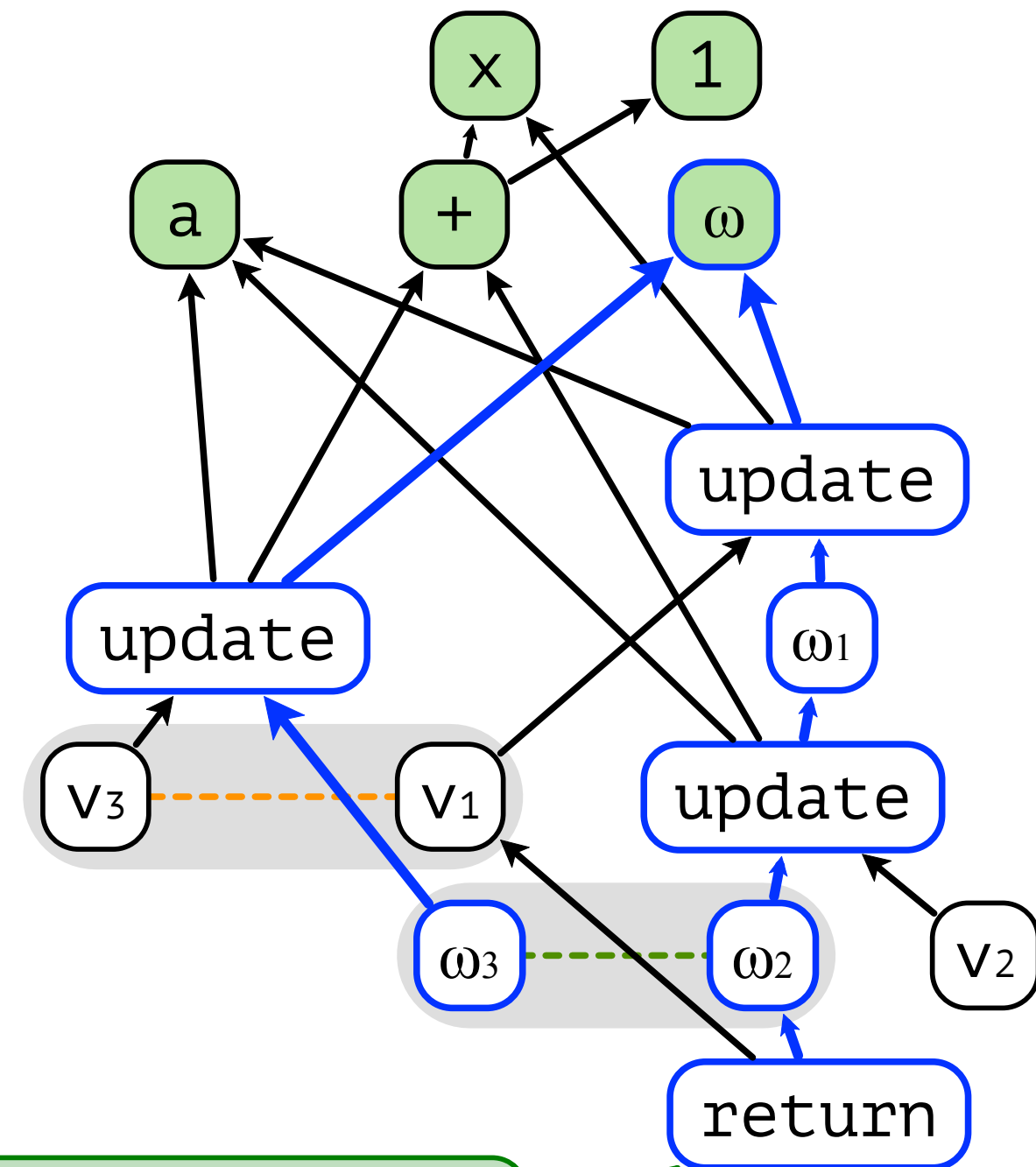
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

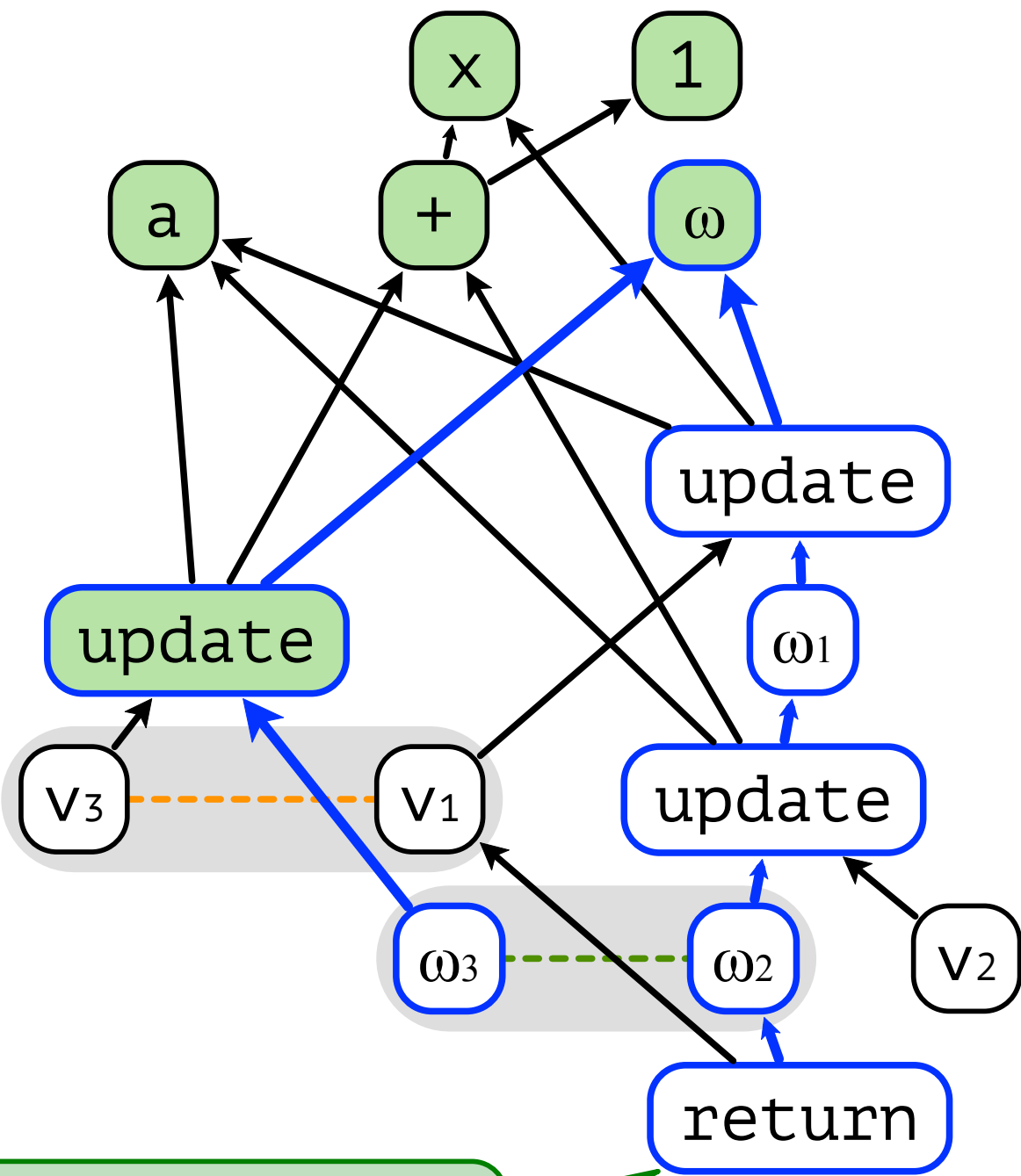
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$$v_1 \equiv v_3$$

$$\omega_2 \equiv \omega_3$$


**extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

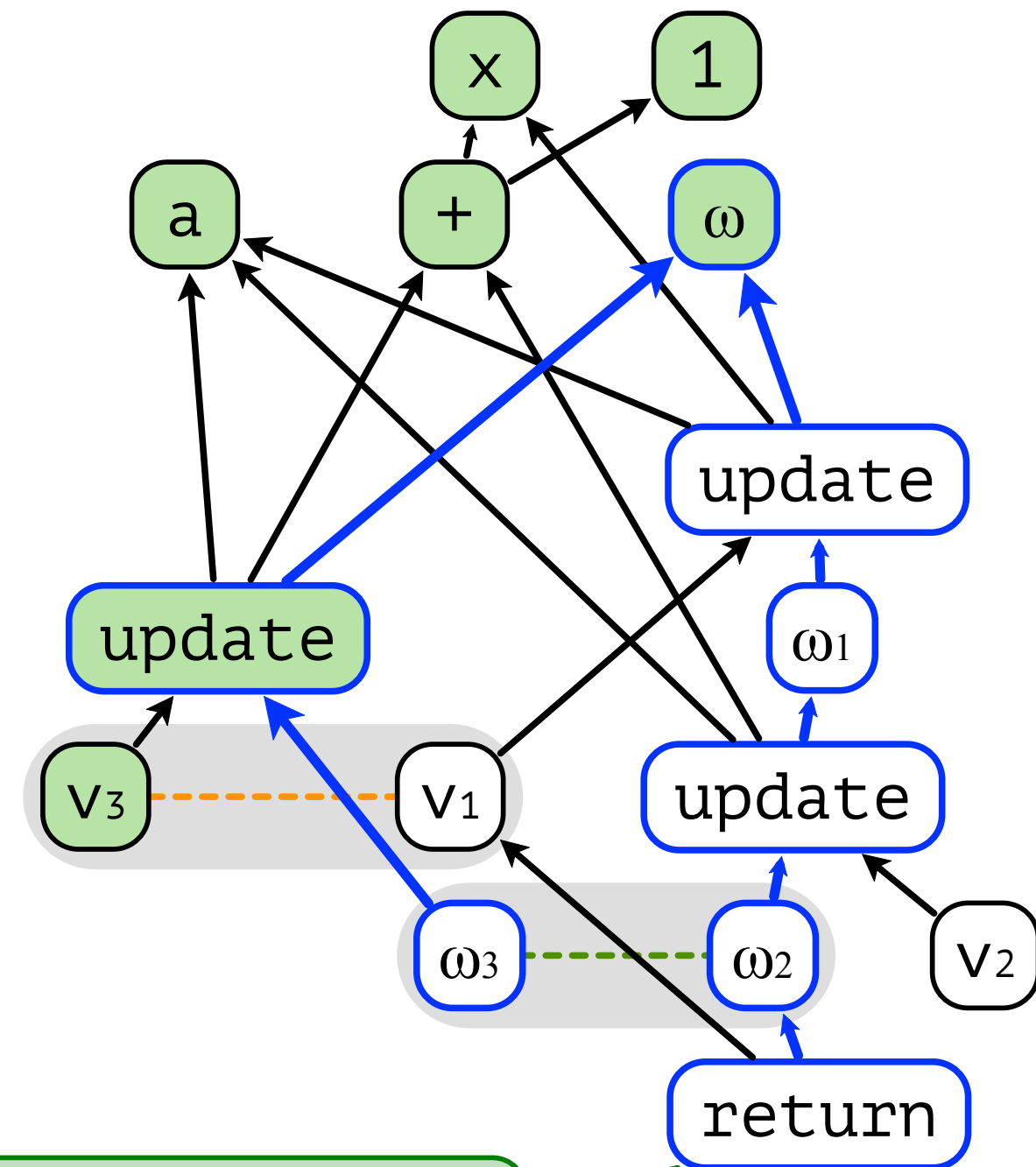
```
  return (v1, ω2)
```

```
(v1, ω1) = update(a, x, ω)
```

```
(v2, ω2) = update(a, y, ω1)
```

```
(v3, ω3) = update(a, y, ω)
```

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

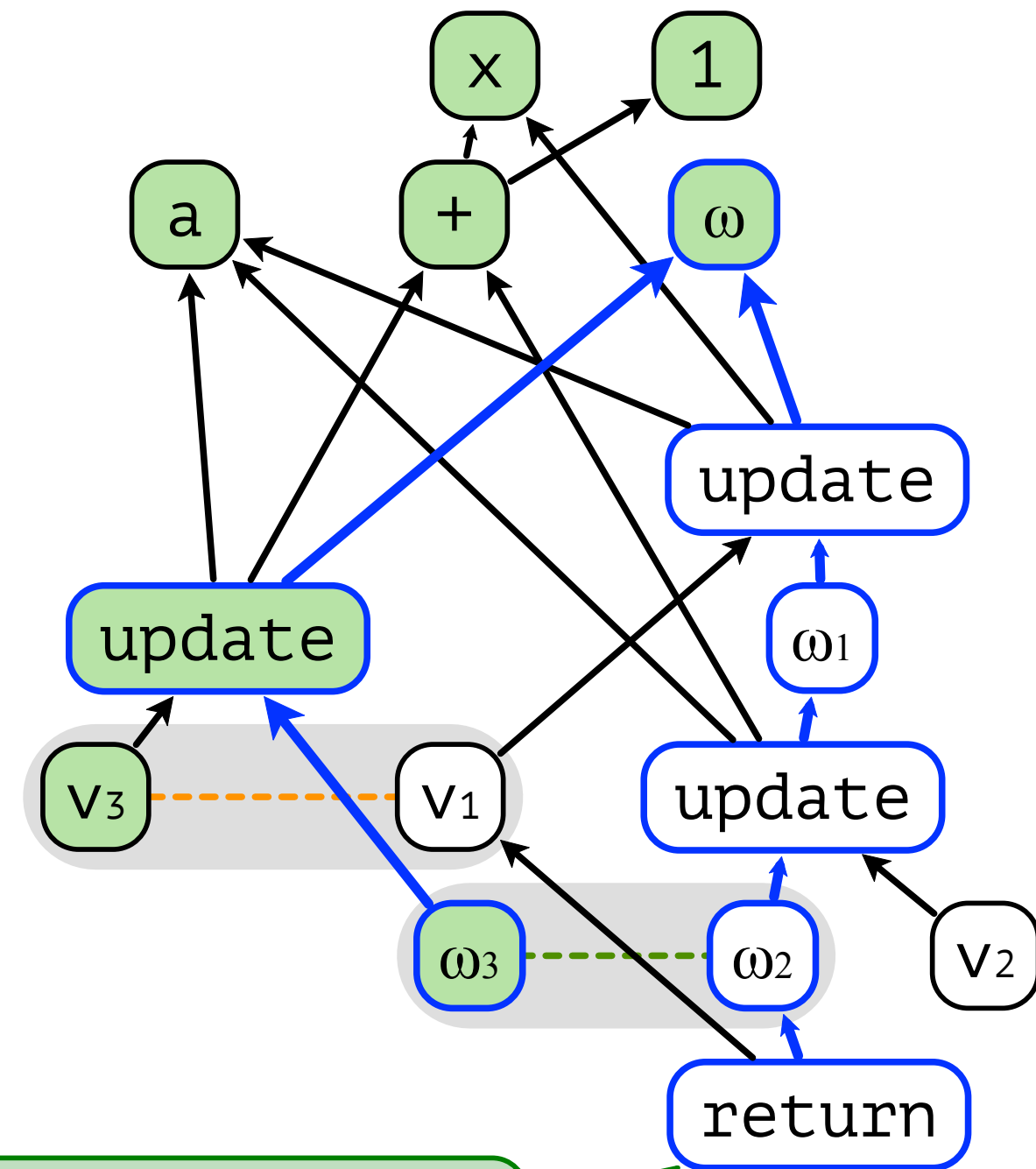
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

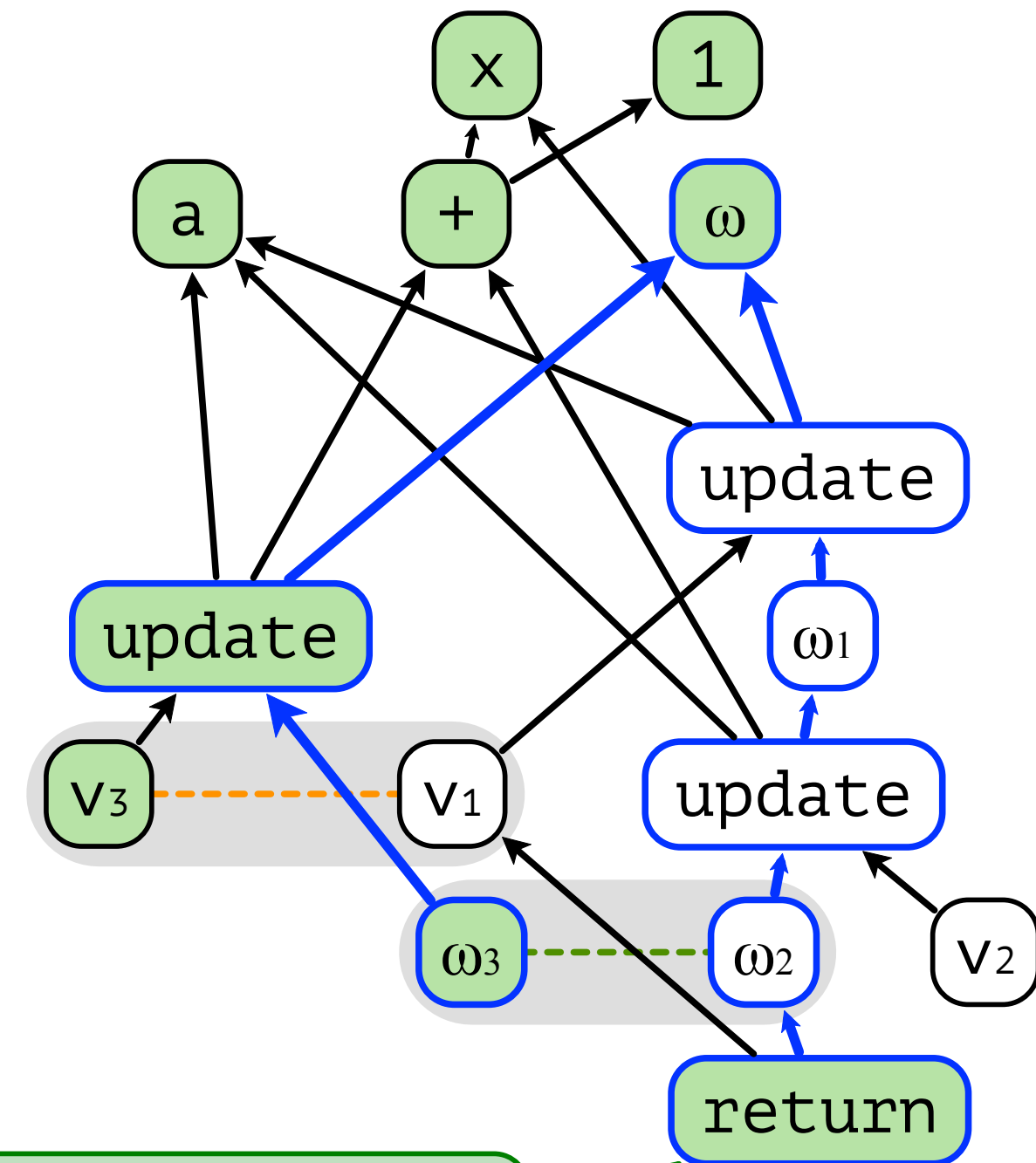
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



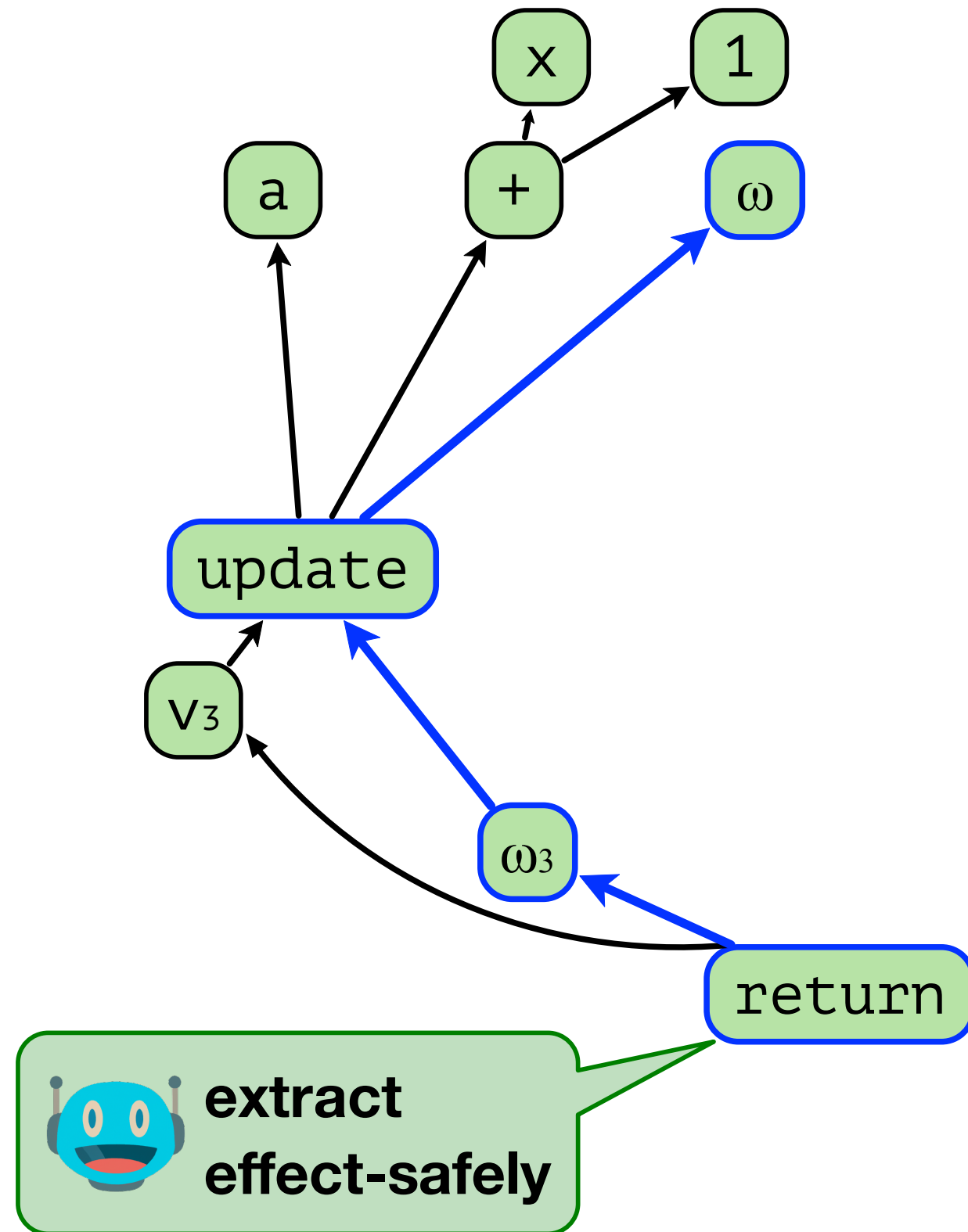
 **extract
effect-safely**

The Effectful E-Graph

```
fun foo(a, x, ω):
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$
$$(v_2, \omega_2) = \text{update}(a, x+1, \omega_1)$$

```
return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$
$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$
$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$
$$V_1 \equiv V_3 \qquad \omega_2 \equiv \omega_3$$


The Effectful E-Graph

```
fun foo(a, x, ω):
```

```
  (v1, ω1) = update(a, x, ω)
```

```
  (v2, ω2) = update(a, x+1, ω1)
```

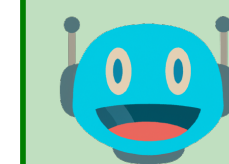
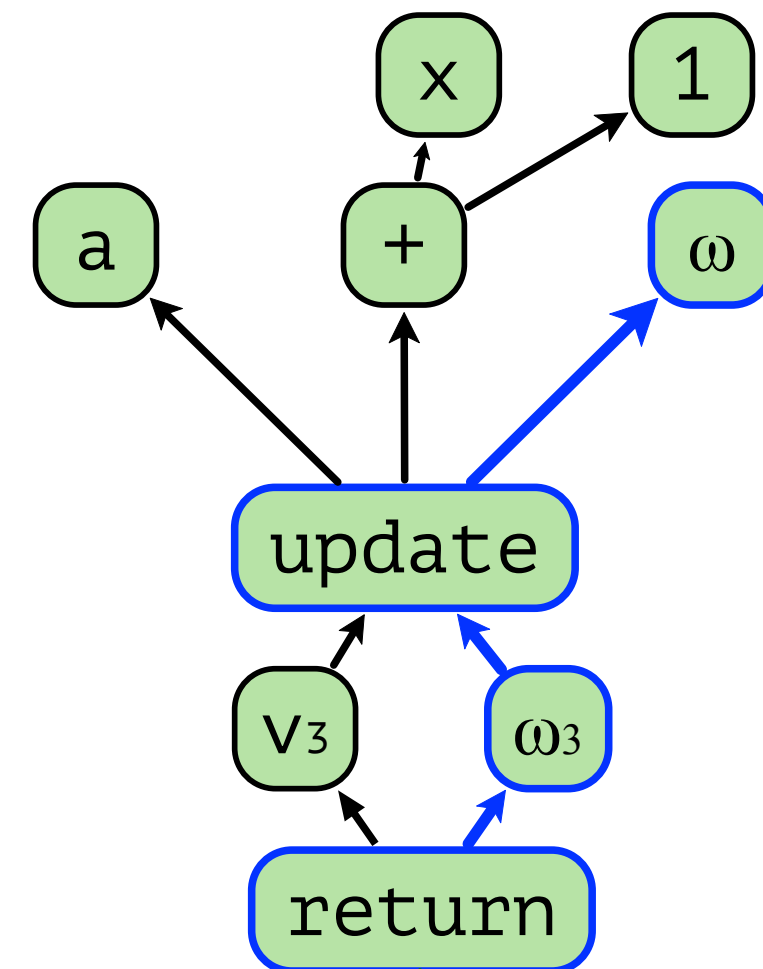
```
  return (v1, ω2)
```

$$(v_1, \omega_1) = \text{update}(a, x, \omega)$$

$$(v_2, \omega_2) = \text{update}(a, y, \omega_1)$$

$$(v_3, \omega_3) = \text{update}(a, y, \omega)$$

$v_1 \equiv v_3$ $\omega_2 \equiv \omega_3$



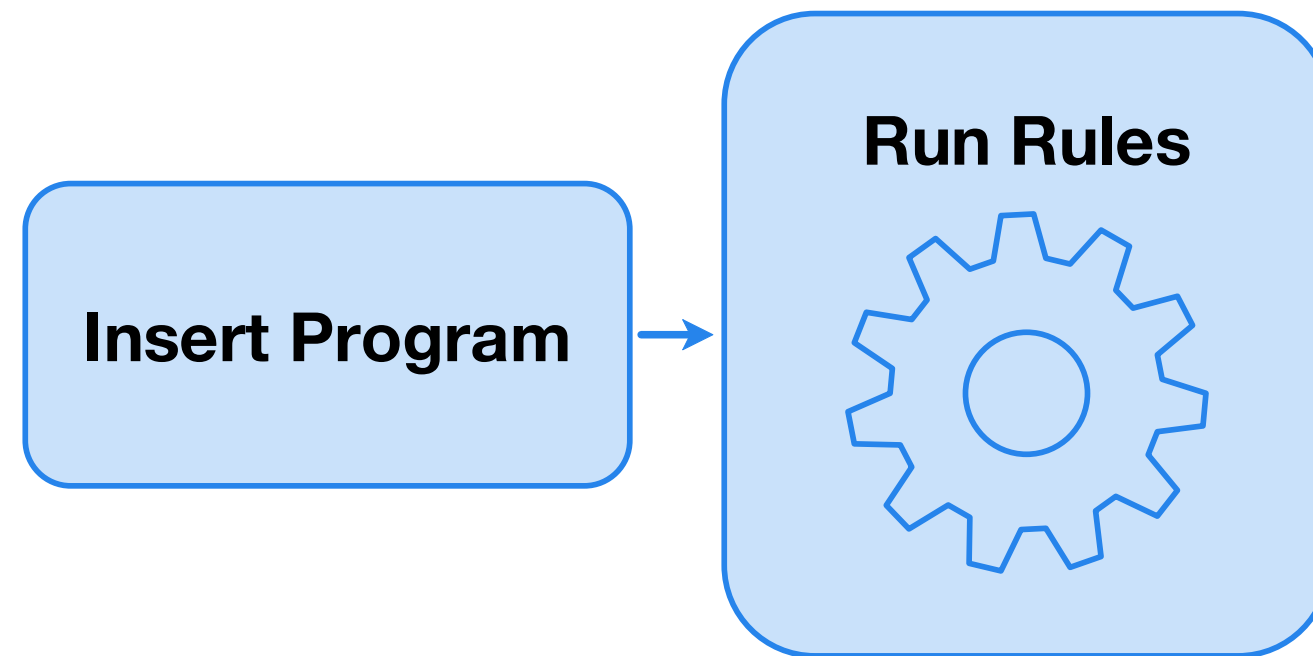
**extract
effect-safely**

E-Graphs Explore a Big Space

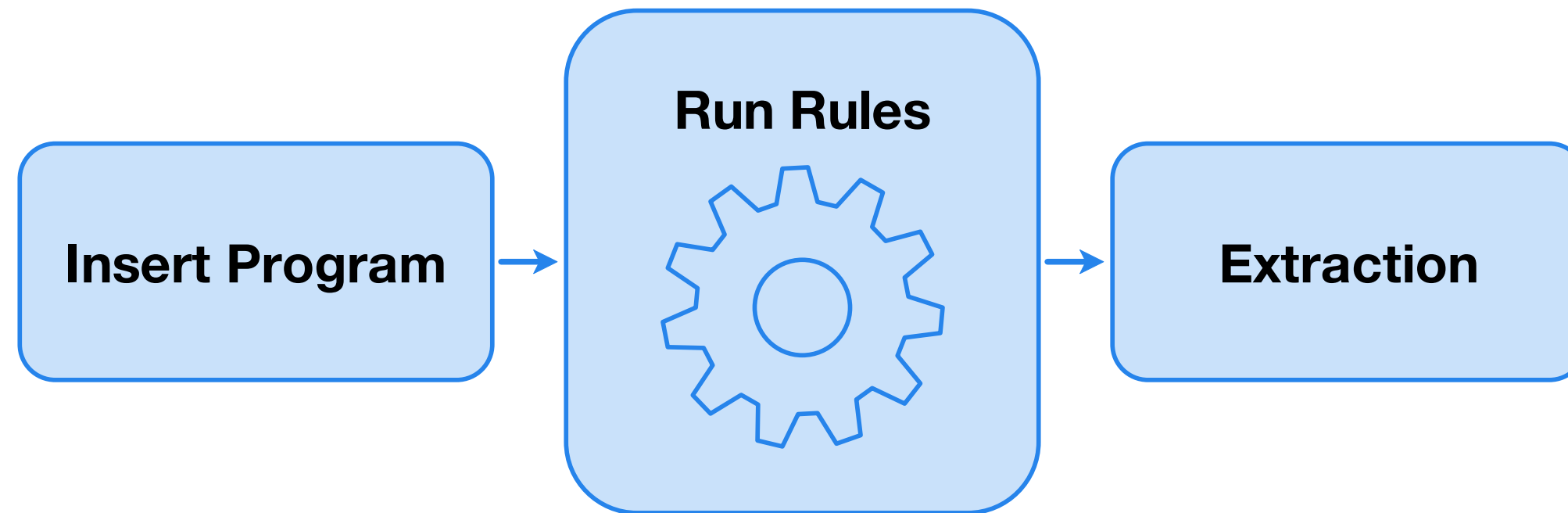
E-Graphs Explore a Big Space

Insert Program

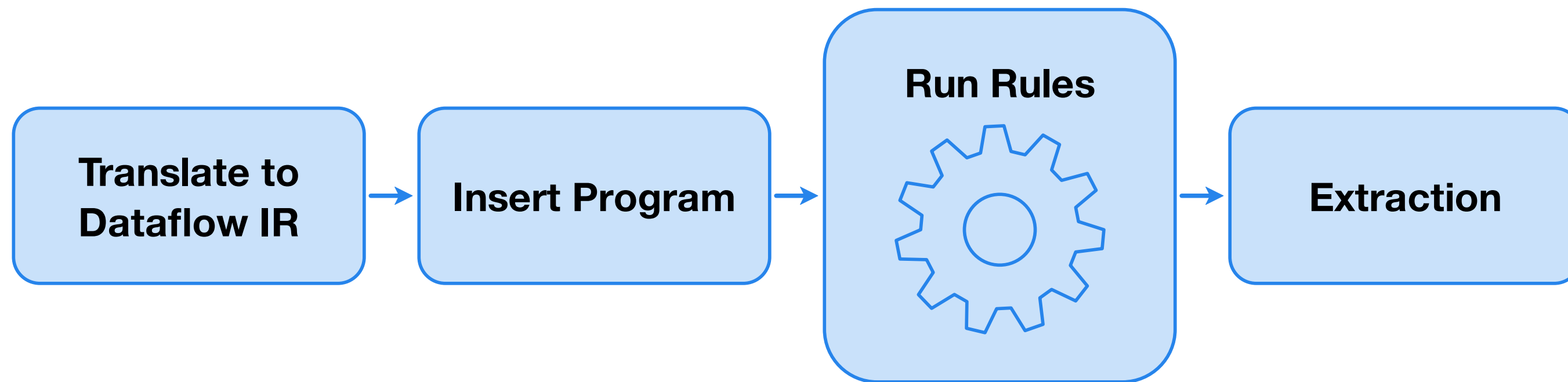
E-Graphs Explore a Big Space



E-Graphs Explore a Big Space

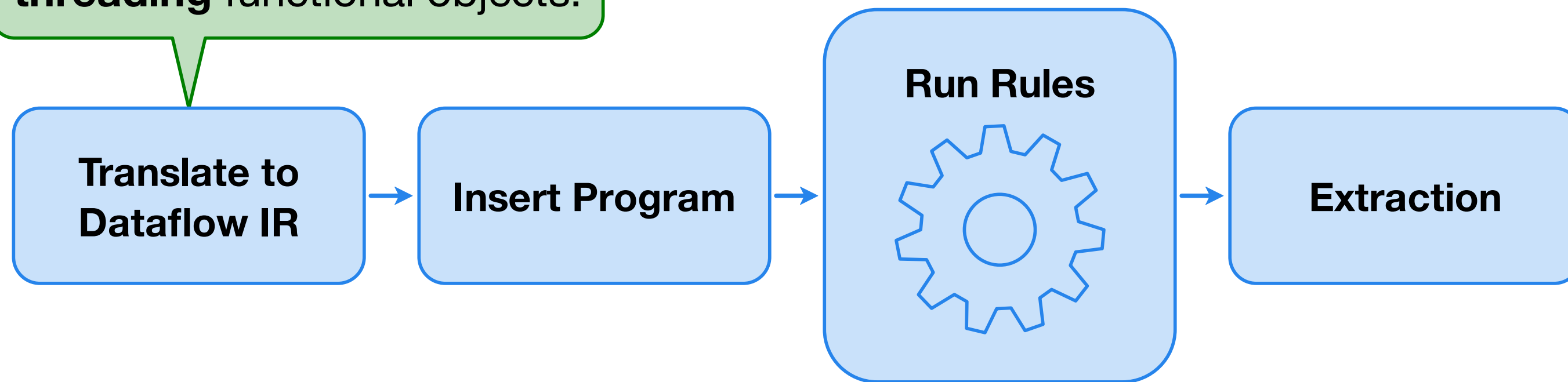


E-Graphs Explore a Big Space

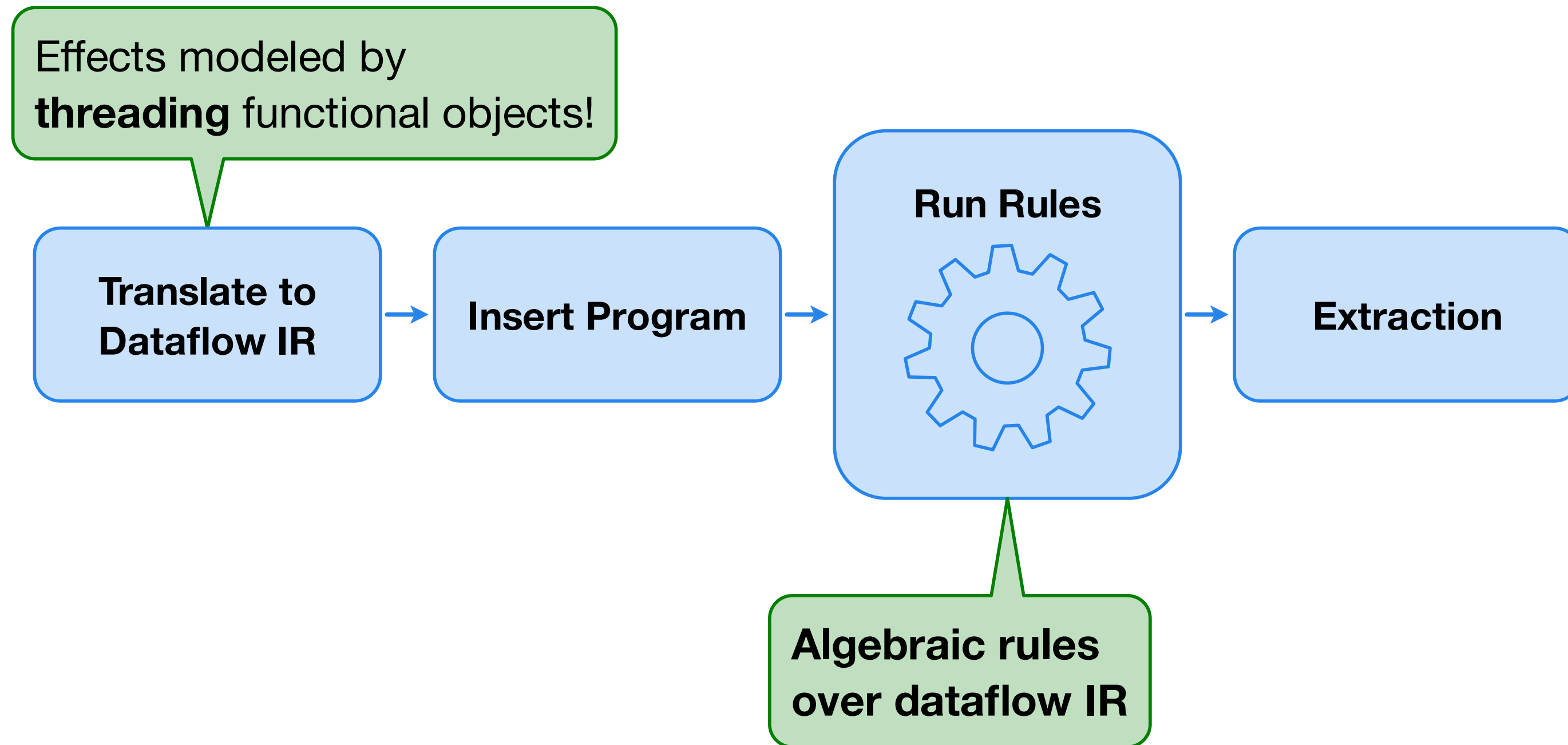


E-Graphs Explore a Big Space

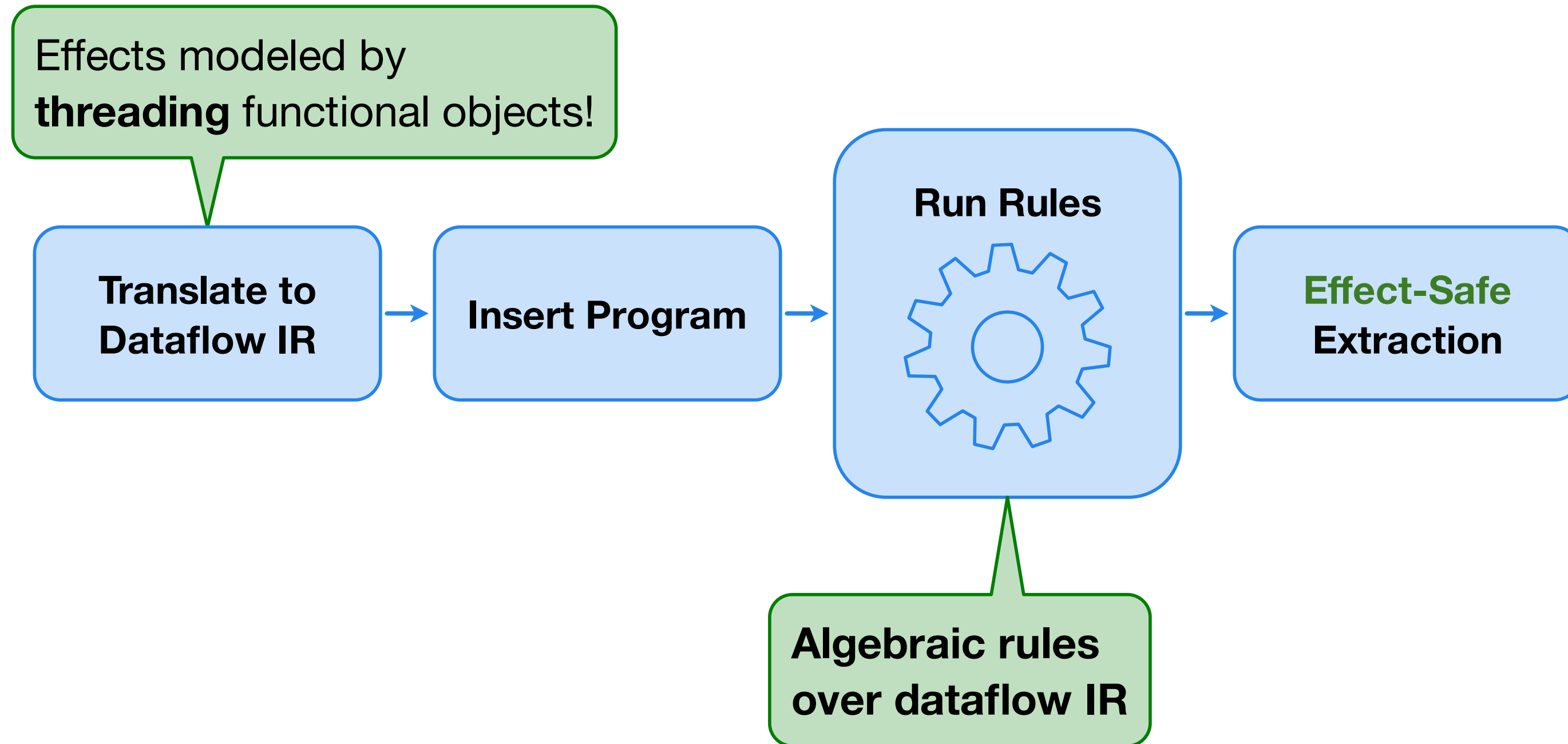
Effects modeled by
threading functional objects!



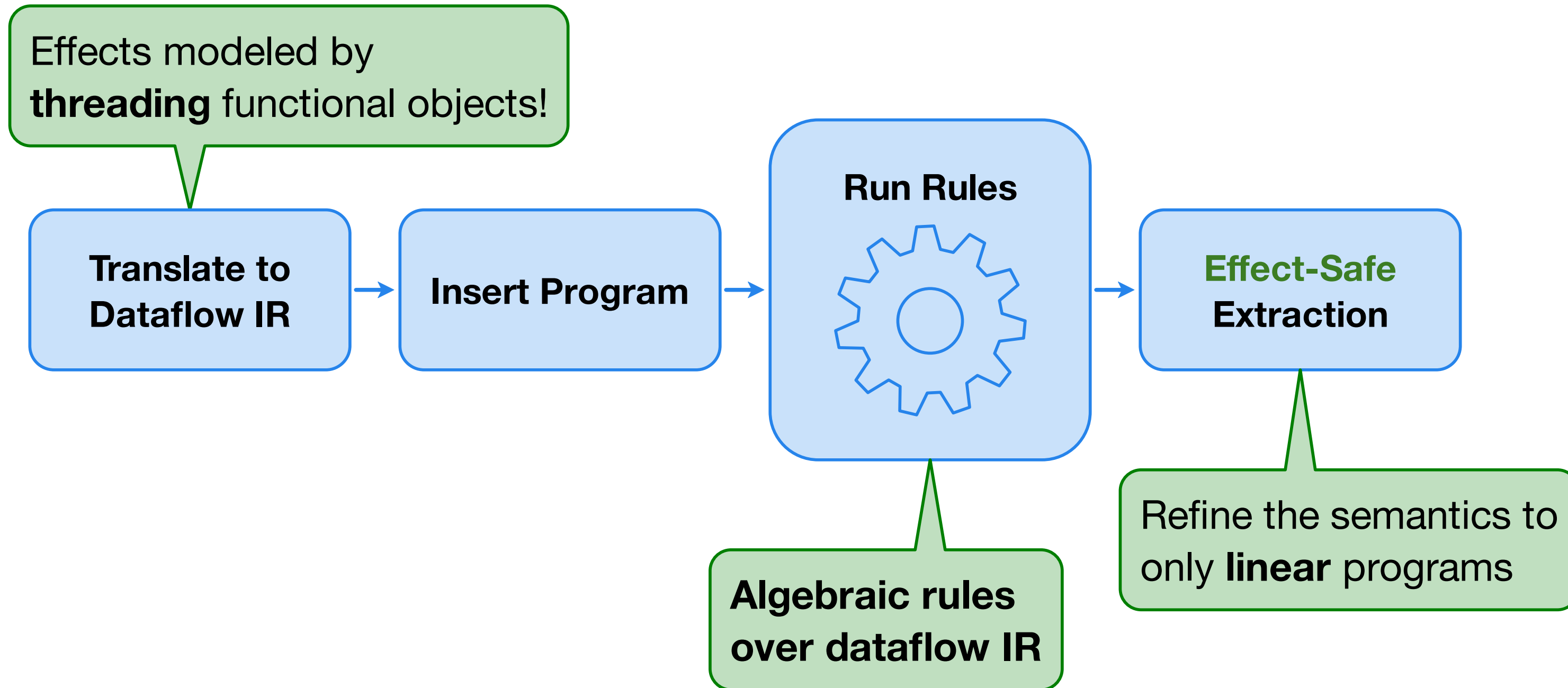
E-Graphs Explore a Big Space



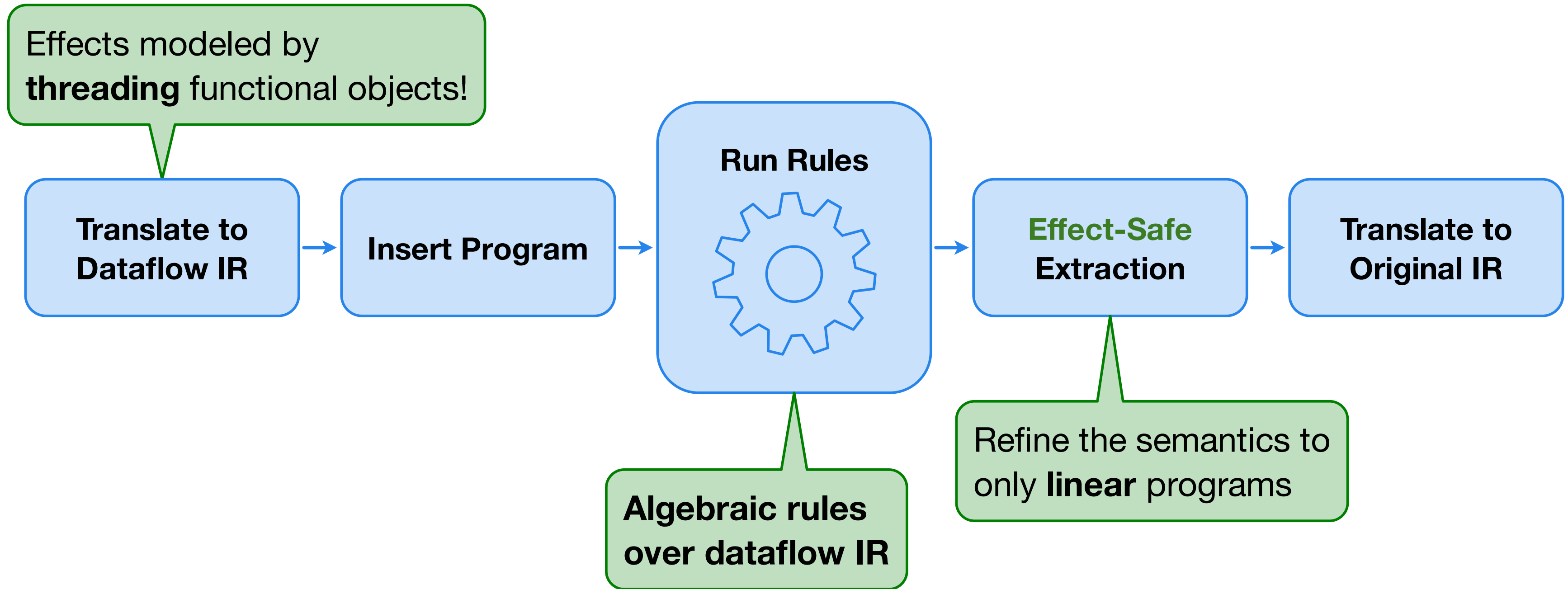
E-Graphs Explore a Big Space



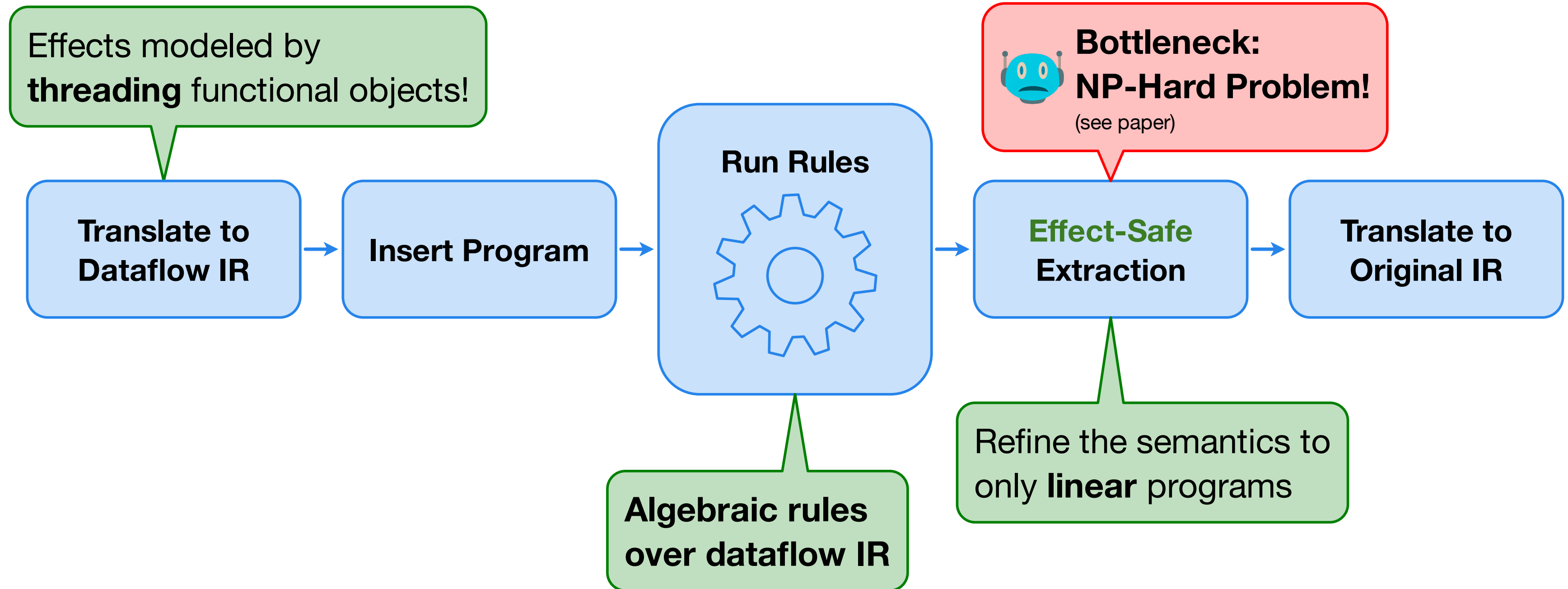
E-Graphs Explore a Big Space

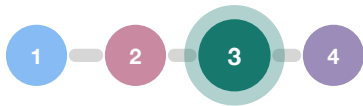


E-Graphs Explore a Big Space



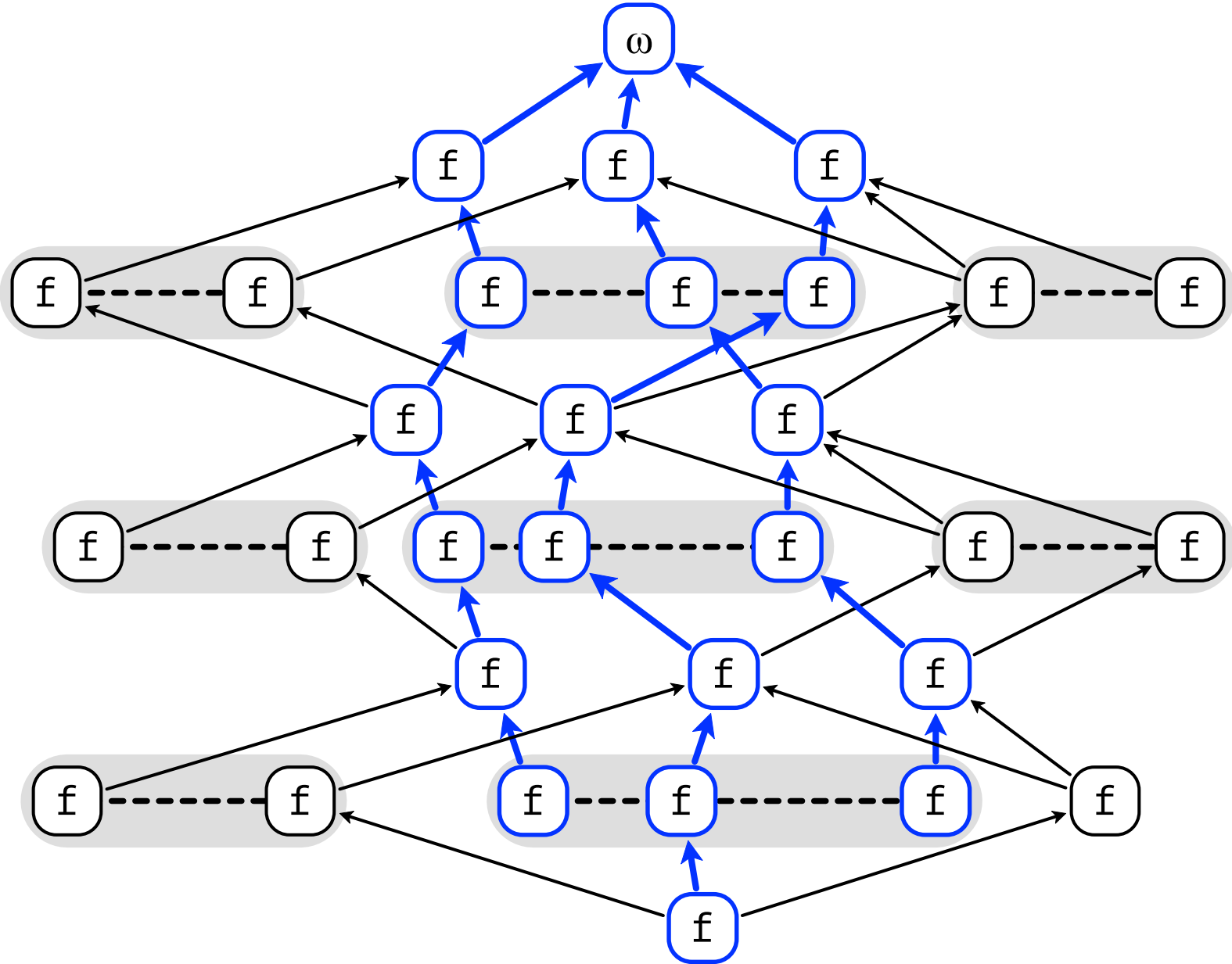
E-Graphs Explore a Big Space



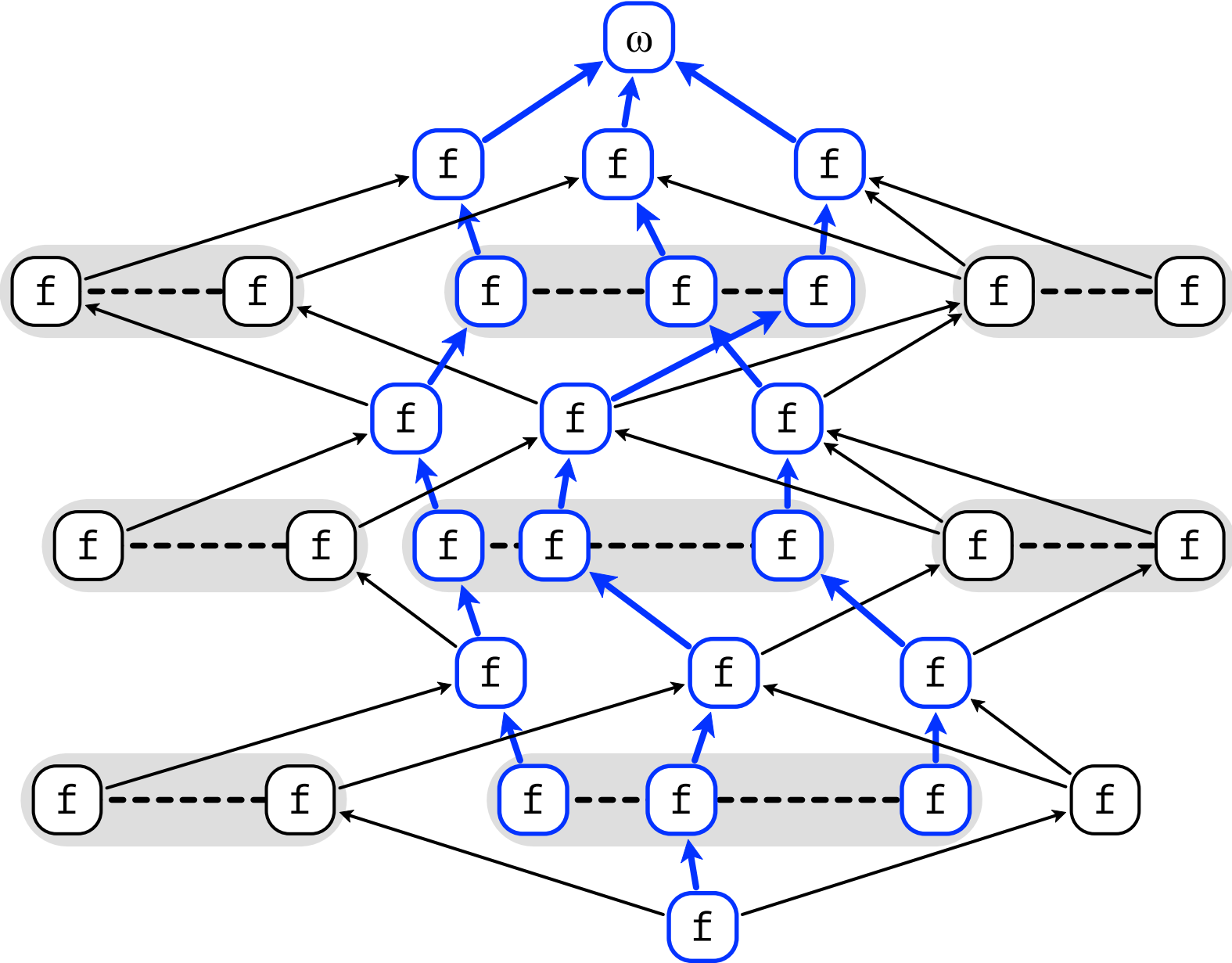


Give Up?

Give Up?

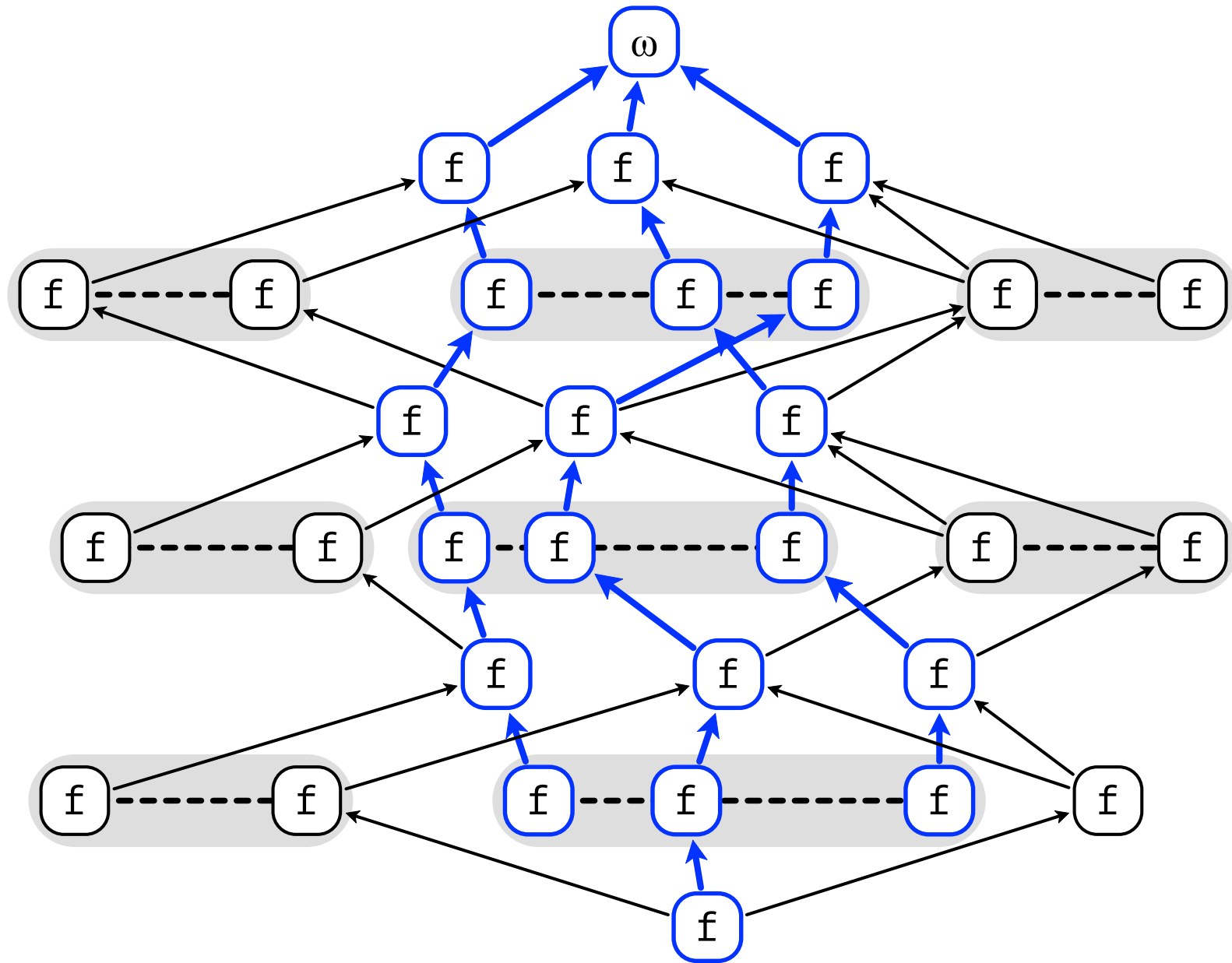


Give Up?

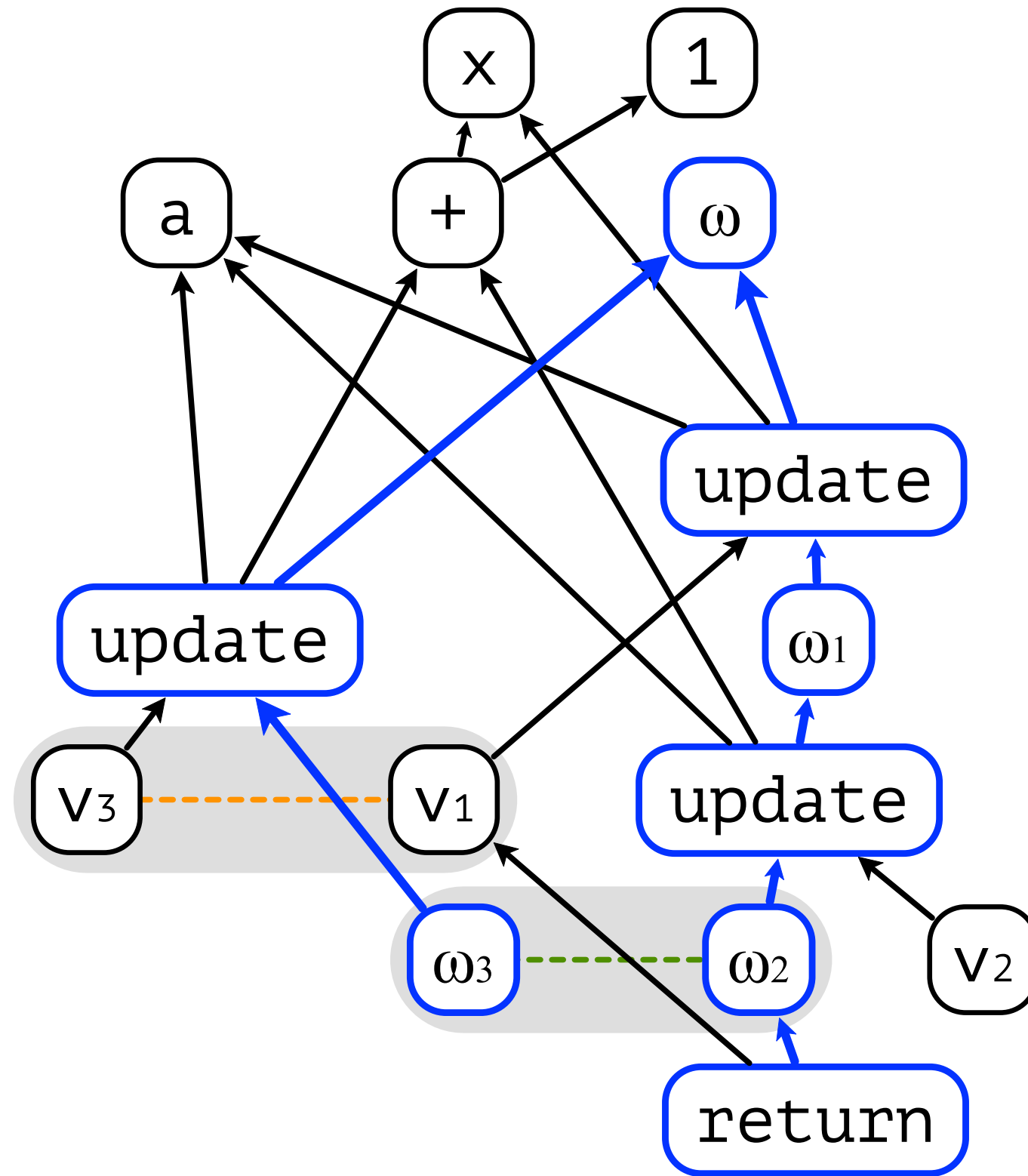


Worst Case E-Graph

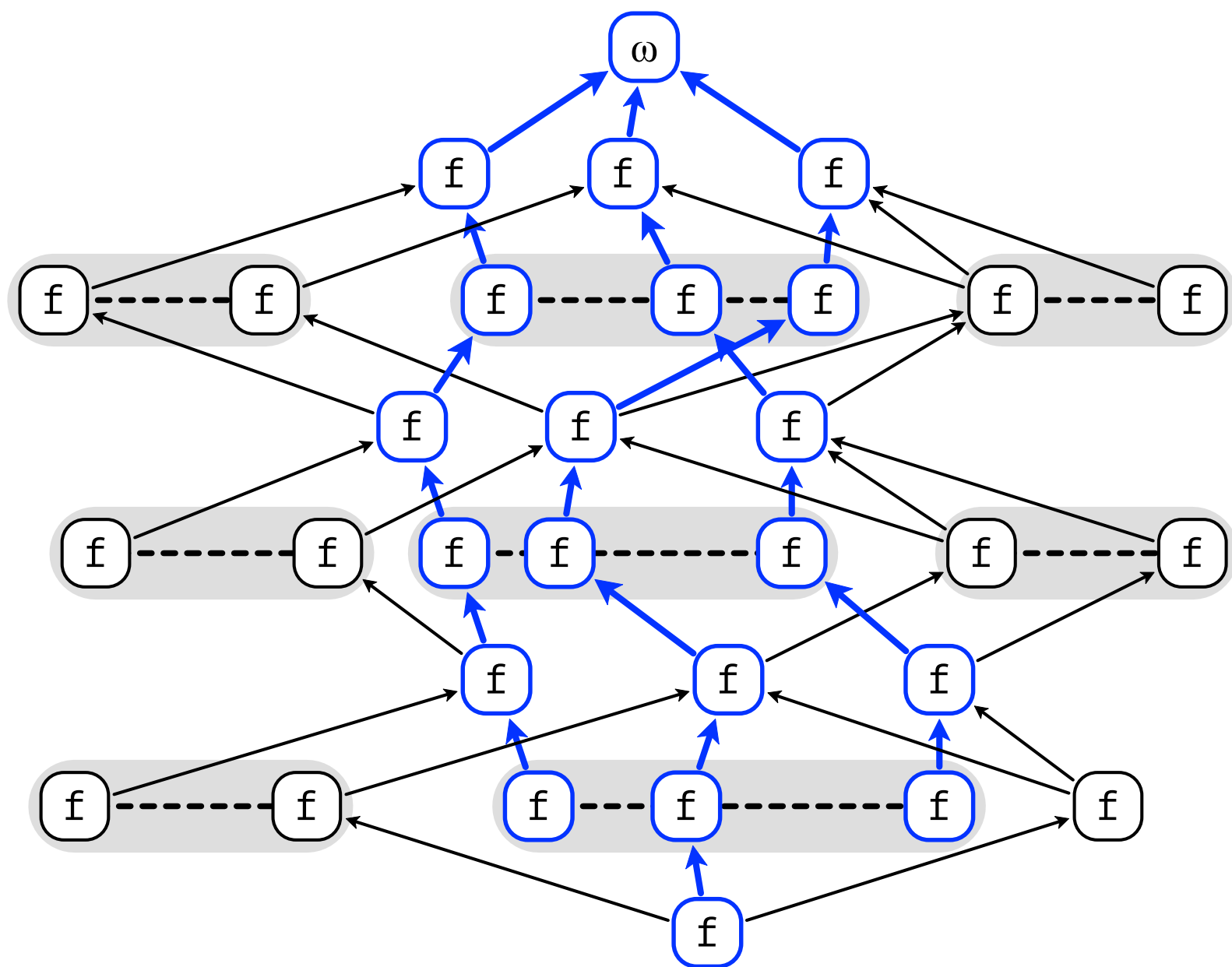
Give Up?



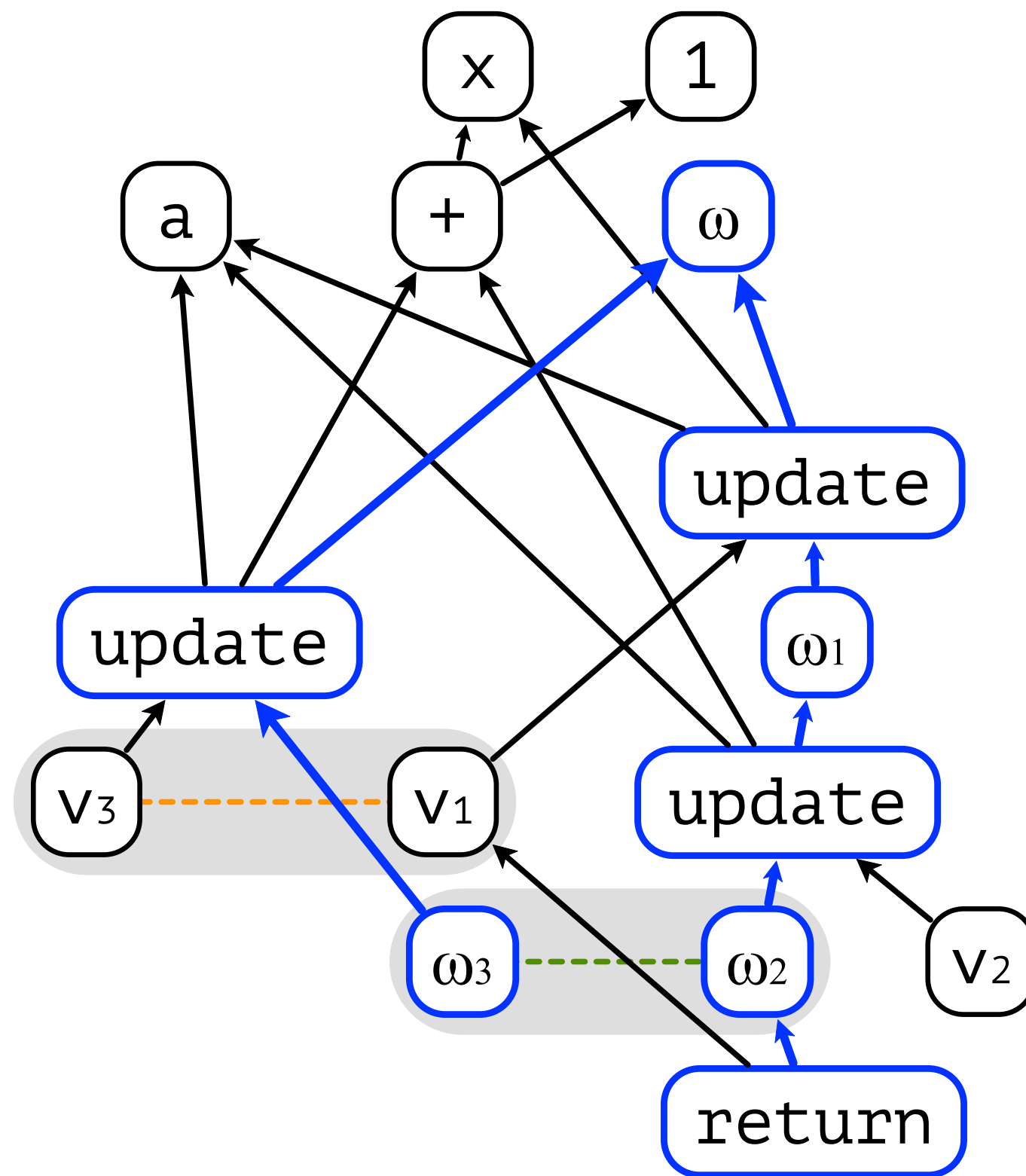
Worst Case E-Graph



Give Up?

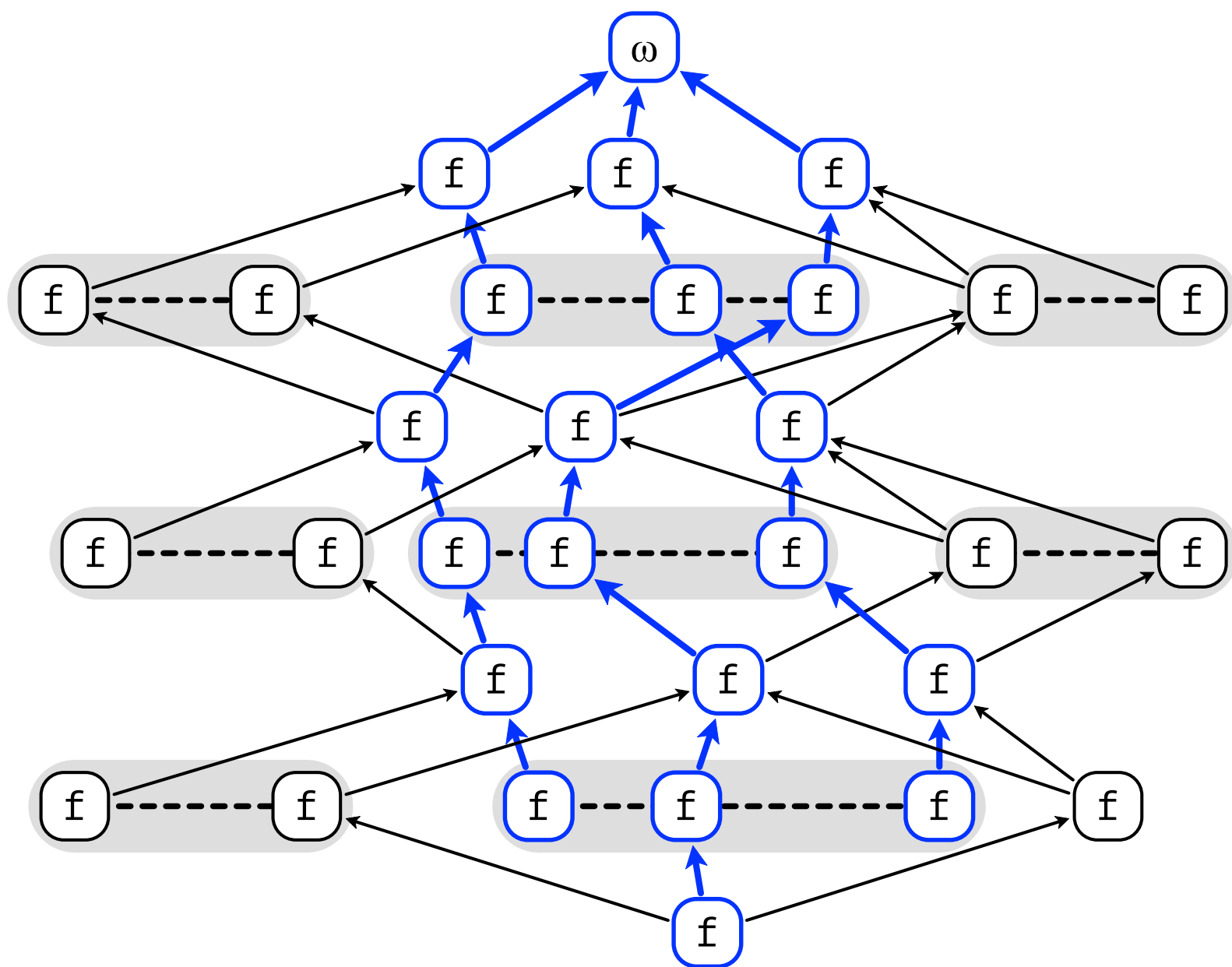


Worst Case E-Graph

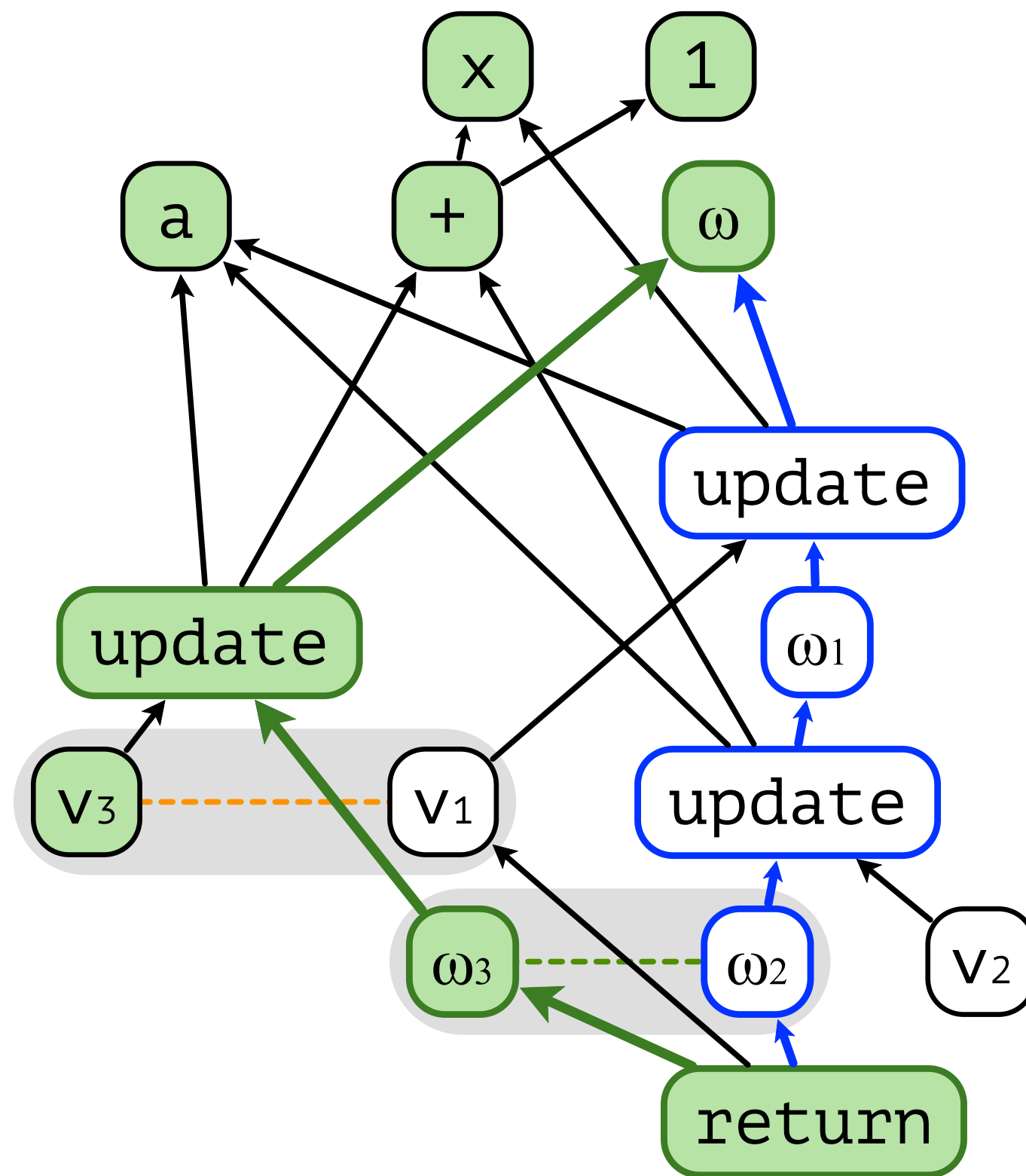


Realistic E-Graph

Give Up?

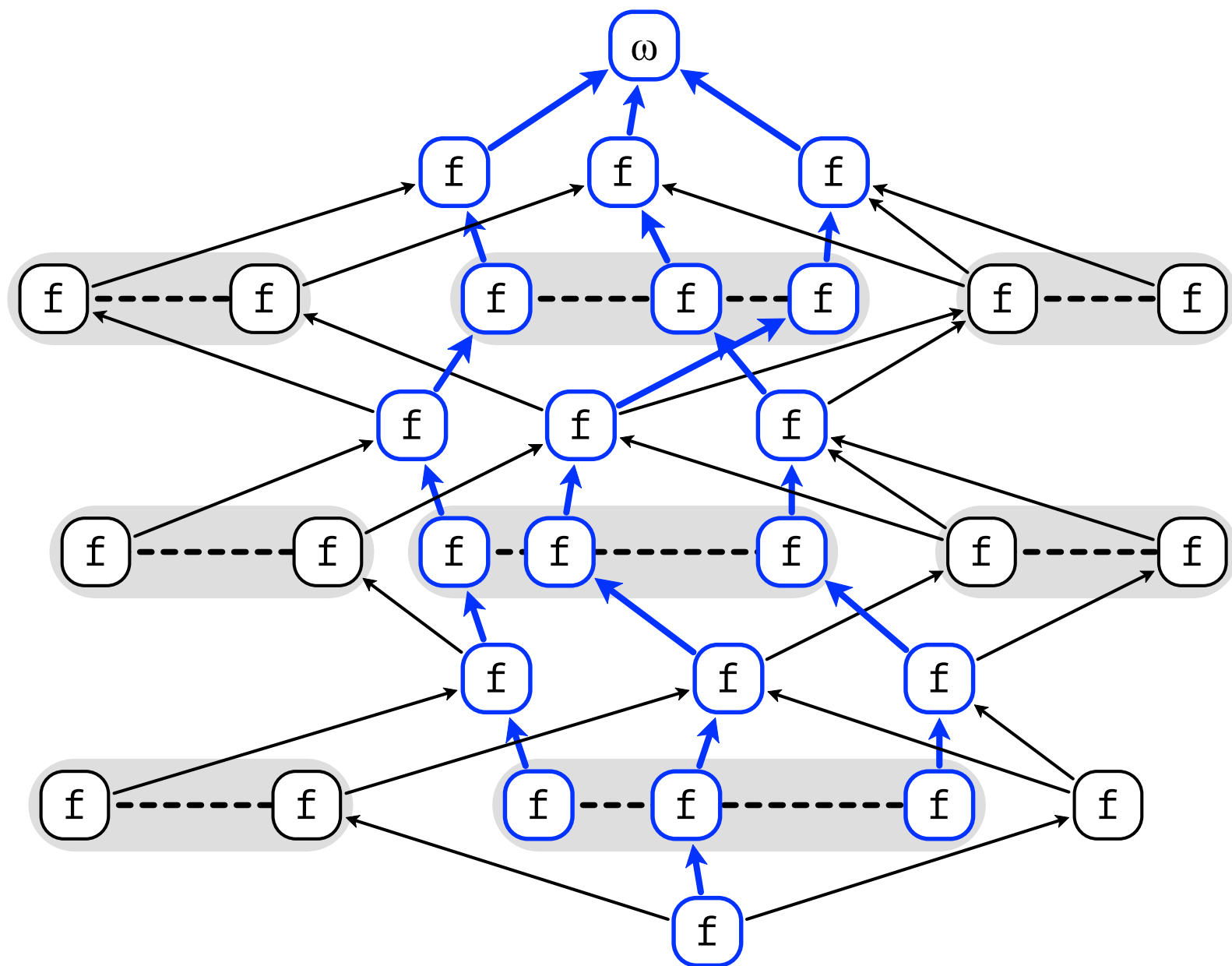


Worst Case E-Graph

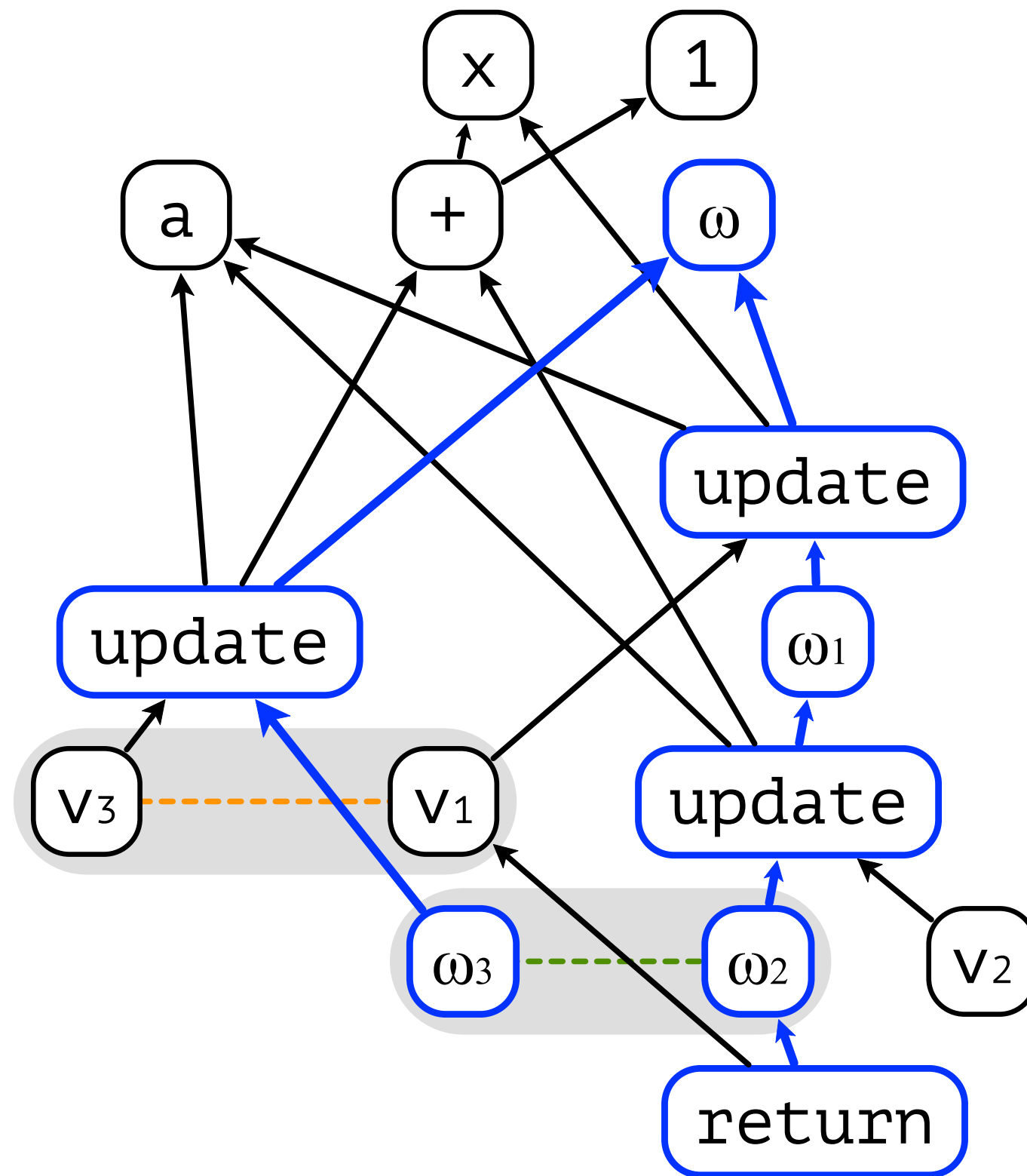


Realistic E-Graph

Give Up?

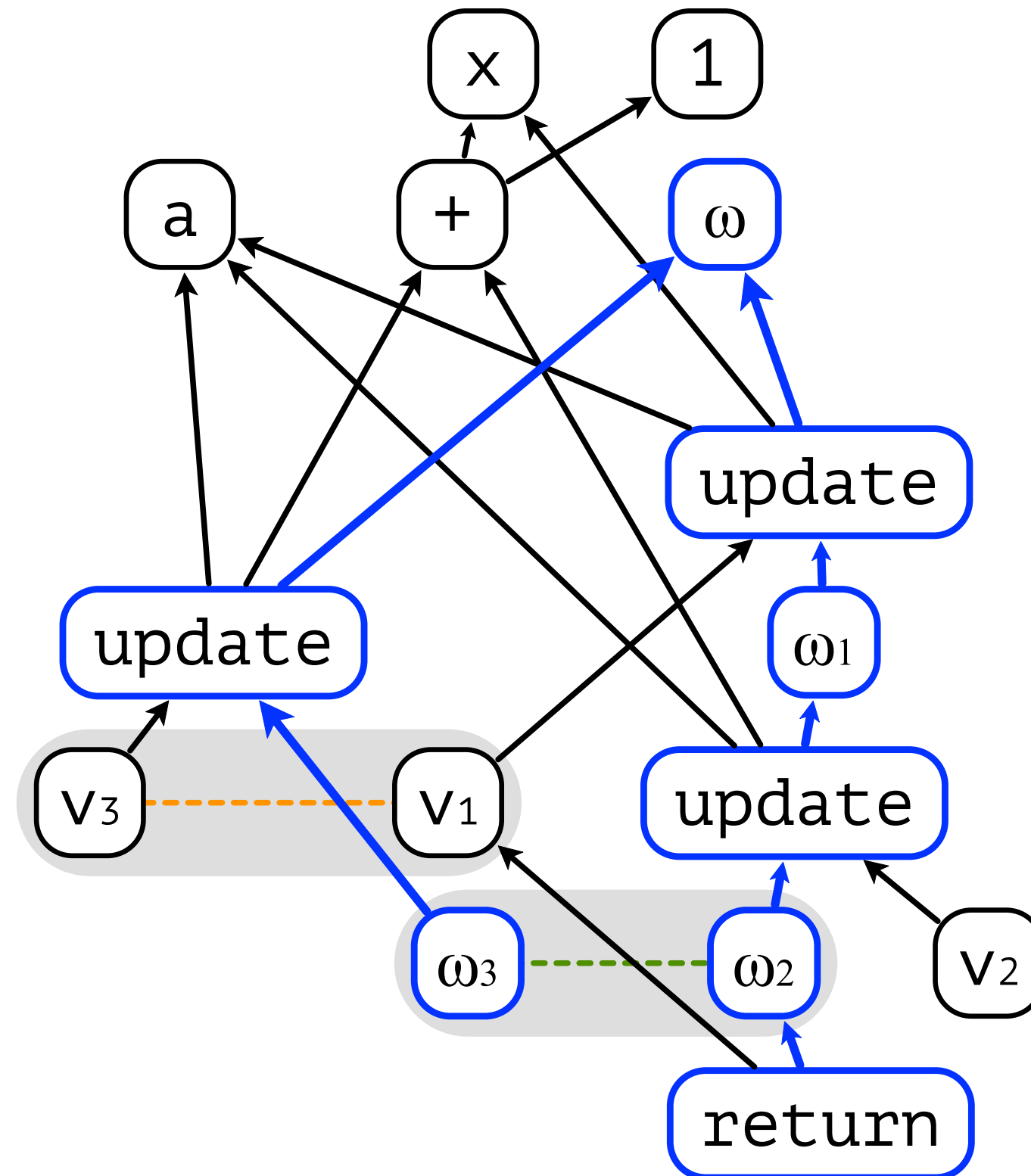


Worst Case E-Graph

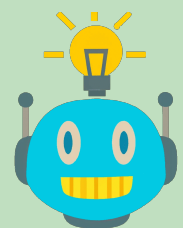


Realistic E-Graph

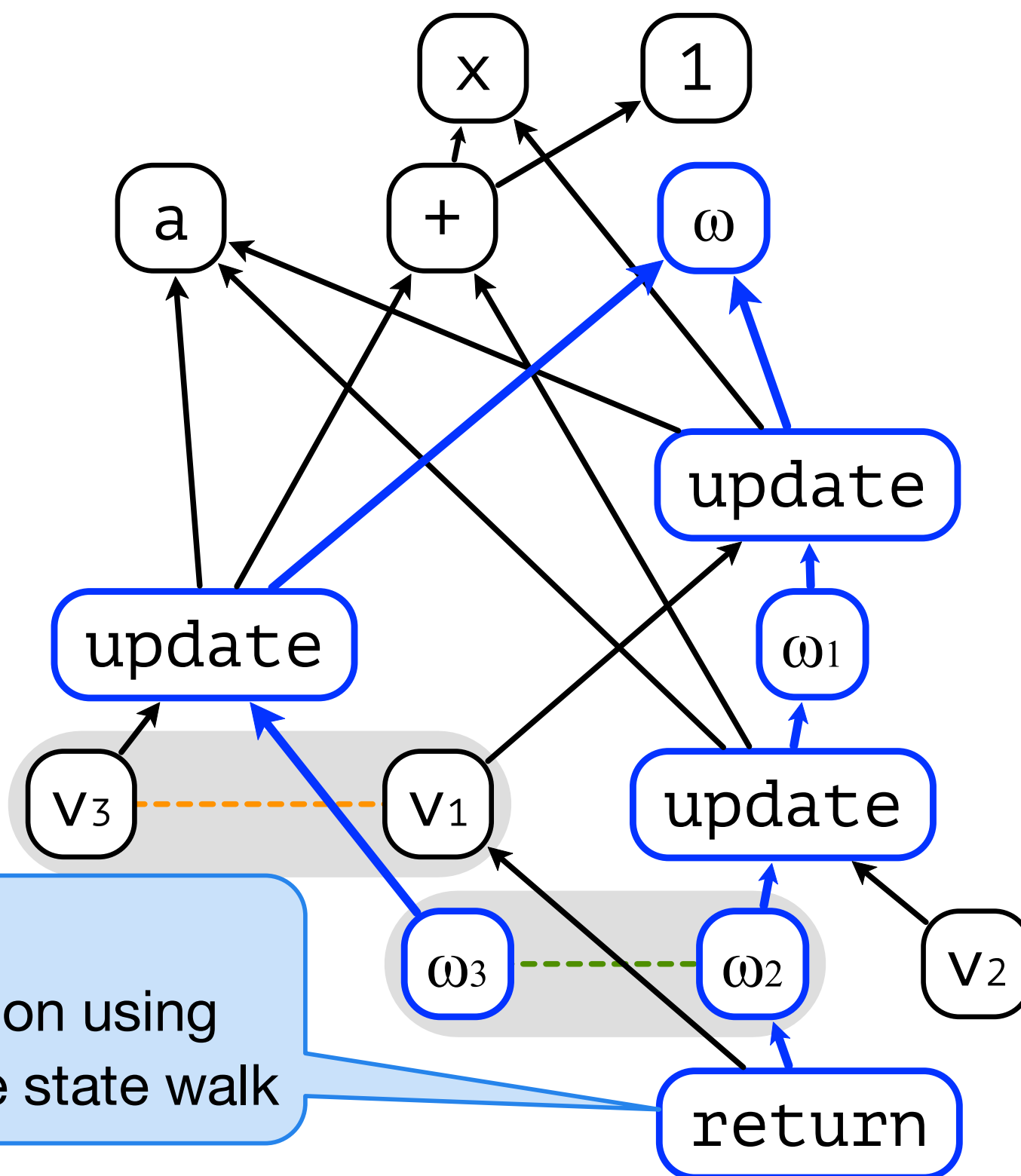
The Statewalk DP Algorithm



The Statewalk DP Algorithm

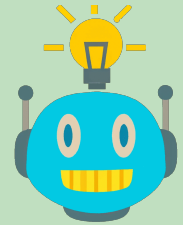


Key idea:
keep **multiple**
statewalks
per e-class

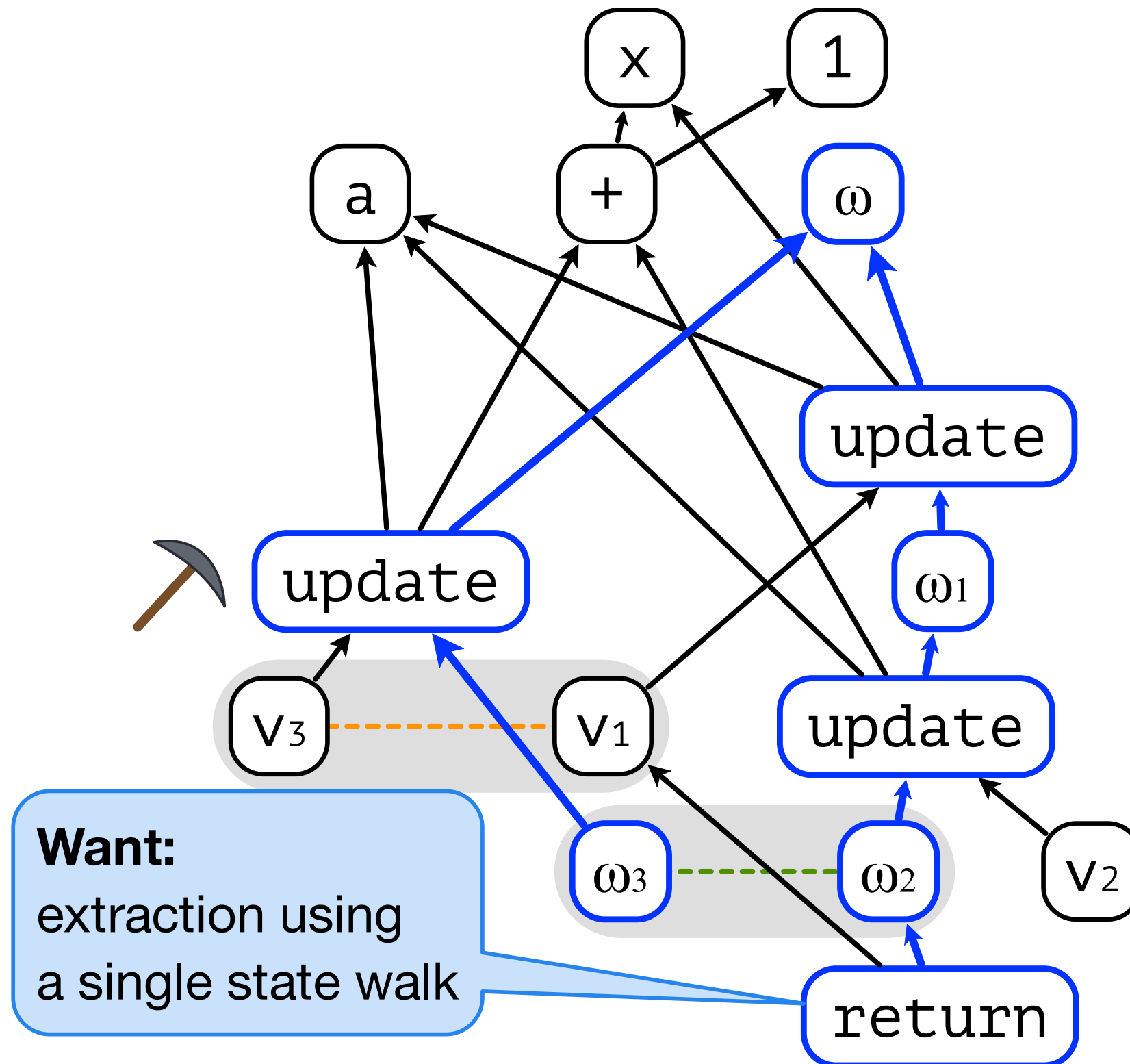


Want:
extraction using
a single state walk

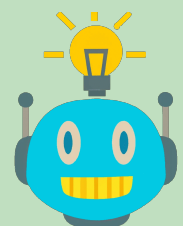
The Statewalk DP Algorithm



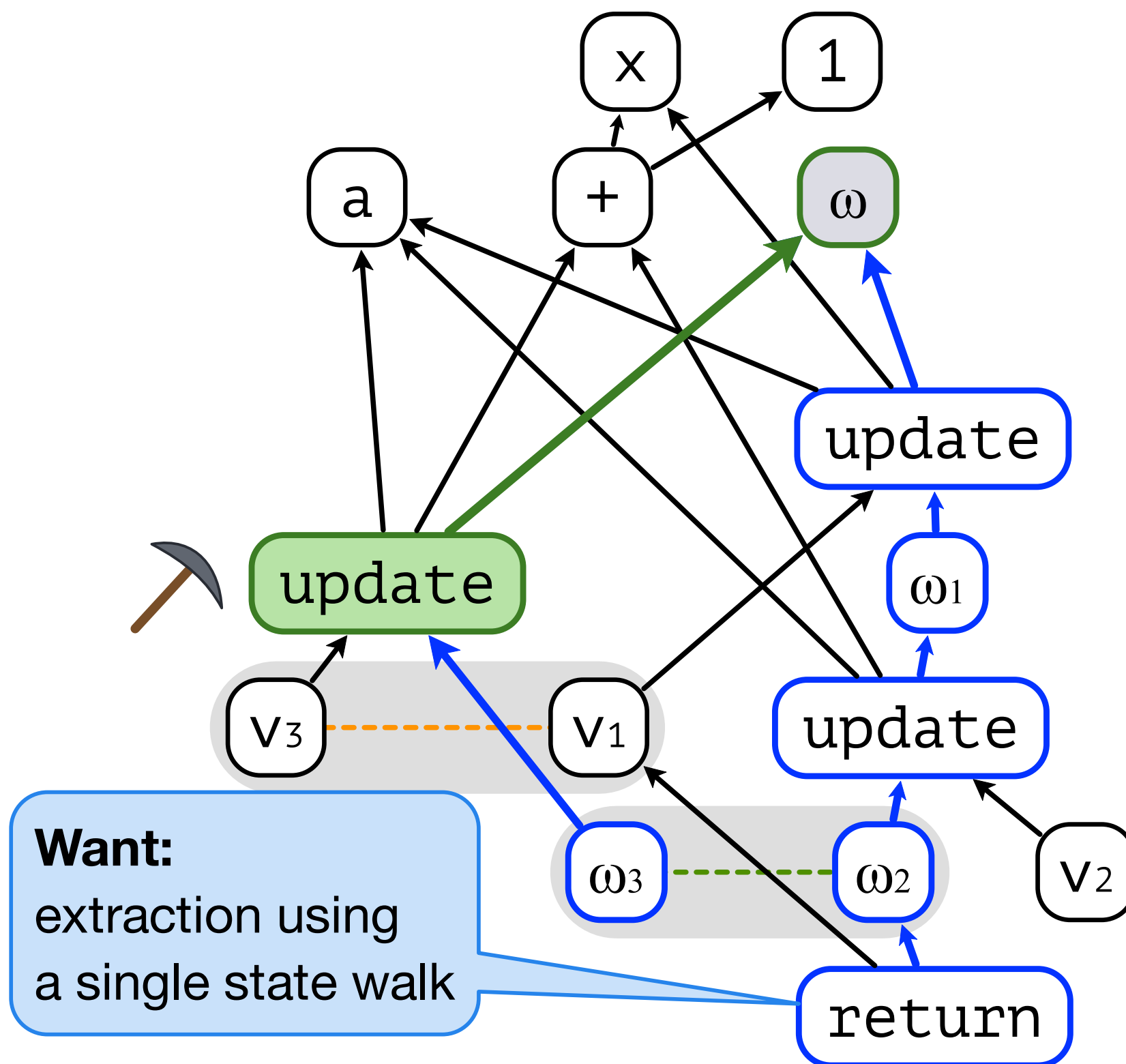
Key idea:
keep **multiple**
statewalks
per e-class



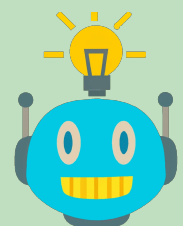
The Statewalk DP Algorithm



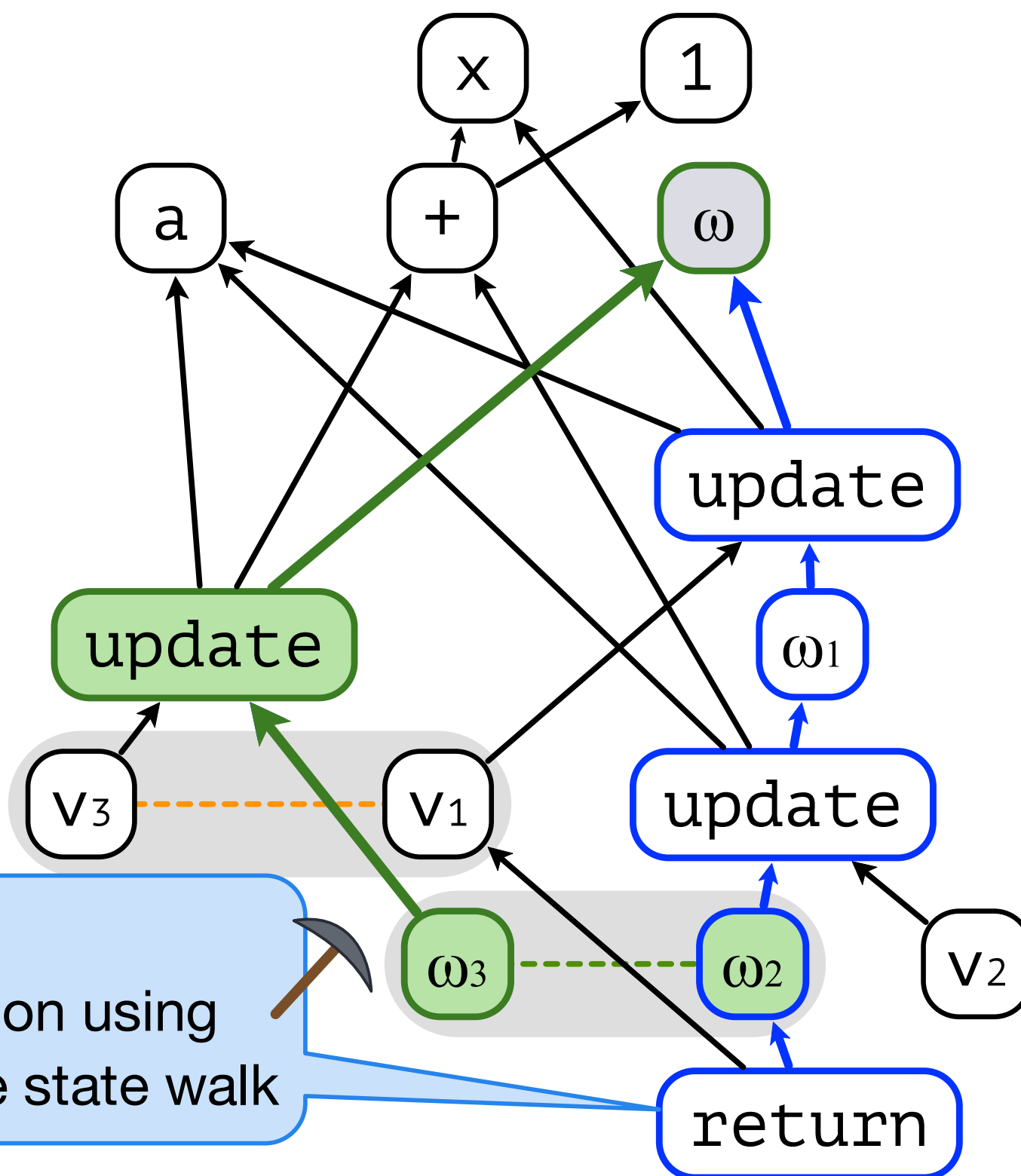
Key idea:
keep **multiple**
statewalks
per e-class



The Statewalk DP Algorithm

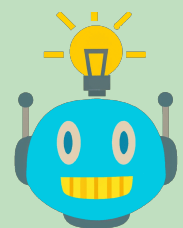


Key idea:
keep **multiple**
statewalks
per e-class

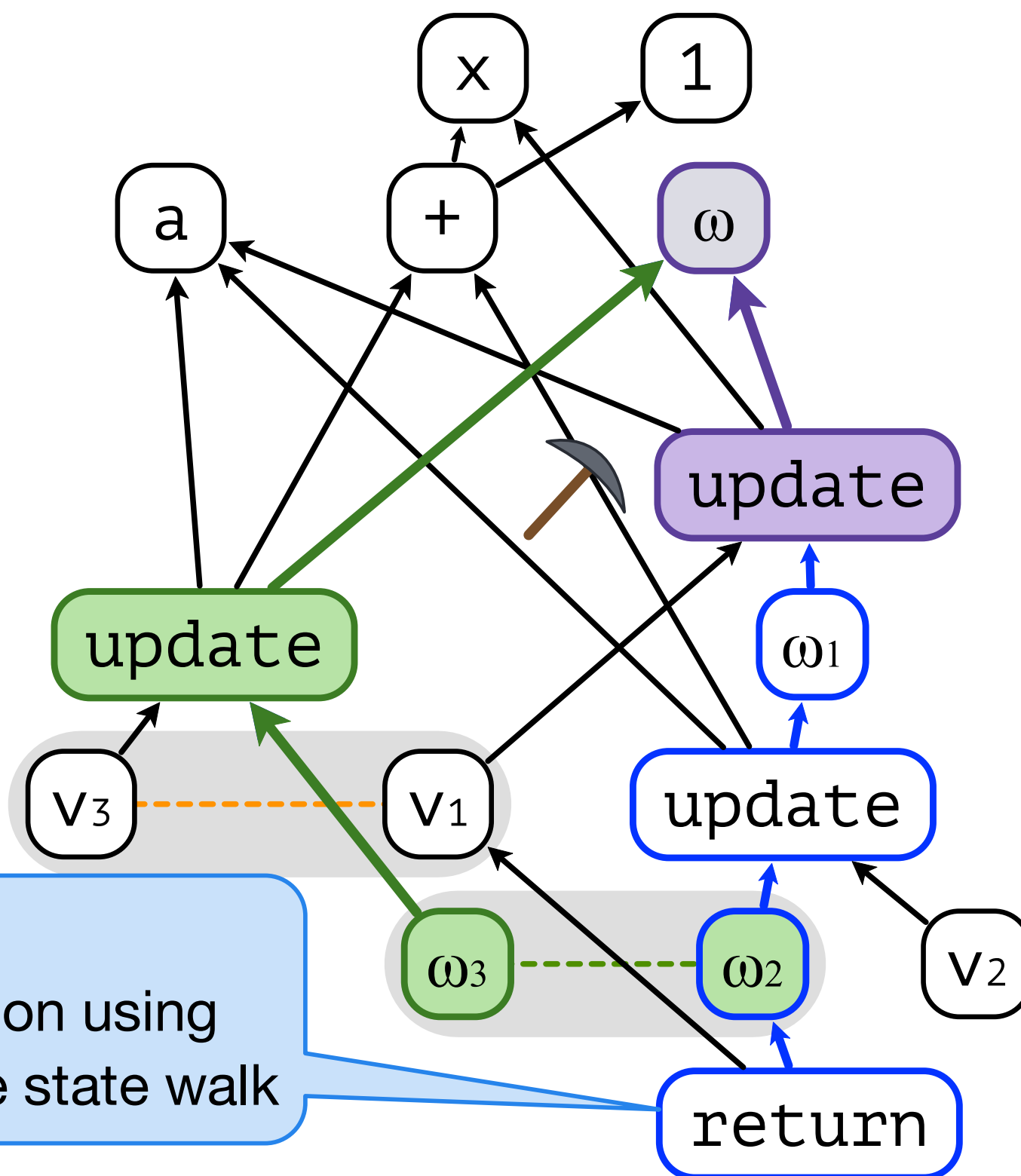


Want:
extraction using
a single state walk

The Statewalk DP Algorithm

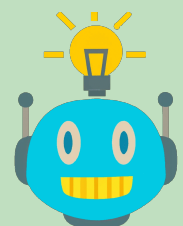


Key idea:
keep **multiple**
statewalks
per e-class

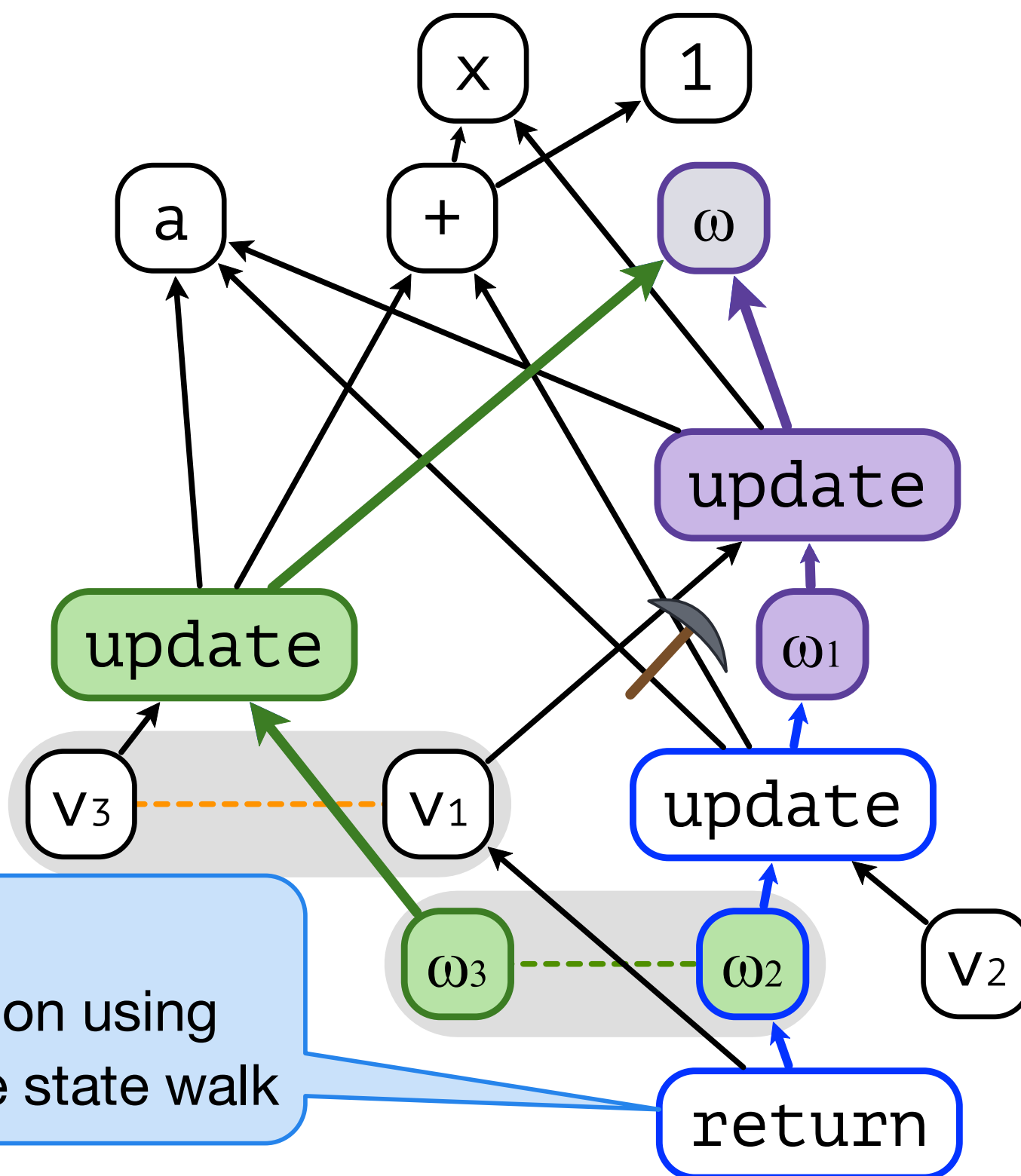


Want:
extraction using
a single state walk

The Statewalk DP Algorithm

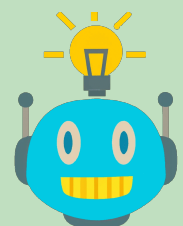


Key idea:
keep **multiple**
statewalks
per e-class

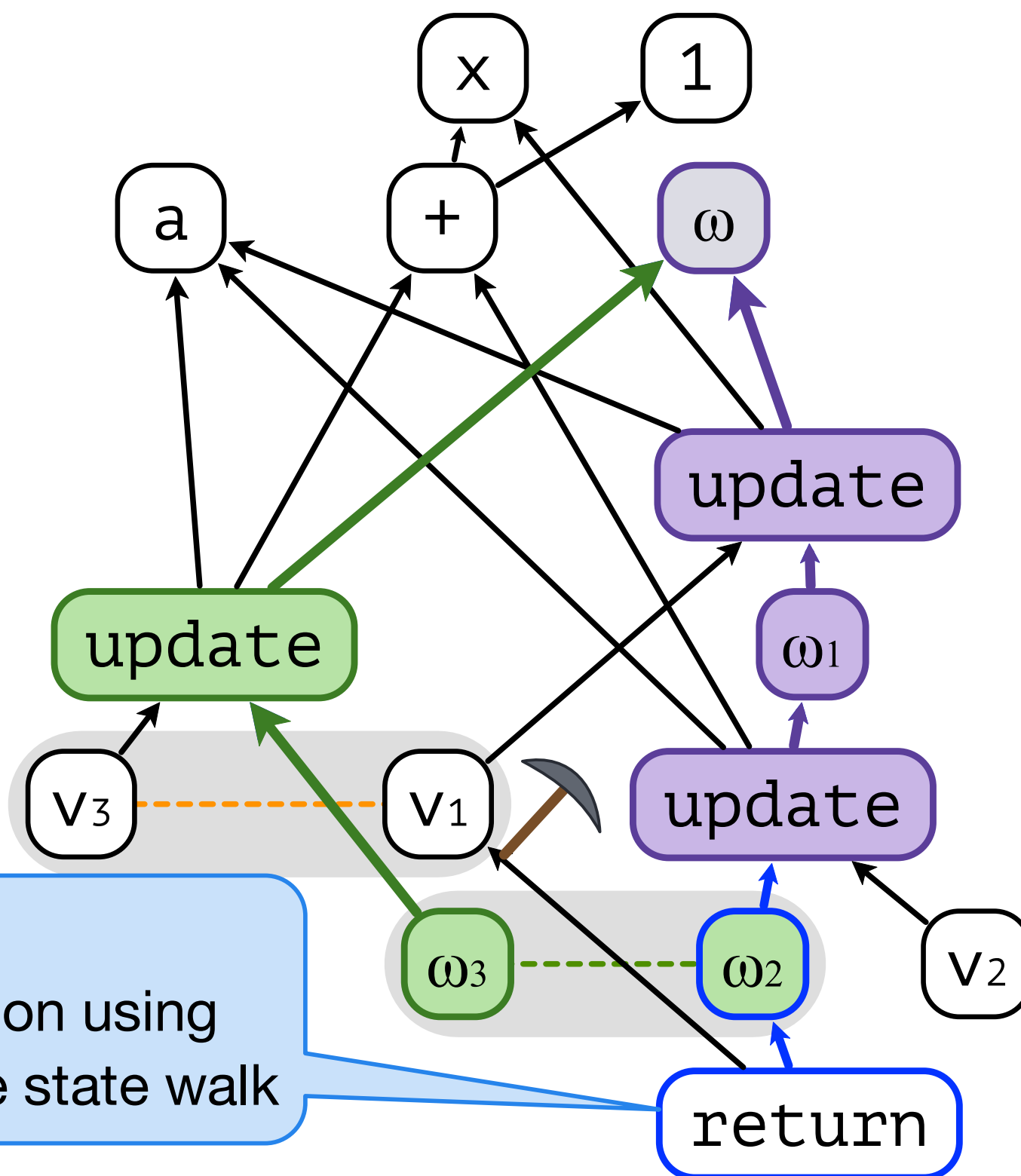


Want:
extraction using
a single state walk

The Statewalk DP Algorithm

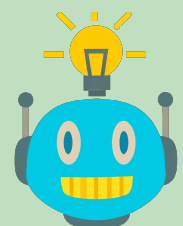


Key idea:
keep **multiple**
statewalks
per e-class

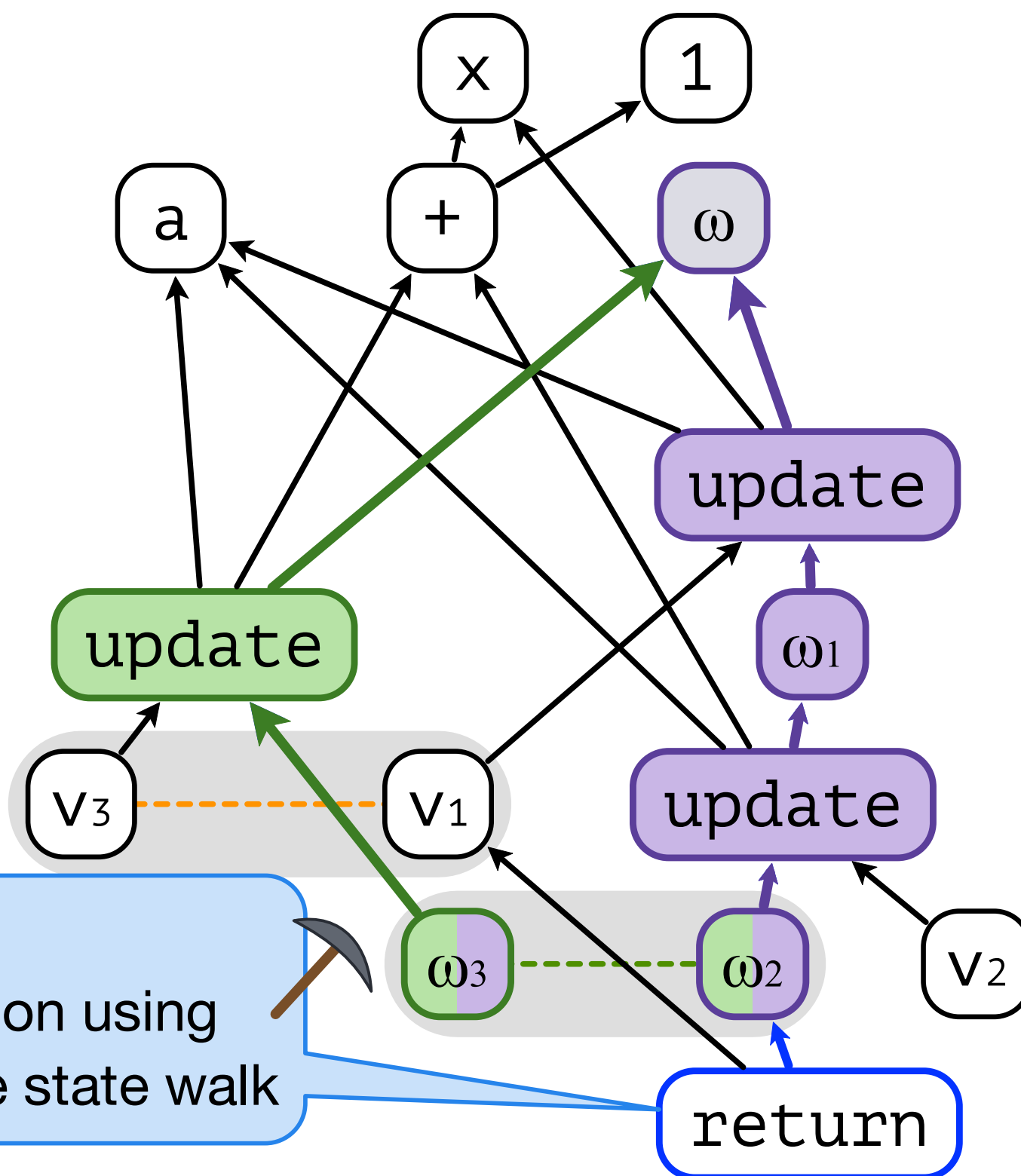


Want:
extraction using
a single state walk

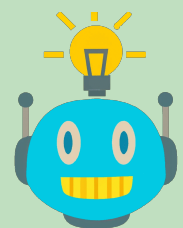
The Statewalk DP Algorithm



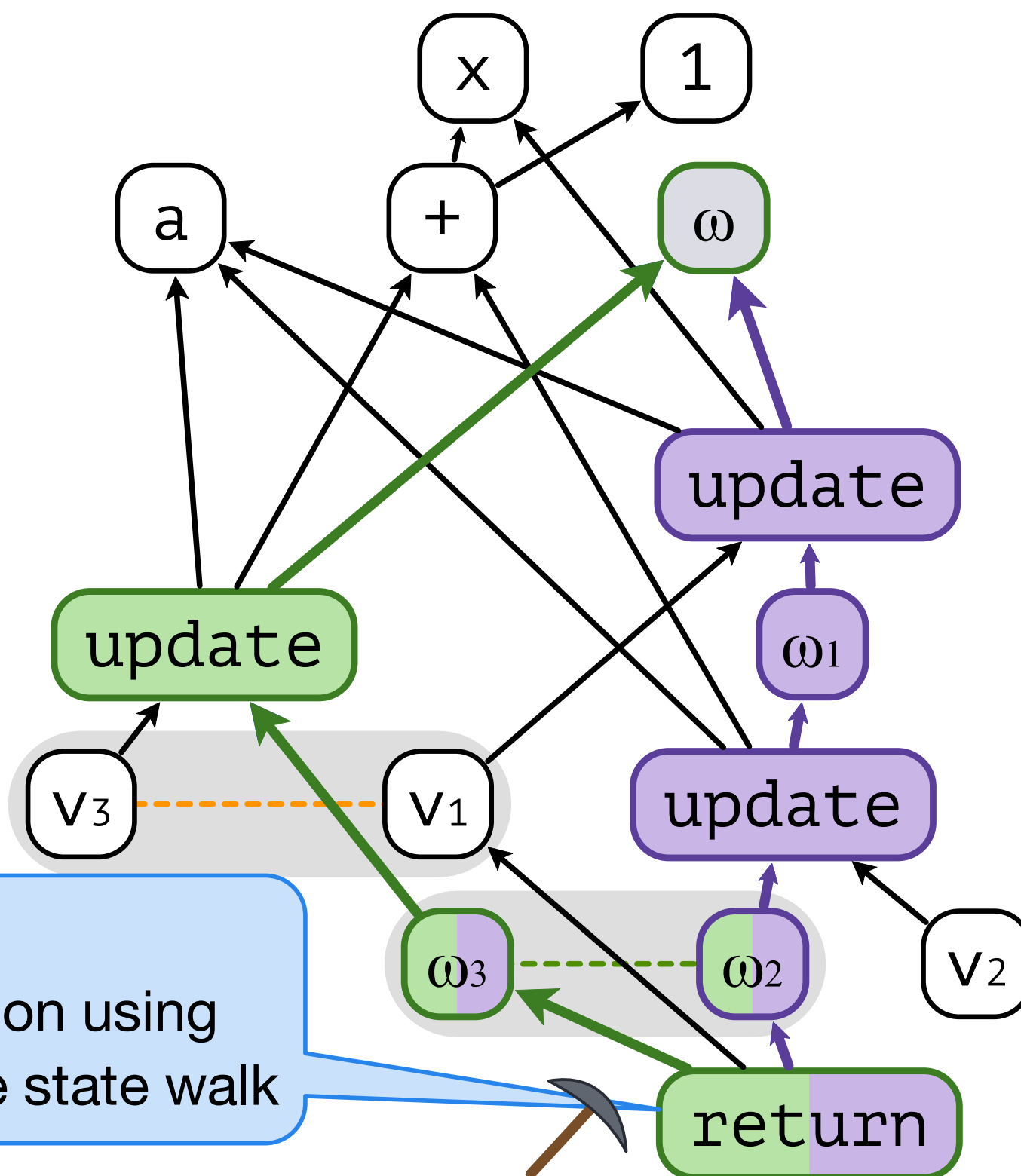
Key idea:
keep **multiple**
statewalks
per e-class



The Statewalk DP Algorithm

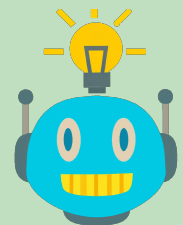


Key idea:
keep **multiple**
statewalks
per e-class

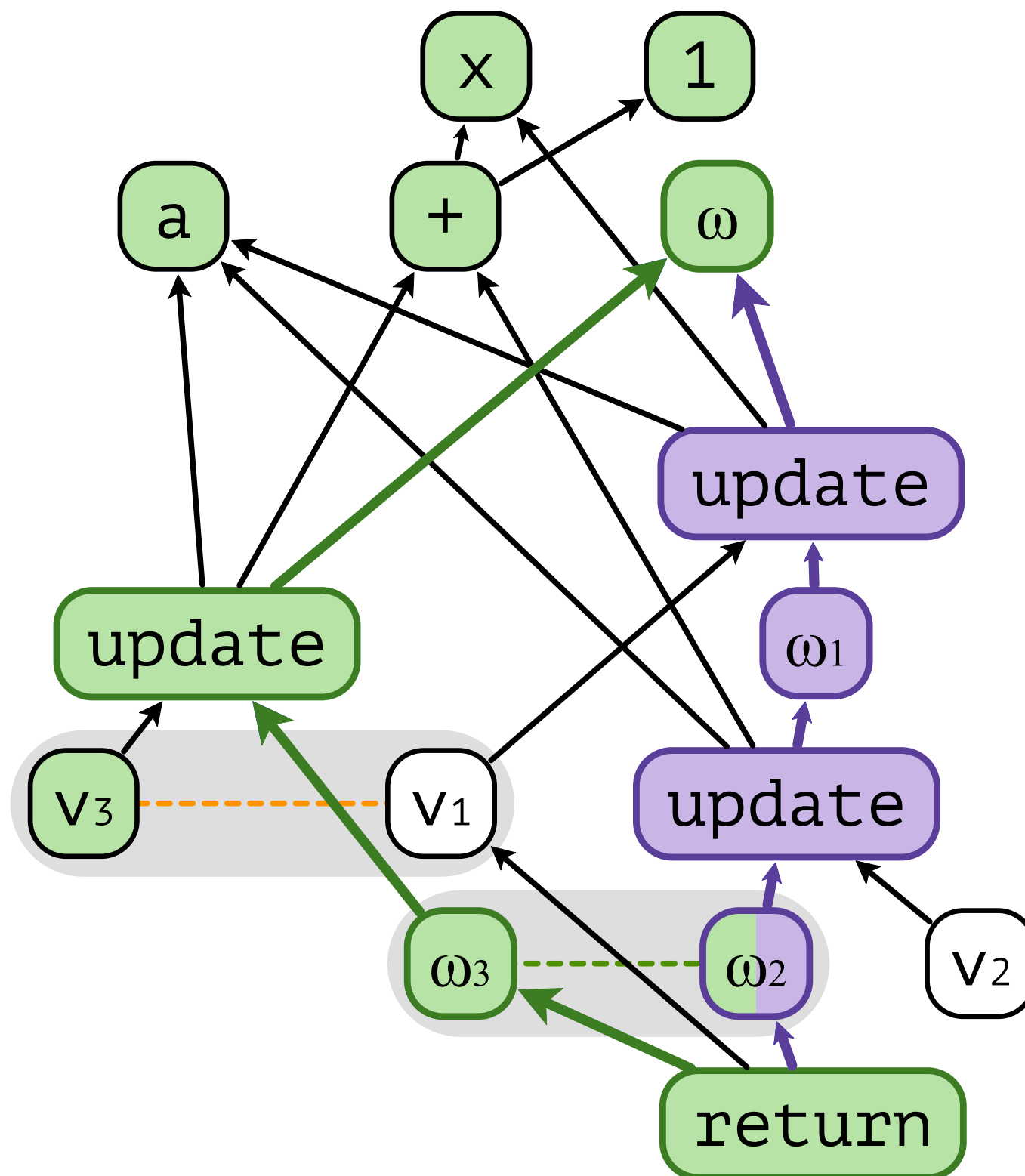


Want:
extraction using
a single state walk

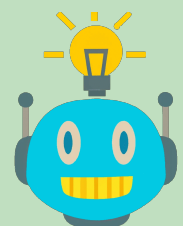
The Statewalk DP Algorithm



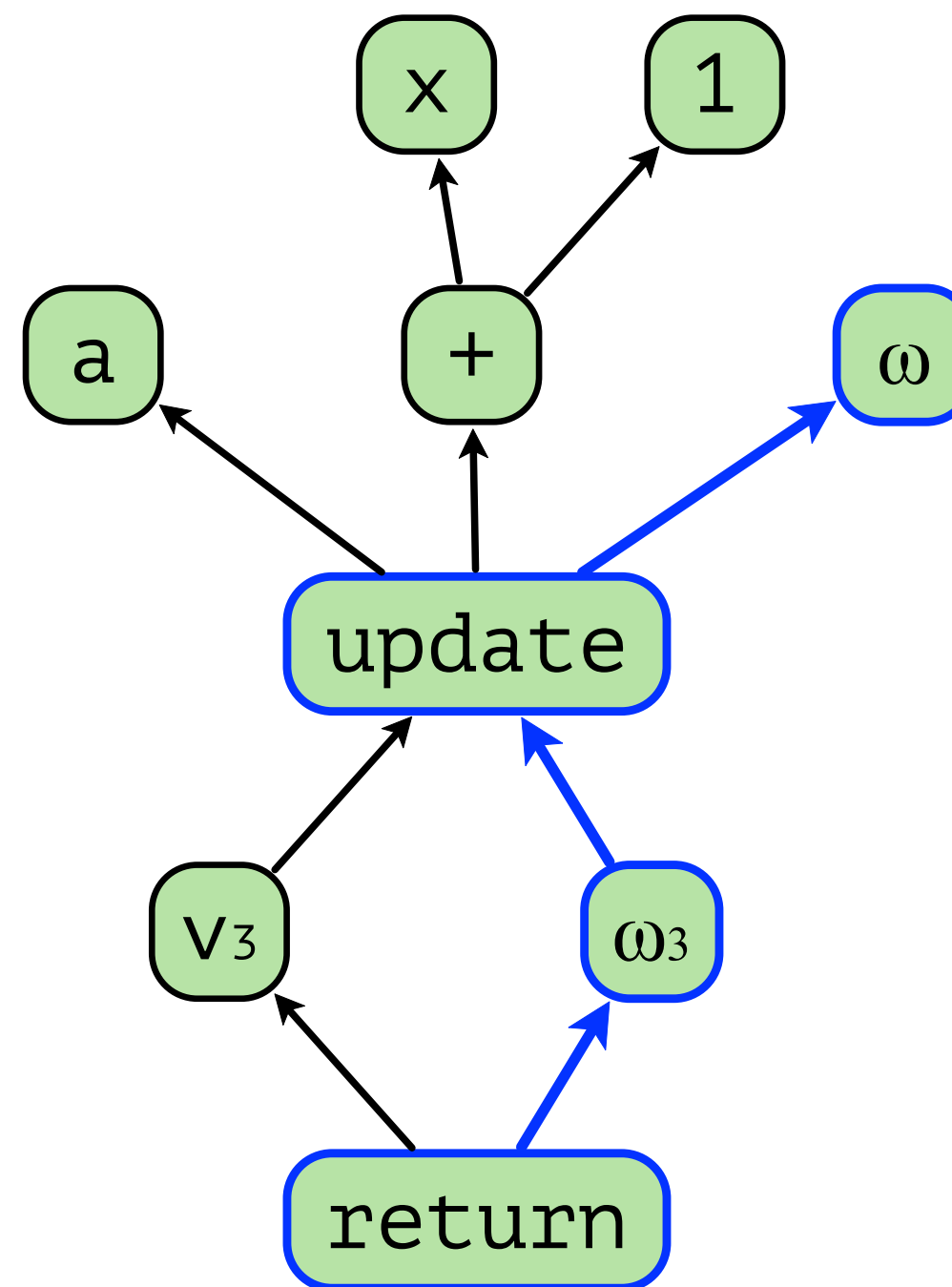
Key idea:
keep **multiple**
statewalks
per e-class



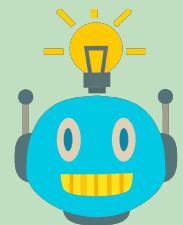
The Statewalk DP Algorithm



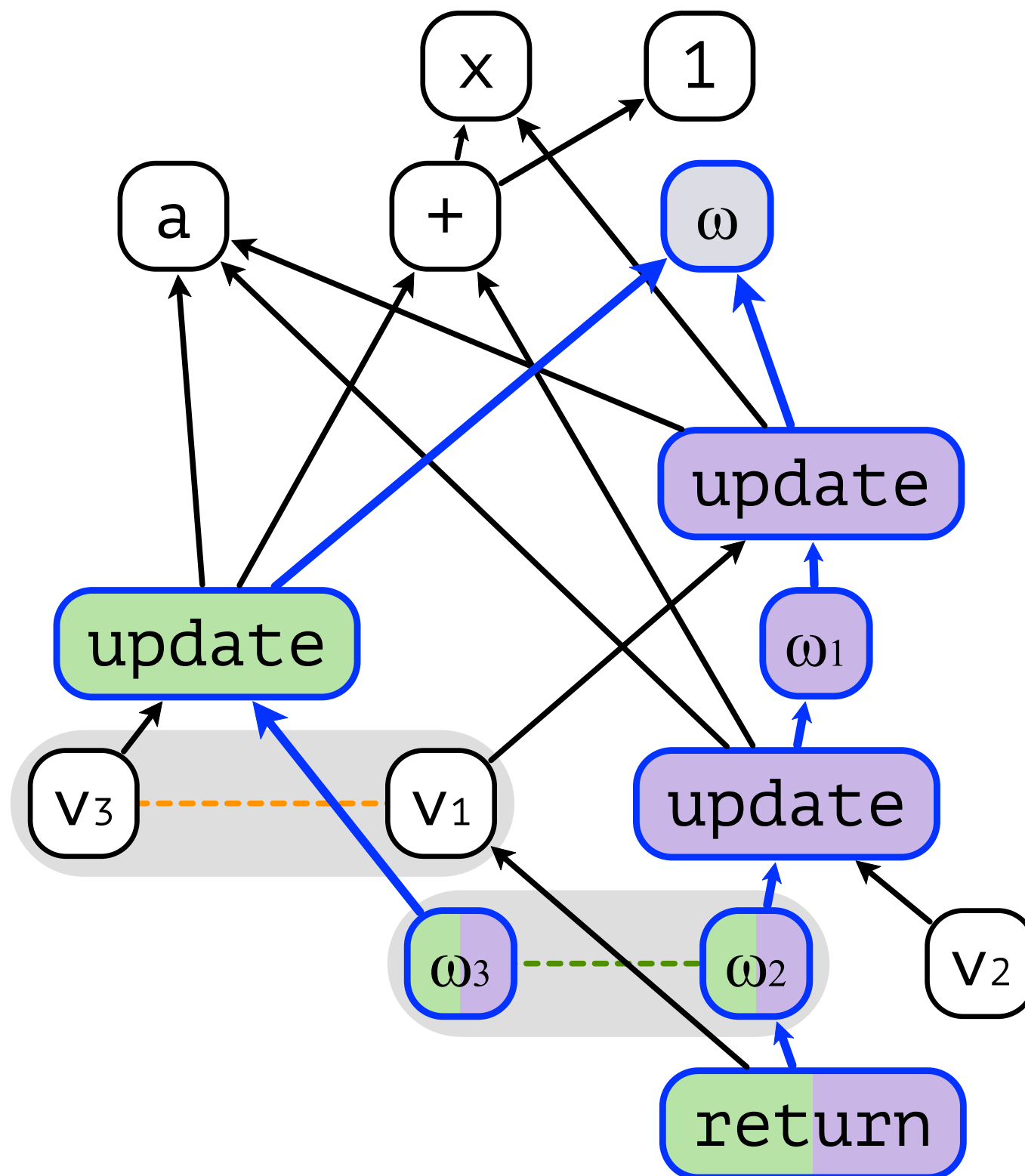
Key idea:
keep **multiple**
statewalks
per e-class



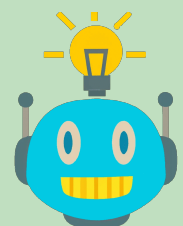
The Statewalk DP Algorithm



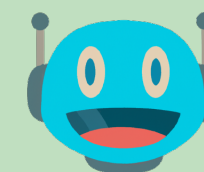
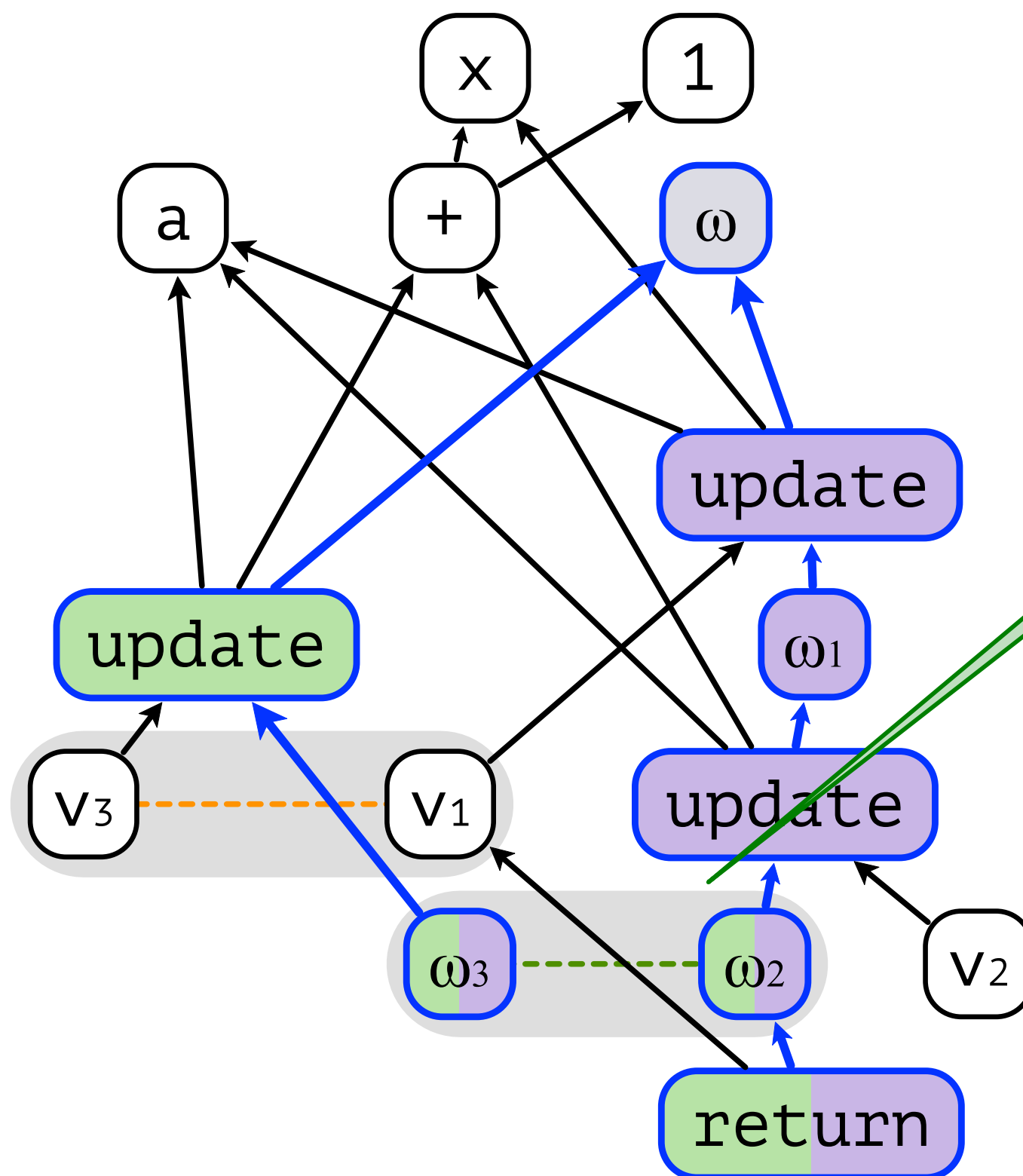
Key idea:
keep **multiple**
statewalks
per e-class



The Statewalk DP Algorithm

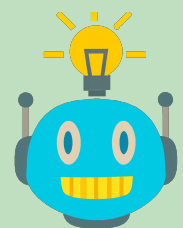


Key idea:
keep **multiple**
statewalks
per e-class

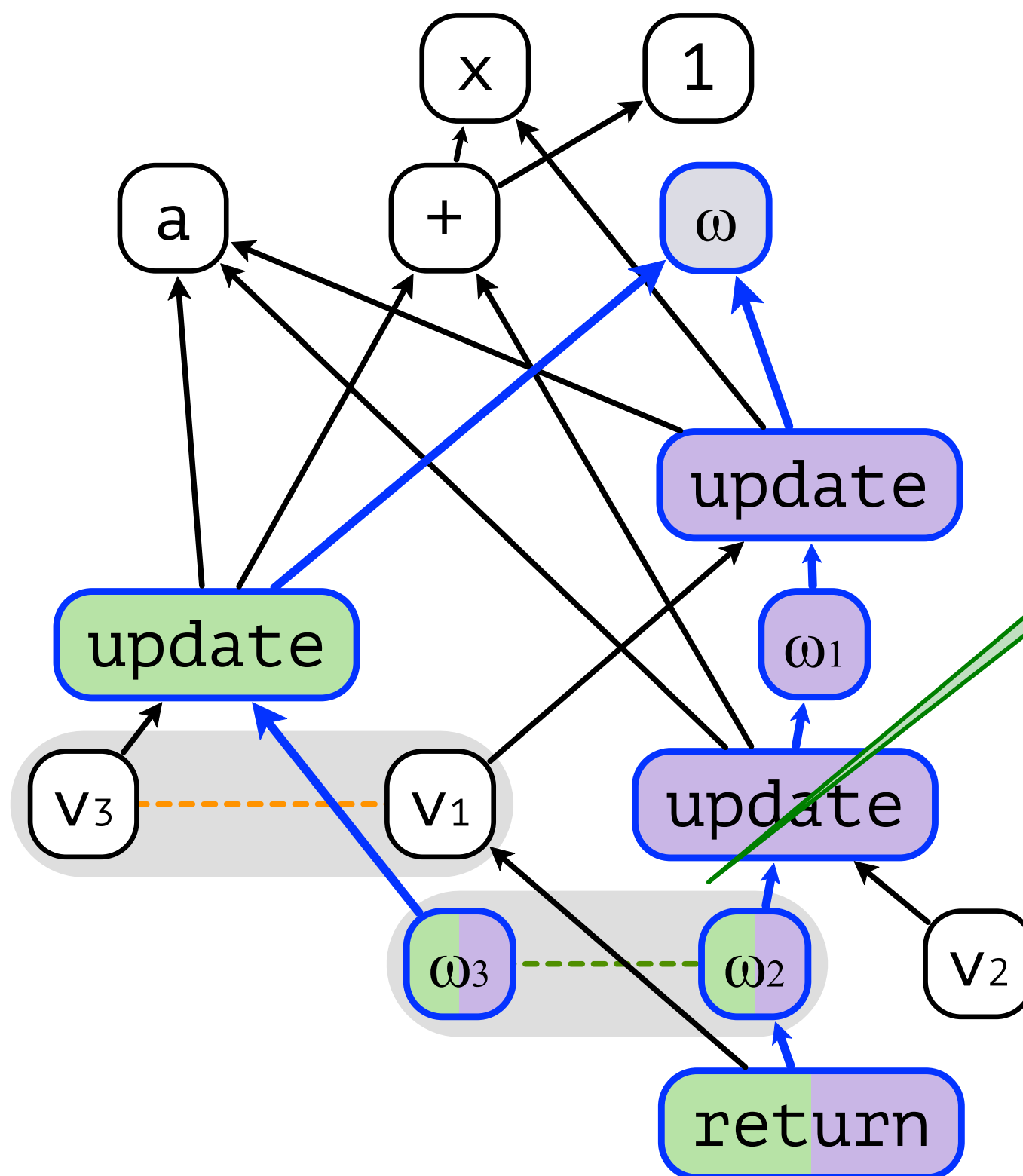


Good news:
time bounded by
number of statewalks!

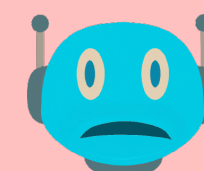
The Statewalk DP Algorithm



Key idea:
keep **multiple**
statewalks
per e-class

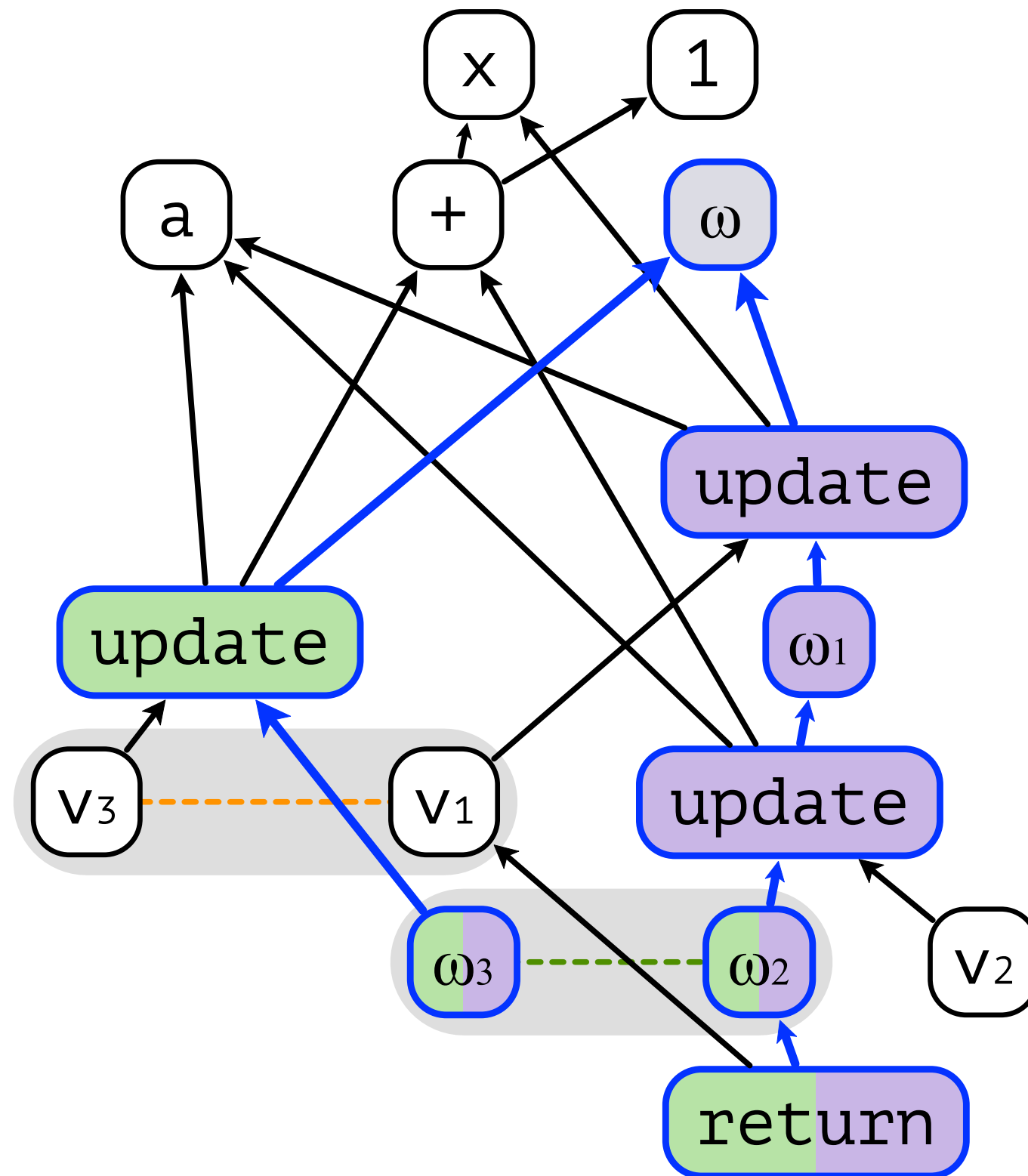


Good news:
time bounded by
number of statewalks!



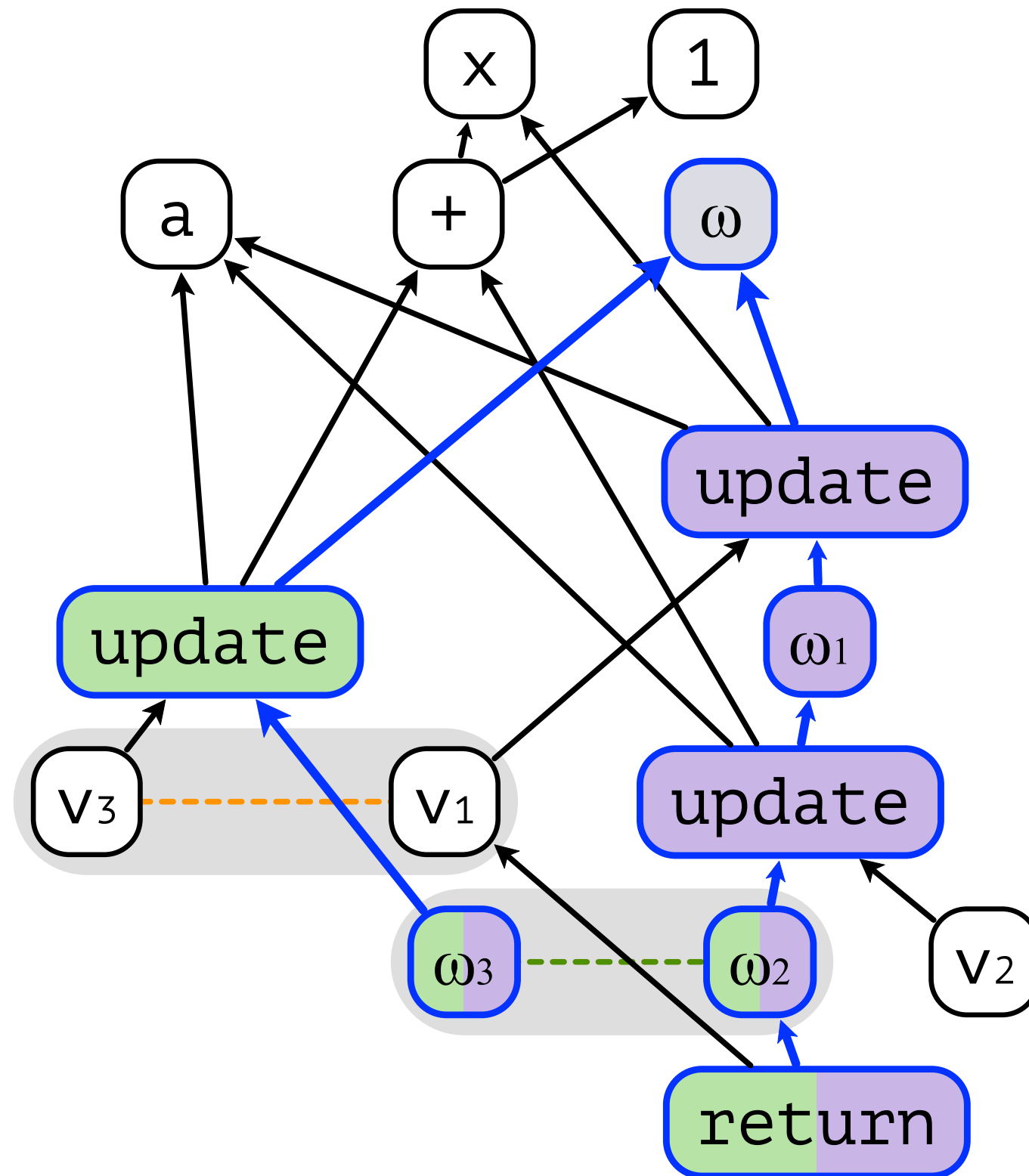
Bad news:
exponential number
of statewalks

Collapsing Statewalks



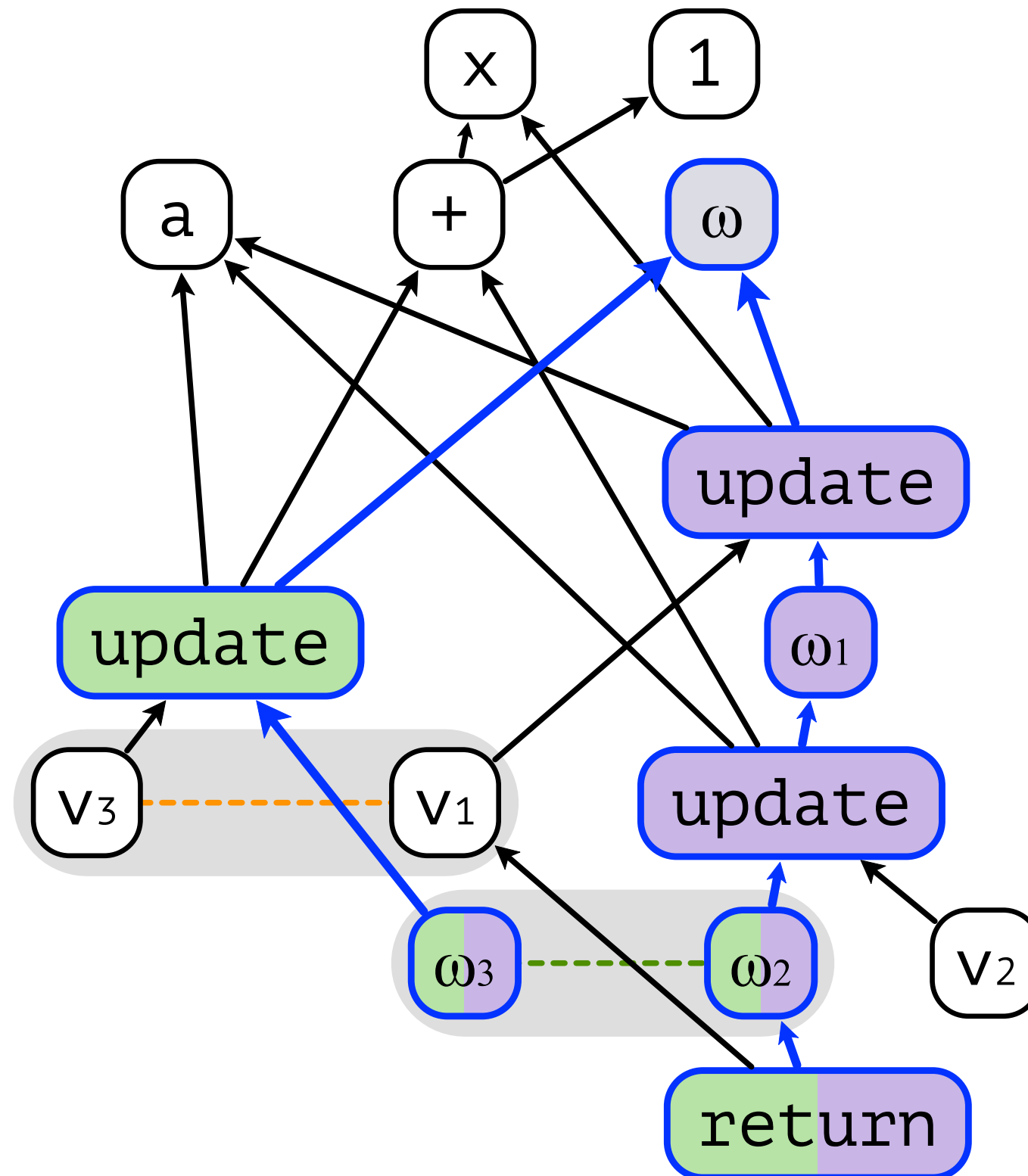
Collapsing Statewalks

Optimization:
collapse by unique
extractable set



Collapsing Statewalks

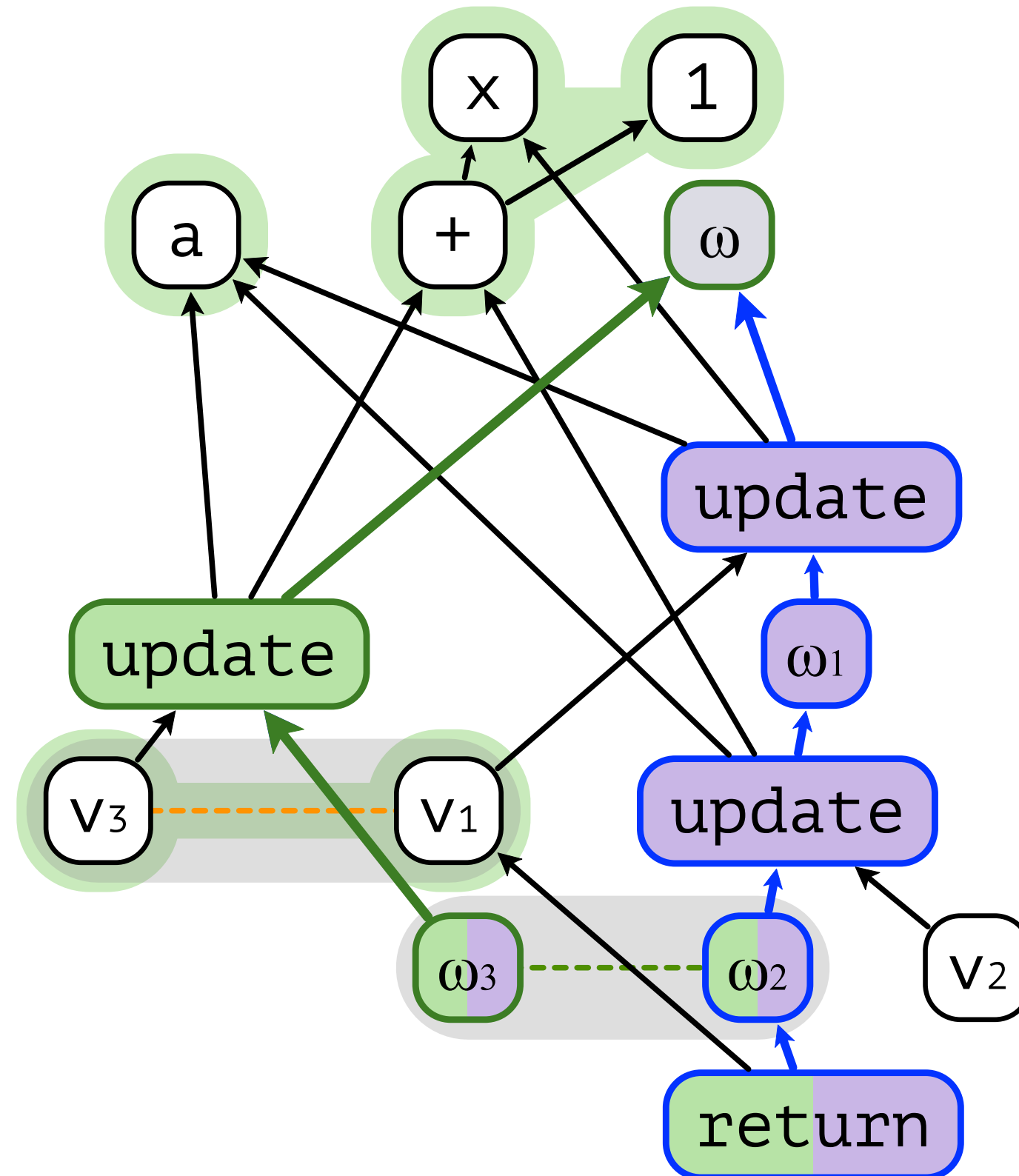
Optimization:
collapse by unique
extractable set



extractable set:
e-classes we could extract
using a given **state walk**

Collapsing Statewalks

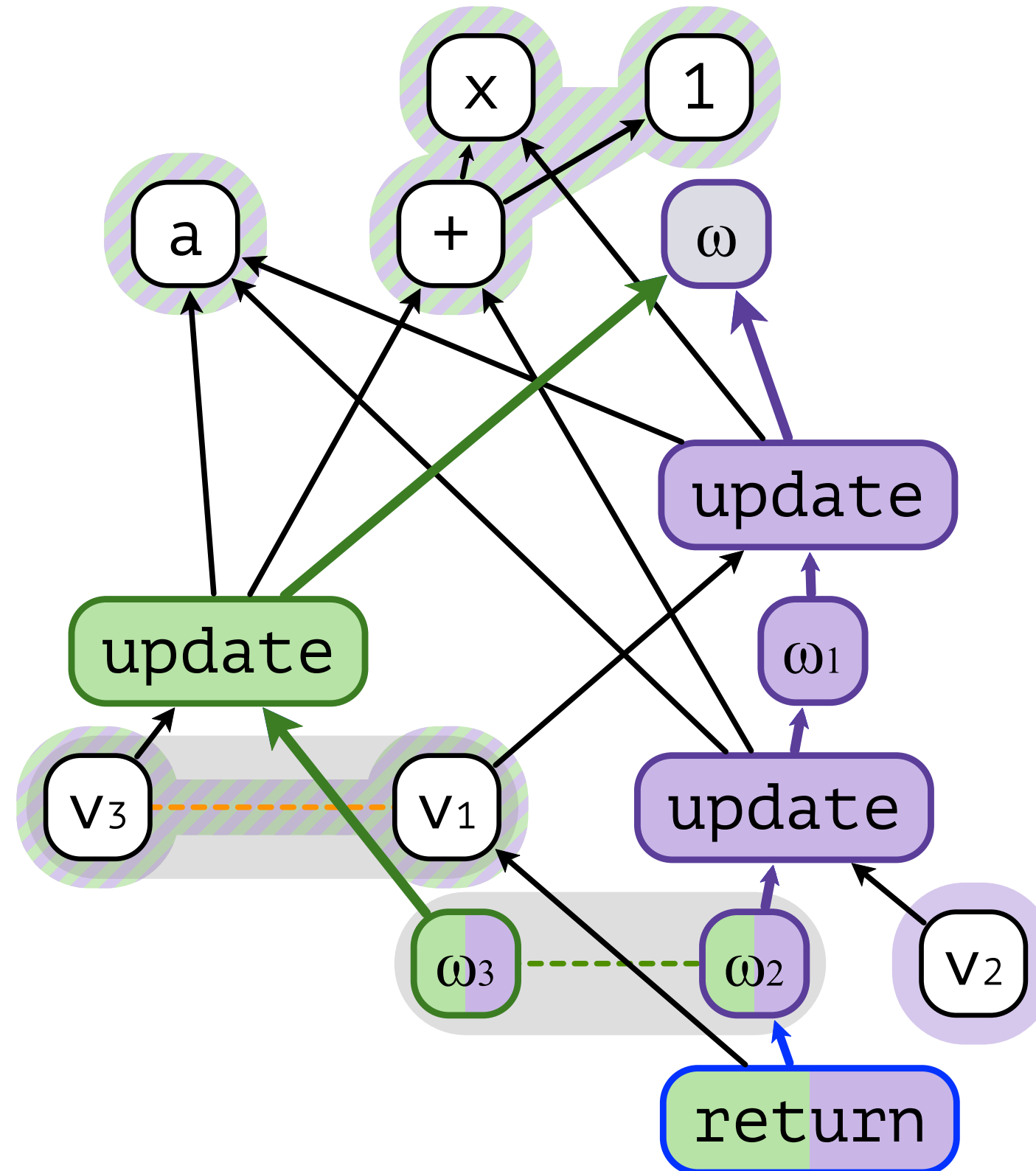
Optimization:
collapse by unique
extractable set



extractable set:
e-classes we could extract
using a given **state walk**

Collapsing Statewalks

Optimization:
collapse by unique
extractable set



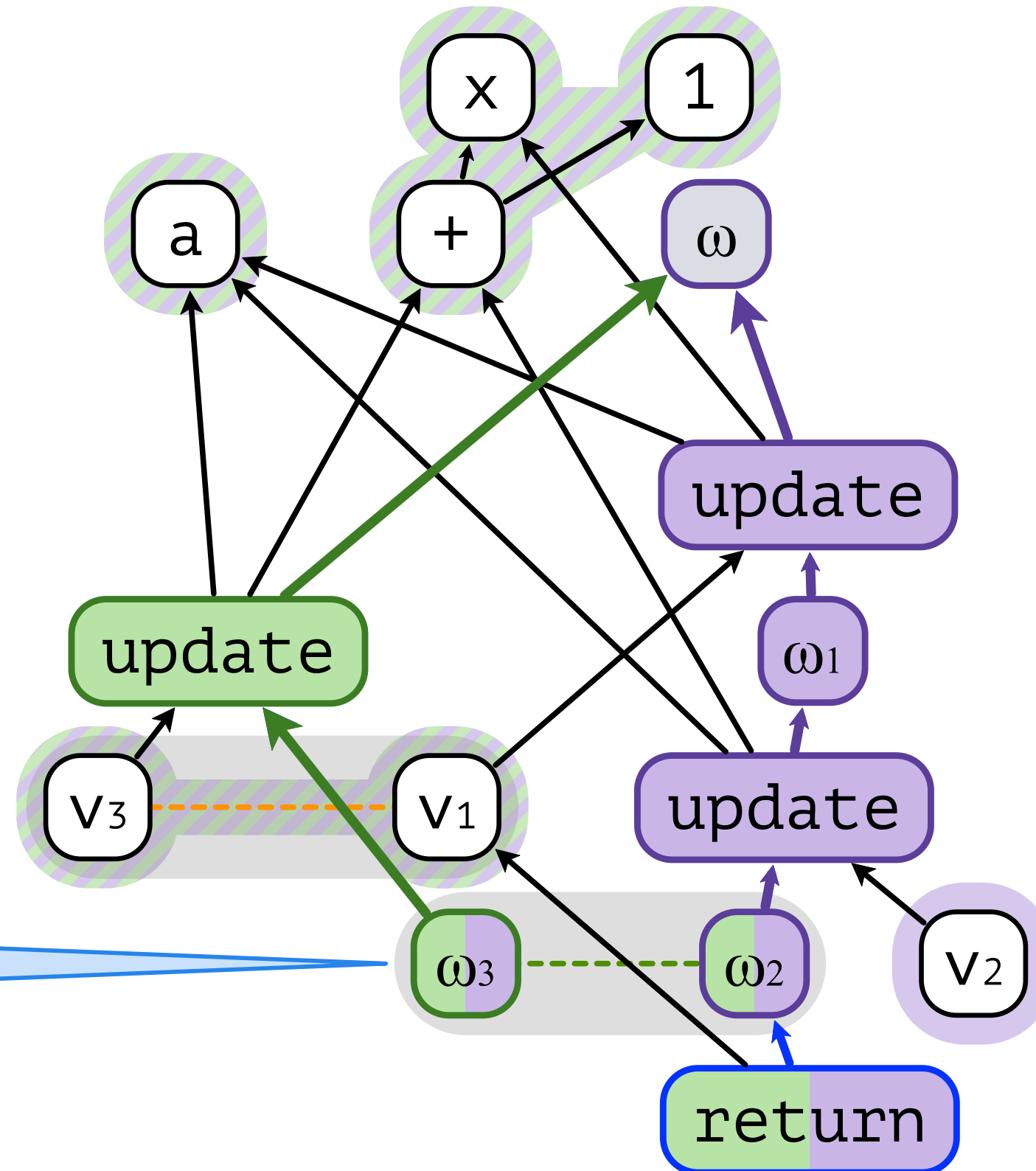
extractable set:
e-classes we could extract
using a given **state walk**

Collapsing Statewalks

Optimization:
collapse by unique
extractable set

extractable set:
e-classes we could extract
using a given **state walk**

different
extractable sets

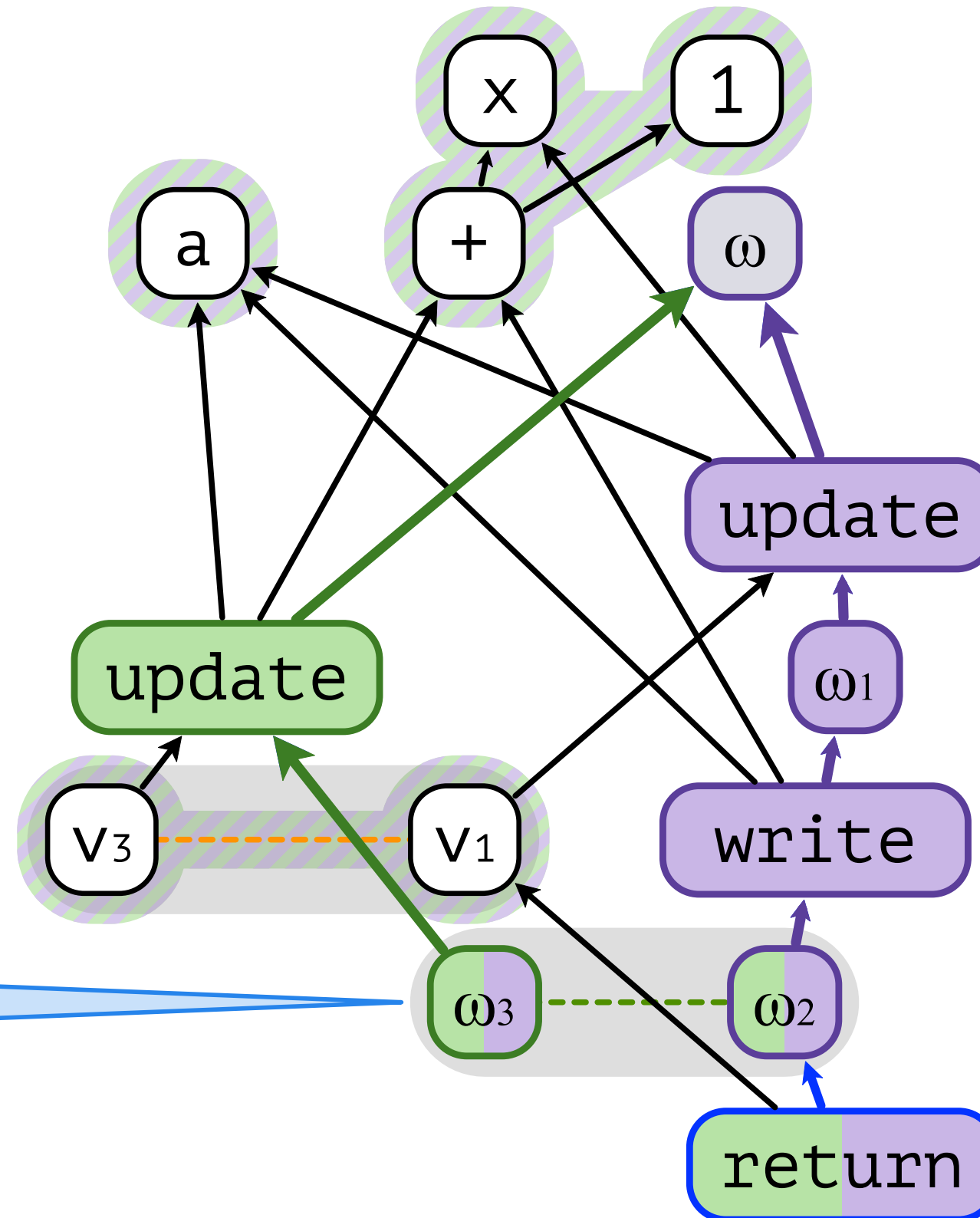


Collapsing Statewalks

Optimization:
collapse by unique
extractable set

extractable set:
e-classes we could extract
using a given **state walk**

different
extractable sets

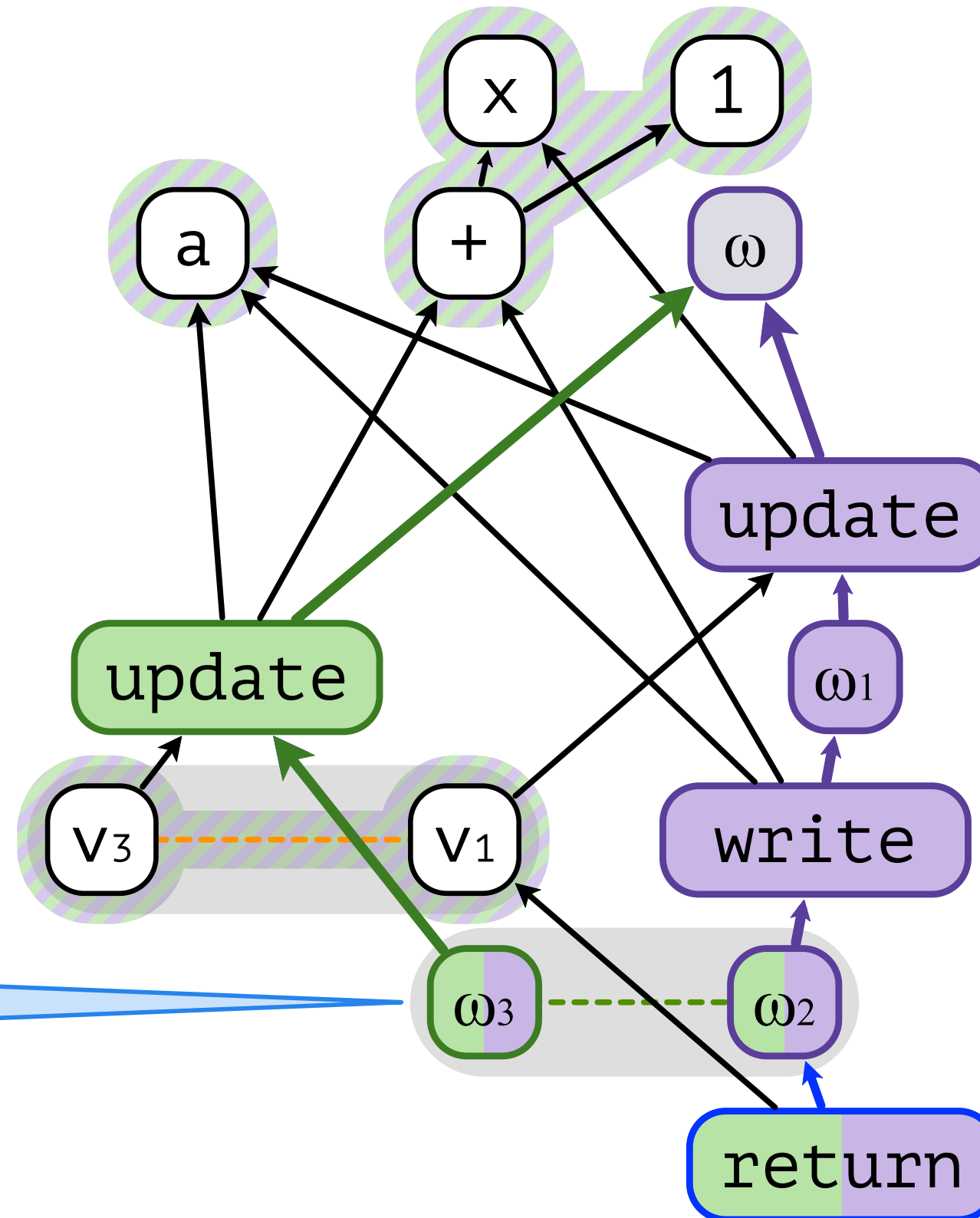


Collapsing Statewalks

Optimization:
collapse by unique
extractable set

extractable set:
e-classes we could extract
using a given **state walk**

same
extractable set

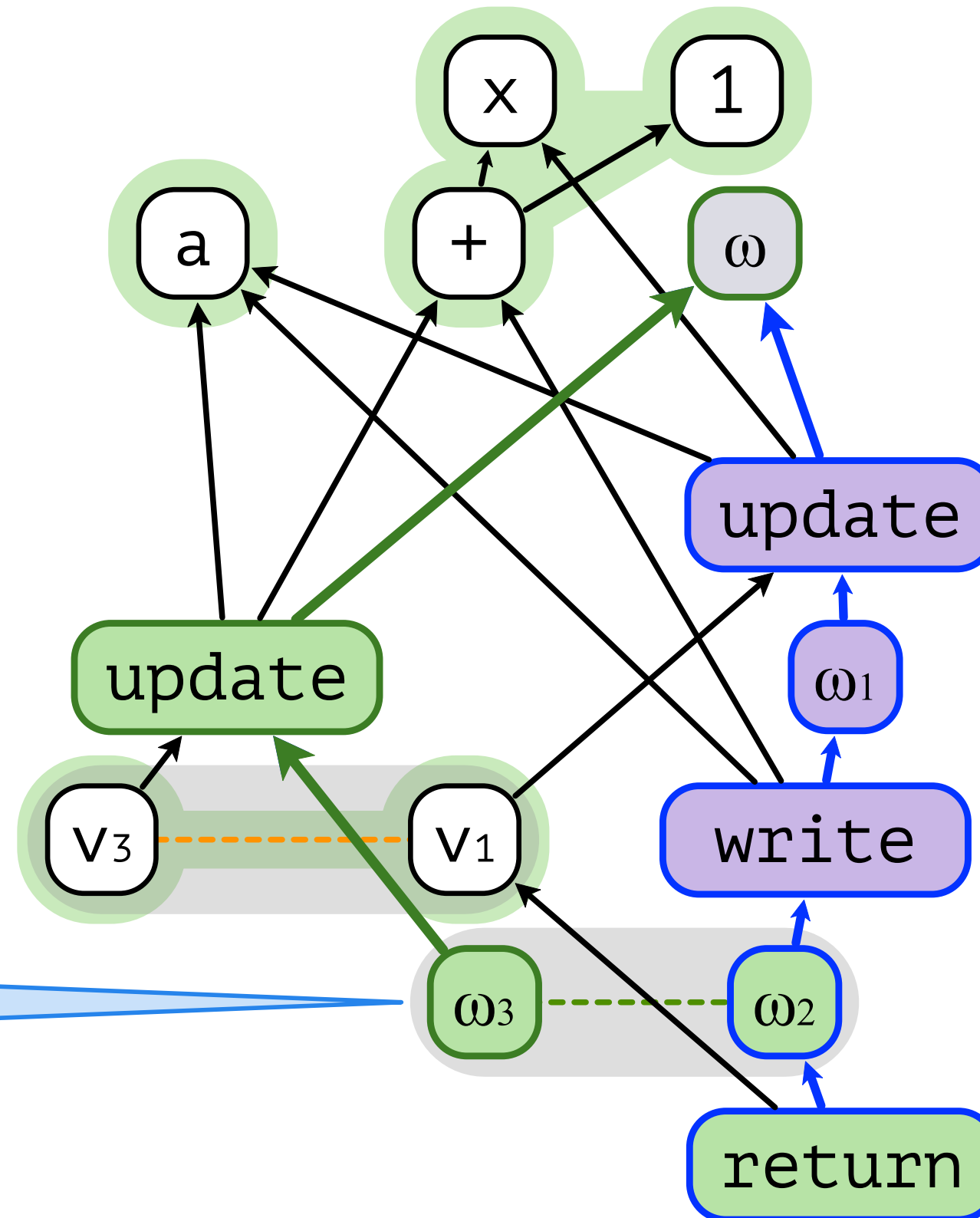


Collapsing Statewalks

Optimization:
collapse by unique
extractable set

extractable set:
e-classes we could extract
using a given **state walk**

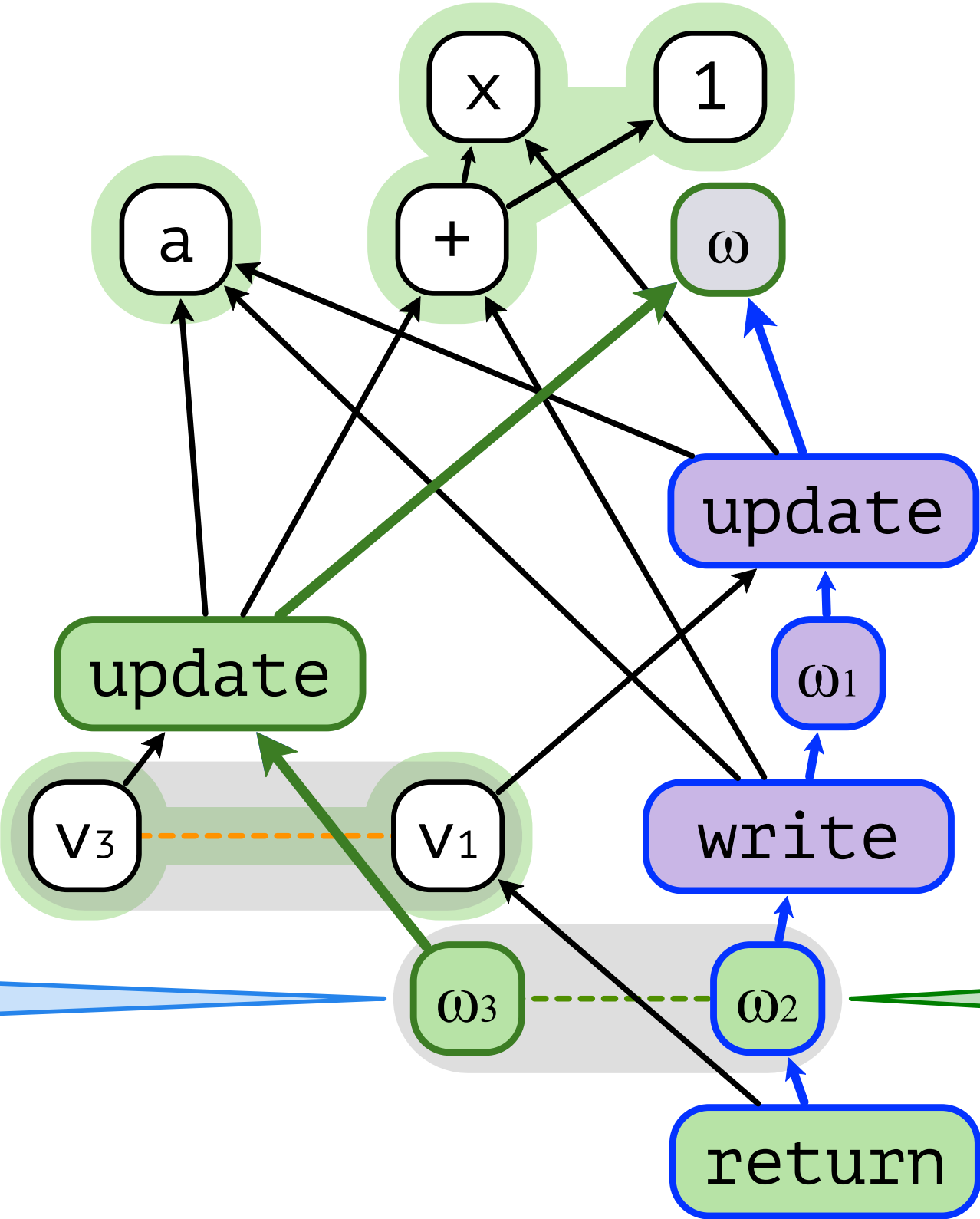
same
extractable set



Collapsing Statewalks

Optimization:
collapse by unique
extractable set

extractable set:
e-classes we could extract
using a given **state walk**



same
extractable set

few in practice

Collapsing Statewalks

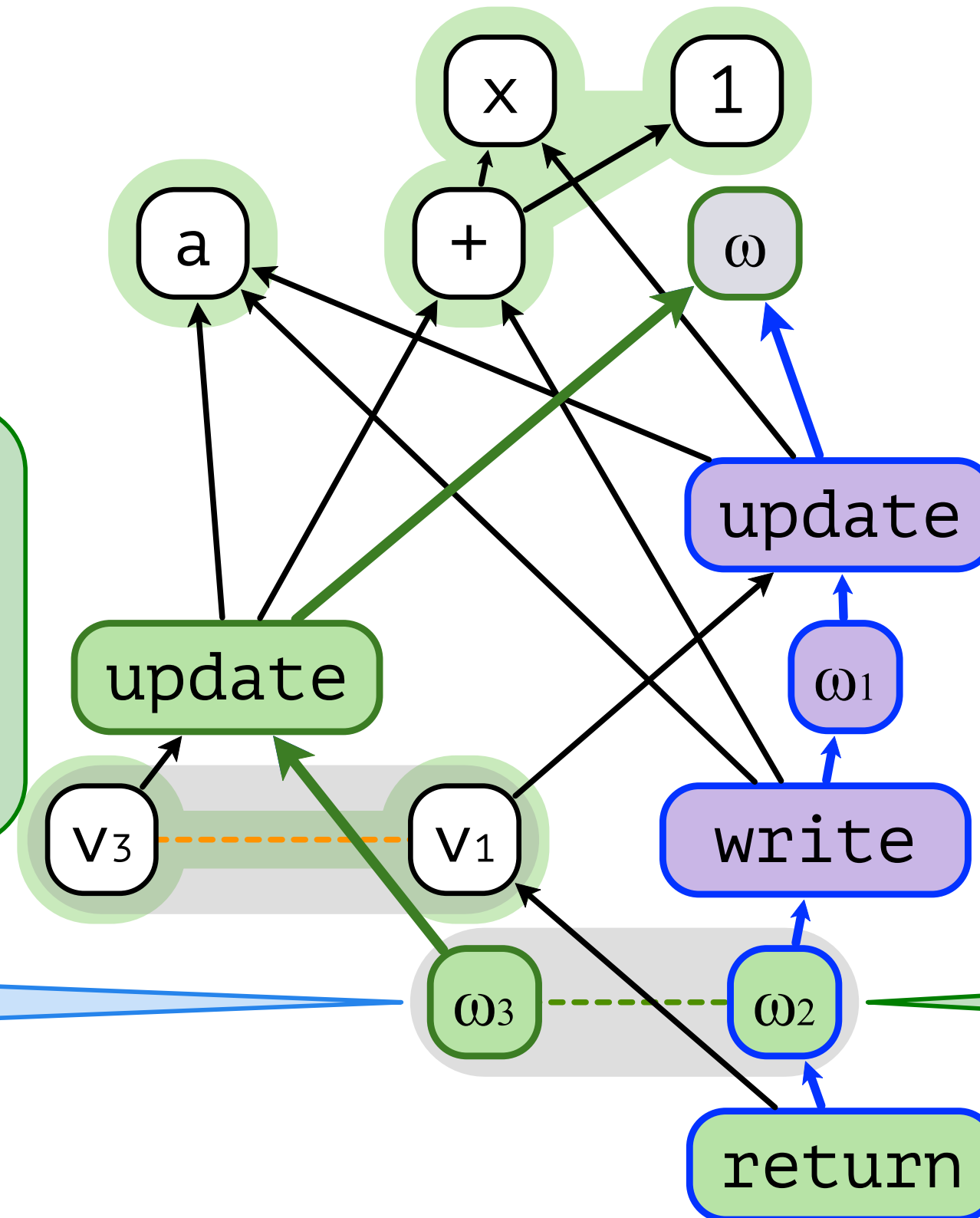
Optimization:
collapse by unique
extractable set

*parameterized
algorithms* can
provide practical
solutions to
NP-hard problems

extractable set:
e-classes we could extract
using a given **state walk**

same
extractable set

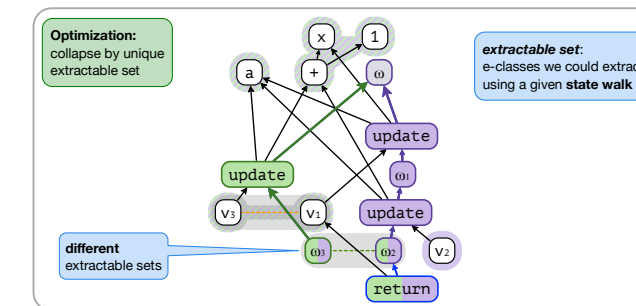
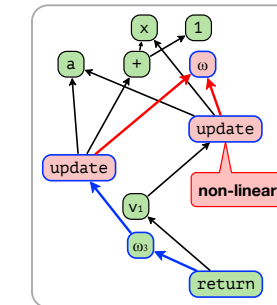
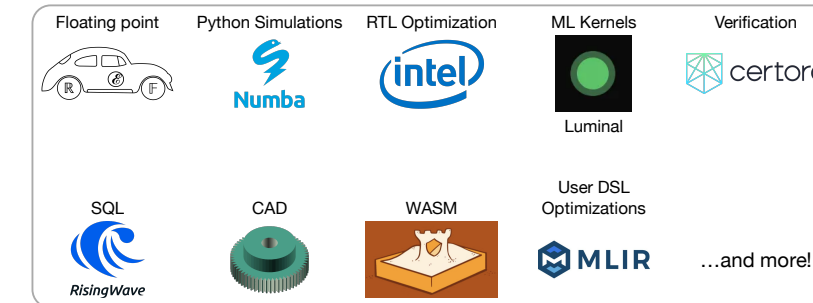
few in practice

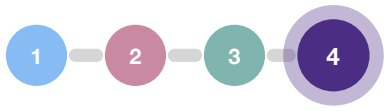




Fast Extraction Anyways

- 1 Why Effectful E-Graphs
- 2 An NP-Hard Problem
- 3 Fast Extraction Anyways
- 4 Eggcc, A New Compiler

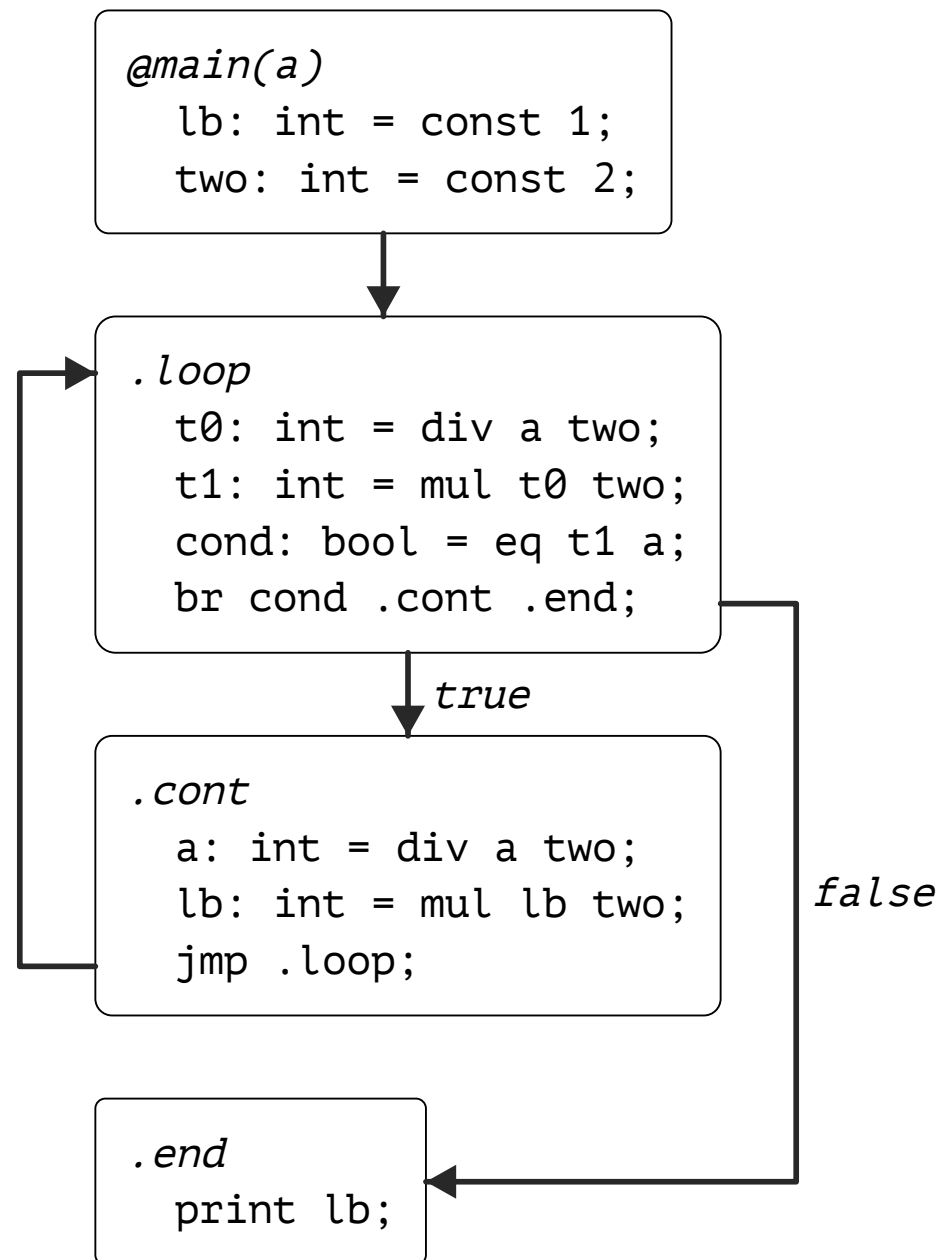




Case Study: eggcc

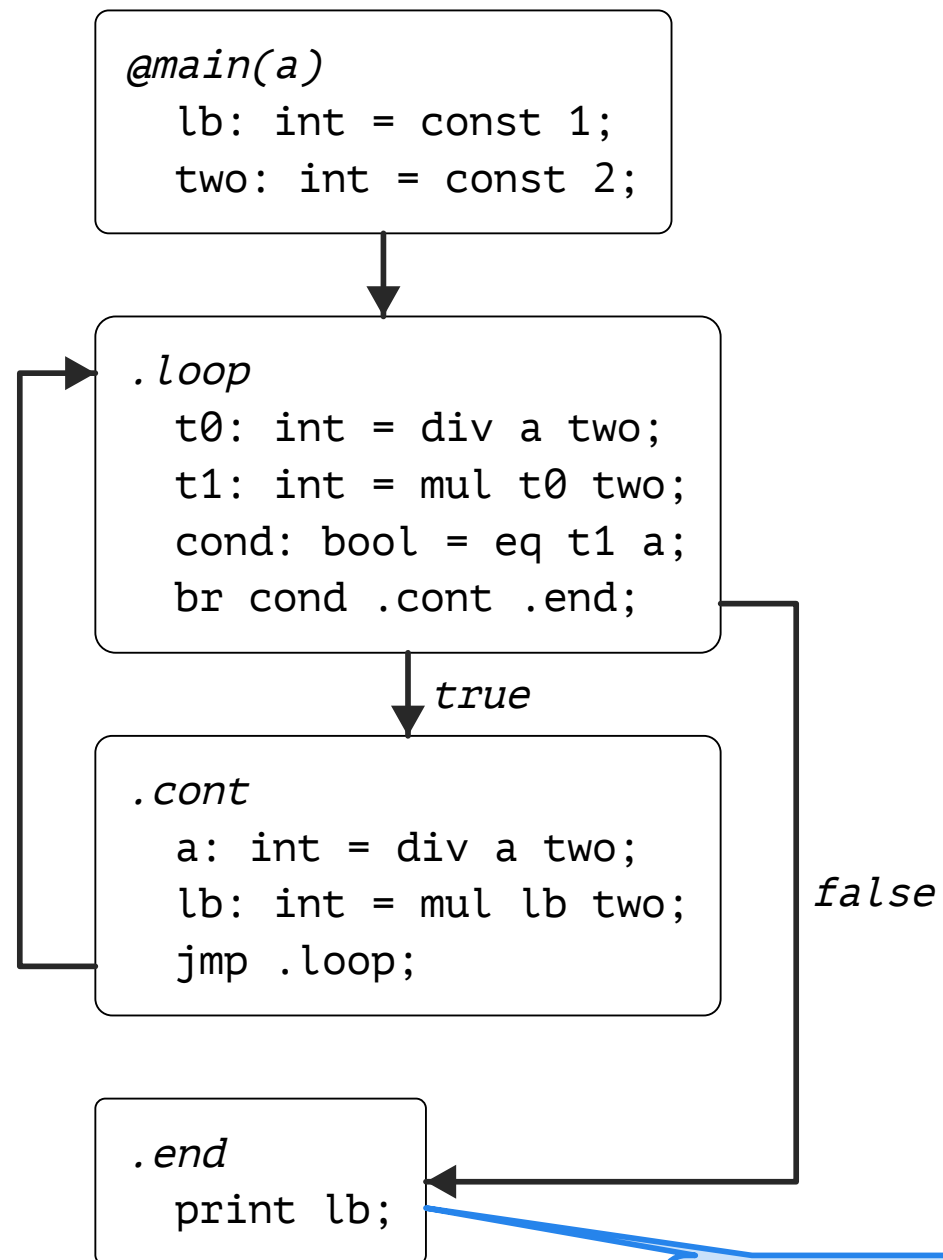
Case Study: eggcc

a Bril CFG



Case Study: eggcc

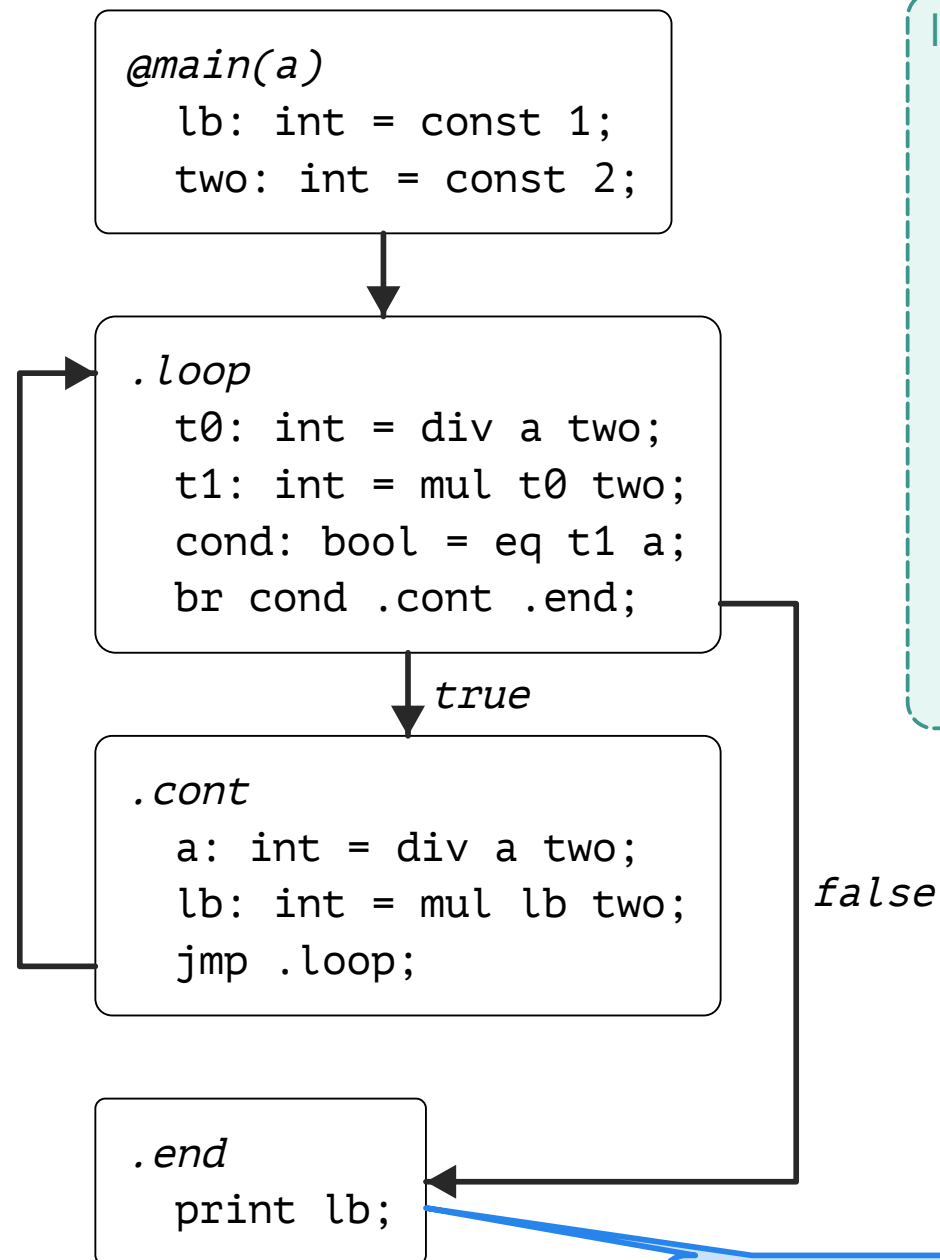
a Bril CFG



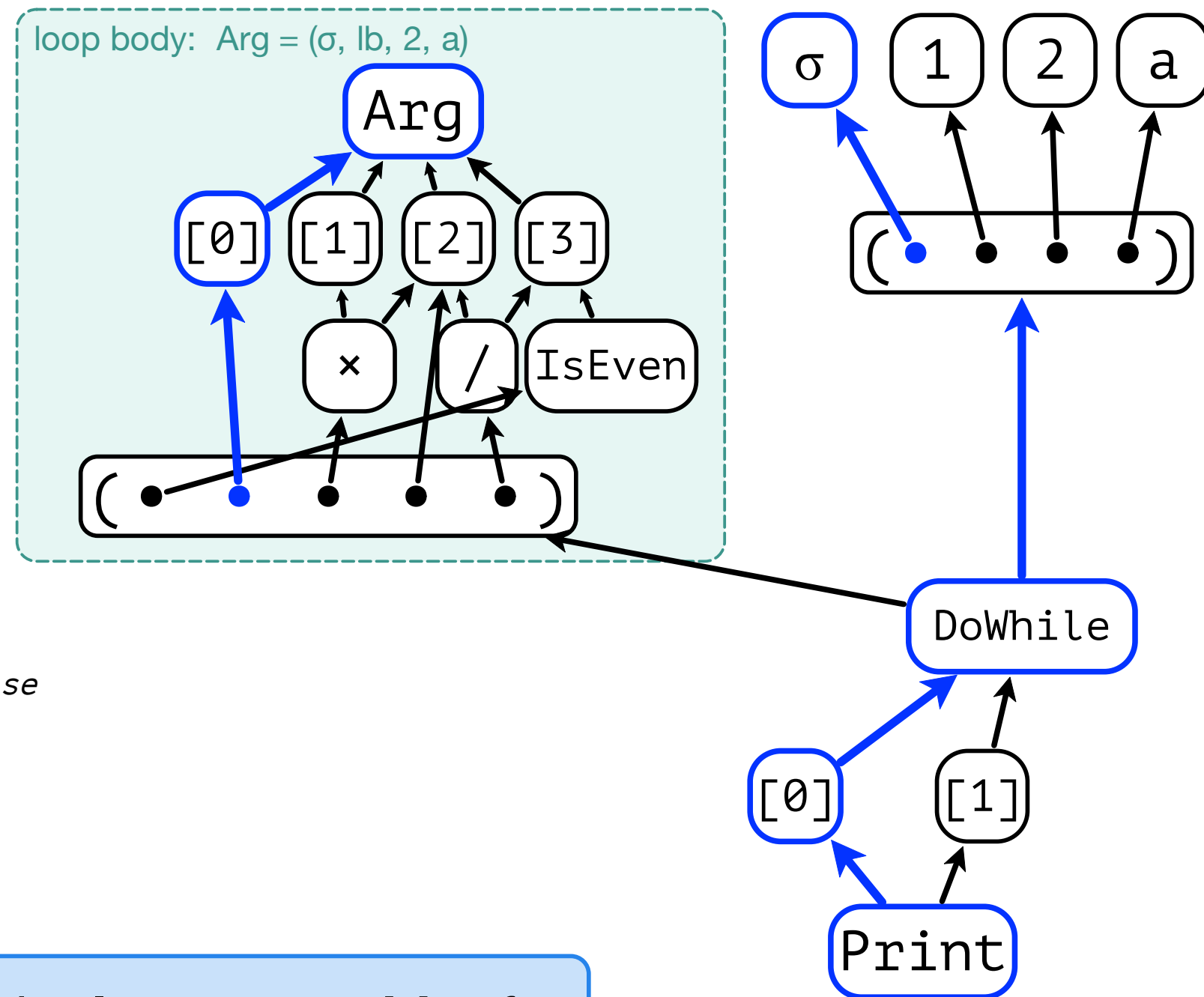
finds the **lowest set bit** of *a*

Case Study: eggcc

a Brill CFG

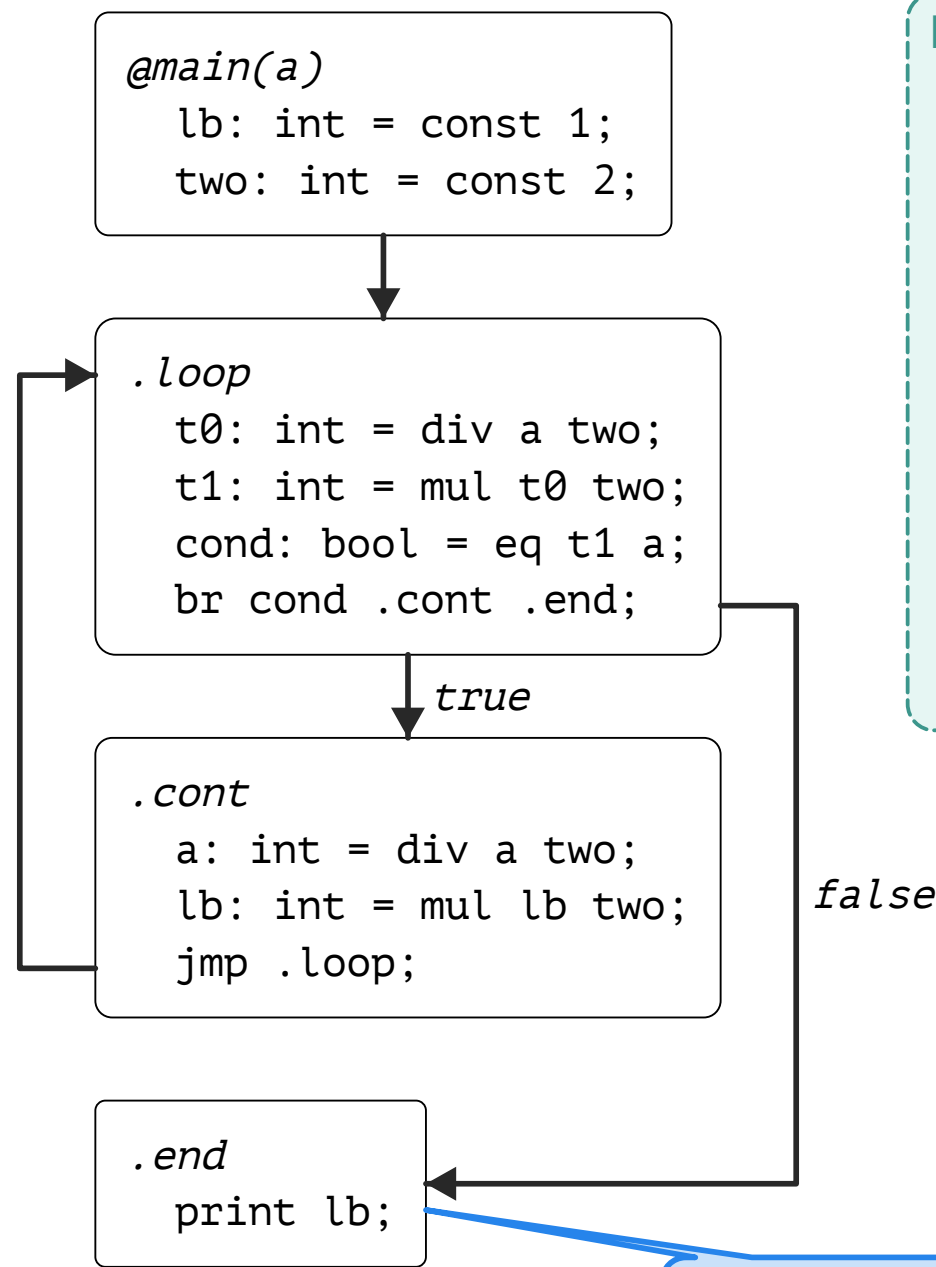


its e-graph



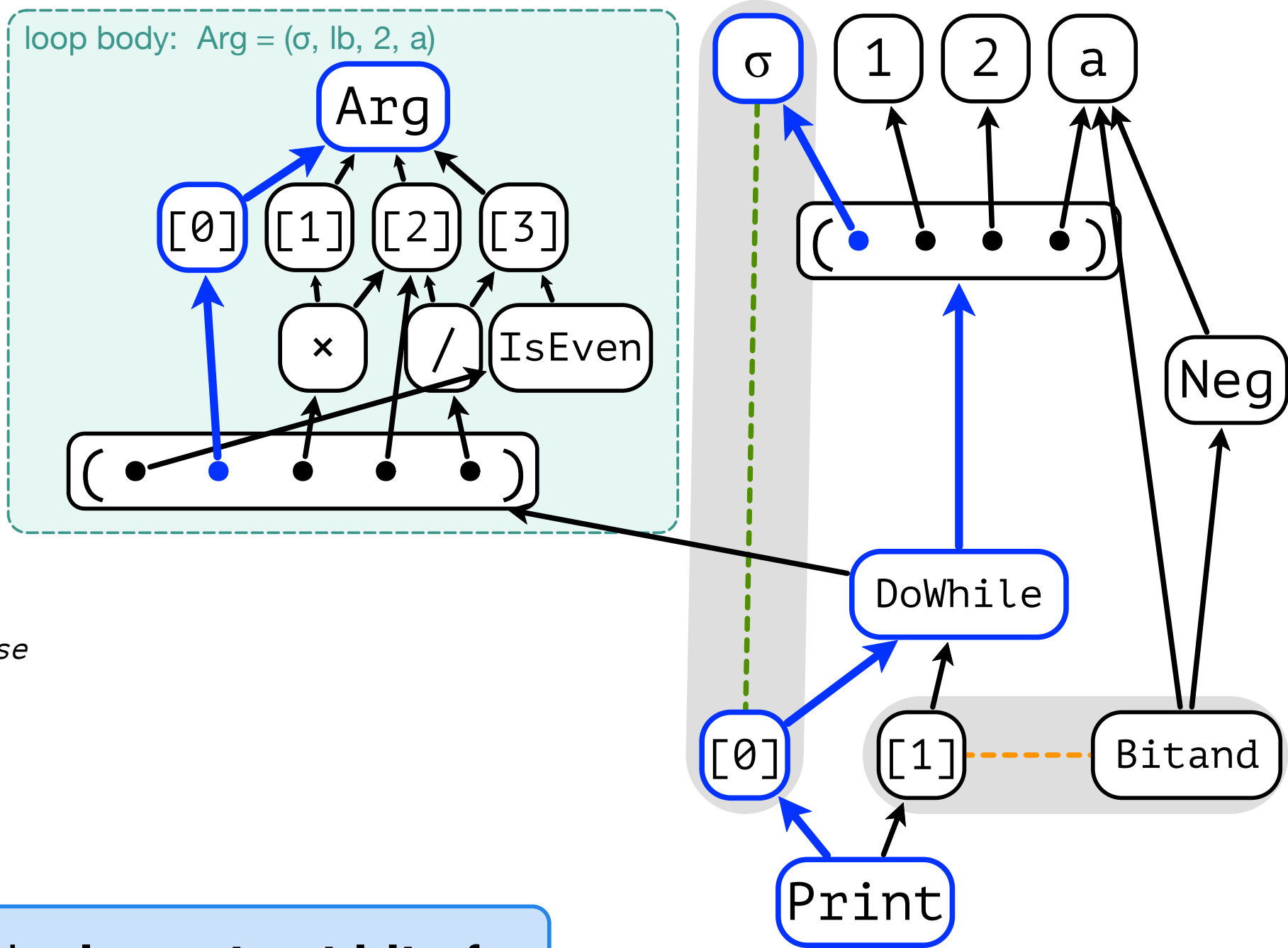
Case Study: eggcc

a Bril CFG



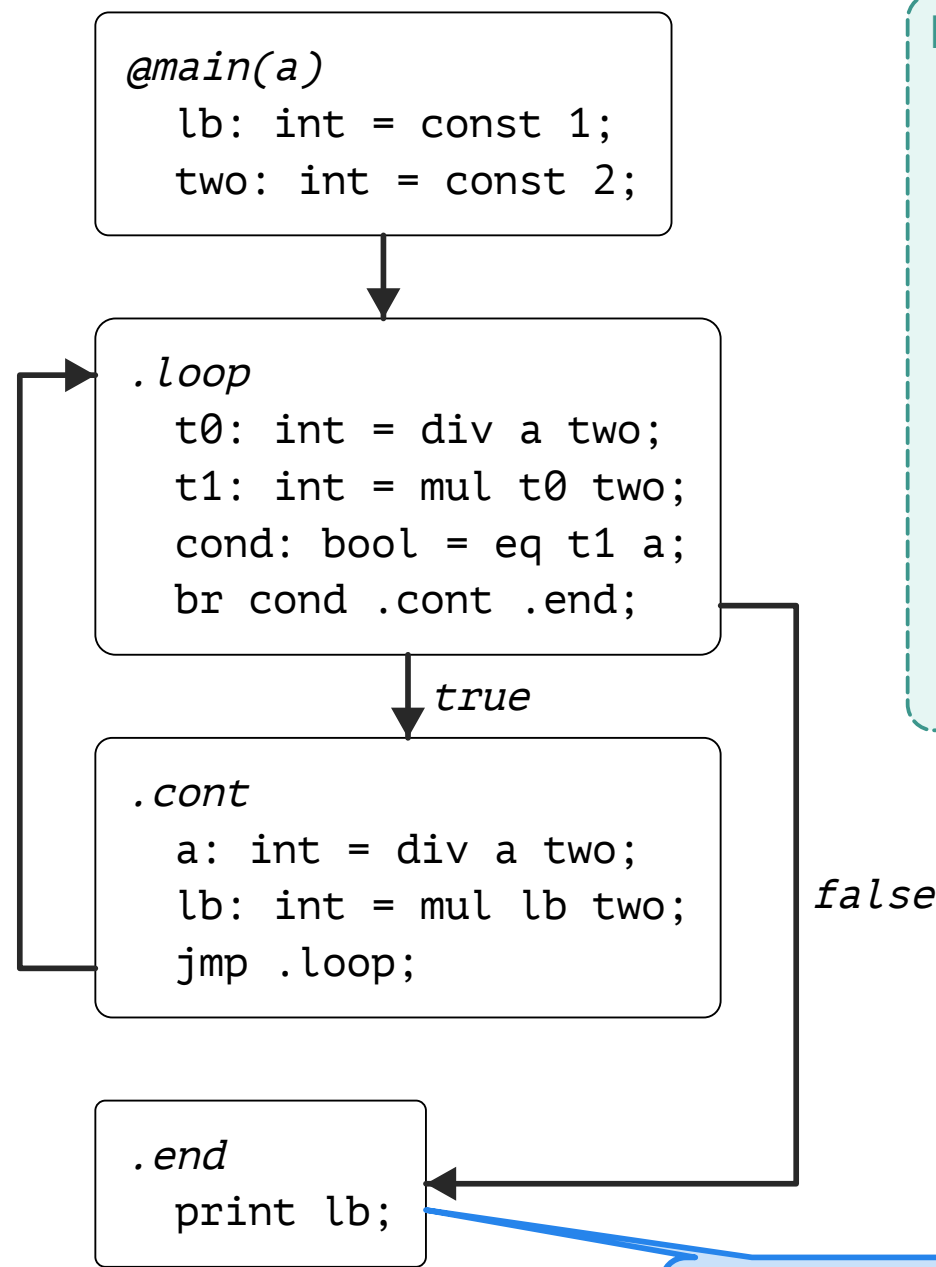
finds the **lowest set bit** of *a*

its e-graph



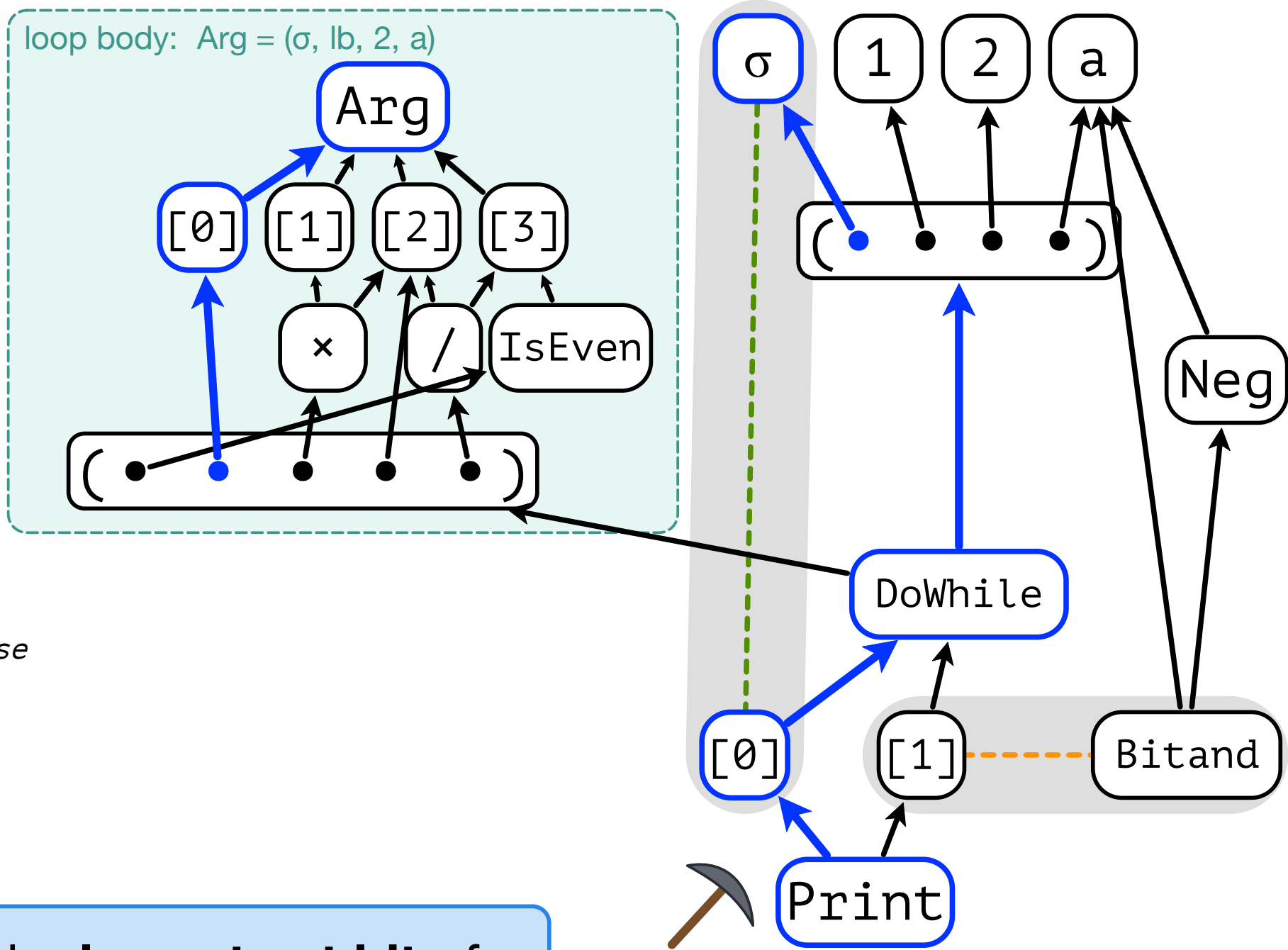
Case Study: eggcc

a Bril CFG



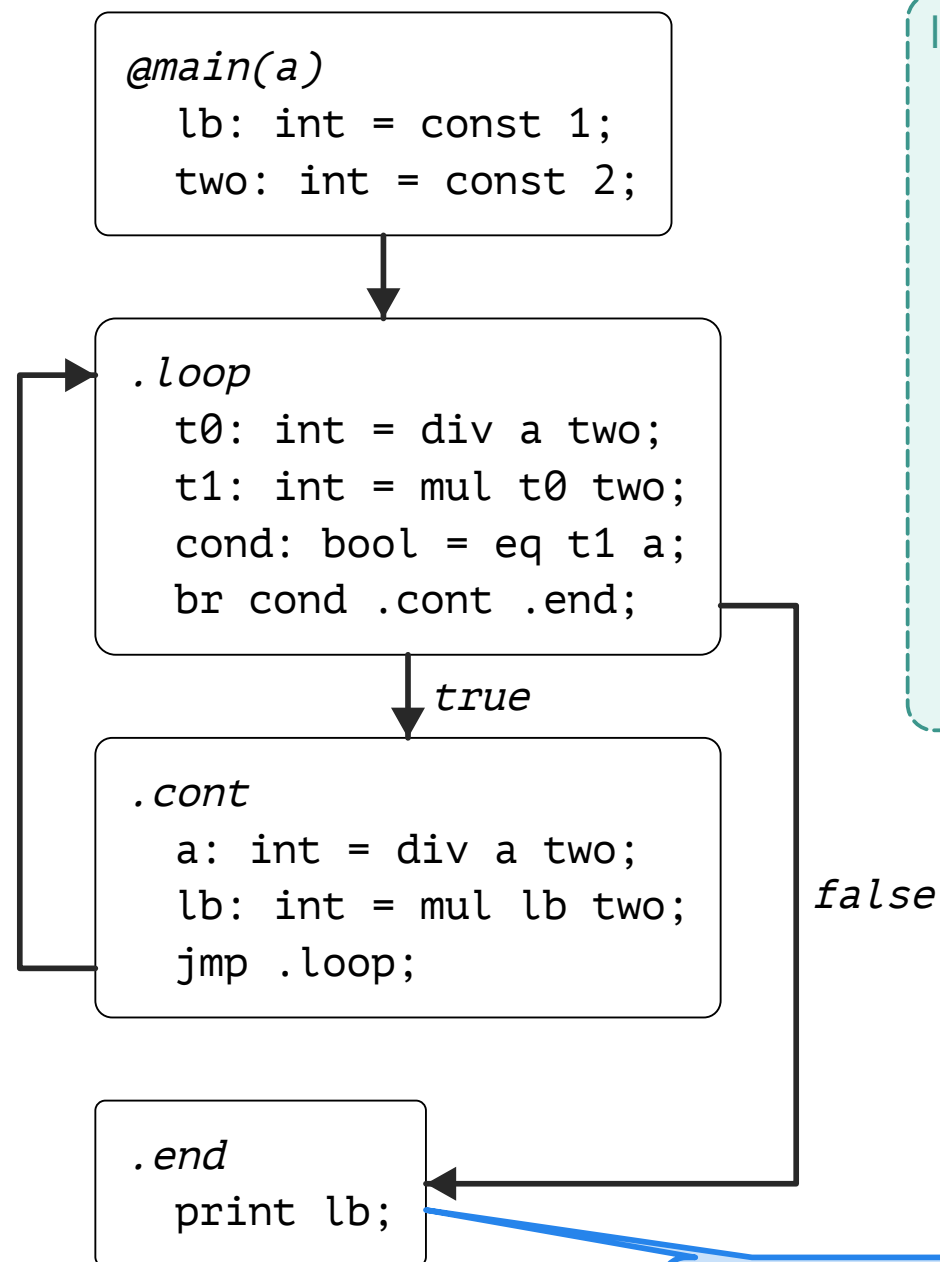
finds the **lowest set bit** of *a*

its e-graph

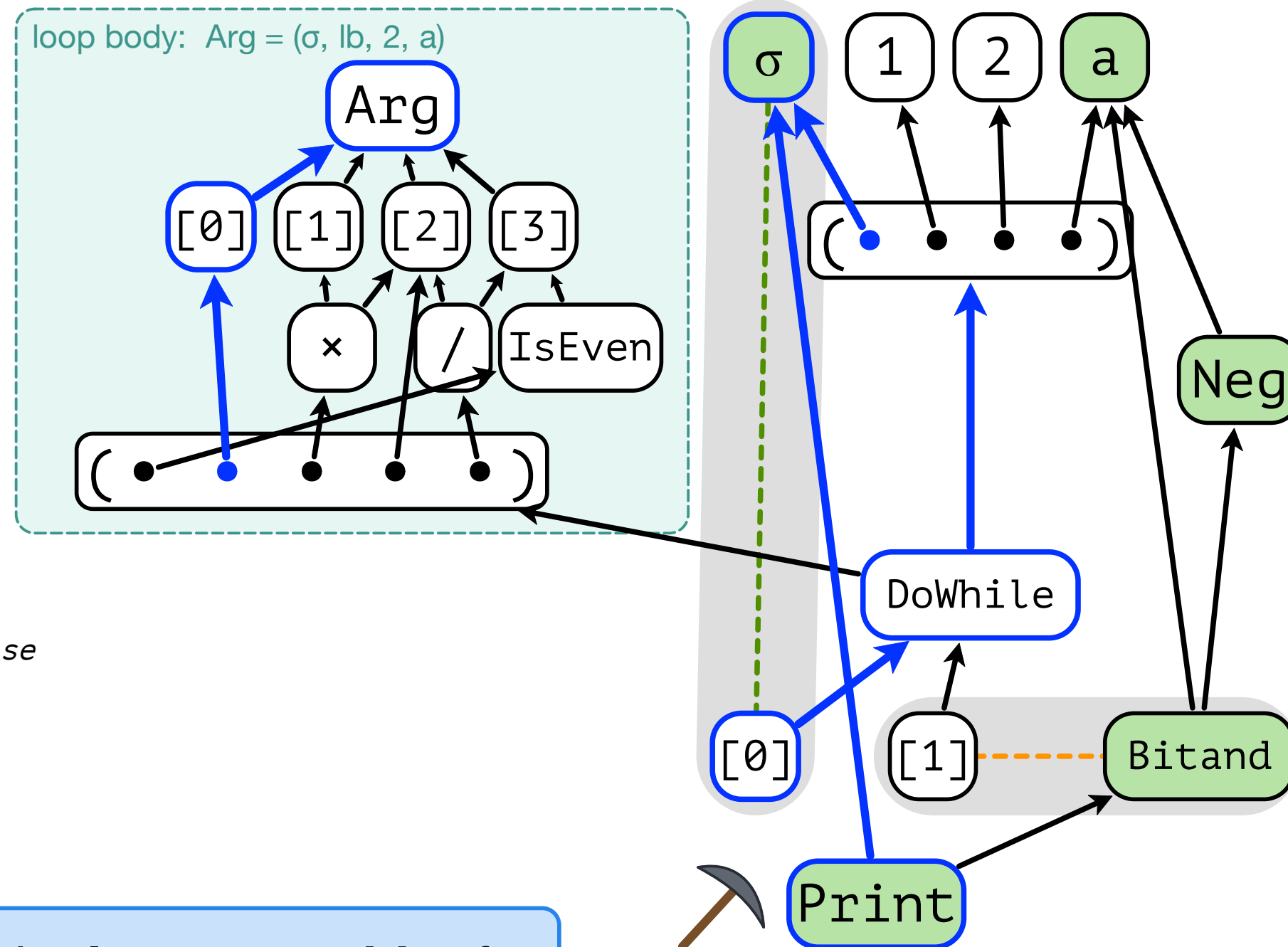


Case Study: eggcc

a Bril CFG



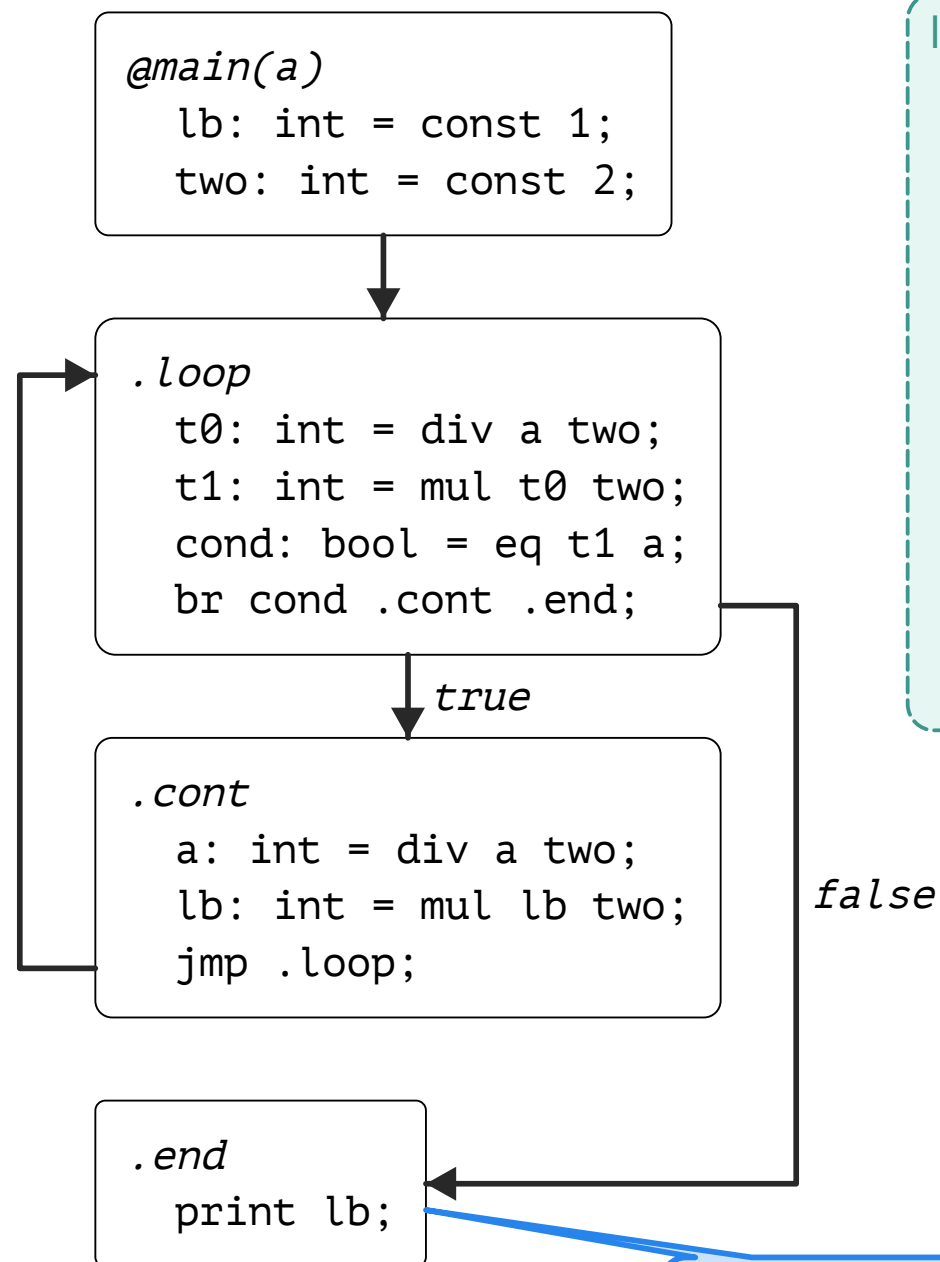
its e-graph



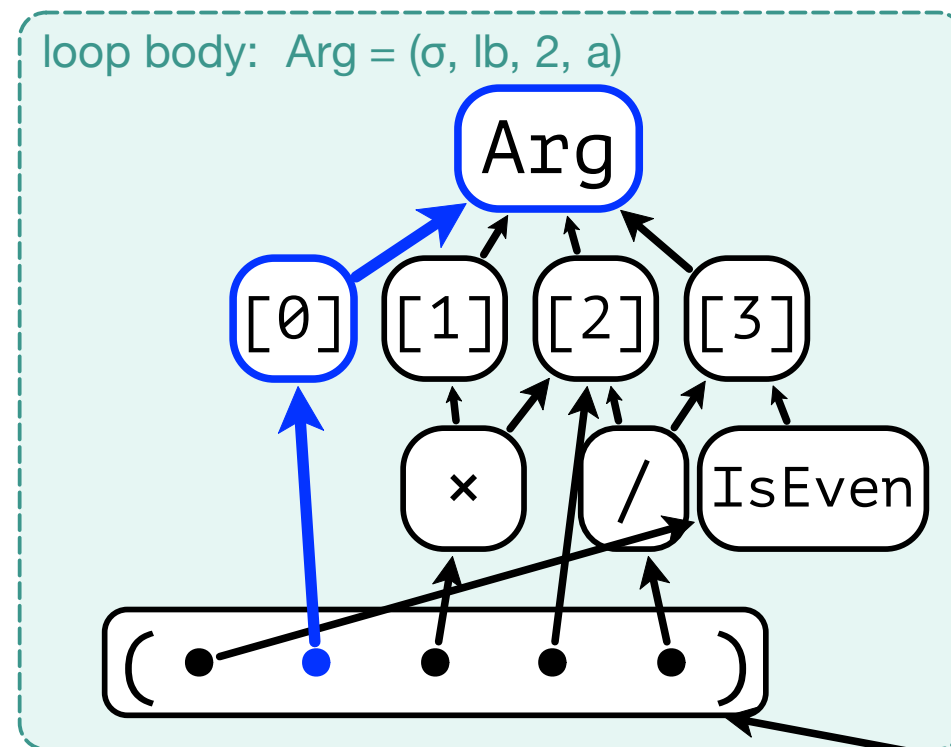
finds the **lowest set bit** of *a*

Case Study: eggcc

a Bril CFG



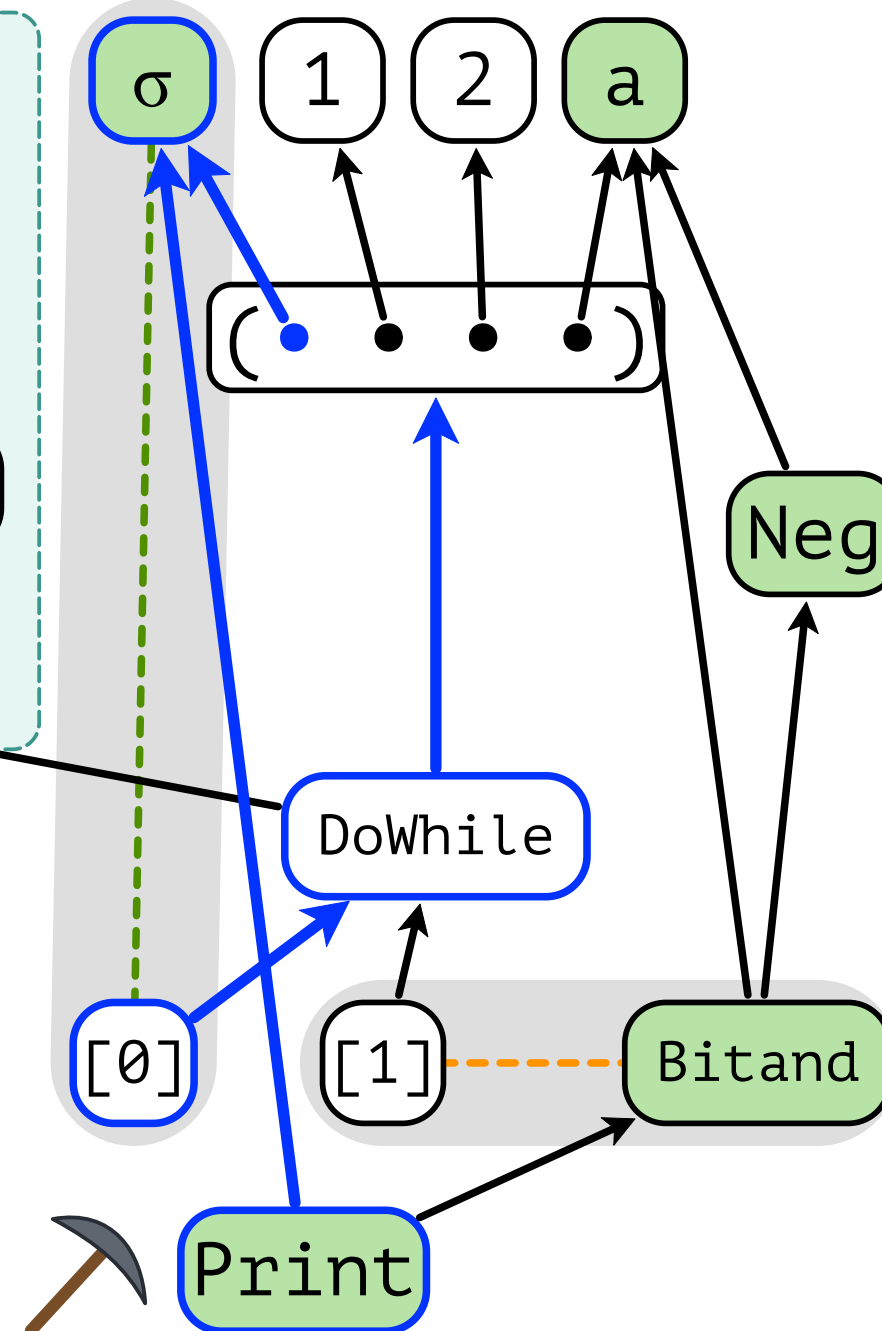
its e-graph



optimized program

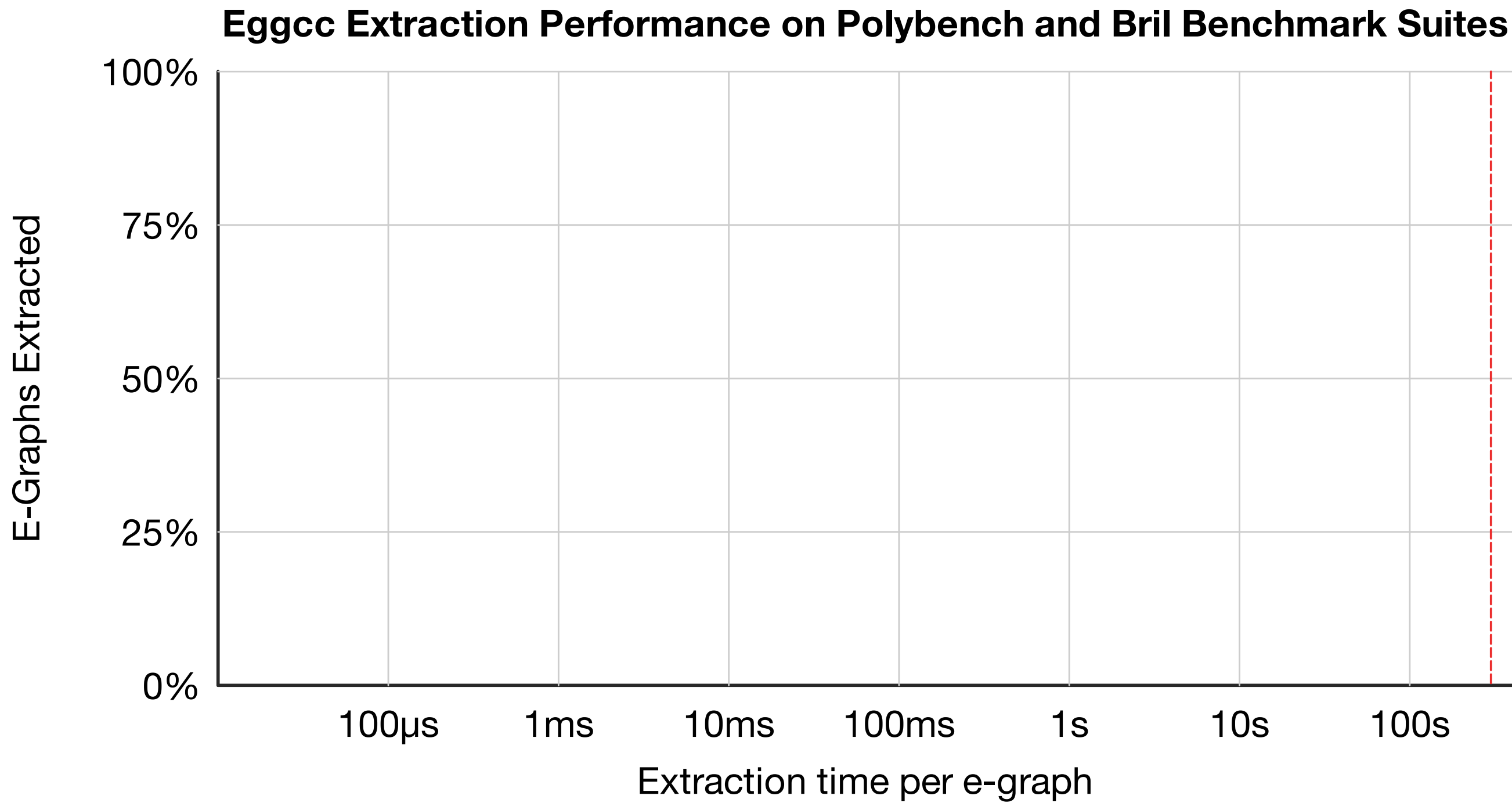
```

@main(a)
  t: int = neg a;
  lb: int = bitand a t;
  print lb;
  
```

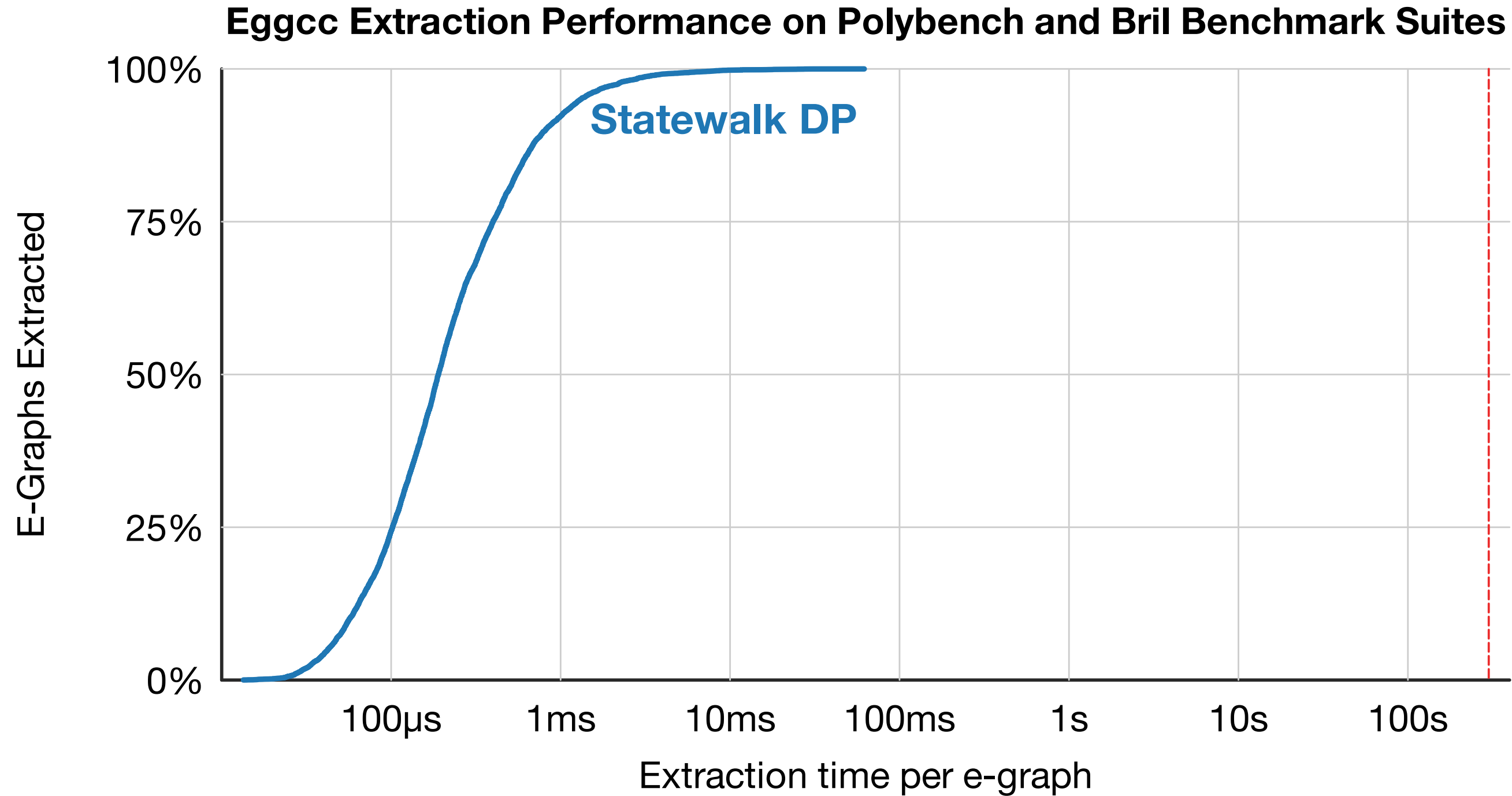


finds the **lowest set bit** of `a`

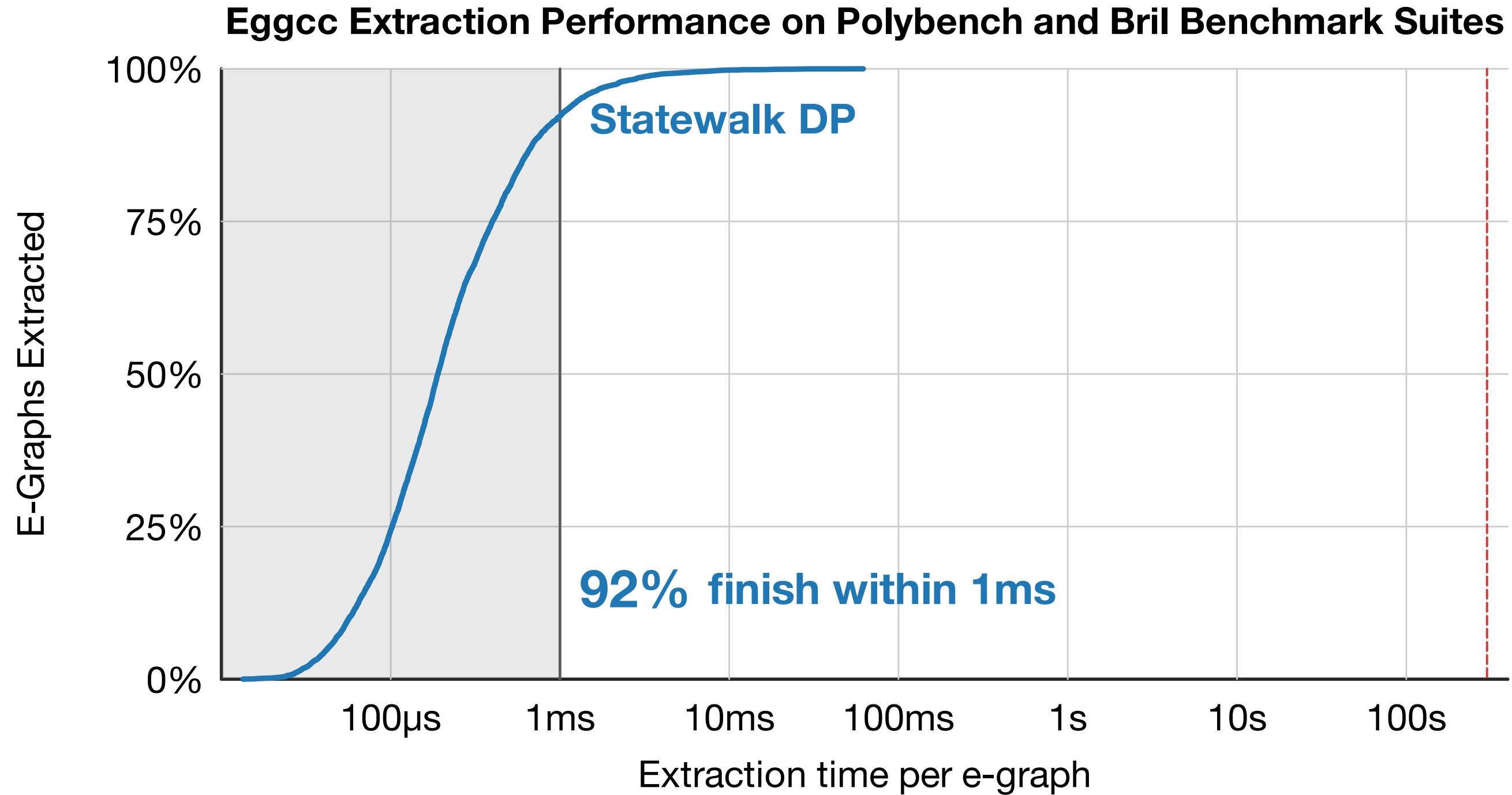
Extraction is Fast in Practice



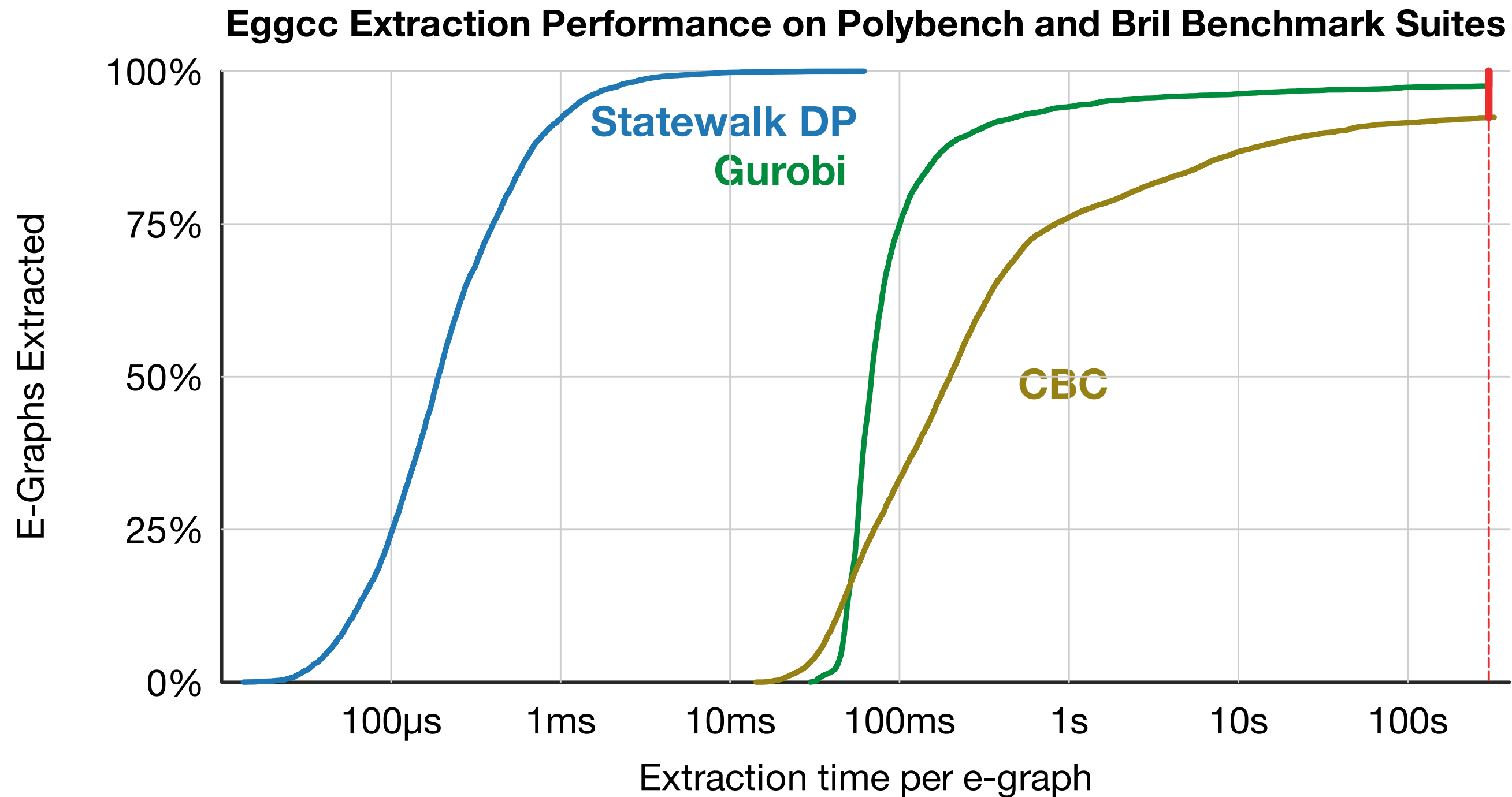
Extraction is Fast in Practice



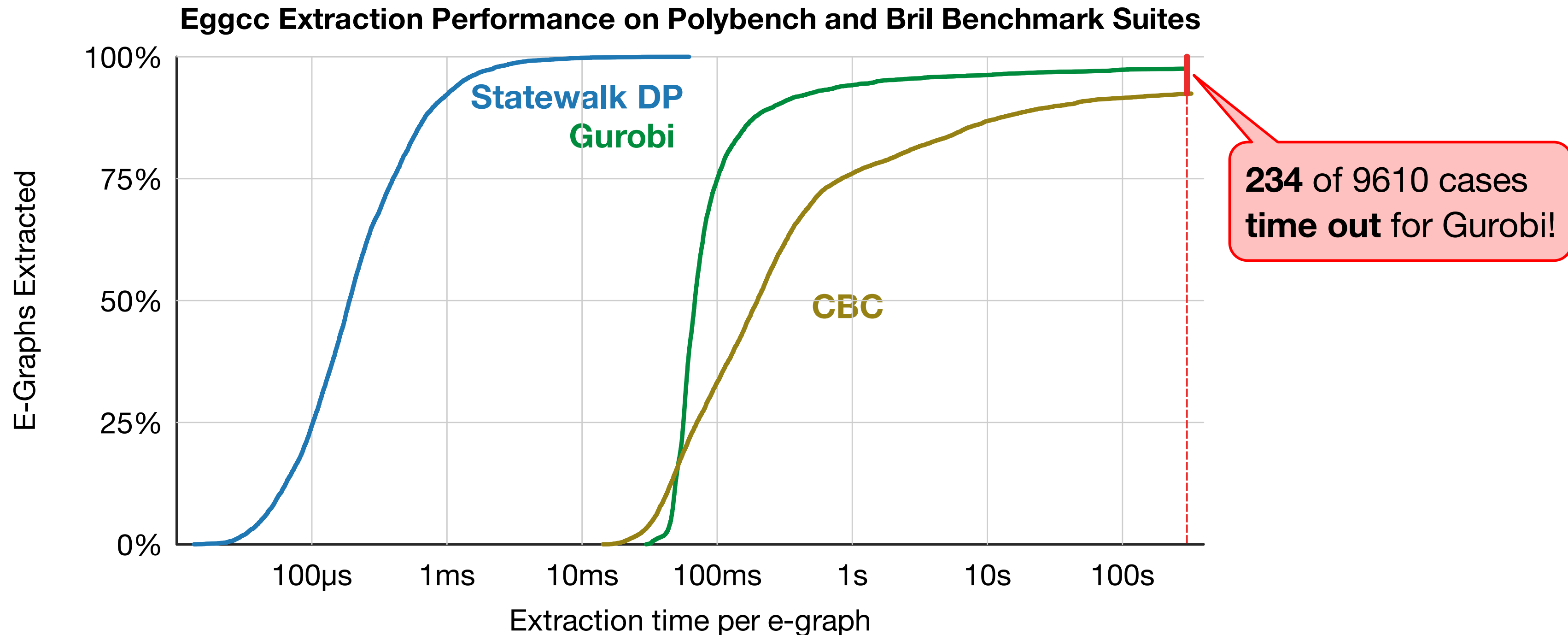
Extraction is Fast in Practice



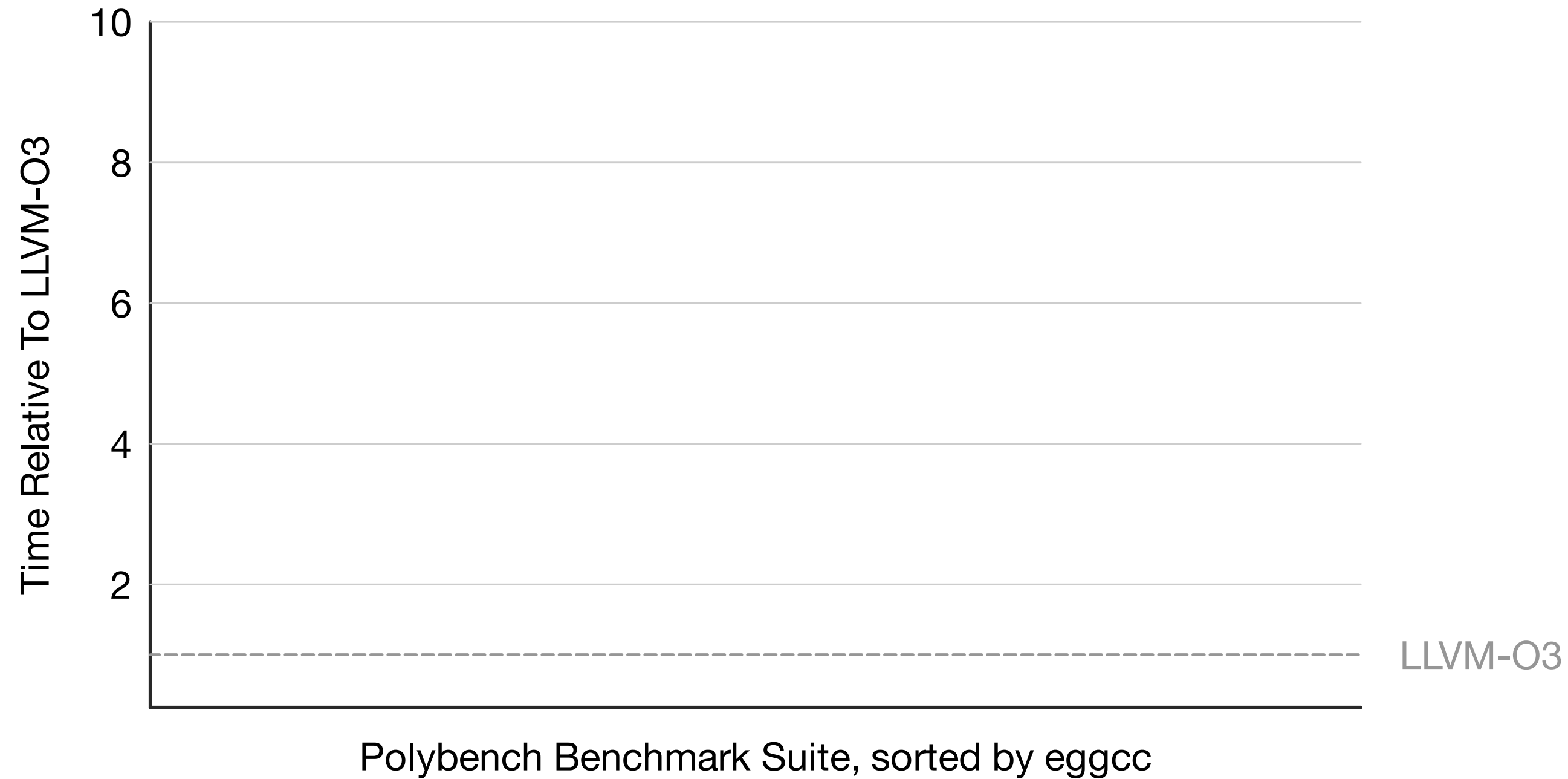
Extraction is Fast in Practice



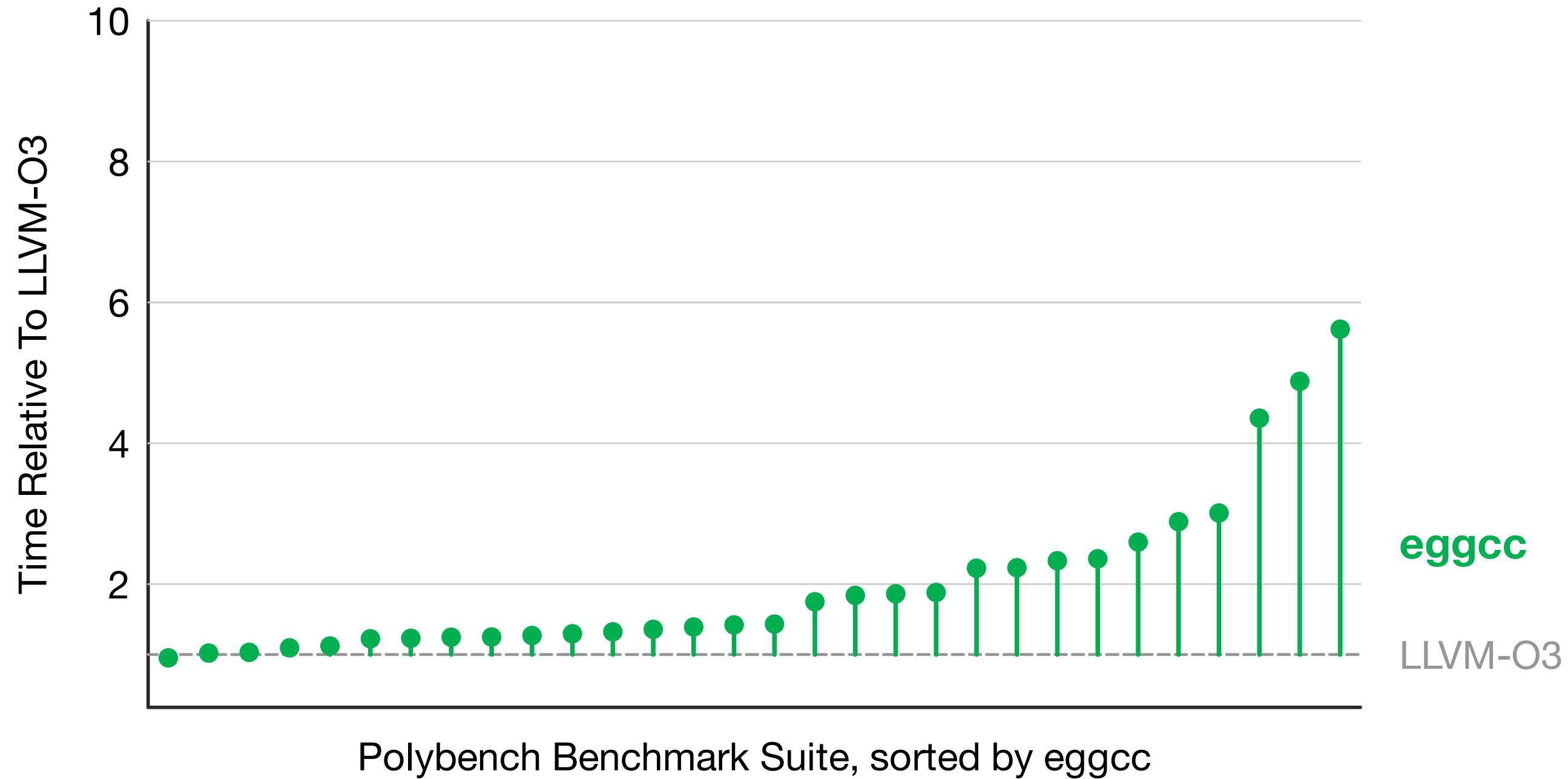
Extraction is Fast in Practice



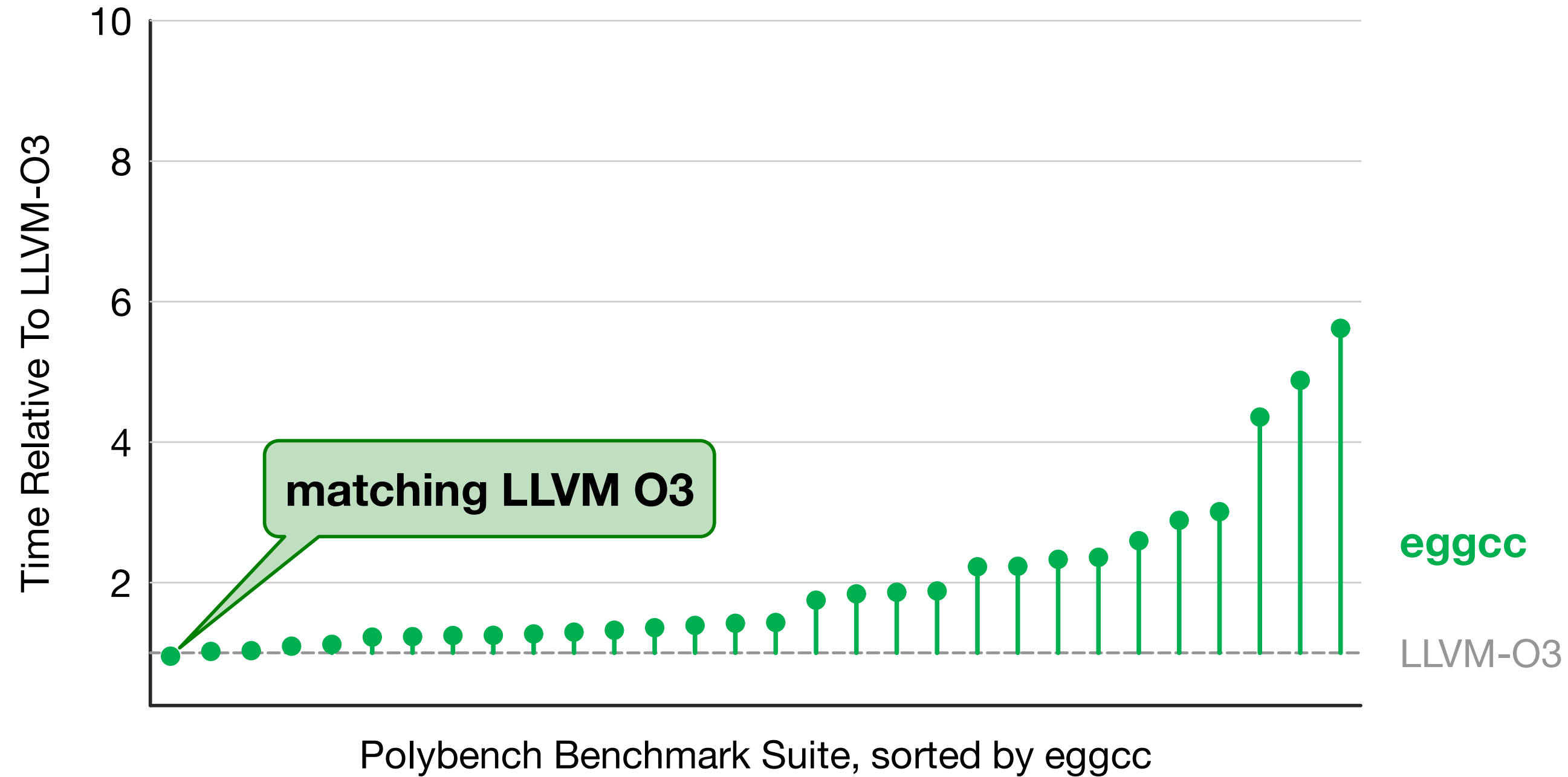
Fast Effect-safe Extraction in Practice



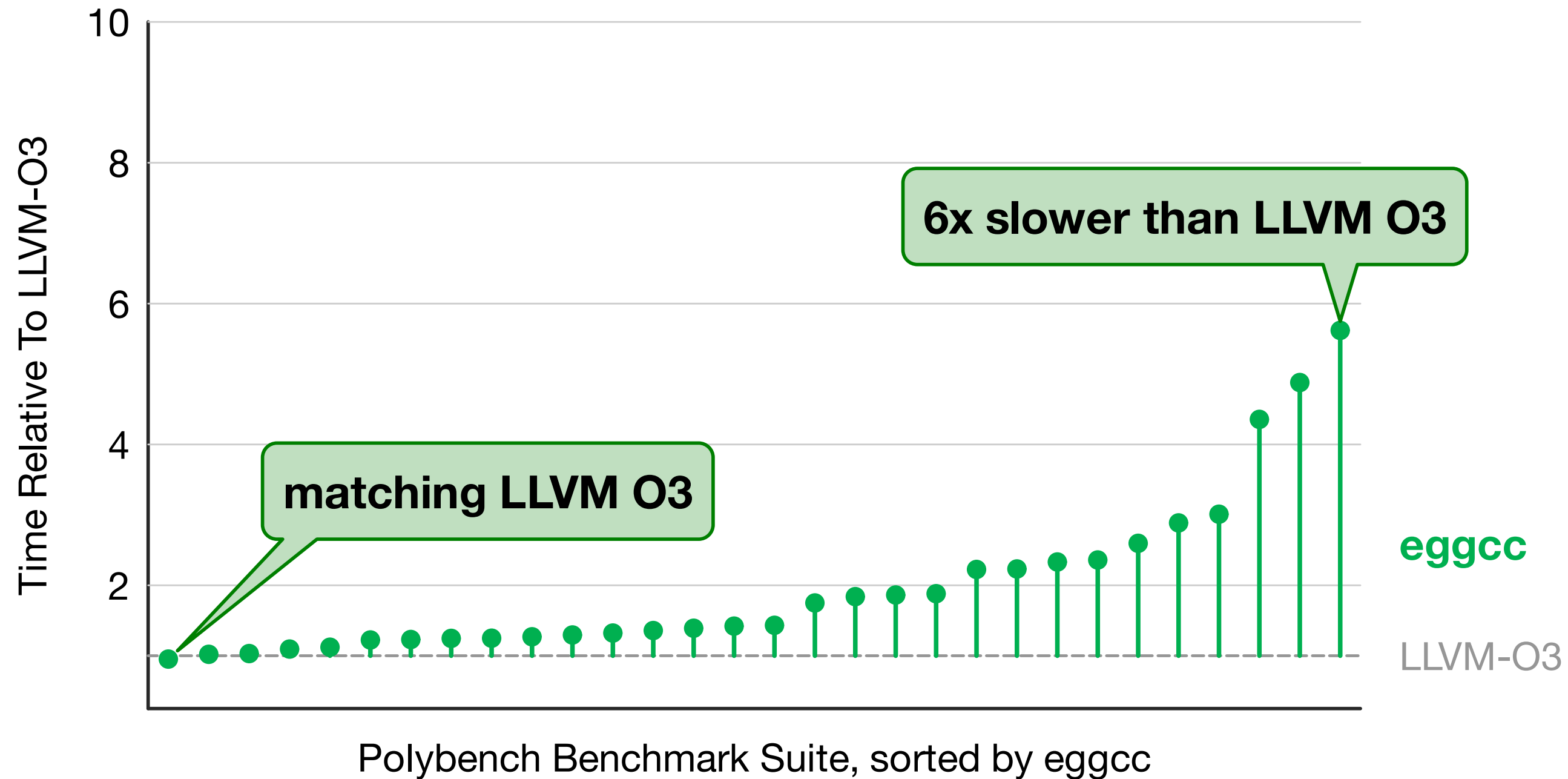
Fast Effect-safe Extraction in Practice



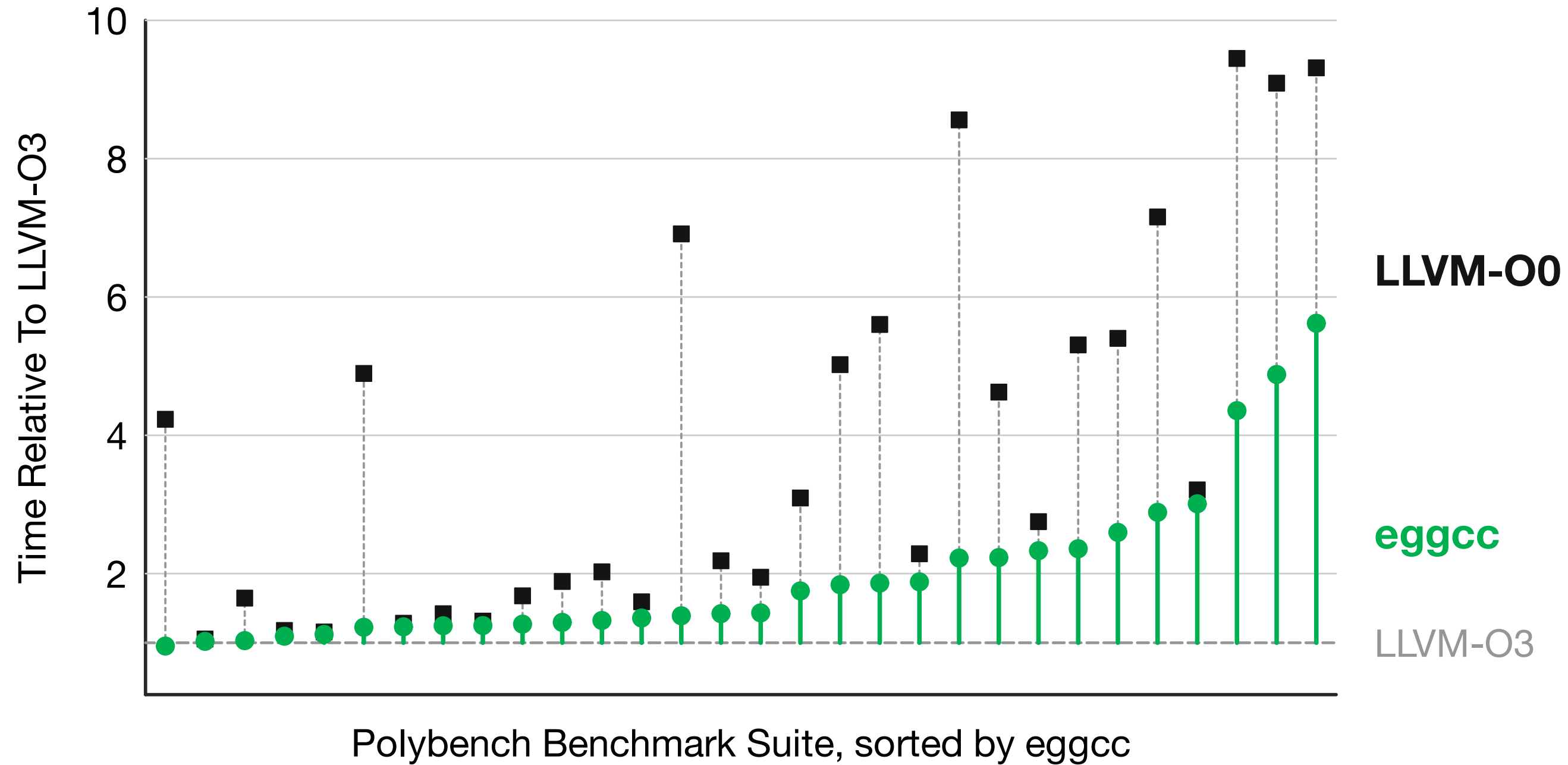
Fast Effect-safe Extraction in Practice



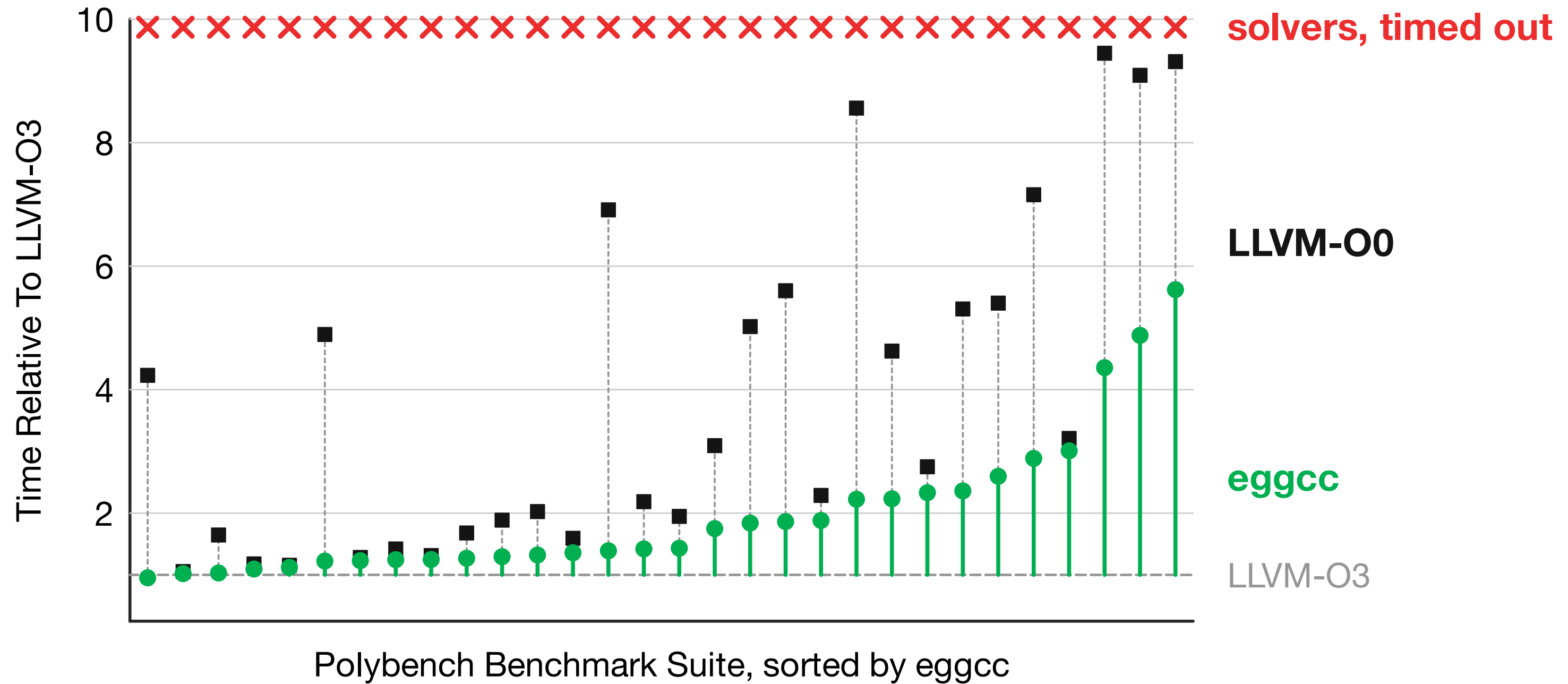
Fast Effect-safe Extraction in Practice



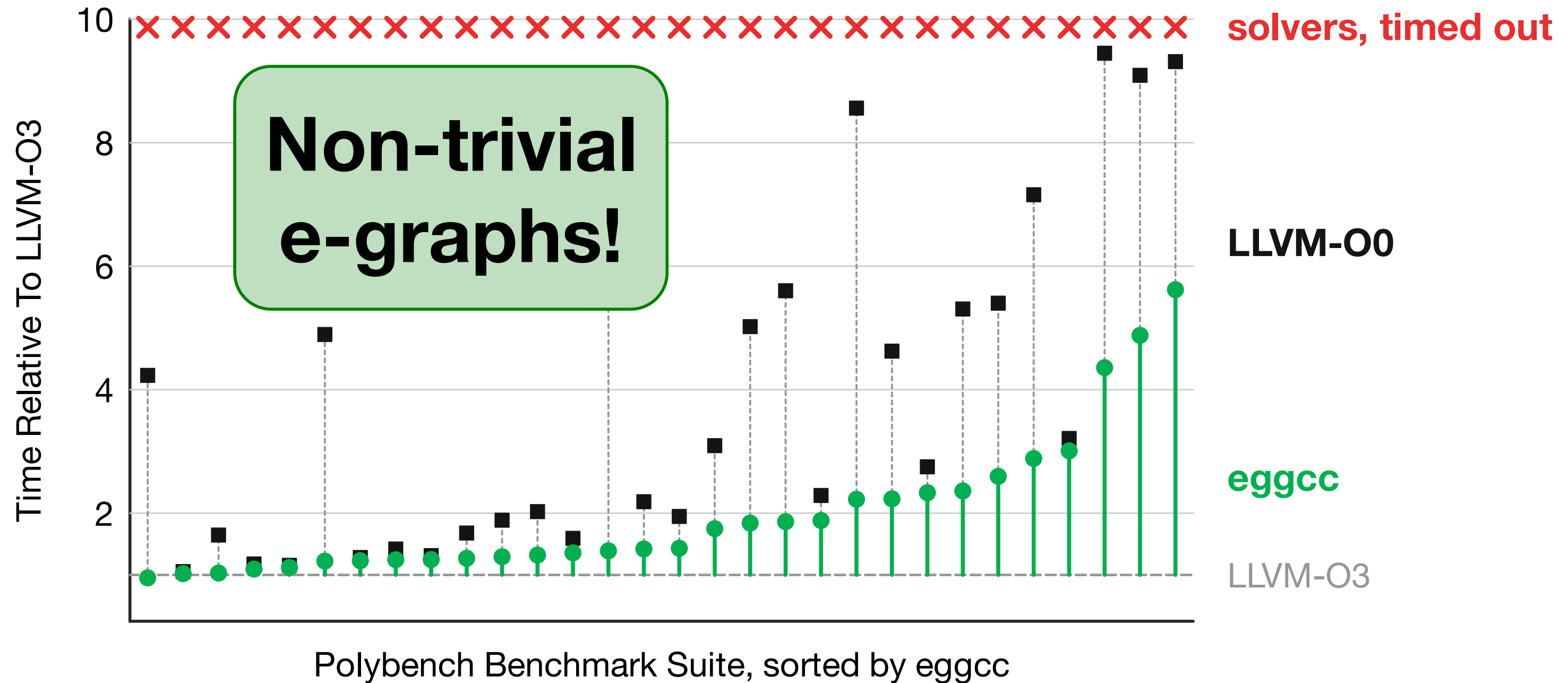
Fast Effect-safe Extraction in Practice



Fast Effect-safe Extraction in Practice



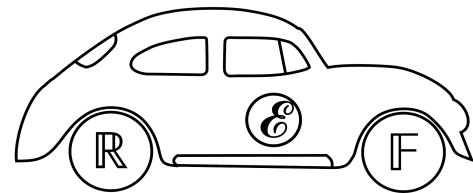
Fast Effect-safe Extraction in Practice



Want: Effectful E-Graphs

E-Graphs are Super Useful!

Floating point

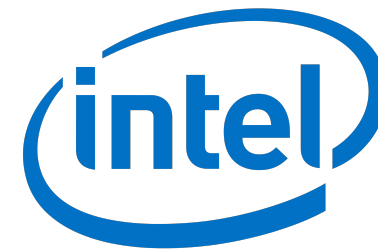


(Herbie)

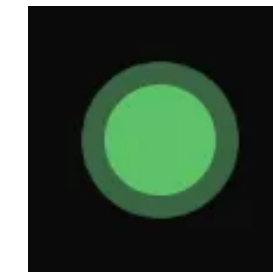
Python Simulations



RTL Optimization



ML Kernels



Luminal

Verification



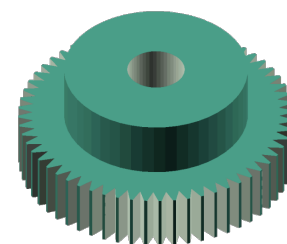
certora

SQL



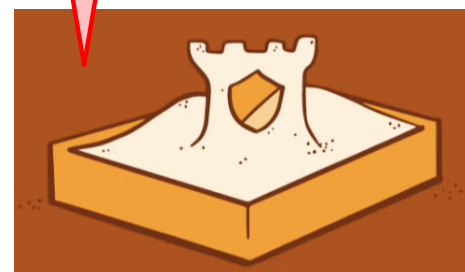
RisingWave

CAD



(Szalinski)

WASM



(Craneflift)

User DSL
Optimizations



...and more!

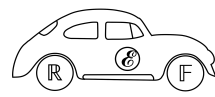


limited to pure
optimizations!

**Want:
Effectful
E-Graphs**

E-Graphs are Super Useful!

Floating point



(Herbie)

Python Simulations



(Szalinski)

RTL Optimization



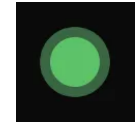
**limited to pure
optimizations!**

WASM



(Cranelift)

ML Kernels



Luminal

User DSL
Optimizations



Verification



...and more!

Problem: Extraction Bottleneck

E-Graphs Explore a Big Space

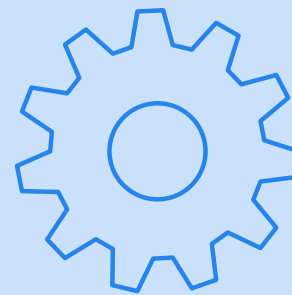
Want
Effectful
E-Graphs

Effects modeled by
threading functional objects!

Translate to
Dataflow IR

Insert Program

Run Rules



Algebraic rules
over dataflow IR



**Bottleneck:
NP-Hard Problem!**
(see paper)

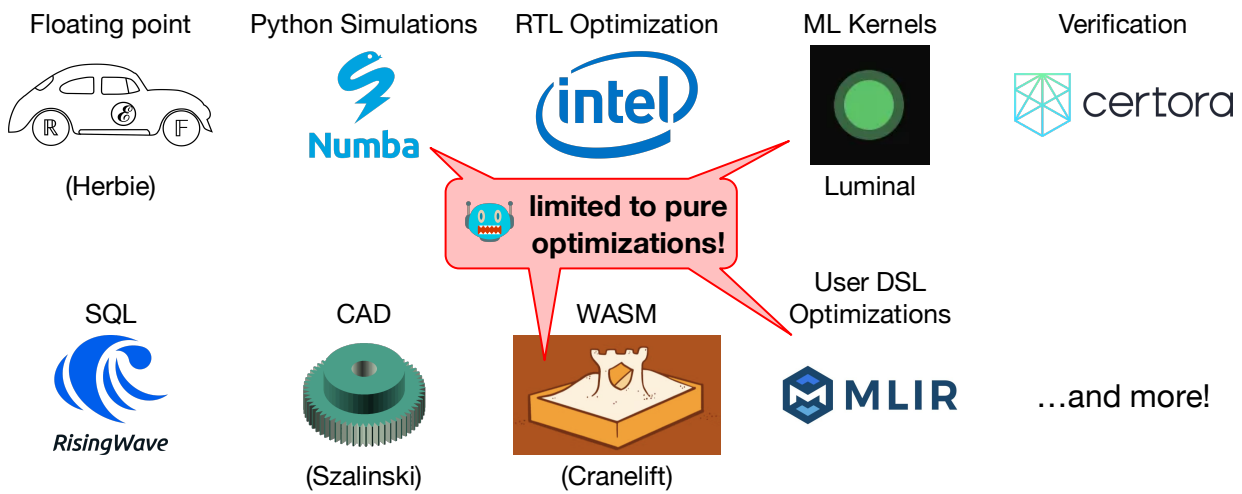
**Effect-Safe
Extraction**

Refine the semantics to
only **linear** programs

Translate to
Original IR

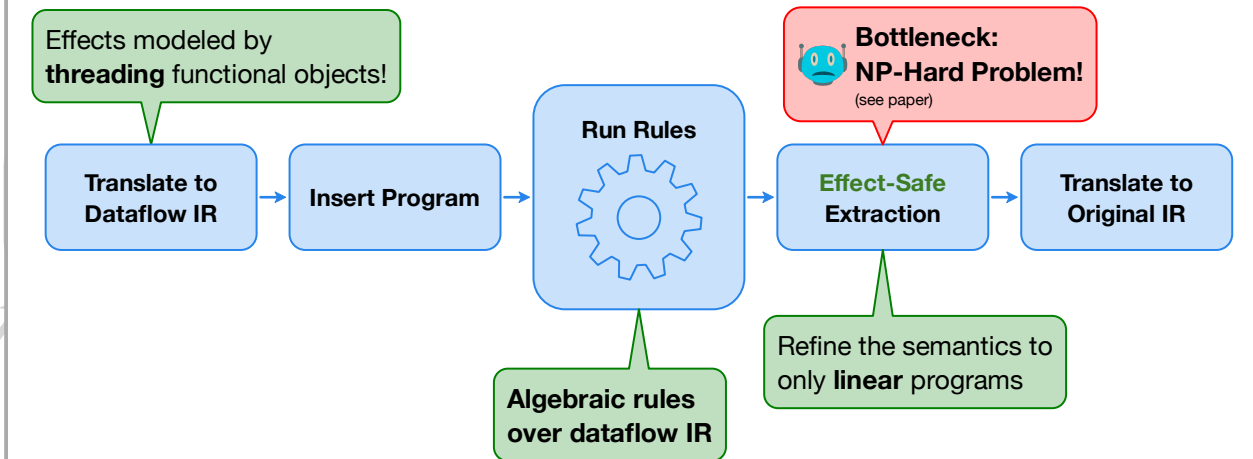
**Want:
Effectful
E-Graphs**

E-Graphs are Super Useful!



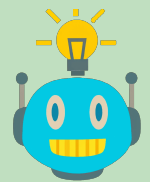
**Problem:
Extraction
Bottleneck**

E-Graphs Explore a Big Space

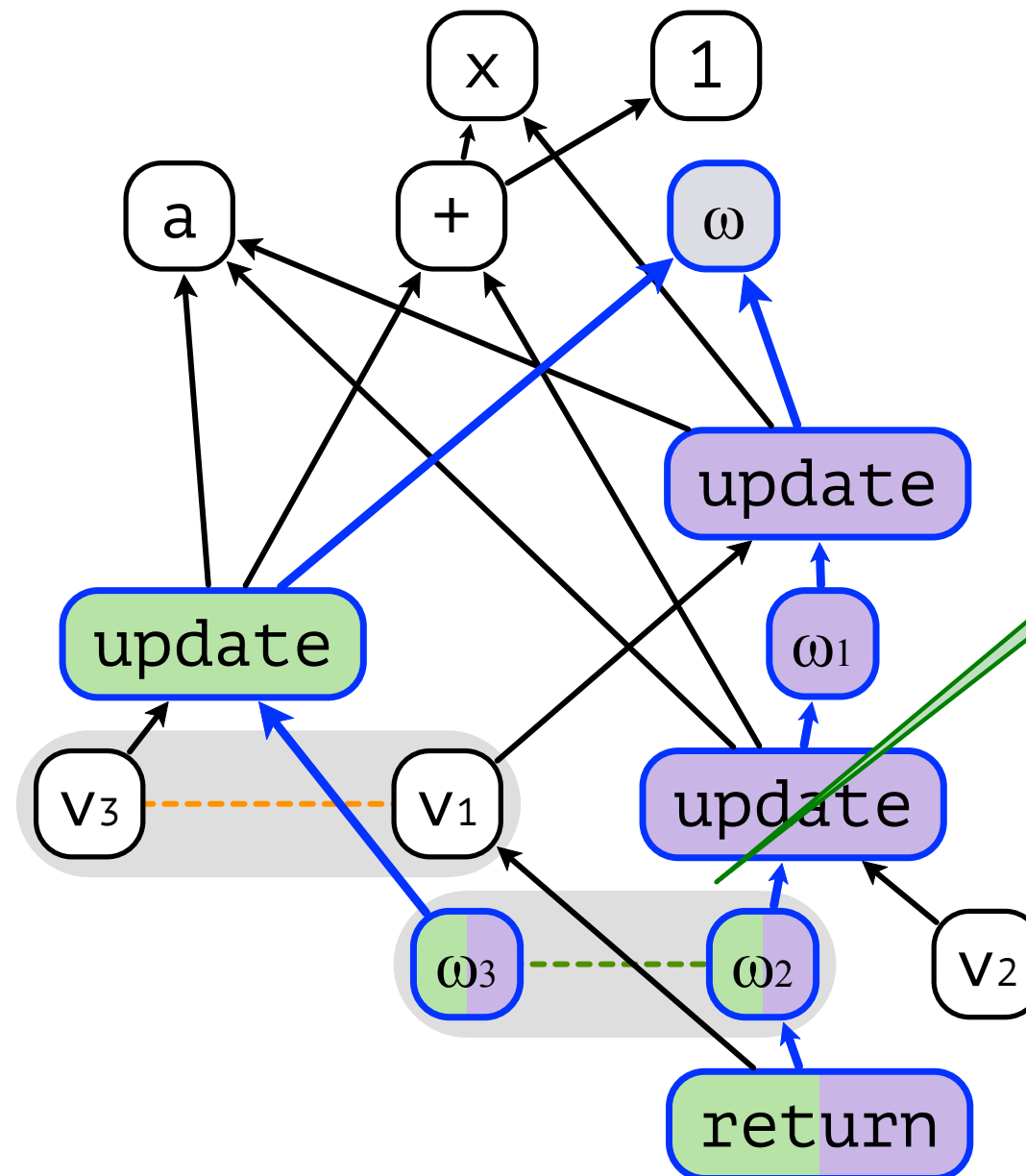


Solution: Statewalk DP

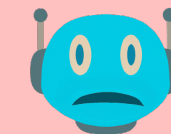
The Statewalk DP Algorithm



Key idea:
keep **multiple**
statewalks
per e-class



Good news:
time bounded by
number of statewalks!



Bad news:
exponential number
of statewalks

Want
Effecti
E-Grap

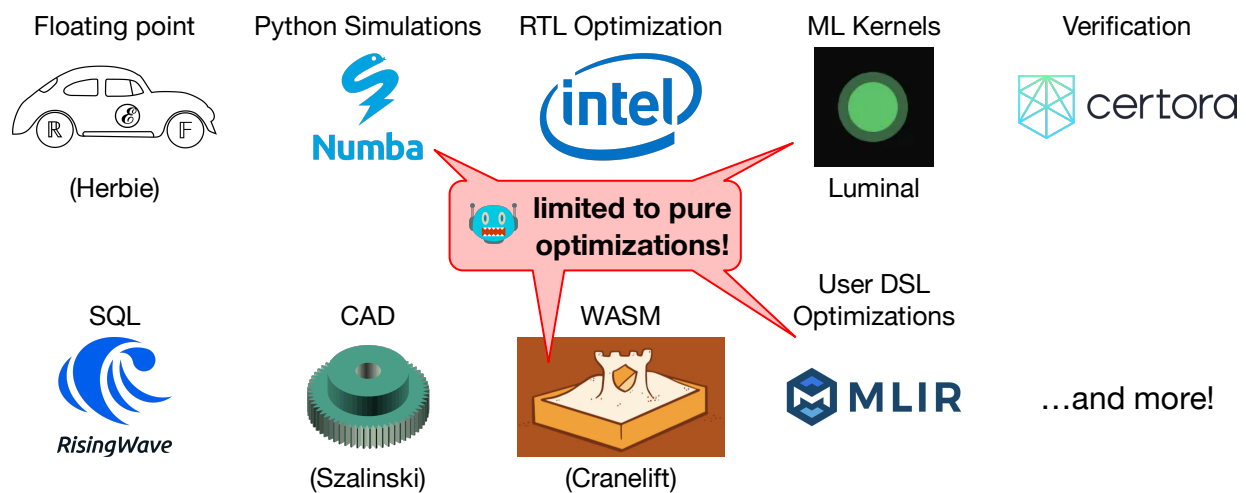
check:
Problem!

Translate to
Original IR

antics to
grams

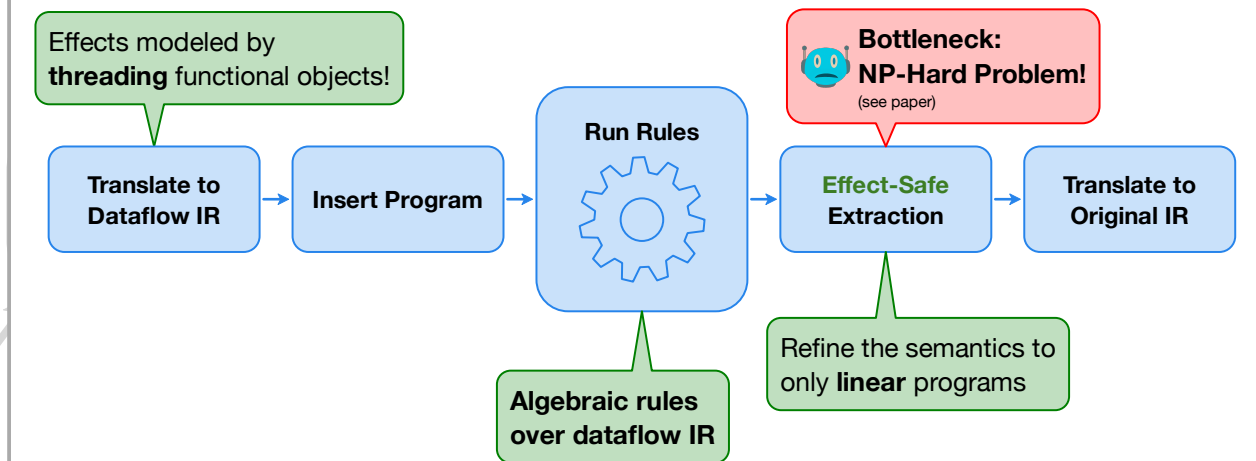
Want:
Effectful
E-Graphs

E-Graphs are Super Useful!

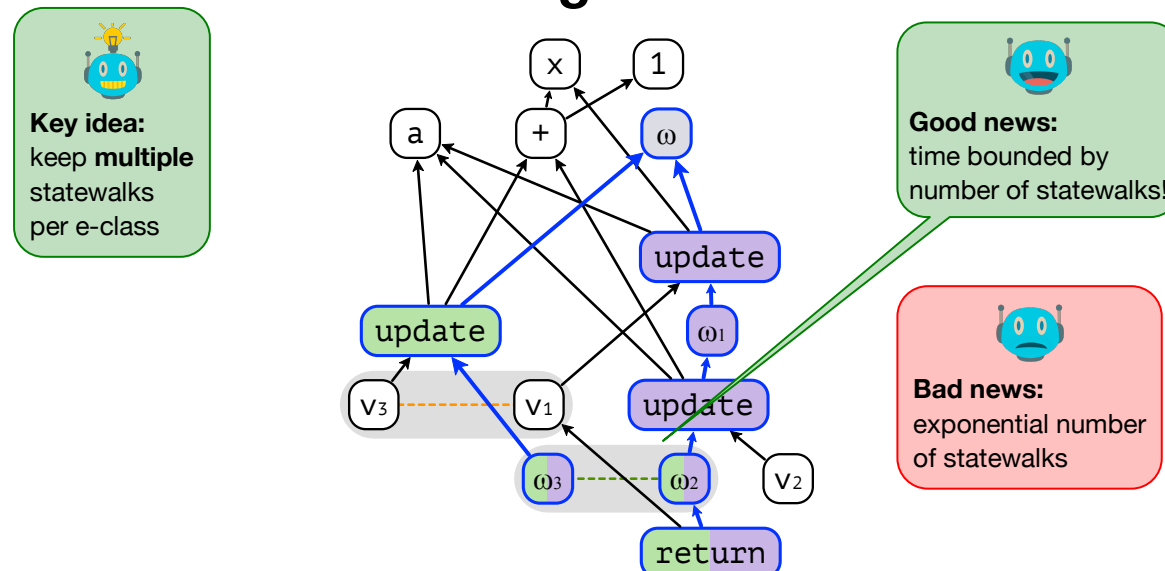


Problem:
Extraction
Bottleneck

E-Graphs Explore a Big Space



The Statewalk DP Algorithm



Solution:
Statewalk DP

Future: Powerful E-Graph Optimizers

Want
Effective
E-Graph

Solution
Statewalk I

Check:
No Problem!

Translate to
Original IR

Antics to
grams

Future: Powerful E-Graph Optimizers

Want
Effective
E-Graph



Solution
Statewalk I

Check:
Problem!

Translate to
Original IR

Antics to
grams

Future: Powerful E-Graph Optimizers



Want
Effective
E-Graph

Solution
Statewalk I

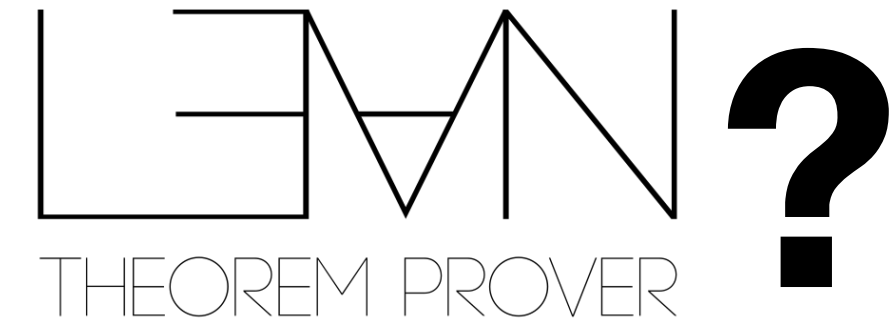
Check:
Problem!

Translate to
Original IR

Antics to
grams

Future: Powerful E-Graph Optimizers

Want
Effective
E-Graph



Check:
Problem!

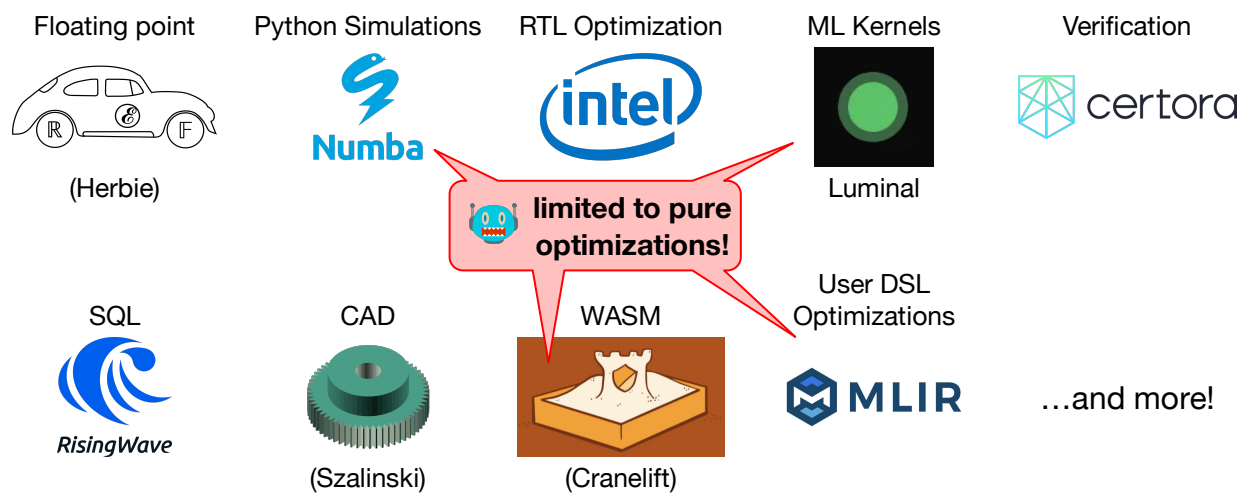
Translate to
Original IR

Antics to
grams

Solution
Statewalk I

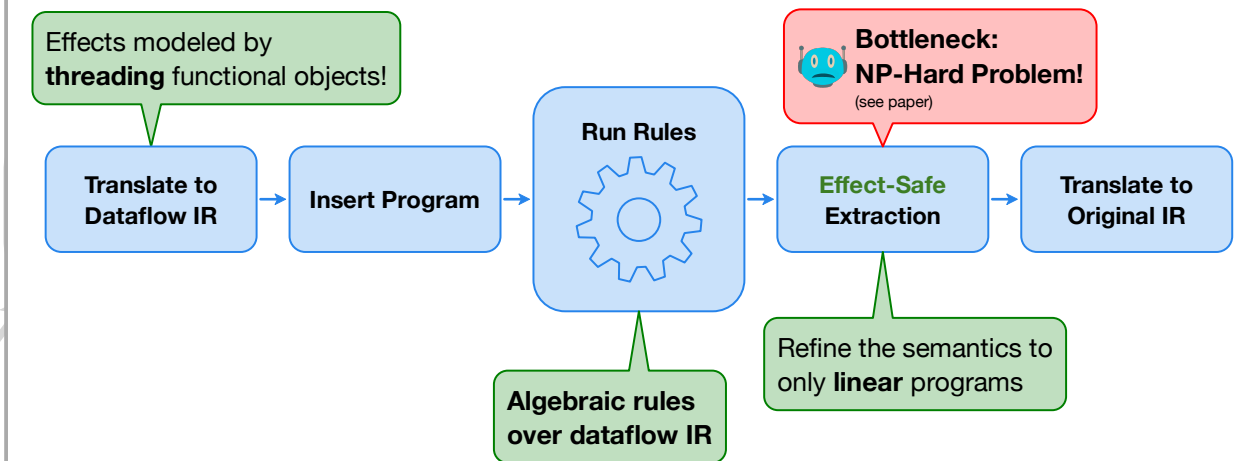
Want:
Effectful
E-Graphs

E-Graphs are Super Useful!

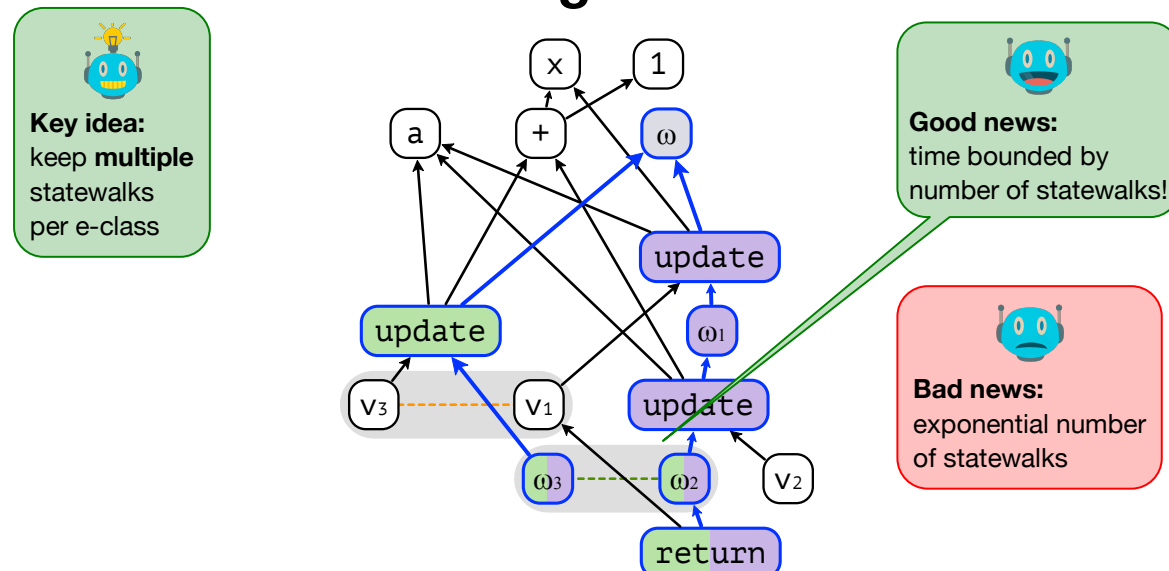


Problem:
Extraction
Bottleneck

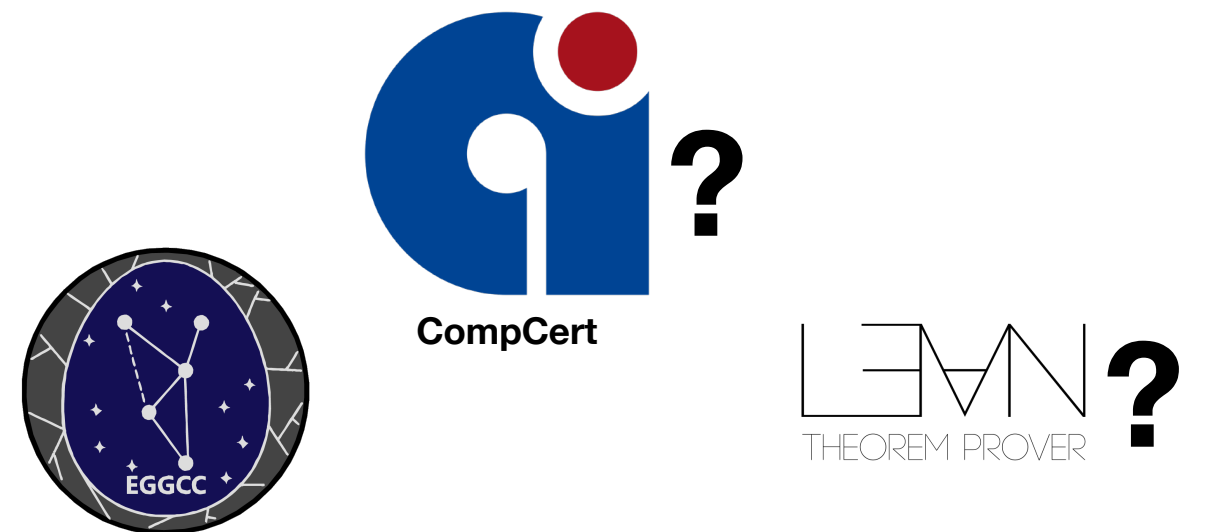
E-Graphs Explore a Big Space



The Statewalk DP Algorithm

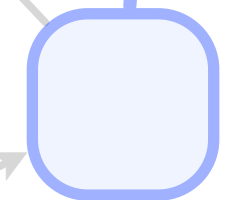
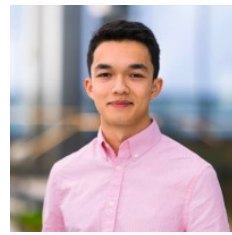


Future:
Powerful
E-Graph
Optimizers



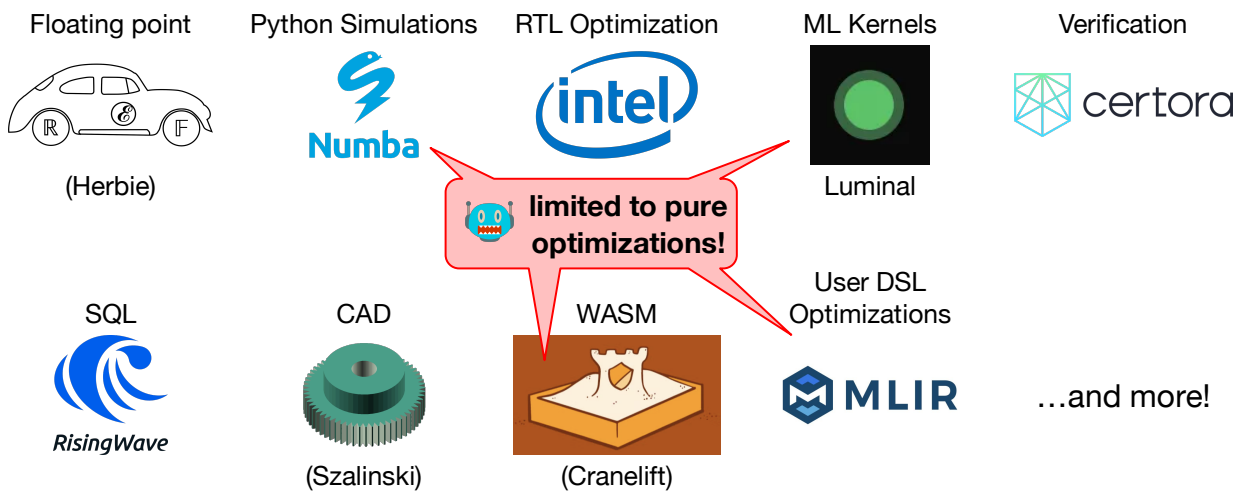
Solution:
Statewalk DP

Efficient Extraction from Effectful E-Graphs



Want:
Effectful
E-Graphs

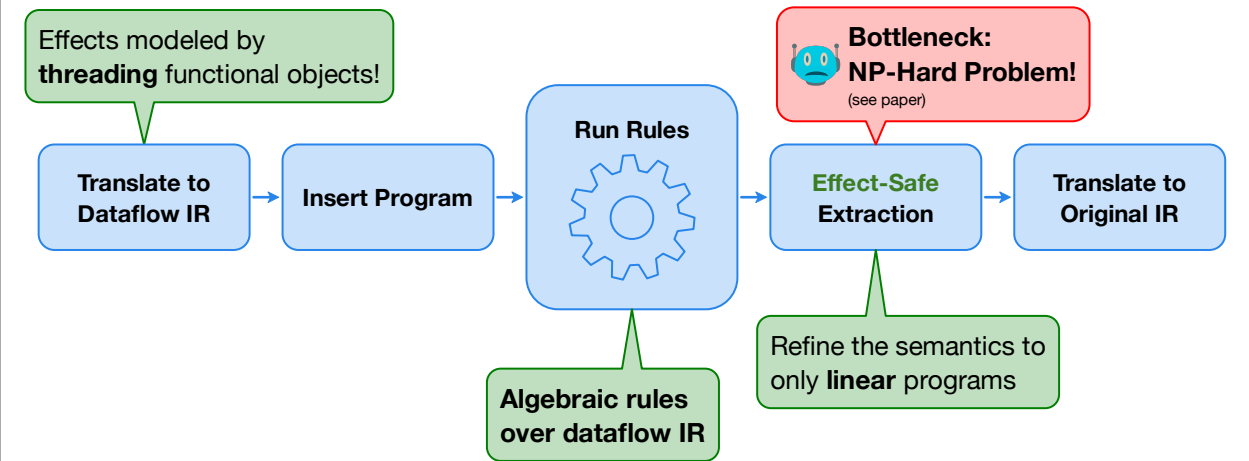
E-Graphs are Super Useful!



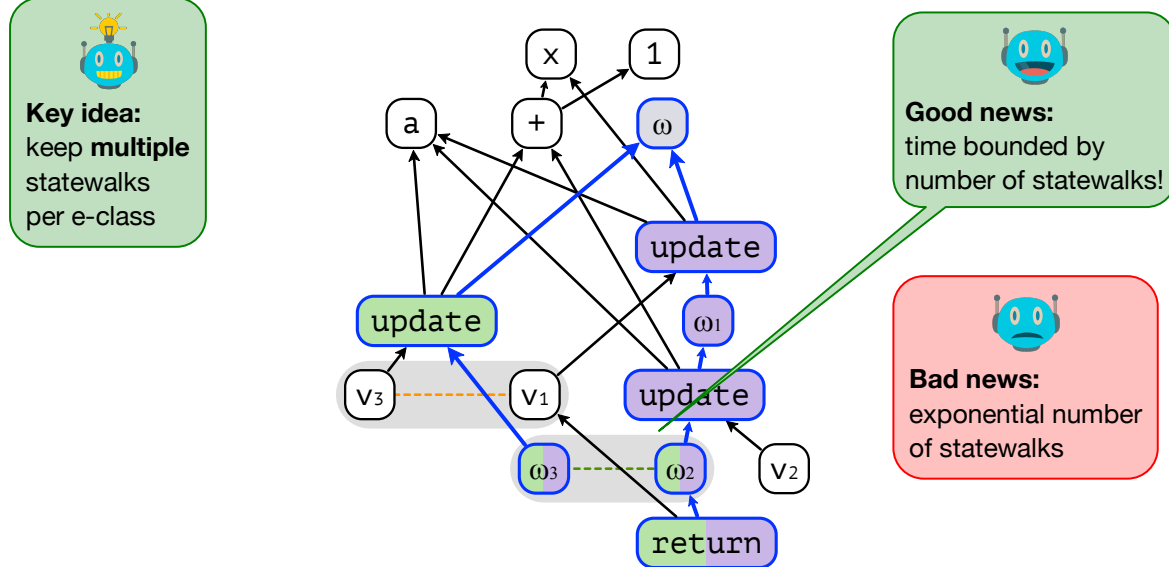
Problem:

Extraction Bottleneck

E-Graphs Explore a Big Space



The Statewalk DP Algorithm



Solution:
Statewalk DP

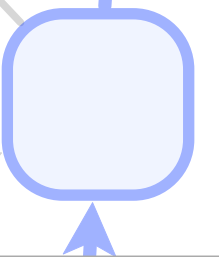
Future:
Powerful
E-Graph
Optimizers



LEMN ?

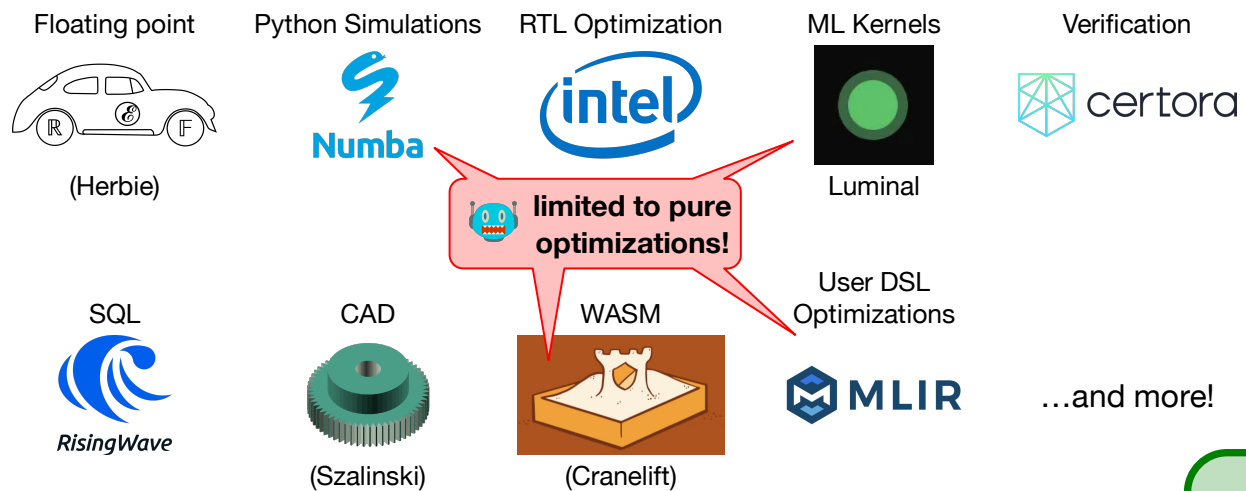
THEOREM PROVER

Efficient Extraction from Effectful E-Graphs



Want:
Effectful
E-Graphs

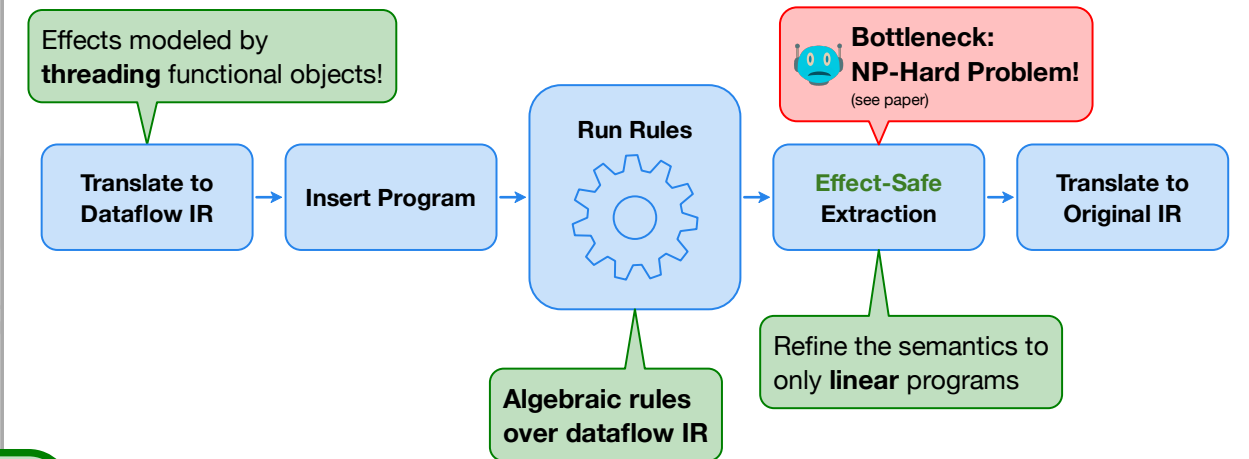
E-Graphs are Super Useful!



Problem:

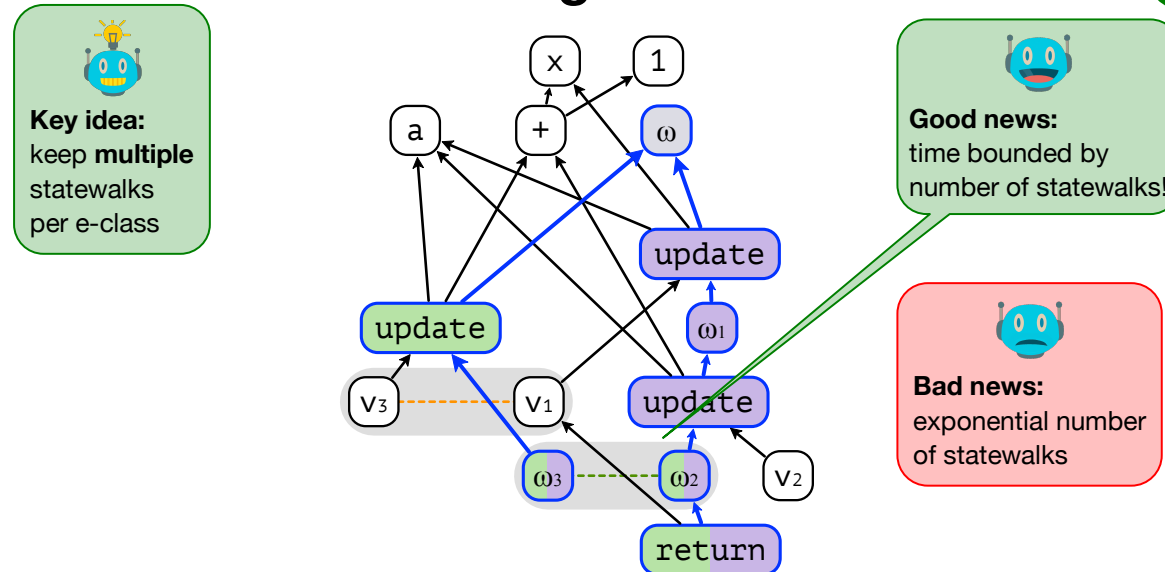
Extraction Bottleneck

E-Graphs Explore a Big Space



Questions?

The Statewalk DP Algorithm



Solution:
Statewalk DP

Future:
Powerful
E-Graph
Optimizers



Effect-Safe Extraction is NP-Hard

Effect-Safe Extraction is NP-Hard

$$\phi = \neg x_0 \wedge (\neg x_1 \vee x_0) \wedge (x_2 \vee x_1 \vee \neg x_2)$$

Effect-Safe Extraction is NP-Hard

$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

X_0

$\neg X_0$

X_1

$\neg X_1$

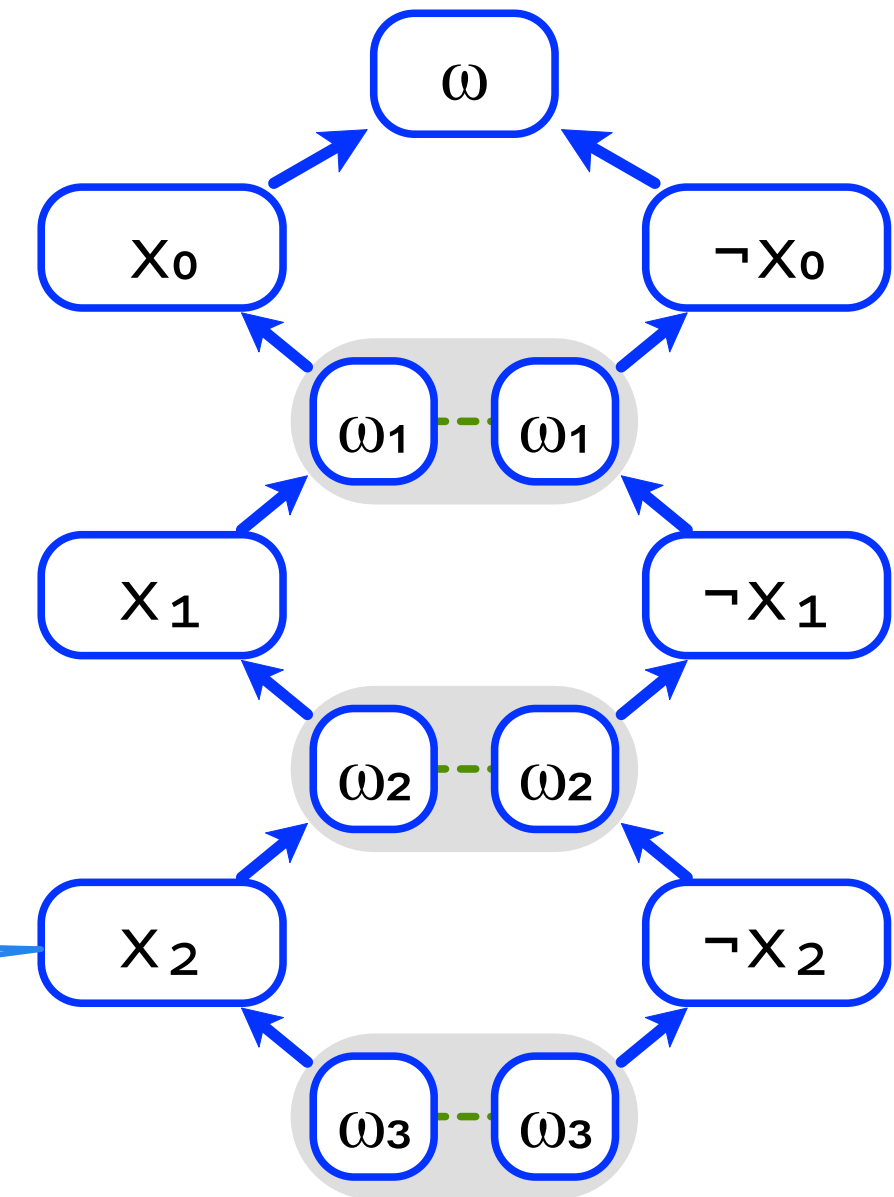
X_2

$\neg X_2$

Effect-Safe Extraction is NP-Hard

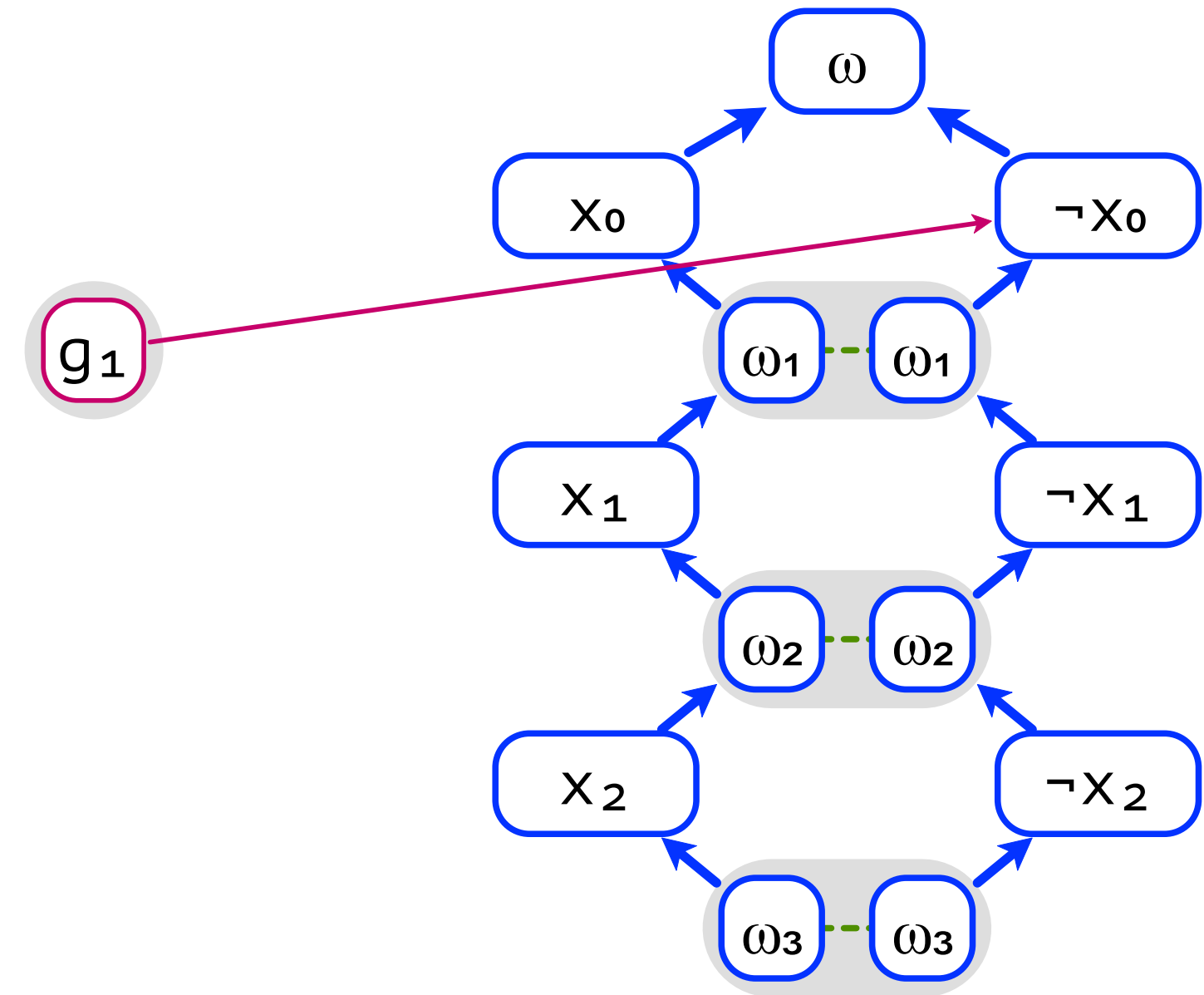
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

state walks encode
assignments



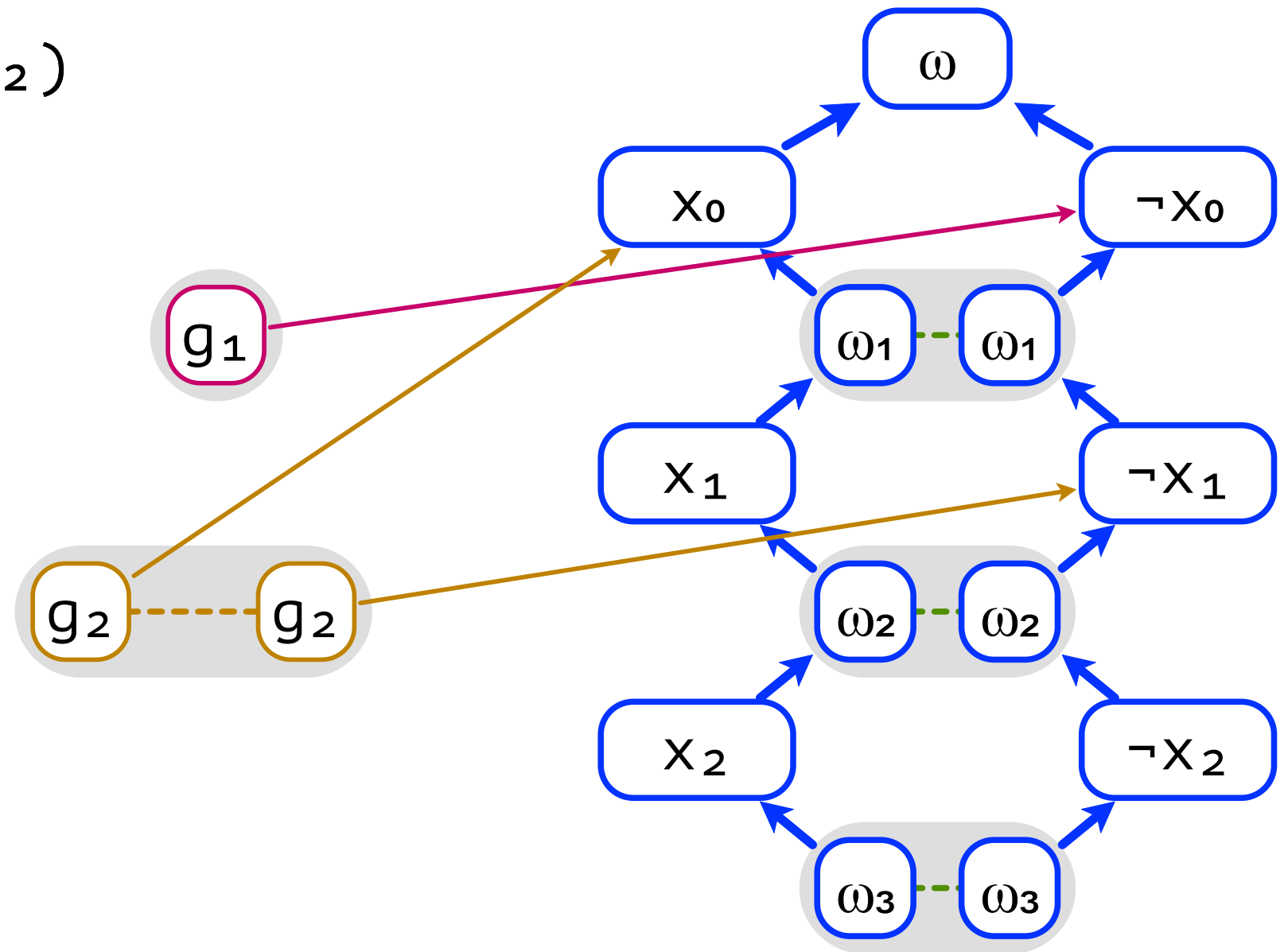
Effect-Safe Extraction is NP-Hard

$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



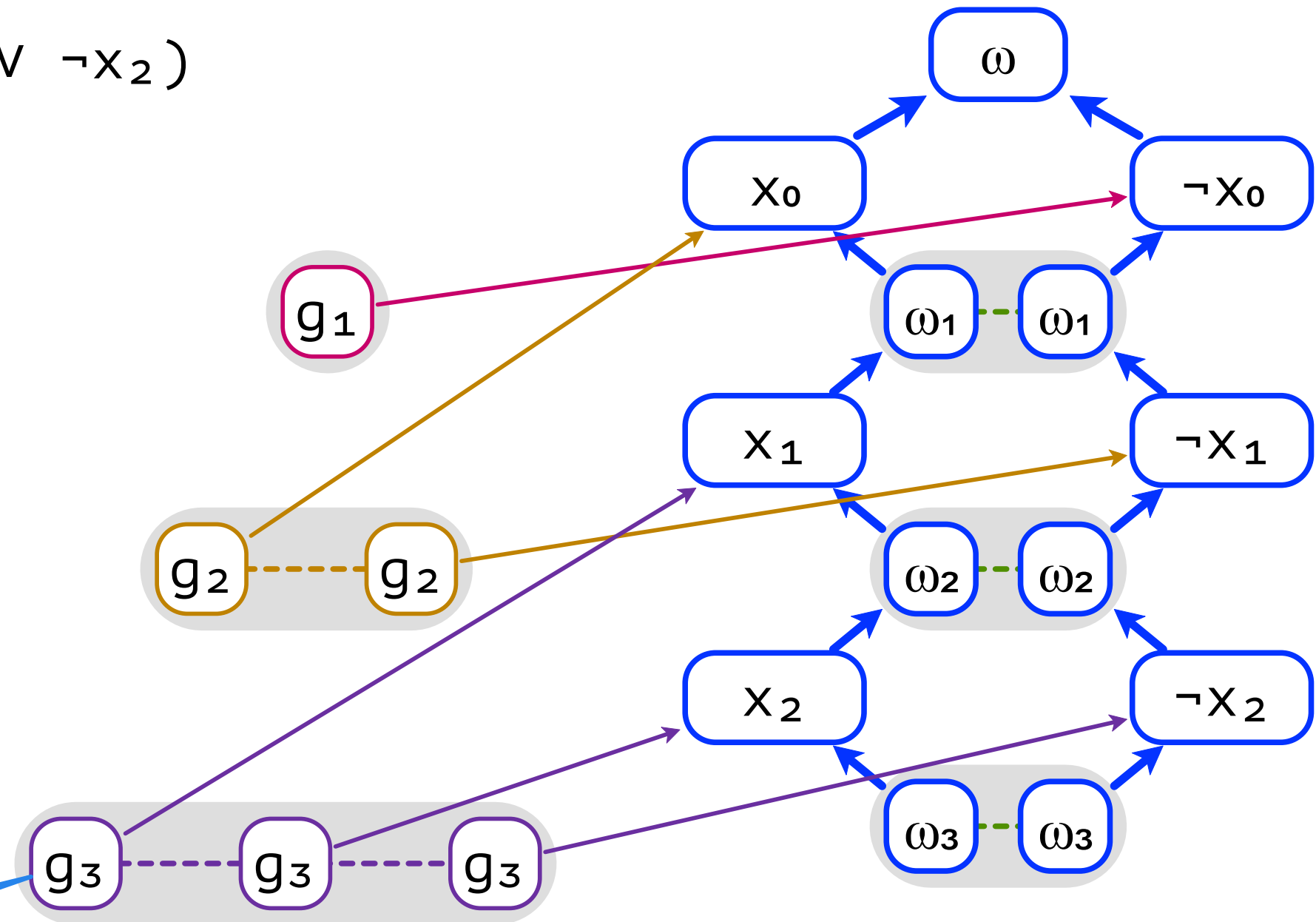
Effect-Safe Extraction is NP-Hard

$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



Effect-Safe Extraction is NP-Hard

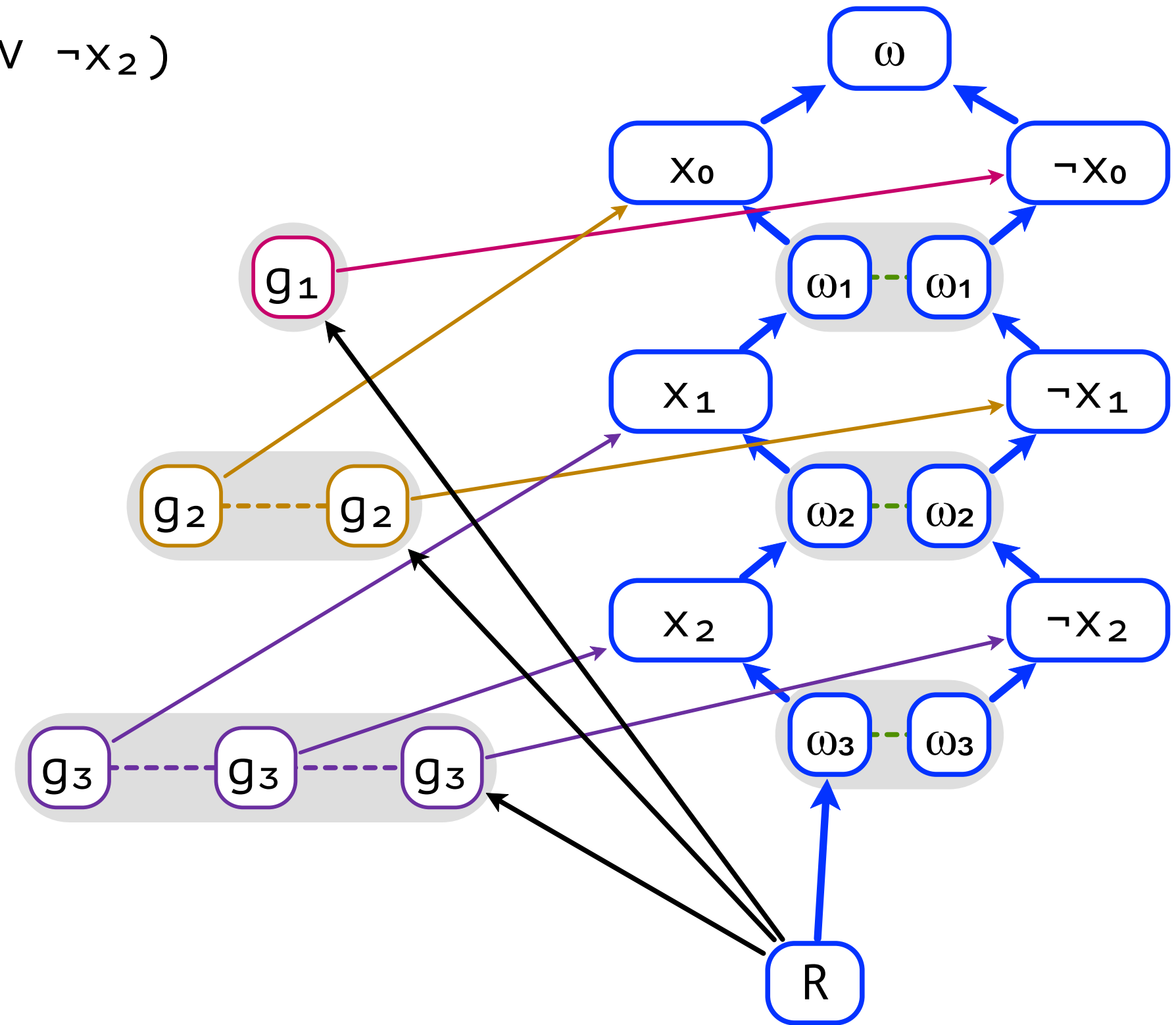
$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$



e-classes encode clauses

Effect-Safe Extraction is NP-Hard

$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

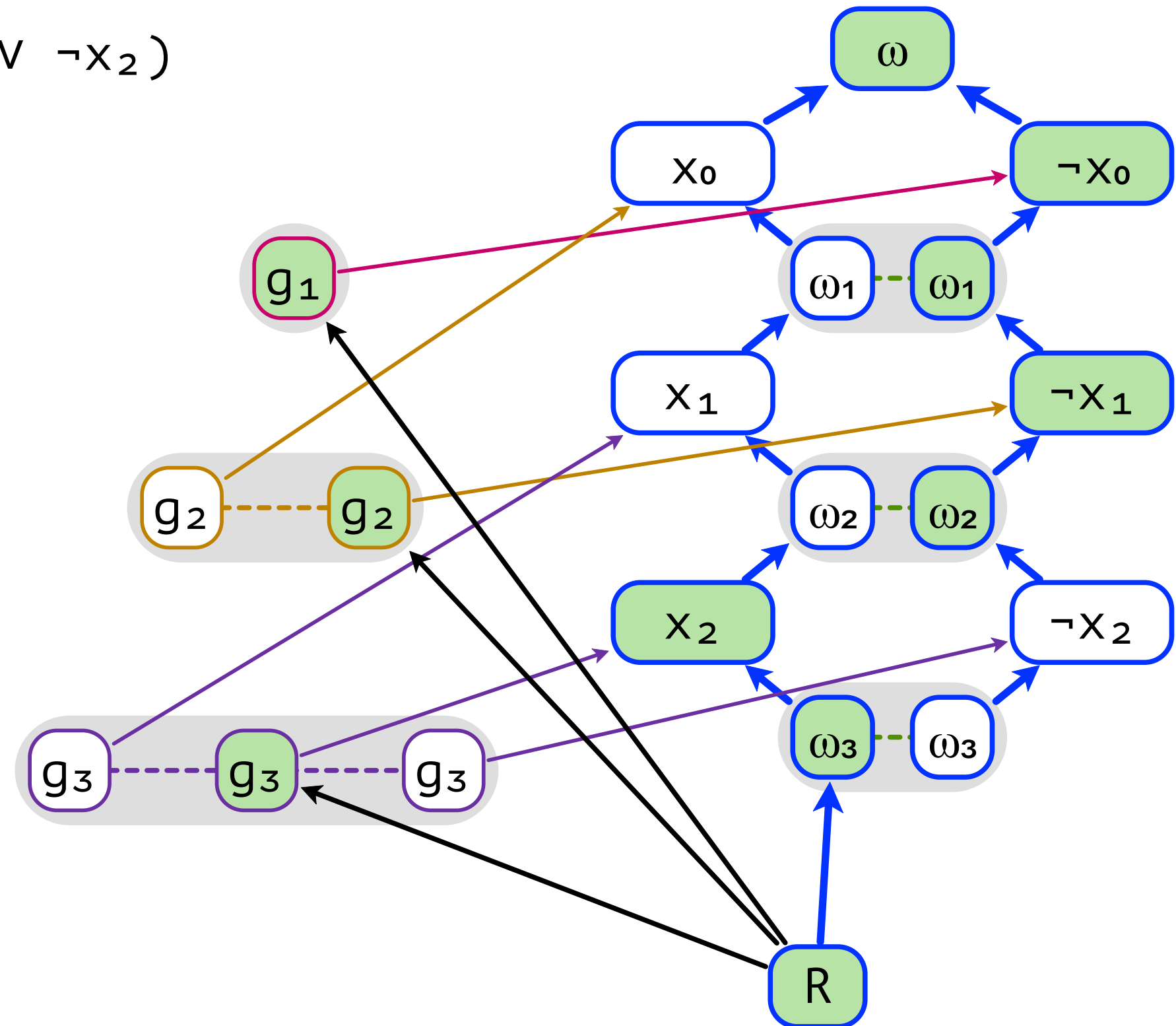


Effect-Safe Extraction is NP-Hard

$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

Satisfying assignment:

$$X_0 = \perp \quad X_1 = \perp \quad X_2 = \top$$

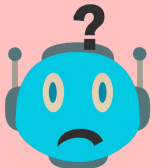


Effect-Safe Extraction is NP-Hard

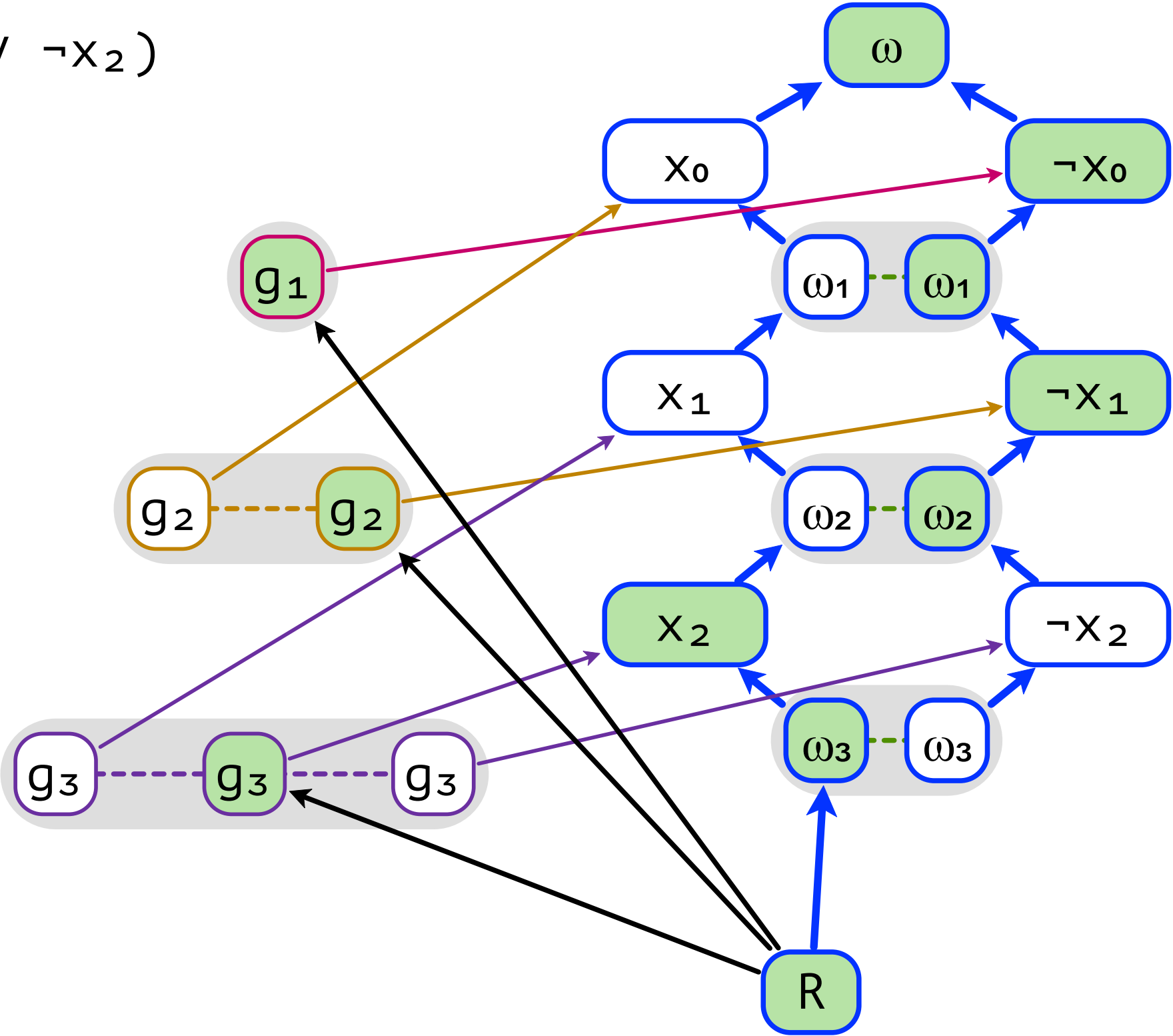
$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$

Satisfying assignment:

$X_0 = \perp \qquad X_1 = \perp \qquad X_2 = \top$



Problem is NP-hard!
Give up??

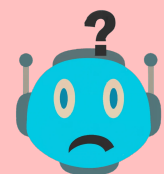


Effect-Safe Extraction is NP-Hard

$$\phi = \neg X_0 \wedge (\neg X_1 \vee X_0) \wedge (X_2 \vee X_1 \vee \neg X_2)$$

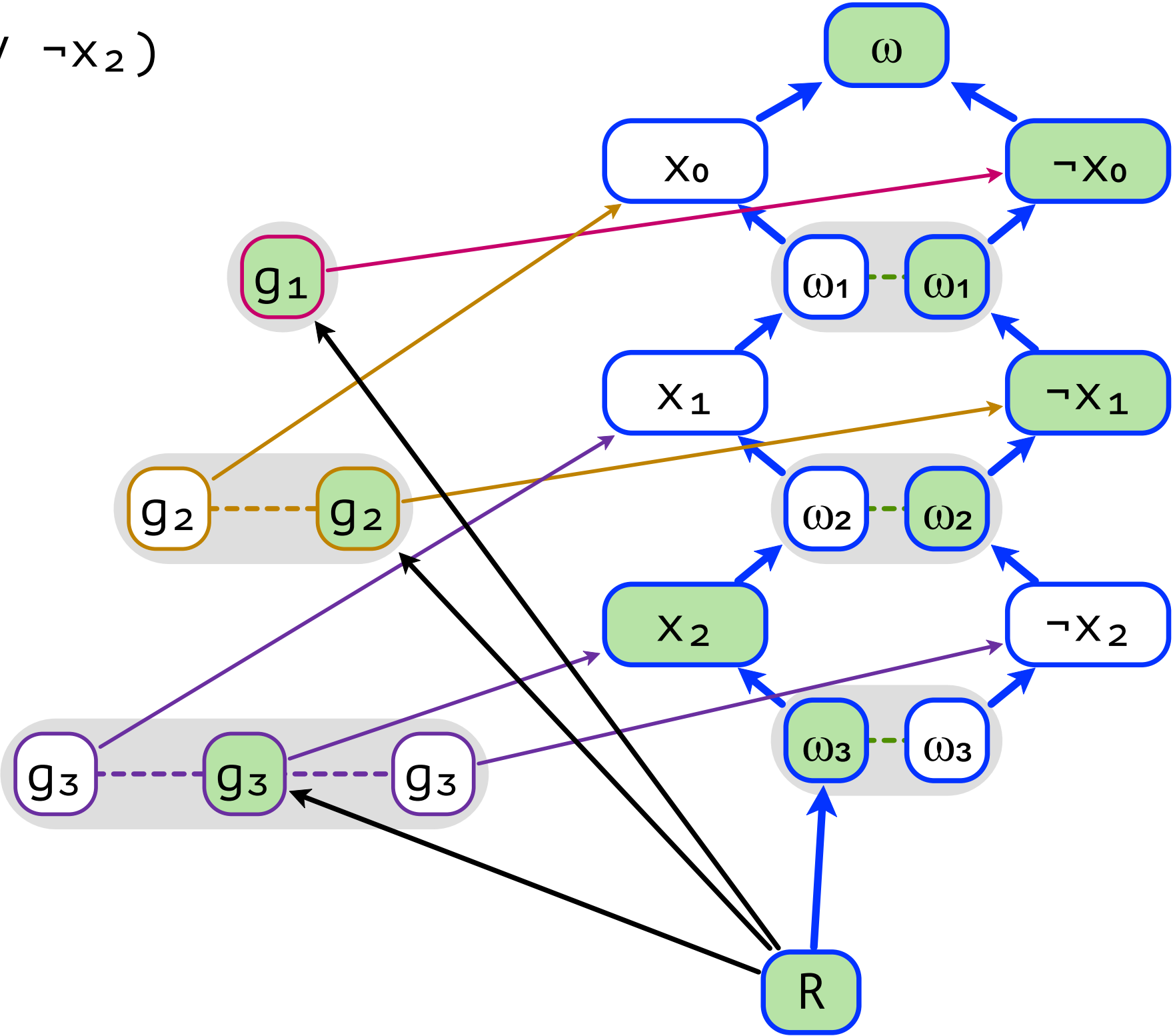
Satisfying assignment:

$$X_0 = \perp \qquad X_1 = \perp \qquad X_2 = \top$$



Problem is NP-hard!
Give up??

No! Real-world e-graphs
are easier, measured
with a *parameter*



eggcc vs LLVM Optimizations

eggcc vs LLVM Optimizations

eggcc implements

optimizations

- redundant load elimination
- load-after-store forwarding
- dead loop deletion
- loop invariant code motion
- conditional invariant code motion
- loop unswitching
- conditional splitting
- loop strength reduction
- loop unrolling
- flow-sensitive constant folding
- if conversion
- function inlining (two levels)
- redundant branch elimination
- peephole optimizations

analyses

- interval analysis
- pointer analysis (simple)
- loop count analysis
- loop invariant analysis
- purity analysis

eggcc vs LLVM Optimizations

eggcc implements

optimizations

- redundant load elimination
- load-after-store forwarding
- dead loop deletion
- loop invariant code motion
- conditional invariant code motion
- loop unswitching
- conditional splitting
- loop strength reduction
- loop unrolling
- flow-sensitive constant folding
- if conversion
- function inlining (two levels)
- redundant branch elimination
- peephole optimizations

analyses

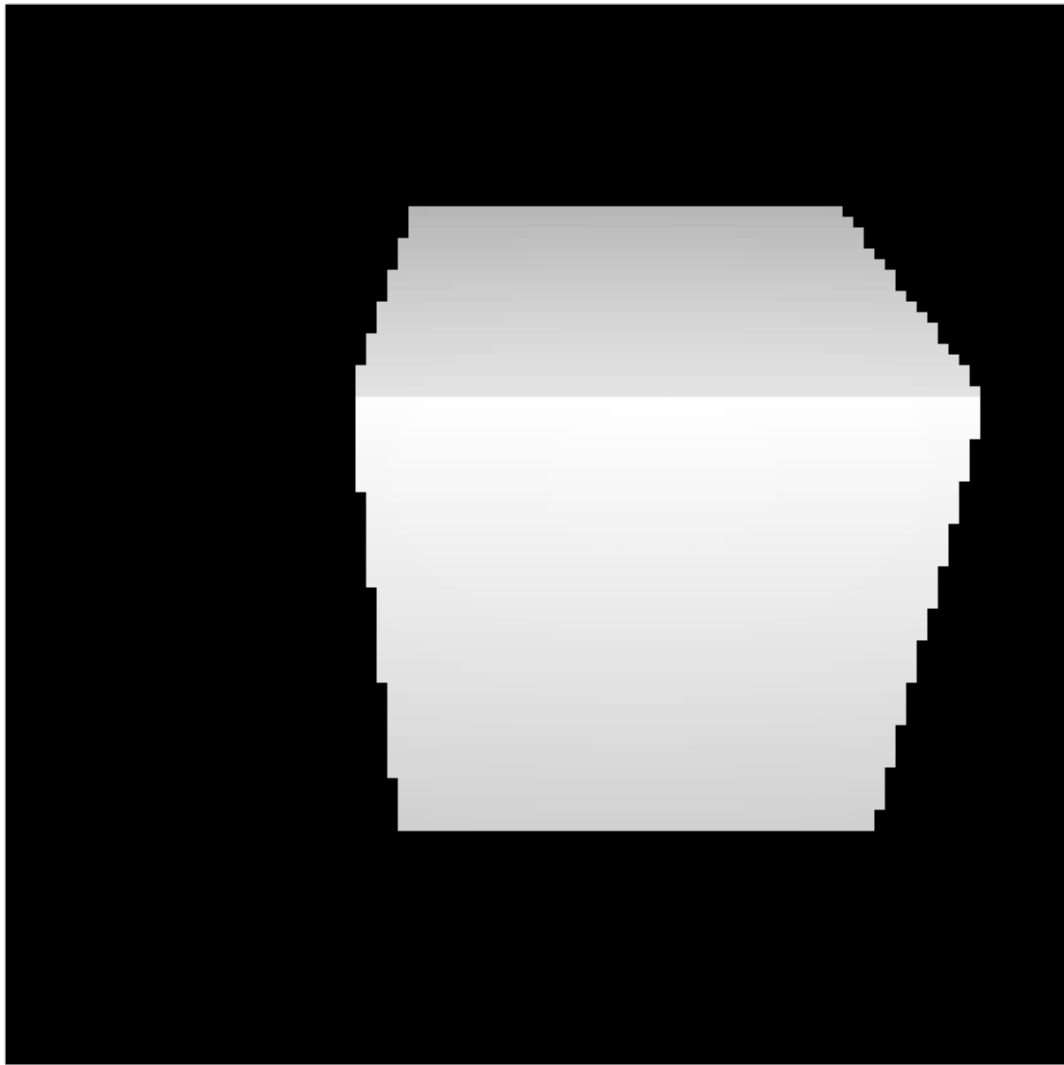
- interval analysis
- pointer analysis (simple)
- loop count analysis
- loop invariant analysis
- purity analysis

Missing, vs LLVM

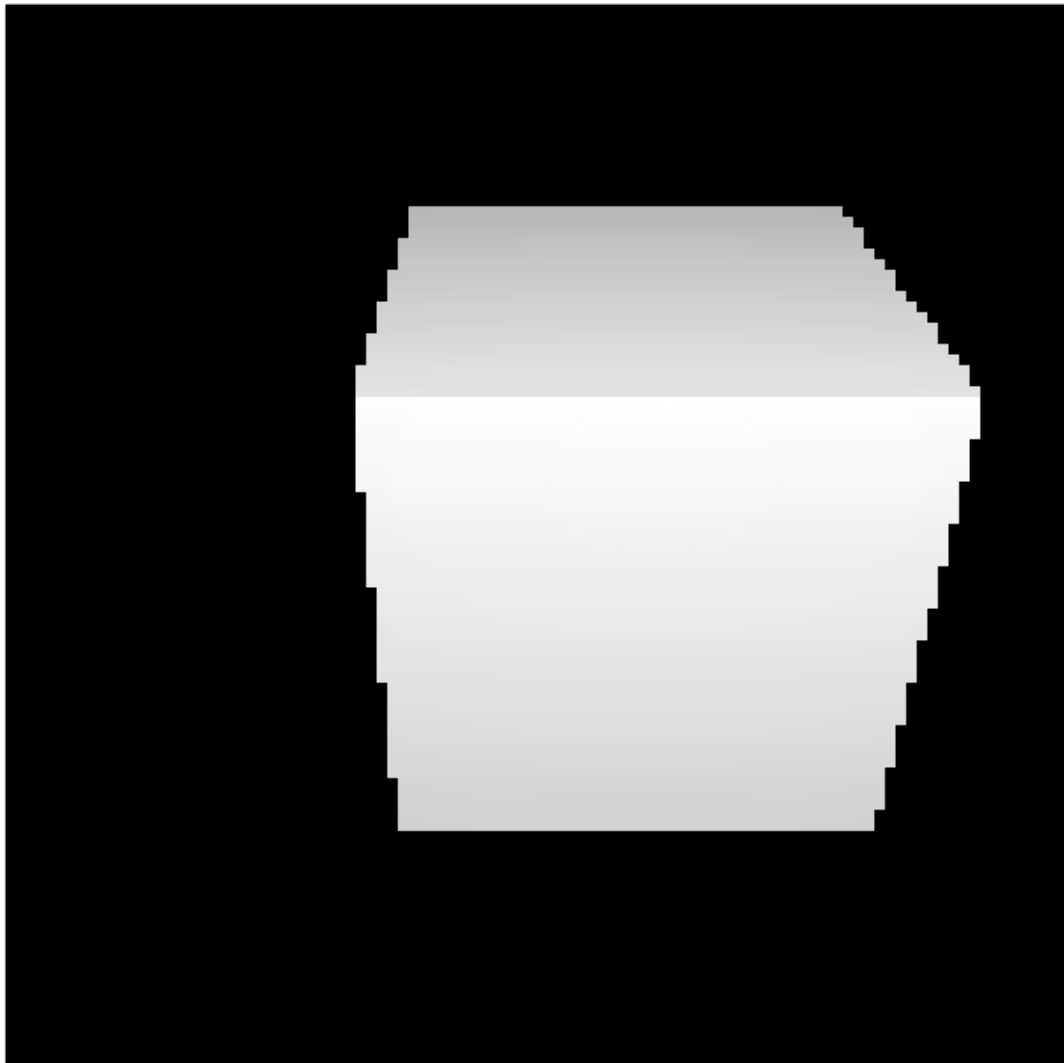
- scalar promotion of memory
- generalized loop unrolling
- stack coloring (reuse of allocations)
- aggressive inlining
- loop peeling
- branch hoisting
- if-true specialization
- many peepholes, e.g. $a - (a/2) \cdot 2 \rightarrow a \ \& \ 1$

eggcc Scales to Big-ish Programs

eggcc Scales to Big-ish Programs

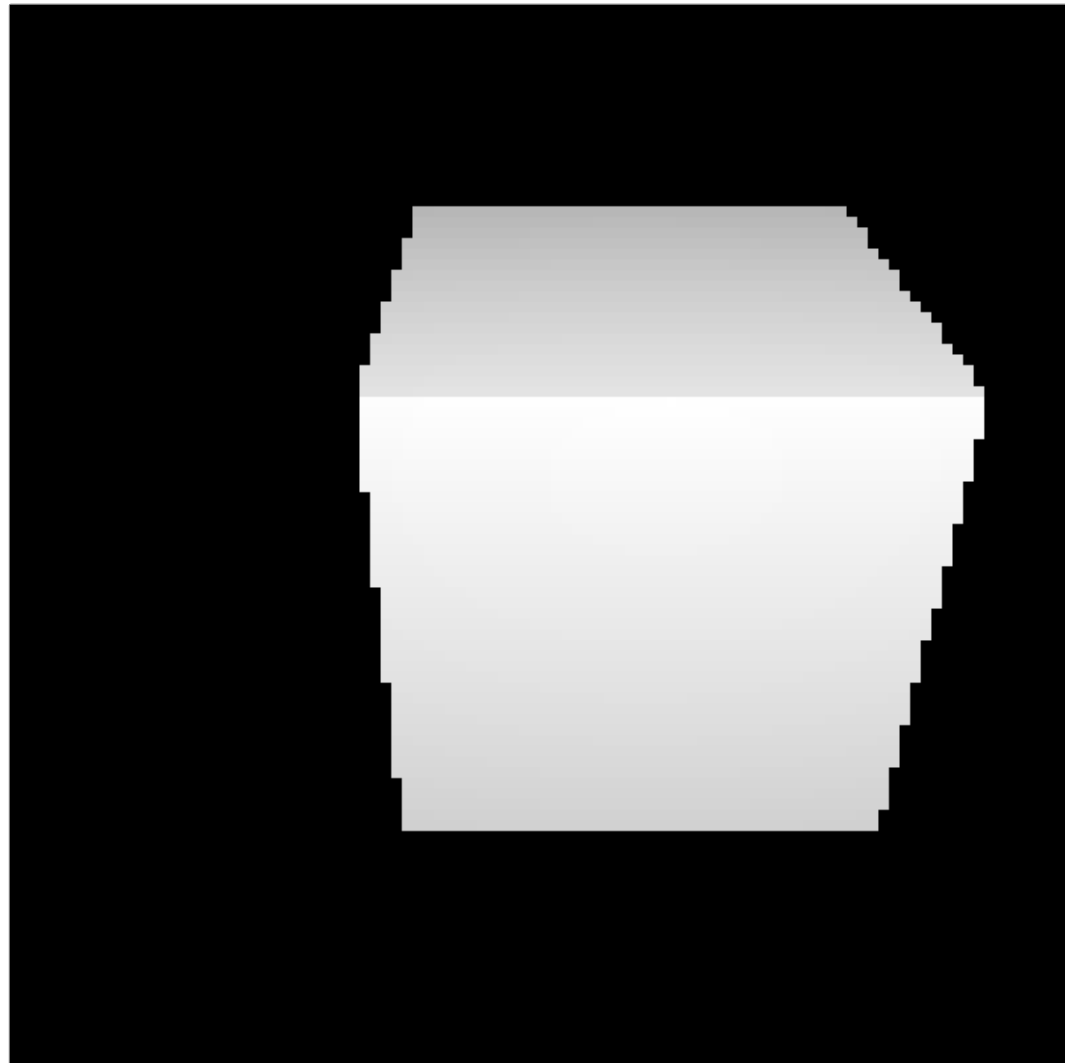


eggcc Scales to Big-ish Programs



a raytracer: 821 lines of Rust, 2503 of Bril

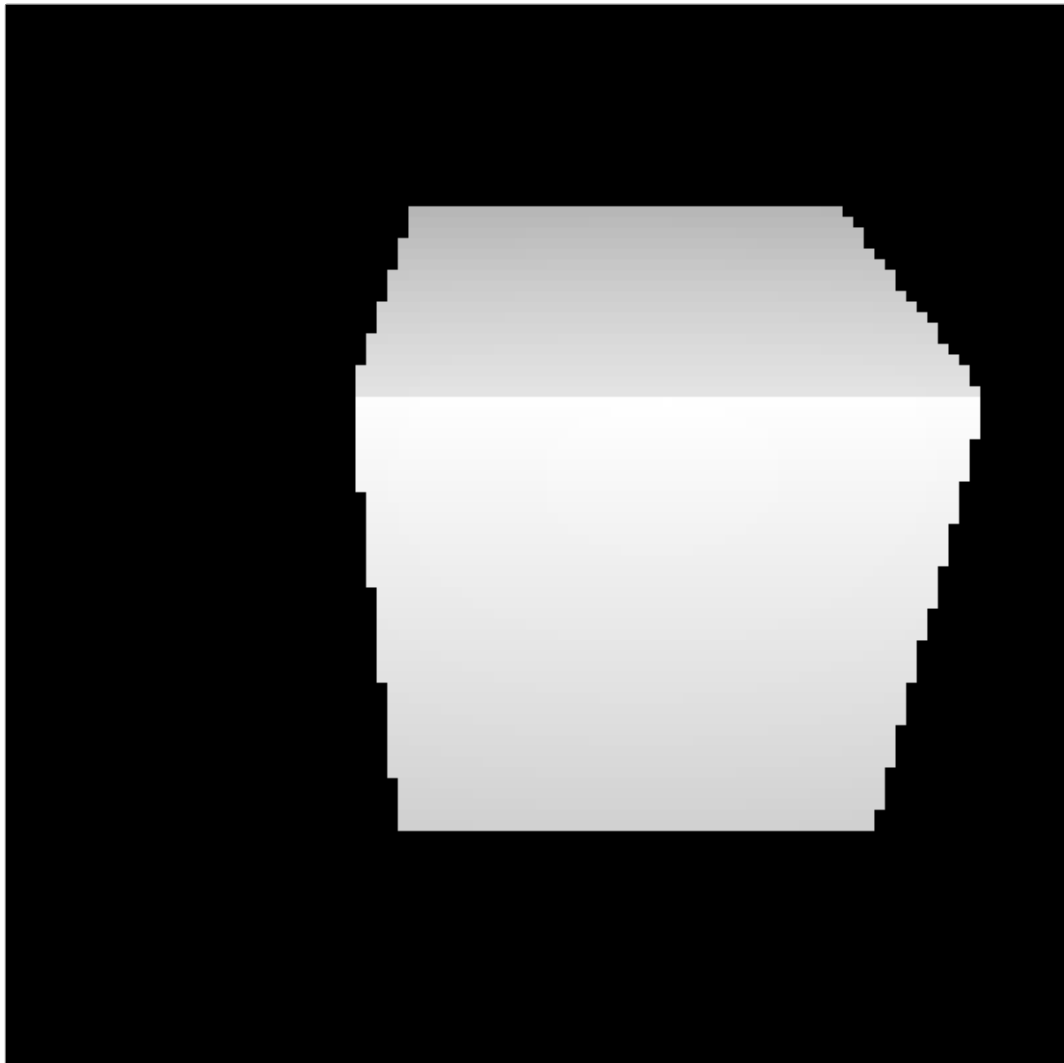
eggcc Scales to Big-ish Programs



a raytracer: 821 lines of Rust, 2503 of Bril

724 regions to extract from, the largest of **4,016 terms**

eggcc Scales to Big-ish Programs

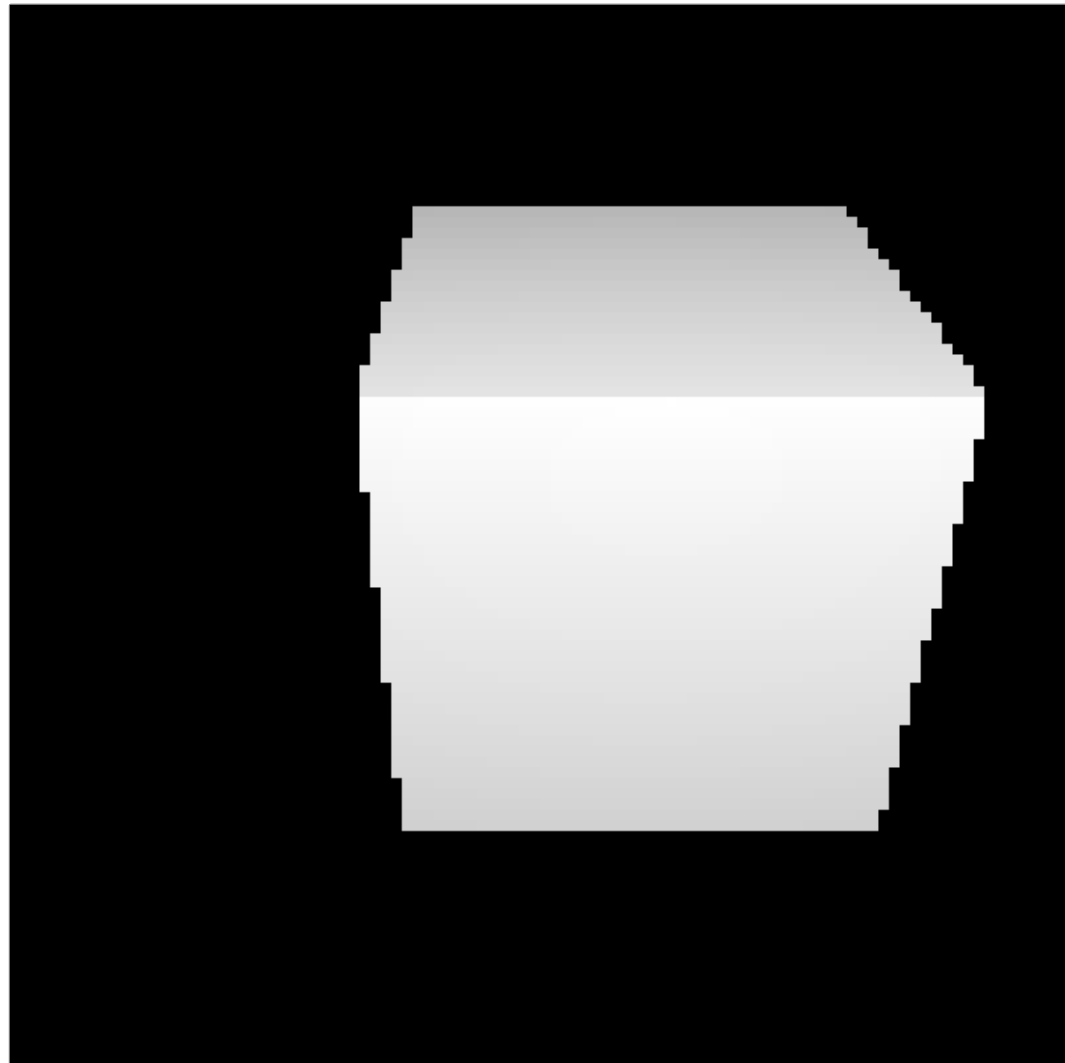


a raytracer: 821 lines of Rust, 2503 of Bril

724 regions to extract from, the largest of **4,016 terms**

compiled in **3¼ minutes**, 23 s of it extraction

eggcc Scales to Big-ish Programs



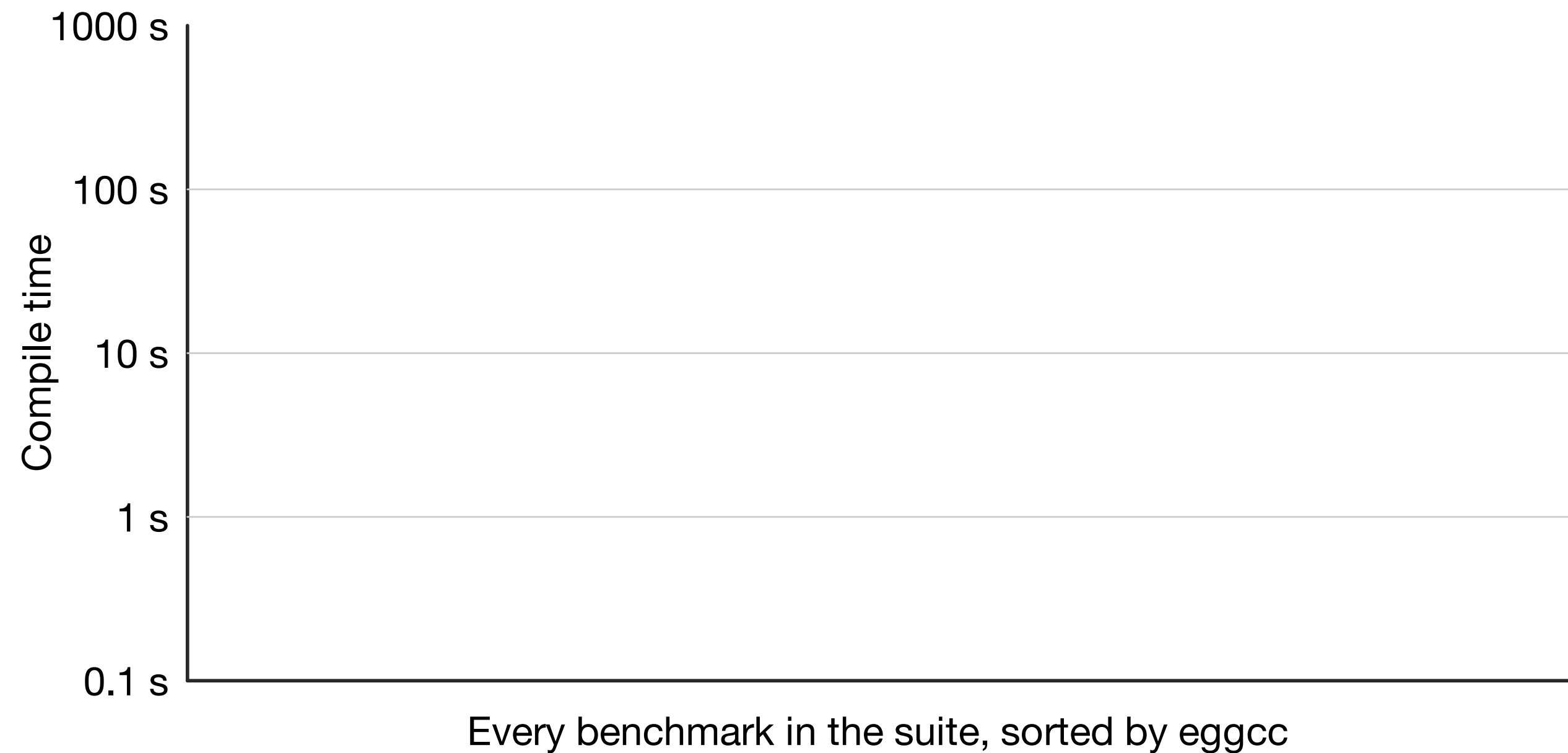
a raytracer: 821 lines of Rust, 2503 of Bril

724 regions to extract from, the largest of **4,016 terms**

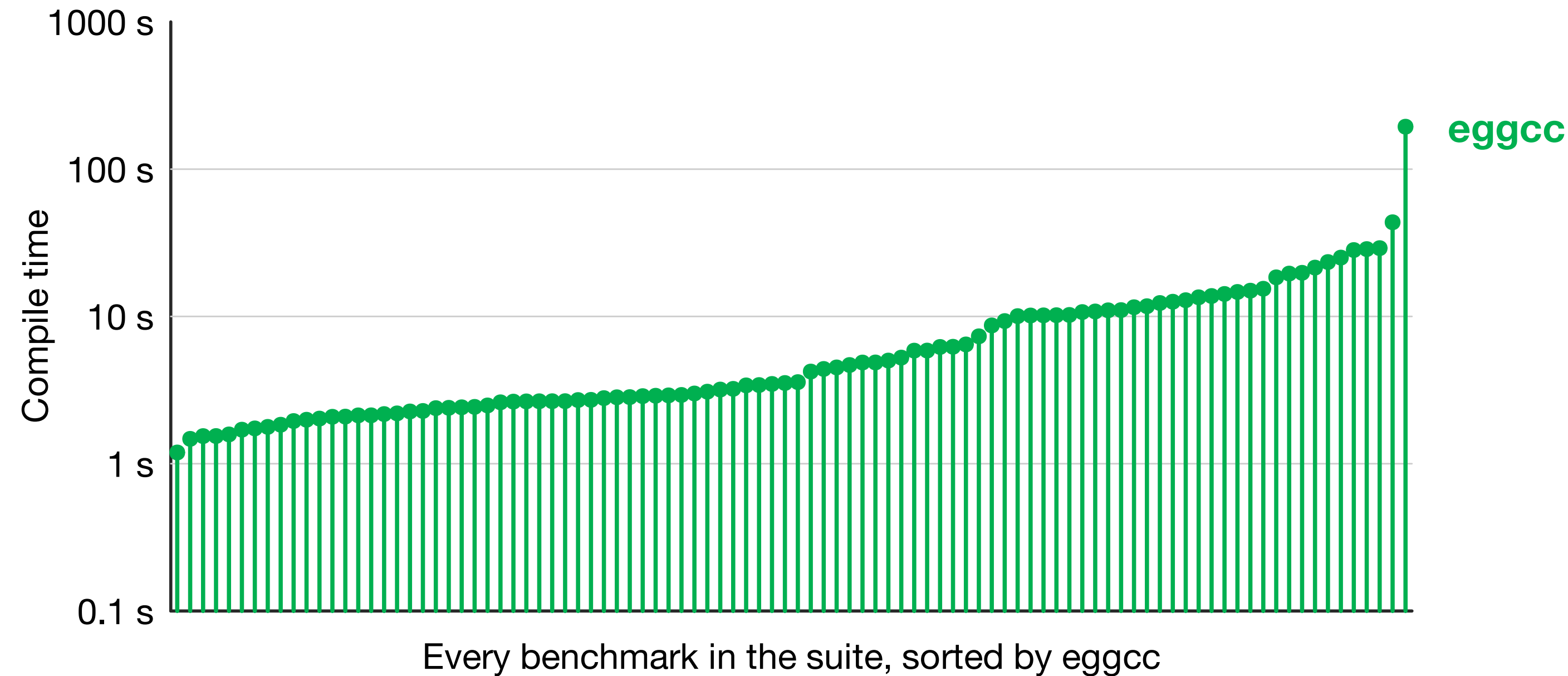
compiled in **3¼ minutes**, 23 s of it extraction

the ILP solver timed out on **35** of the 724

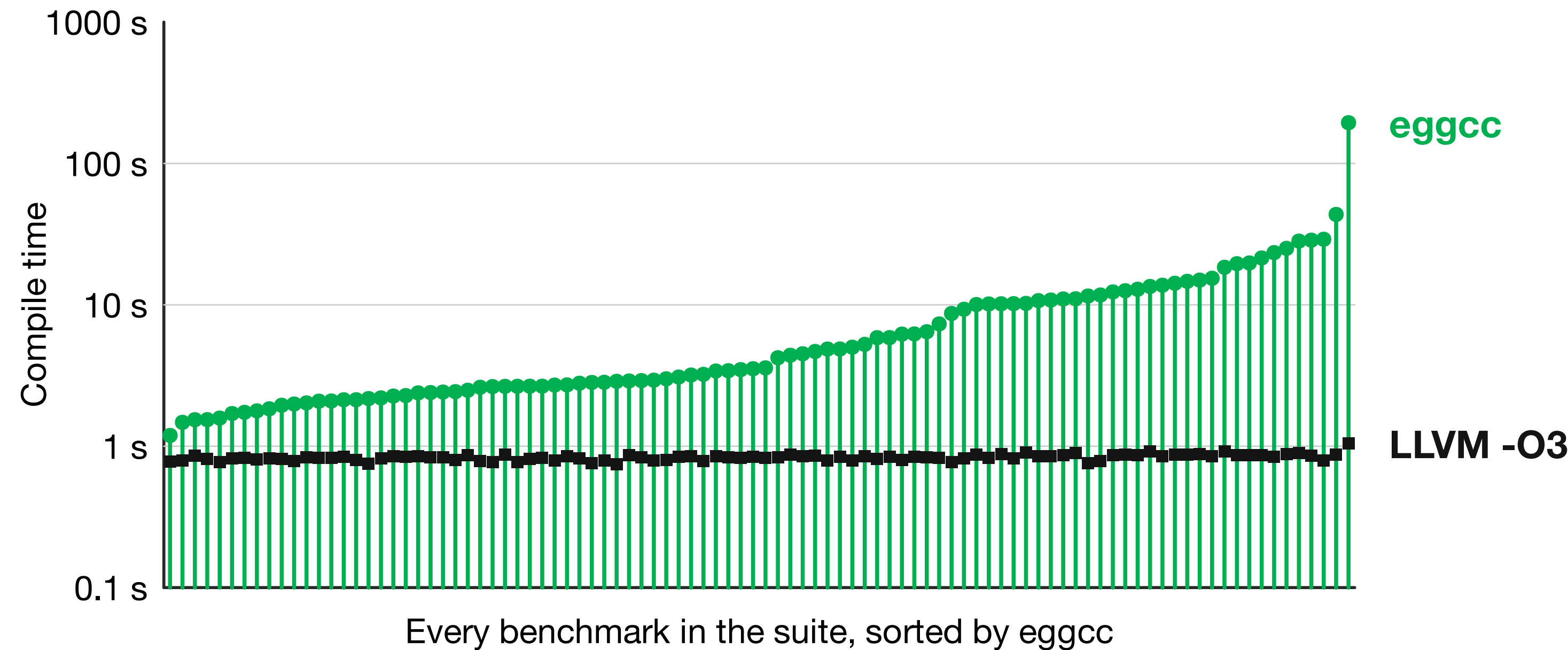
eggcc is Slow to Compile



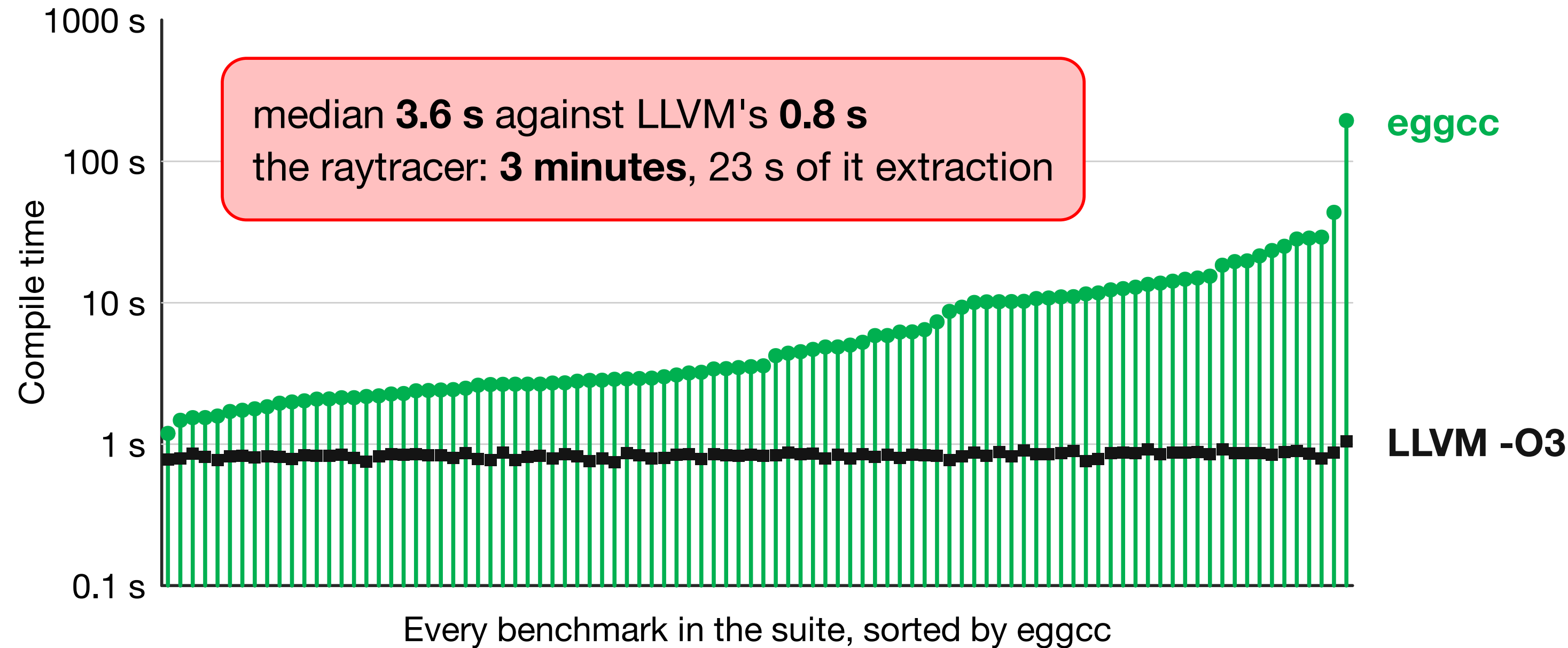
eggcc is Slow to Compile



eggcc is Slow to Compile

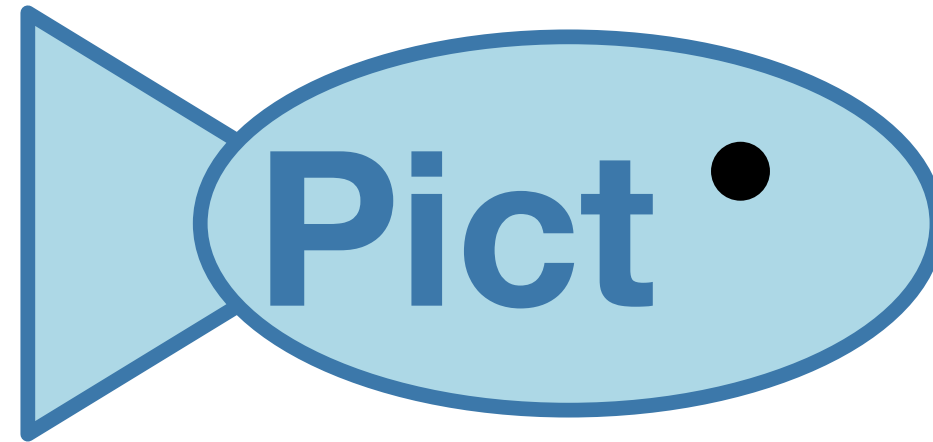


eggcc is Slow to Compile

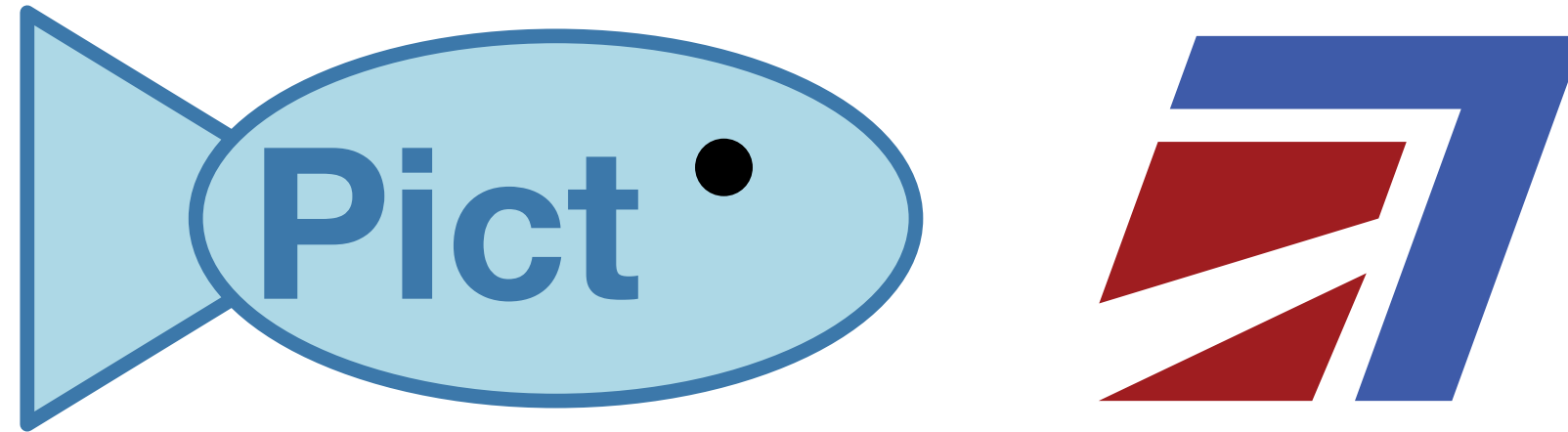


How was this slideshow made?

How was this slideshow made?

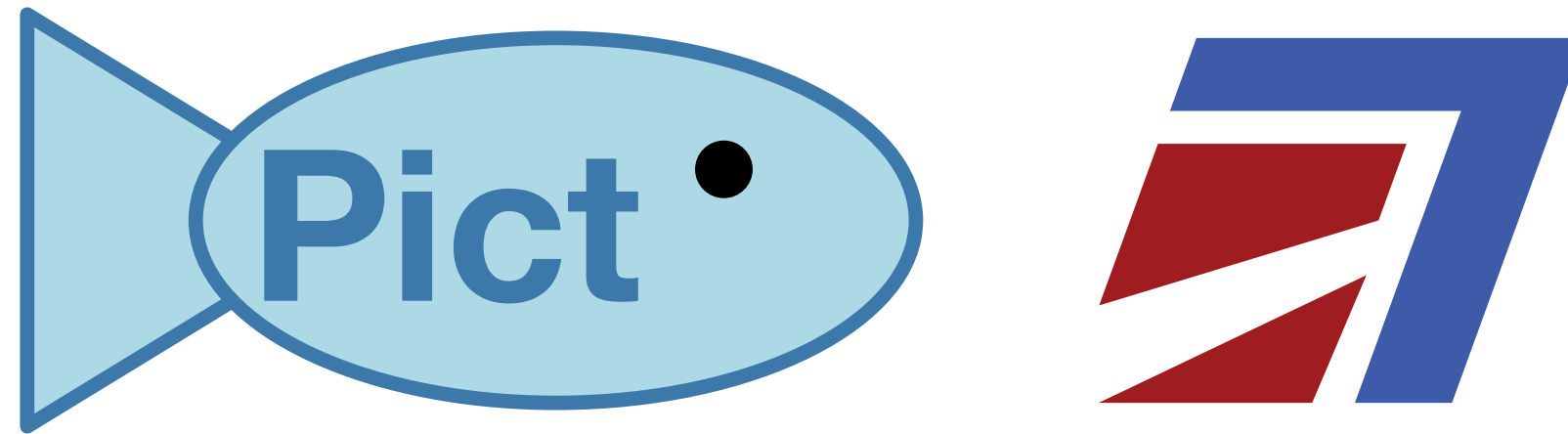


How was this slideshow made?



Rhombus + pict: animated pictures as values

How was this slideshow made?



Rhombus + pict: animated pictures as values

Animated Pictures for Slide Presentations, ICFP 2026